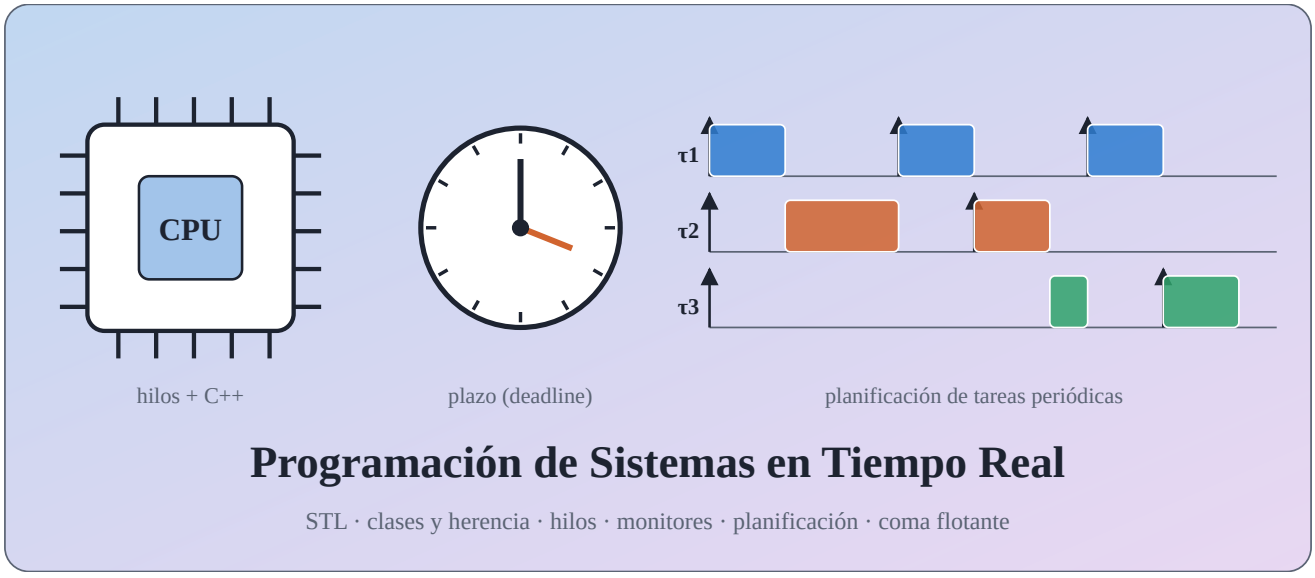
Programación de Sistemas en Tiempo Real

Guía de estudio completa

Teoría desde cero, prácticas, exámenes resueltos, demostraciones, autoevaluación y simulacros

Versión completa con todas las soluciones visibles.



Los simuladores interactivos están en la versión web de la guía; aquí aparecen como capturas.

Última actualización: 2026-11-08

Índice general

1 Empieza aquí **4**
1.1 De qué va esta asignatura . . . 4
1.2 Qué necesitas saber antes de empezar . . . 4
1.3 Cómo usar la guía . . . 5
1.4 Tu progreso . . . 5

2 El examen y cómo estudiar **6**
2.1 Formato del examen . . . 6
2.2 Qué cae más . . . 6
2.3 Estrategia para el examen . . . 6

3 T0 · Sistemas de tiempo real **8**
3.1 Qué es un sistema de tiempo real . . . 8
3.2 Plazos, periodos y jitter . . . 8
3.3 Dos filosofías de diseño . . . 9
3.4 Ciclo de vida de un sistema de tiempo real . . . 9
3.5 Qué necesita un sistema operativo de tiempo real . . . 9

4 T1 · Repaso de C y C++ **11**
4.1 Valores, referencias y punteros . . . 11
4.2 Flujos de entrada y salida . . . 12
4.3 string, vector, map y set . . . 12
4.4 Lambdas y algoritmos . . . 15
4.5 Plantillas y enlaces estructurados . . . 15
4.6 Trucos y ejemplos explicados . . . 16

5 T2 · Clases y objetos **18**
5.1 Clase, objeto y encapsulación . . . 18
5.2 Cuándo hay que escribir copia y destructor (regla de 0, 3 y 5) . . . 19
5.3 Sobrecarga de operadores . . . 20
5.4 Miembros estáticos y composición . . . 20
5.5 Ejemplo completo y plantilla de clase . . . 21

6 T3 · Herencia y polimorfismo **23**
6.1 Herencia: la relación «es un» . . . 23
6.2 Polimorfismo: un puntero a la base, muchos comportamientos . . . 23
6.3 Clases abstractas y destructor virtual . . . 24
6.4 Ejemplo completo . . . 25
6.5 Trampas del polimorfismo, con código . . . 26

7 T4 · Hilos y exclusión mutua **29**
7.1 Crear hilos con `std::thread` . . . 29
7.2 La condición de carrera . . . 29
7.3 Mutex y bloqueos . . . 30
7.4 Interbloqueo (deadlock) . . . 31
7.5 Tareas periódicas: `sleep_until` frente a `sleep_for` . . . 32

7.6 Plantillas y trucos de hilos . . . 33

8 T5 · Monitores y variables condición 36
8.1 El problema: esperar a que se cumpla una condición . . . 36
8.2 `condition_variable`: cómo se usa siempre . . . 36
8.3 Monitor . . . 37
8.4 Cómo resolver un ejercicio de monitor en el examen . . . 38
8.5 Plantilla de monitor genérica . . . 39

9 T6 · Planificación 41
9.1 Qué hay que decidir: quién ejecuta y cuándo . . . 41
9.2 Ejecutivo cíclico . . . 41
9.3 Prioridades fijas: RM y DM . . . 42
9.4 Recursos compartidos: inversión de prioridad . . . 44
9.5 EDF y utilización . . . 45
9.6 Cuando RM falla y EDF no: ejemplo explicado . . . 46

10 T7 · Coma flotante y casos reales 49
10.1 Números reales en el ordenador . . . 49
10.2 Caso 1: el misil Patriot (Dhahran, 1991) . . . 50
10.3 Caso 2: Mars Pathfinder (1997) . . . 51
10.4 Experimentos con coma flotante . . . 52

11 Prácticas 54
11.1 Práctica 1 · Programación basada en objetos: strings y colecciones . . . 54
11.2 Práctica 2 · Programación orientada a objetos . . . 57
11.3 Práctica 3 · Programación concurrente . . . 60

12 Exámenes resueltos 66
12.1 Empieza por aquí . . . 66
12.2 Practicar por tema . . . 66
12.3 Buscar un examen: enunciados por curso . . . 66
12.4 Resoluciones paso a paso . . . 123

13 Hojas de problemas 152
13.1 Hoja 1 · Cálculo numérico y bucles . . . 152
13.2 Hoja 2 · Conjuntos y cadenas . . . 153
13.3 Hoja 3 · Clases . . . 154

14 Demostraciones 157
14.11 · Cota de Liu–Layland para dos tareas . . . 157
14.22 · Análisis de tiempo de respuesta . . . 158
14.33 · EDF es óptimo con $U \leq 1$ . . . 159
14.44 · Protocolo de techo inmediato (ICPP) . . . 160
14.55 · Restricciones del ejecutivo cíclico . . . 161
14.66 · RM con periodos armónicos . . . 161
14.77 · Pérdida de actualización . . . 162
14.88 · `push_back` en $O(1)$ amortizado . . . 163
14.99 · El error del Patriot . . . 164

15 Autoevaluación **165**
15.1 Test con corrección . . . 165
15.2 Tarjetas de repaso . . . 171

16 Simulacros **173**
16.1 Simulacro 1 · Biblioteca y parking . . . 173
16.2 Simulacro 2 · Matriz y tarea periódica . . . 176
16.3 Simulacro 3 · Planificación . . . 179

17 Formularios **181**
17.1 T0 · Sistemas de tiempo real . . . 181
17.2 T1 · Repaso de C y C++ . . . 181
17.3 T2 · Clases y objetos . . . 181
17.4 T3 · Herencia y polimorfismo . . . 181
17.5 T4 · Hilos y exclusión mutua . . . 182
17.6 T5 · Monitores y variables condición . . . 182
17.7 T6 · Planificación . . . 182
17.8 T7 · Coma flotante y casos reales . . . 182
17.9 Prácticas . . . 182

18 Glosario **184**

19 Plan de estudio **187**
19.1 Plan por semanas . . . 187

1 Empieza aquí

1.1 De qué va esta asignatura

Programación de Sistemas en Tiempo Real te enseña dos cosas a la vez:

1. **Programar bien en C++**: clases, herencia, polimorfismo y la biblioteca estándar (strings, vectores, mapas).
2. **Programar cosas que van a la vez y con plazos**: varios hilos que comparten datos, esperan unos a otros y tienen que acabar a tiempo.

Un sistema de tiempo real no es «un sistema muy rápido». Es un sistema que tiene que **responder dentro de un plazo**. Un airbag que se infla un segundo tarde es un airbag que no funciona.

Idea clave

En tiempo real importa **cuándo** se obtiene un resultado, no solo **cuál** es. Un resultado correcto fuera de plazo es un resultado incorrecto.

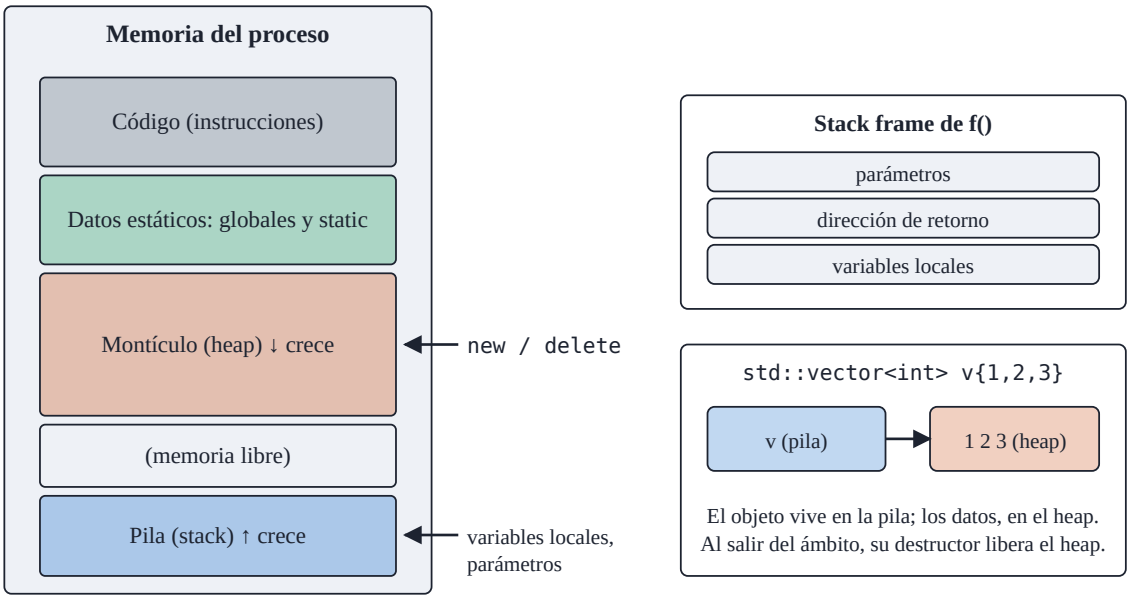
El examen mezcla las dos mitades: un ejercicio grande de **programación orientada a objetos** (el 70 %) y uno de **concurrencia** (el 30 %). En convocatorias antiguas también caía **planificación** con cálculo de tiempos de respuesta. La guía cubre todo.

1.2 Qué necesitas saber antes de empezar

Si partes de cero, estos cinco conceptos son suficientes para entrar en el tema T1. Los repasamos aquí en pocas líneas.

Variable, tipo y memoria

Una **variable** es un trozo de memoria con nombre y con un **tipo** que dice cuántos bytes ocupa y cómo se interpretan. `int n = 5;` reserva 4 bytes (normalmente) y guarda el número 5.



Función

Una **función** recibe valores, hace algo y puede devolver un valor. En C++ los parámetros se **copian** por defecto. Para no copiar (o para modificar el original) se pasa una **referencia**: `void f(int& x)`.

Compilar y ejecutar

El **compilador** (`g++`) traduce el texto `.cpp` a un programa ejecutable. Si hay un error de sintaxis, no hay programa. En los exámenes recientes, **si el programa no compila, la nota se divide por 2**. Compilar tu

código a menudo es la mejor costumbre que puedes tener.

```
g++ -std=c++17 -Wall -Wextra -pthread programa.cpp -o programa
./programa
```

Objeto y clase

Una **clase** es un molde: dice qué datos (atributos) y qué operaciones (métodos) tiene algo. Un **objeto** es una pieza concreta hecha con ese molde. `std::string` es una clase; `std::string s{"hola"};` es un objeto.

Hilo

Un **hilo** es una secuencia de instrucciones que el sistema operativo puede ejecutar a la vez que otras. Dos hilos que tocan la misma variable sin cuidado producen resultados impredecibles. Todo el bloque de concurrencia va de evitar eso.

1.3 Cómo usar la guía

- **Orden recomendado:** T0 → T1 → T2 → T3 (programación, 70 % del examen), después T4 → T5 (concurrencia) y, si tu convocatoria lo pide, T6 (planificación).
- Cada tema tiene teoría con intuición, ejemplos, un simulador, un recuadro **«Qué suele fallar»** y las fórmulas o plantillas clave.
- En **Exámenes resueltos**, abre cada paso solo después de intentarlo tú. Los pasos están plegados a propósito.
- Marca tu progreso abajo. Se guarda en tu navegador.
- Pulsa / para buscar en cualquier momento.

Qué suele fallar al estudiar esta asignatura

Leer soluciones no es lo mismo que saber escribirlas. El examen es **escribir código a mano** (o en un editor sin ayuda) con una hoja de referencia de la biblioteca estándar. Practica escribiendo, no leyendo.

1.4 Tu progreso

Marca a mano lo que vayas terminando (la versión web lo guarda sola).

☐ He entendido qué es tiempo real duro, firme y blando (T0)
☐ Sé usar string, vector, map, set e iteradores sin mirar la guía (T1)
☐ Sé escribir una clase con constructor, `operator<<` y regla de 0/3/5 (T2)
☐ Sé escribir una jerarquía con clase abstracta, `virtual`, destructor virtual y contenedor de punteros (T3)
☐ Sé crear hilos, usar `mutex`, `lock_guard` y `scoped_lock` (T4)
☐ Sé escribir un monitor con `condition_variable` y predicado (T5)
☐ Sé calcular prioridades, bloqueos y tiempos de respuesta (T6)
☐ Sé explicar el error del Patriot y por qué 0,1 no es exacto en binario (T7)
☐ He escrito y ejecutado las tres prácticas
☐ He resuelto sin ayuda los cuatro exámenes recientes de «Empieza por aquí»
☐ He hecho al menos dos simulacros completos con cronómetro
☐ P1: escribí y ejecuté el histograma de palabras
☐ P2: escribí complejo, mensajes y actividades
☐ P3: escribí suma por filas, filósofos y almacén

2 El examen y cómo estudiar

2.1 Formato del examen

Lo que dicen los exámenes más recientes del material (cursos 2023-24 a 2025-26):

Dato	Valor
Duración	2 h 30 min
Ejercicio 1	Programación orientada a objetos (**70 %**)
Ejercicio 2	Concurrencia (**30 %**)
Compilación	En 2026: «si el programa no compila, la puntuación se divide por 2»
Ayuda	Hoja con referencia de clases de la biblioteca estándar (string, pair, vector, map, set…)

Antes (cursos 2015-16 a 2016-17) el examen duraba 3 h y tenía además un ejercicio de **planificación** (prioridades, techos, bloqueos y tiempos de respuesta), una pregunta de desarrollo y un ejercicio de prácticas.

Evaluación del curso 2025-26

Parte	Peso	Condición
Examen de programación	70 %	Nota mínima 4 sobre 10
Seguimiento de prácticas (3 cuestionarios)	20 %	Sin nota mínima
Trabajo de planificación	10 %	Sin nota mínima

Se aprueba si la nota del examen supera el mínimo y la suma de las partes es al menos 5. Hay dos convocatorias de recuperación: el 8 de junio (tarde) y el 20 de julio (mañana). Se mantienen las notas de las actividades con nota superior a la mínima.

Compruébalo

Esas fechas y pesos son las del curso 2025-26. Si cursas otro año, confirma en la información oficial de la asignatura.

2.2 Qué cae más

Contamos cuántos ejercicios de cada tema aparecen en los 26 exámenes del material.

Tema	Ejercicios	Peso	Desde 2022-23	Antes
C++ básico y STL (strings, vector, map, set, flujos)	43	34 %	0	43
Clases y objetos	28	22 %	7	21
Herencia y polimorfismo	14	11 %	4	10
Hilos y exclusión mutua	13	10 %	1	12
Monitores y variables condición	22	17 %	8	14
Planificación de tiempo real	7	6 %	0	7

Conteo automático por palabras clave sobre los 127 ejercicios con enunciado de los exámenes; cada ejercicio cuenta una vez, en su tema principal. Es una estimación, no una estadística oficial.

Lectura rápida:

- **C++ con la biblioteca estándar, clases y herencia** suman la mayor parte. Son el 70 % del examen reciente.
- En los últimos años la concurrencia es **siempre** un monitor con `mutex` y `condition_variable` (o una variante con varios `mutex` y `scoped_lock`).
- La **planificación** solo aparece en los exámenes antiguos, pero sigue siendo materia del curso (trabajo de planificación).

2.3 Estrategia para el examen

1. **Lee los dos ejercicios enteros** (5 minutos). Decide el orden: empieza por el que tengas más claro.

2. **Reparte el tiempo** según el peso: unos 105 min para POO y 45 min para concurrencia, más 10-15 min de revisión.
3. **Escribe el esqueleto primero**: `#include`, clases con atributos y firmas de métodos. Rellena después.
4. **Compila mentalmente** al final: `;` tras las llaves de clase, `const` donde corresponda, `#include` de todo lo que uses, `override` y `virtual`.
5. **Haz el `main`** que pide el enunciado, aunque sea corto. Suele puntuar.
6. En concurrencia, **dibuja** los estados compartidos: qué variable protege cada `mutex` y qué condición espera cada `condition_variable`.

Qué suele fallar en el examen

- Olvidar el `;` al final de una clase.
- Usar `map[clave]` para **consultar** (crea la entrada). Usa `find` o `at`.
- Borrar de un `vector` dentro de un `for` sin actualizar el iterador.
- Esperar en una `condition_variable` sin predicado (se despierta en falso o se pierde el aviso).
- Dos `mutex` bloqueados en orden distinto: interbloqueo.

3 T0 · Sistemas de tiempo real

3.1 Qué es un sistema de tiempo real

Un **sistema de tiempo real** es un sistema cuya corrección depende de dos cosas: que el resultado sea correcto **y** que llegue **a tiempo**. El tiempo no es un adorno: forma parte de la especificación.

Piensa en tres ejemplos:

- El **airbag** de un coche. Si se infla 50 ms tarde, ya no sirve.
- Un **reproductor de vídeo**. Si un fotograma llega tarde, se nota un salto, pero la película sigue.
- Un **cajero** que responde en 2 s en lugar de 1 s. Molesta, nada más.

Los tres son «tiempo real», pero fallar tiene consecuencias muy distintas. De ahí la clasificación.

Tiempo real duro, firme y blando

Tipo	Si pierde el plazo…	Ejemplo
Duro (*hard*)	Fallo catastrófico. Nunca debe pasar.	Airbag, control de vuelo
Firme (*firm*)	El resultado tardío no sirve y se descarta, pero el sistema sigue. Se toleran pocos fallos.	Fotograma de una cadena de producción que ya pasó
Blando (*soft*)	El resultado tardío sirve, pero con menos valor.	Vídeo en streaming, interfaz de usuario

Intuición

Pregúntate: «¿qué pasa si esta tarea acaba tarde?». Si la respuesta es «alguien puede morir o se rompe una máquina», es duro. Si es «el resultado ya no vale», es firme. Si es «vale menos», es blando.

Vídeo para entrar en materia (4 min, en inglés): qué significa «tiempo real» y por qué no es «ir rápido».

Vídeo recomendado

What is a Real-Time System?
JLabs · 4:45
youtube.com/watch?v=ocHjg6bVZX0

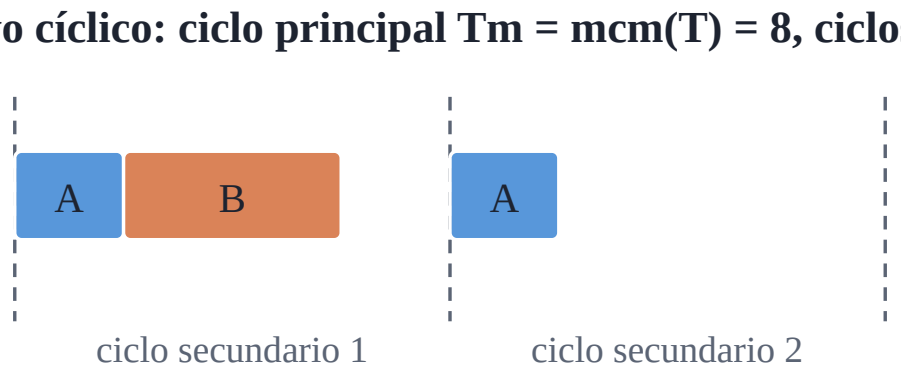
3.2 Plazos, periodos y jitter

Una **tarea** es una unidad de trabajo del sistema. Tres números la describen:

- **C**: tiempo de cómputo en el peor caso.
- **T**: periodo (cada cuánto se activa, si es periódica) o separación mínima entre activaciones (si es esporádica).
- **D**: plazo relativo (cuánto tiempo tiene desde que se activa hasta que debe acabar).

Normalmente $D \leq T$. Si $D = T$, el plazo de una activación termina justo cuando llega la siguiente.

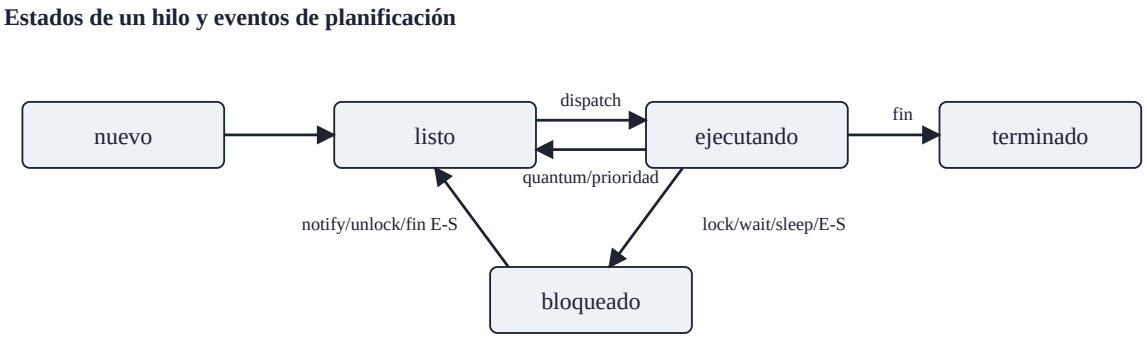
El **jitter** es la variación del instante real en que una tarea periódica arranca o termina respecto al instante ideal. En el código, `sleep_until` produce periodos estables, mientras que `sleep_for` acumula deriva (lo vemos en T4).



3.3 Dos filosofías de diseño

- **Dirigido por tiempo** (*time-triggered*): todo ocurre en instantes fijados de antemano (un reloj manda). Es predecible y fácil de analizar. Se implementa con un **ejecutivo cíclico** (T6).
- **Dirigido por eventos** (*event-triggered*): las tareas se activan cuando ocurre algo (una interrupción, un mensaje). Es flexible y responde rápido, pero analizarlo es más difícil. Se implementa con **prioridades** y un planificador expulsivo (T6).

3.4 Ciclo de vida de un sistema de tiempo real



Se parte de los **requisitos** (incluidos los temporales), se hace el **diseño** y la **implementación**, y se **verifica** que se cumplen los plazos. Esa verificación no se hace «probando a ver si va»: se hace con **análisis** (T6), porque un fallo de plazo puede aparecer una vez cada millón de ejecuciones.

3.5 Qué necesita un sistema operativo de tiempo real

- **Determinismo**: tiempos de respuesta acotados y conocidos para sus servicios.
- **Planificación por prioridades con expulsión**: la tarea más urgente interrumpe a una menos urgente.
- **Temporizadores precisos** y mecanismos para tareas periódicas.
- **Sincronización** sin inversión de prioridad no acotada (T6).
- **Gestión de memoria** predecible: evitar la reserva dinámica impredecible durante la parte crítica.

Qué suele fallar

- Confundir «rápido» con «tiempo real». Un sistema lento pero predecible puede ser de tiempo real; uno rápido en media pero con picos no.
- Diseñar con el tiempo **medio** en lugar del **peor caso** (C es siempre el peor caso).

Definiciones clave

- Tarea periódica: (C, T, D) con $D \leq T$ habitual.
- Utilización de una tarea: $U_i = C_i/T_i$.
- Utilización del sistema: $U = \sum_i C_i/T_i$.

4 T1 · Repaso de C y C++

4.1 Valores, referencias y punteros

Cuando llamas a una función, ¿trabaja con tu variable o con una copia? De eso depende casi todo en C++.

Forma	Sintaxis	¿Modifica el original?
Por valor	`void f(int x)`	No: trabaja con una copia
Por referencia	`void f(int& x)`	Sí: `x` es otro nombre de la variable
Por referencia constante	`void f(const string& s)`	No, y **sin copiar** (se usa para objetos grandes)
Por puntero	`void f(int* p)`	Sí: `*p` es lo apuntado; `p` puede ser `nullptr`

CPP

```cpp
#include <iostream>
using namespace std;

void por_valor([[maybe_unused]] int x) { x = 99; }  // modifica una copia
void por_referencia(int& x) { x = 99; }       // modifica el original
void por_puntero(int* p)   { *p = 99; }       // modifica lo apuntado

int main() {
    int a = 1, b = 1, c = 1;
    por_valor(a); por_referencia(b); por_puntero(&c);
    cout << "a=" << a << " b=" << b << " c=" << c << '\n';

    int* d = new int{7};          // memoria dinámica (heap)
    cout << "d apunta a " << *d << '\n';
    delete d;                     // hay que liberarla
    d = nullptr;

    int arr[3] = {10, 20, 30};
    int* p = arr;                 // un array decae a puntero a su primer elemento
    cout << *(p + 2) << ' ' << arr[2] << '\n';
    const int k = 5;              // no se puede modificar
    cout << "k=" << k << '\n';
}
```

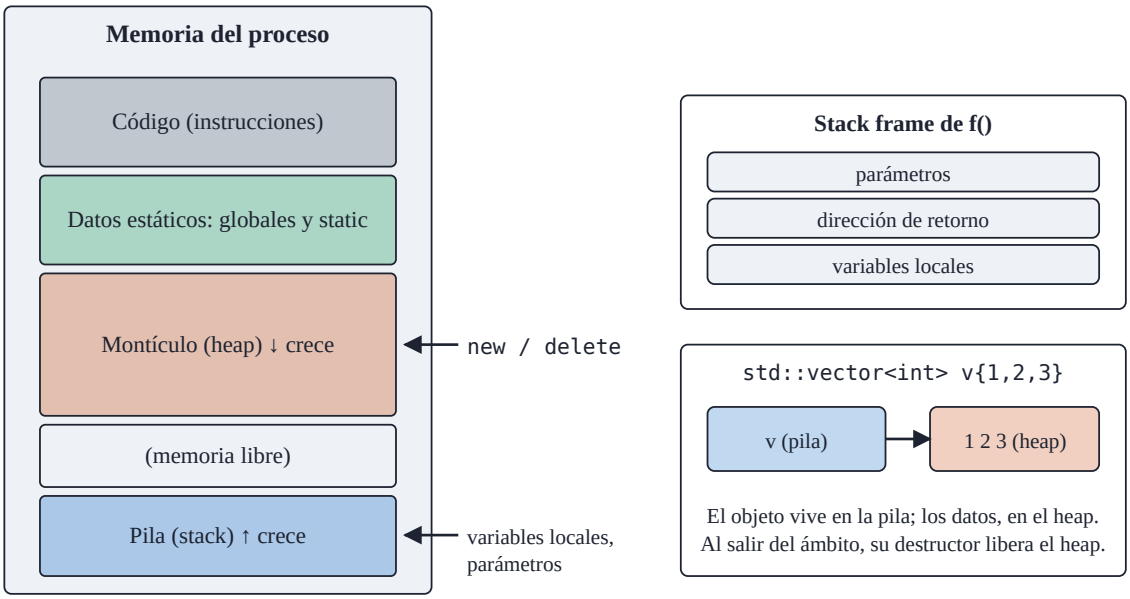
Salida

```
a=1 b=99 c=99
d apunta a 7
30 30
k=5
```

Regla práctica

Objetos pequeños (`int`, `double`, `char`): por valor. Objetos grandes que solo lees (`string`, `vector`, tus clases): `const T&`. Si la función debe modificar el argumento: `T&`.

Pila y montón (stack y heap)



Las variables locales viven en la **pila** (stack): se crean al entrar en la función y desaparecen al salir. La memoria que pides con `new` vive en el **montón** (heap) hasta que haces `delete`. Si pierdes el puntero sin hacer `delete`, tienes una **fuga de memoria**. Si usas el puntero después de hacer `delete`, tienes un **puntero colgante**: comportamiento indefinido.

Qué suele fallar

- Devolver una referencia o un puntero a una variable local: la variable muere al salir de la función.
- `new` sin `delete` (fuga) o `delete` dos veces (error).
- `new[]` se libera con `delete[]`, no con `delete`.
- Un array pasado a una función pierde su tamaño: es solo un puntero.

4.2 Flujos de entrada y salida

`cout` escribe, `cin` lee. `cin >> x` lee una palabra (se detiene en espacios). `getline(cin, s)` lee la línea entera. Un `stringstream` es un flujo que lee o escribe en un `string`: es la herramienta para trocear texto.

CPP

```
istringstream in("Ana, E, 8");
string nombre, tipo; int hora; char coma;
getline(in, nombre, ',');   // "Ana"
in >> ws;                    // salta el espacio
getline(in, tipo, ',');     // "E"
in >> hora;                  // 8
```

Para volcar un objeto en un flujo se define `operator<<` (lo vemos en T2). Los ficheros usan `ifstream` y `ofstream` de `<fstream>` con la misma interfaz.

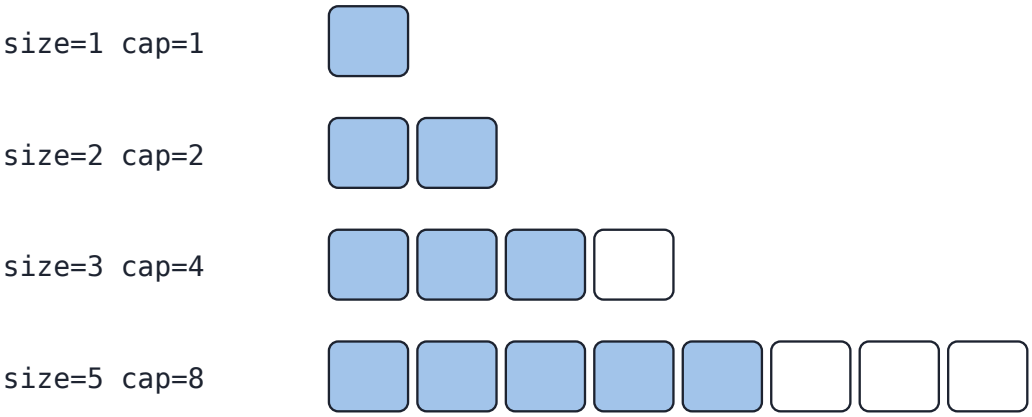
4.3 string, vector, map y set

Contenedor	Qué es	Coste típico
`string`	Texto que crece solo	Acceso `s[i]` O(1)
`vector<T>`	Array que crece al final	`push_back` O(1) amortizado; insertar/borrar en medio O(n)
`map<K,V>`	Pares clave→valor ordenados por clave	Buscar, insertar, borrar: O(log n)
`set<T>`	Conjunto ordenado sin repetidos	O(log n)

vector: size y capacity

`size()` son los elementos que hay. `capacity()` es el espacio reservado. Cuando `size` llega a `capacity`, el vector reserva un bloque mayor, **copia** todo y libera el antiguo. Por eso `reserve(n)` evita copias si sabes cuántos elementos vas a meter.

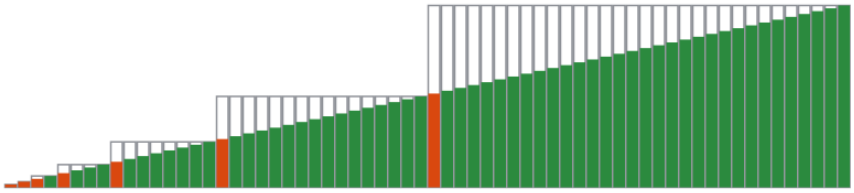
push_back sobre un vector: la capacidad se duplica y los datos se copian a otra zona



Al crecer (cap 4 → 8) se reserva memoria nueva, se copian los 4 elementos y se libera la vieja:
los punteros, referencias e iteradores a elementos del vector antiguo quedan INVALIDADOS.

std::vector: capacidad y copias con push_back

push_back × crecimiento factor 2 (GCC) reserve inicial 0



size final = **64**, capacity final = **64**, realocaciones = **7** (barras naranjas), elementos copiados = **63**.
Con `reserve(64)` serían 1 realocación y 0 copias.

Captura del simulador interactivo (disponible en la versión web de la guía).

map: ojo con `[]`

`m[clave]` **crea** la entrada con valor por defecto si no existía. Para consultar sin modificar, usa `find`. Para contar, `++m[clave]` es perfecto.

CPP

```cpp
#include <algorithm>
#include <iostream>
#include <map>
#include <set>
#include <sstream>
#include <string>
#include <vector>
using namespace std;

int main() {
    // 1) vector: size frente a capacity
```

```
    vector<int> v;
    size_t cap = v.capacity();
    for (int i = 1; i <= 17; ++i) {
        v.push_back(i);
        if (v.capacity() != cap) { cap = v.capacity(); cout << "size " << v.size() << " -> capacity " << cap << '\n'; }
    }
    // 2) map: contar palabras. find no crea entradas; [] sí
    map<string, int> hist;
    istringstream in("el que lee el que sabe el");
    string w;
    while (in >> w) ++hist[w];
    for (const auto& [palabra, n] : hist) cout << palabra << ": " << n << '\n';
    cout << "tamaño antes: " << hist.size() << '\n';
    if (hist.find("zzz") == hist.end()) cout << "zzz no está\n";
    int x = hist["zzz"];   // ¡crea la entrada con valor 0!
    cout << "tamaño después de hist[\"zzz\"]: " << hist.size() << " (x=" << x << ")\n";
    // 3) borrar del vector con iteradores
    vector<int> n{1, 2, 3, 4, 5, 6};
    for (auto it = n.begin(); it != n.end();) {
        if (*it % 2 == 0) it = n.erase(it);  // erase devuelve el siguiente
        else ++it;
    }
    for (int e : n) cout << e << ' ';
    cout << '\n';
    // 4) sort con lambda y set
    vector<string> s{"pera", "Manzana", "kiwi", "uva"};
    stable_sort(s.begin(), s.end(), [](const string& a, const string& b) { return a.size() < b.size(); });
    for (auto& e : s) cout << e << ' ';
    cout << '\n';
    set<int> st{5, 1, 3, 1, 5};
    cout << "set: ";
    for (int e : st) cout << e << ' ';
    cout << '\n';
}
```

Salida

```
size 1 -> capacity 1
size 2 -> capacity 2
size 3 -> capacity 4
size 5 -> capacity 8
size 9 -> capacity 16
size 17 -> capacity 32
el: 3
lee: 1
que: 2
sabe: 1
tamaño antes: 4
zzz no está
tamaño después de hist["zzz"]: 5 (x=0)
1 3 5
uva pera kiwi Manzana
set: 1 3 5
```

Iteradores y borrado

Un **iterador** es un «puntero inteligente» a un elemento de un contenedor. `begin()` apunta al primero y `end()` a **una posición después del último**. Para recorrer: `for (auto it = v.begin(); it != v.end(); ++it)`. Para borrar mientras recorres, usa el valor que devuelve `erase`:

CPP

```
for (auto it = v.begin(); it != v.end();) {
    if (condicion(*it)) it = v.erase(it);   // erase devuelve el siguiente
    else ++it;
}
```

Qué suele fallar

- Después de `erase(it)`, `it` **ya no vale**. Hacer `++it` es comportamiento indefinido.
- `push_back` puede reubicar el vector: invalida iteradores y referencias a sus elementos.
- `v.size()` es **sin signo**: `v.size() - 1` con un vector vacío da un número enorme.

¿Qué contenedor de la STL uso?

`vector`
secuencia con acceso por índice;
el valor por defecto

`map`
clave → valor, ordenado por clave;
contar con ++m[k]

`set`
elementos únicos y ordenados;
¿está o no está?

`unordered_map`
clave → valor sin orden;
búsqueda media O(1)

`Recorrer y modificar: for (auto& x : v) · ordenar: sort(begin,end,lambda) · borrar: erase(remove_i`

Vídeo recomendado (en inglés): repaso de los contenedores de la STL con ejemplos; fíjate en cuándo conviene `vector`, `map` y `set`.

Vídeo recomendado

Back To Basics: C++ Containers
javidx9 · 31:41
youtube.com/watch?v=6OoSgY6NVVk

4.4 Lambdas y algoritmos

Una **lambda** es una función sin nombre que escribes en el sitio donde la usas: `[captura](parámetros){ cuerpo }`. La captura `[&]` toma las variables locales por referencia; `[=]`, por copia.

CPP

```
int umbral = 3;
auto n = count_if(v.begin(), v.end(), [&](int x) { return x > umbral; });
auto it = find_if(v.begin(), v.end(), [](int x) { return x % 2 == 0; });
sort(v.begin(), v.end(), [](int a, int b) { return a > b; });   // de mayor a menor
```

Algoritmos útiles de `<algorithm>`: `sort`, `stable_sort`, `find`, `find_if`, `count_if`, `max_element`, `min_element`, `remove_if` (con `erase`, el «borrado-eliminación»), `accumulate` (en `<numeric>`).

CPP

```
v.erase(remove_if(v.begin(), v.end(), [](int x){ return x < 0; }), v.end());
```

4.5 Plantillas y enlaces estructurados

Una **plantilla** es código que funciona con cualquier tipo: `template<typename T> T maximo(T a, T b) { return a > b ? a : b; }`. El compilador genera una versión por cada tipo con que la uses. `vector<T>` y `map<K,V>` son plantillas.

Los **enlaces estructurados** de C++17 desempaquetan un `pair` o una estructura: `for (const auto& [clave, valor] : mapa)`.

Más a fondo

Por qué `vector` crece con factor multiplicativo. Si cada realocación añadiera un número fijo de huecos, copiar n elementos cada k inserciones daría un coste total cuadrático. Con factor 2, las copias suman $1 + 2 + 4 + \cdots + n/2 < n$, es decir, el coste **amortizado** de `push_back` es constante. Puedes comprobarlo con el simulador: con factor 2 y 17 inserciones se copian $1 + 2 + 4 + 8 + 16 = 31$ elementos. Referencia: Stroustrup, *The C++ Programming Language* (4.ª ed.), capítulo sobre contenedores.

Plantillas rápidas

- Contar palabras: `++hist[w];`
- ¿Existe la clave?: `if (m.find(k) != m.end())`
- Recorrer mapa: `for (const auto& [k, v] : m)`

- Borrar con condición: `it = v.erase(it);` o `v.erase(remove_if(...), v.end());`
- Ordenar por criterio: `sort(b, e, [](const T& a, const T& b){ return ...; });`

4.6 Trucos y ejemplos explicados

Nueve trucos que resuelven el 80 % de los ejercicios de STL. Léelos con el código delante: cada uno está marcado con un comentario `TRUCO n`.

CPP

```cpp
#include <algorithm>
#include <iostream>
#include <map>
#include <sstream>
#include <string>
#include <vector>
using namespace std;

int main() {
    // TRUCO 1: partir una línea en palabras (cin >> corta por espacios)
    istringstream linea{"hola mundo hola tiempo real mundo hola"};
    vector<string> palabras;
    for (string p; linea >> p;) palabras.push_back(p);
    cout << "1) " << palabras.size() << " palabras\n";

    // TRUCO 2: contar con map: ++m[clave] crea la entrada con 0 si no existía
    map<string, int> cuenta;
    for (const auto& p : palabras) ++cuenta[p];
    cout << "2) ";
    for (const auto& [palabra, n] : cuenta) cout << palabra << '=' << n << ' ';
    cout << '\n';

    // TRUCO 3: un map se ordena por CLAVE; para ordenar por VALOR, copia a vector<pair> y sort con lambda
    vector<pair<string, int>> v(cuenta.begin(), cuenta.end());
    sort(v.begin(), v.end(), [](const auto& a, const auto& b) { return a.second != b.second ? a.second > b.second :
    ↪  a.first < b.first; });
    cout << "3) mas frecuente: " << v[0].first << " (" << v[0].second << ")\n";

    // TRUCO 4: borrar por condición = erase(remove_if(...), end)
    vector<int> n{5, 12, 7, 20, 3, 15};
    n.erase(remove_if(n.begin(), n.end(), [](int x) { return x > 10; }), n.end());
    cout << "4) tras borrar los > 10:";
    for (int x : n) cout << ' ' << x;
    cout << '\n';

    // TRUCO 5: borrar dentro de un bucle: el iterador que devuelve erase ya apunta al siguiente
    vector<int> m{1, 2, 3, 4, 5, 6};
    for (auto it = m.begin(); it != m.end();) {
        if (*it % 2 == 0) it = m.erase(it);   // NO avanzar: erase ya lo hizo
        else ++it;
    }
    cout << "5) impares:";
    for (int x : m) cout << ' ' << x;
    cout << '\n';

    // TRUCO 6: `for (auto x : v)` COPIA cada elemento; `auto&` modifica el original
    vector<int> w{1, 2, 3};
    for (auto x : w) x *= 10;          // no cambia w
    cout << "6a) copia: " << w[0] << ' ' << w[1] << ' ' << w[2] << '\n';
    for (auto& x : w) x *= 10;         // cambia w
    cout << "6b) referencia: " << w[0] << ' ' << w[1] << ' ' << w[2] << '\n';

    // TRUCO 7: mezclar >> y getline: >> deja el '\n' en el flujo, getline lo lee y devuelve una línea vacía
    istringstream in{"42\nHola Mundo\n"};
    int edad; string nombre;
    in >> edad;
    getline(in, nombre);
    cout << "7a) sin ignore: [" << nombre << "]\n";
    istringstream in2{"42\nHola Mundo\n"};
    in2 >> edad; in2.ignore(1000, '\n'); getline(in2, nombre);
    cout << "7b) con ignore: [" << nombre << "]\n";

    // TRUCO 8: .at() comprueba límites (lanza excepción); [] no comprueba
    try { w.at(10); } catch (const out_of_range&) { cout << "8) at(10) lanza out_of_range\n"; }
```

```
    // TRUCO 9: find en un map no crea entradas; [] sí
    map<string, int> mm;
    if (mm.find("x") == mm.end()) cout << "9a) tamano tras find: " << mm.size() << '\n';
    (void)mm["x"];
    cout << "9b) tamano tras []: " << mm.size() << '\n';
}
```

Salida

```
1) 7 palabras
2) hola=3 mundo=2 real=1 tiempo=1
3) mas frecuente: hola (3)
4) tras borrar los > 10: 5 7 3
5) impares: 1 3 5
6a) copia: 1 2 3
6b) referencia: 10 20 30
7a) sin ignore: []
7b) con ignore: [Hola Mundo]
8) at(10) lanza out_of_range
9a) tamano tras find: 0
9b) tamano tras []: 1
```

Cómo leer cada truco

1. **Partir una línea en palabras:** `istringstream` + `linea >> p`. Es lo mismo que leer de `cin`, pero de un `string`. Si en el examen te dan «una línea de texto», empieza siempre así.
2. **Contar con `map`:** `++cuenta[p]`. Si `p` no existe, `[]` lo crea con valor 0 y después sube a 1. No necesitas `if (find…)`.
3. **Ordenar por valor:** un `map` siempre está ordenado por **clave**. Para ordenar por frecuencia, copia a `vector<pair<…>>` y usa `sort` con lambda. Añade un **desempate** (`a.first < b.first`) para que el resultado sea siempre el mismo.
4. **Borrar por condición:** `v.erase(remove_if(…), v.end())`. Se lee de dentro hacia fuera: `remove_if` empuja al final lo que se borra y `erase` lo corta.
5. **Borrar dentro de un bucle:** `it = v.erase(it)` y **no** `++it` en ese caso. Si avanzas dos veces, te saltas elementos.
6. **`auto` copia, `auto&` modifica.** Si quieres cambiar el vector, el bucle necesita `&`. Si solo lees objetos grandes, usa `const auto&` para no copiarlos.
7. **`>>` seguido de `getline`:** `>>` deja el salto de línea sin leer; `getline` lo toma y devuelve cadena vacía. Solución: `in.ignore(1000, '\n')` entre ambos.
8. **`at()` frente a `[]`:** `at` comprueba el rango y lanza excepción; `[]` no comprueba nada (comportamiento indefinido).
9. **`find` frente a `[]` en `map`:** `find` solo consulta; `[]` **inserta** si no existe. Para preguntar «¿está?», usa `find` o `count`.

Qué suele fallar (con código)

- `for (auto it = v.begin(); it != v.end(); ++it) v.erase(it);` → **invalida el iterador**. Guarda el resultado de `erase`.
- `v.push_back(x)` dentro de un `for (auto& e : v)` → puede reasignar el vector y dejar `e` colgando.
- `int n = v.size();` con un `vector` vacío en la condición `i < v.size() - 1` → `size()` es sin signo y `0 - 1` es un número enorme. Usa `i + 1 < v.size()`.
- `sort` con un comparador que usa `<=` → comportamiento indefinido. Usa siempre `<` o `>` estrictos.

Plantilla para «lee un fichero y cuenta»

```
ifstream in{nombre};
if (!in) { cerr << "no se pudo abrir\n"; return 1; }
map<string,int> cuenta;
for (string p; in >> p;) ++cuenta[p];
```

Es la base de la práctica 1. Todo lo demás (ordenar, filtrar, imprimir) son los trucos 3 y 4.

5 T2 · Clases y objetos

5.1 Clase, objeto y encapsulación

Una **clase** agrupa **datos** (atributos) y **operaciones** (métodos) que van juntos. La parte `private` esconde los datos; la parte `public` es la «interfaz»: lo único que el resto del programa puede usar. Así puedes cambiar el interior sin romper a nadie y garantizar que el objeto siempre está en un estado válido.

Un ejemplo completo y verificado, el del gestor de cuentas bancarias:

CPP

```cpp
#include <iostream>
#include <string>
using namespace std;

class C_Cuenta {
public:
    C_Cuenta(string titular, int numero) : titular_{std::move(titular)}, numero_{numero} {}
    C_Cuenta() : C_Cuenta("sin titular", 0) {}                 // constructor delegado
    int numero() const { return numero_; }
    double saldo() const { return saldo_; }
    bool ingresar(double c) {
        if (c <= 0) return false;
        saldo_ += c; return true;
    }
    bool reintegrar(double c) {
        if (c <= 0 || saldo_ - c < 0) return false;
        saldo_ -= c; return true;
    }
    bool transferencia(double c, C_Cuenta& destino) {
        if (!reintegrar(c)) return false;
        destino.ingresar(c); return true;
    }
    friend ostream& operator<<(ostream& os, const C_Cuenta& c) {
        return os << c.titular_ << " (ccc " << c.numero_ << "): " << c.saldo_ << " euros";
    }
private:
    string titular_;
    int numero_;
    double saldo_ = 0;           // inicializador en la clase
};

int main() {
    C_Cuenta a{"Mario", 12345678}, b{"Lucia", 87654321};
    a.ingresar(150.5);
    cout << a << '\n' << b << '\n';
    cout << "transferencia 50: " << boolalpha << a.transferencia(50, b) << '\n';
    cout << "transferencia 500: " << a.transferencia(500, b) << '\n';
    cout << a << '\n' << b << '\n';
}
```

Salida

```
Mario (ccc 12345678): 150.5 euros
Lucia (ccc 87654321): 0 euros
transferencia 50: true
transferencia 500: false
Mario (ccc 12345678): 100.5 euros
Lucia (ccc 87654321): 50 euros
```

Fíjate en cuatro cosas, todas habituales en los exámenes:

1. Los métodos que **no modifican** el objeto llevan `const` (`saldo() const`). Solo con `const` se pueden llamar sobre un objeto `const` o a través de `const&`.
2. El atributo `saldo_` tiene un **inicializador en la clase** (`= 0`).
3. El `operator<<` es un `friend` que recibe `ostream&` y **devuelve** el mismo flujo (para encadenar `cout << a << b`).
4. `transferencia` recibe la cuenta destino **por referencia** (`C_Cuenta&`): si fuera por valor, ingresaría el dinero en una copia que desaparece.

Constructores

- La **lista de inicialización** (`: titular_{...}, numero_{...}`) construye los atributos directamente. Es lo normal y a veces obligatorio (atributos `const` o referencias).
- Los atributos se inicializan **en el orden en que están declarados** en la clase, no en el orden de la lista.
- Un **constructor delegado** (`C_Cuenta() : C_Cuenta("sin titular", 0) {}`) reutiliza otro constructor.
- Si marcas un constructor con `explicit`, evitas conversiones implícitas no deseadas.

5.2 Cuándo hay que escribir copia y destructor (regla de 0, 3 y 5)

Cuando copias un objeto, C++ copia sus atributos uno a uno. Si el objeto guarda un **puntero** a memoria dinámica, copia el puntero, no la memoria: los dos objetos comparten el bloque, y al destruirse **liberan el mismo bloque dos veces**.

- **Regla de 0:** si tus atributos son `string`, `vector`, `map`, `unique_ptr`..., **no escribas** destructor ni copias. El compilador hace lo correcto. Es lo que quieres casi siempre.
- **Regla de 3:** si escribes uno de estos, escribe los tres: **destructor**, **constructor de copia** y **asignación por copia**.
- **Regla de 5:** añade también **constructor de movimiento** y **asignación por movimiento**.

CPP

```cpp
#include <algorithm>
#include <iostream>
using namespace std;

// Clase que posee memoria dinámica: necesita copia profunda (regla de los 3)
class Buffer {
public:
    explicit Buffer(size_t n) : n_{n}, d_{new int[n]{}} { cout << "ctor(" << n_ << ")\n"; }
    Buffer(const Buffer& o) : n_{o.n_}, d_{new int[o.n_]} { copy(o.d_, o.d_ + n_, d_); cout << "copia\n"; }
    Buffer& operator=(const Buffer& o) {
        if (this != &o) { Buffer tmp{o}; swap(n_, tmp.n_); swap(d_, tmp.d_); }   // copy-and-swap
        cout << "asignacion\n"; return *this;
    }
    ~Buffer() { delete[] d_; cout << "dtor(" << n_ << ")\n"; }
    int& operator[](size_t i) { return d_[i]; }
private:
    size_t n_;
    int* d_;
};

int main() {
    Buffer a{3};
    a[0] = 7;
    Buffer b = a;      // copia
    b[0] = 9;
    cout << a[0] << ' ' << b[0] << '\n';   // 7 9: cada uno tiene su memoria
    Buffer c{1};
    c = a;             // asignación
}
```

Salida

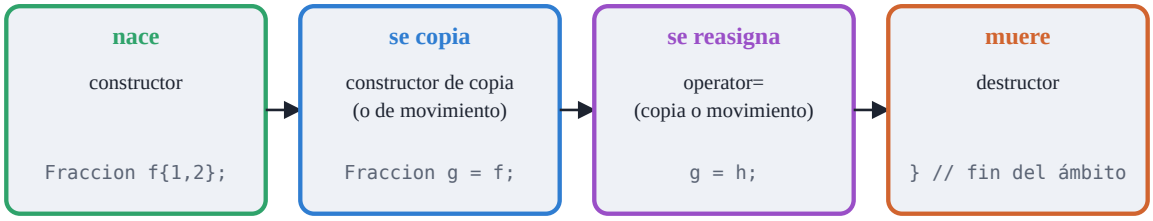
```
ctor(3)
copia
7 9
ctor(1)
copia
dtor(1)
asignacion
dtor(3)
dtor(3)
dtor(3)
```

La asignación usa el truco **copy-and-swap**: copia a un temporal y intercambia. Así es segura aunque falle la copia y aunque hagas `a = a`.

Qué suele fallar

- `Buffer b = a;` con puntero crudo y sin constructor de copia: doble `delete`.
- Escribir la asignación sin proteger contra la autoasignación y borrar antes de copiar.
- Declarar el destructor `private` o olvidarlo `virtual` en una clase base (T3).
- Usar una variable no inicializada: los atributos `int`, `double` o punteros no tienen valor por defecto.

Ciclo de vida de un objeto y qué función especial se ejecuta en cada momento



Regla de los cinco: si escribes UNO de estos cinco, escribe (o prohíbe con =delete) los demás.

`destructor · constr. de copia · = de copia · constr. de movimiento · = de movimiento`

Si la clase solo usa vector/string/unique_ptr: regla del cero (no escribas ninguno).

Vídeo recomendado (en inglés, 5 min): regla de tres y de cinco, es decir, cuándo escribir destructor, copia y asignación.

Vídeo recomendado

Rule Of Three And Rule Of Five In C++ Programming
CppNuts · 4:57
youtube.com/watch?v=RspbK8tfU0Y

5.3 Sobrecarga de operadores

Un operador es una función con un nombre especial. `a + b` es `operator+(a, b)`. Las reglas útiles:

Operador	Forma habitual
`<<` (salida)	Función libre o `friend`: `ostream& operator<<(ostream&, const T&)`
`>>` (entrada)	`istream& operator>>(istream&, T&)` (construye un temporal y haz `swap`)
`+`, `-`	Función libre: `T operator+(const T&, const T&)`. Devuelve un objeto nuevo
`==`, `<`	Función libre o método `const`. Necesarios para `set`, `map`, `sort`
`=`, `[]`, `()`	Siempre métodos

Un `friend`

Un `friend` es una función **fuera** de la clase con permiso para ver sus atributos privados. Si la clase ya tiene métodos `get`, no necesitas `friend`: el operador puede usar los `get`.

5.4 Miembros estáticos y composición

Un atributo `static` es **uno solo para toda la clase** (por ejemplo, un contador de objetos creados). Se declara en la clase y se define fuera (o con `inline static` en C++17).

La **composición** significa que una clase contiene objetos de otras clases: un `Plataforma` tiene un `vector<Suscripcion>`. Es la herramienta más usada; la herencia (T3) se reserva para relaciones «es un».

Más a fondo

Acoplamiento y cohesión. Una clase con **alta cohesión** hace una sola cosa bien; dos clases con **bajo acoplamiento** dependen poco una de otra. Declarar `private` para los atributos y exponer solo operaciones con sentido es la manera práctica de conseguir ambas. Los principios SOLID (en particular, «una clase, una responsabilidad») desarrollan esta idea; el libro de referencia es R. C. Martin, *Clean Architecture*.

Plantilla de clase para el examen

CPP

```cpp
class C_X {
public:
    C_X(tipo a, tipo b) : a_{a}, b_{b} {}          // constructor
    tipo a() const { return a_; }                   // getters const
    friend ostream& operator<<(ostream& os, const C_X& x) { return os << x.a_; }
private:
    tipo a_, b_;
};   // <- el ; final
```

5.5 Ejemplo completo y plantilla de clase

Una clase `Fraccion` que usa casi todo lo que se pide en un examen: constructor con valores por defecto, `const`, operadores aritméticos, comparaciones, ++ prefijo y sufijo, `operator<<` y una excepción. Está comentada con números (1), (2)...

CPP

```cpp
#include <iostream>
#include <numeric>
#include <set>
#include <stdexcept>
using namespace std;

class Fraccion {
public:
    Fraccion(int num = 0, int den = 1) : n_{num}, d_{den} {      // (1) valores por defecto: sirve de constructor vacío
        if (d_ == 0) throw invalid_argument{"denominador 0"};
        simplificar();
    }
    int num() const { return n_; }
    int den() const { return d_; }
    // (2) operadores aritméticos: devuelven un objeto NUEVO y no modifican a los operandos (const)
    Fraccion operator+(const Fraccion& o) const { return {n_ * o.d_ + o.n_ * d_, d_ * o.d_}; }
    Fraccion operator-(const Fraccion& o) const { return {n_ * o.d_ - o.n_ * d_, d_ * o.d_}; }
    Fraccion operator*(const Fraccion& o) const { return {n_ * o.n_, d_ * o.d_}; }
    Fraccion operator/(const Fraccion& o) const { return {n_ * o.d_, d_ * o.n_}; }
    // (3) comparaciones: con == y < ya puedes usar la clase en set, map y sort
    bool operator==(const Fraccion& o) const { return n_ == o.n_ && d_ == o.d_; }   // están simplificadas: basta
     ↪ comparar campos
    bool operator<(const Fraccion& o) const { return n_ * o.d_ < o.n_ * d_; }        // den > 0 siempre
    // (4) ++ prefijo (devuelve referencia) y sufijo (parámetro int ficticio, devuelve copia del valor anterior)
    Fraccion& operator++() { n_ += d_; return *this; }
    Fraccion operator++(int) { Fraccion antes = *this; ++*this; return antes; }
    friend ostream& operator<<(ostream& os, const Fraccion& f) { return f.d_ == 1 ? os << f.n_ : os << f.n_ << '/' <<
     ↪ f.d_; }
private:
    void simplificar() {
        if (d_ < 0) { n_ = -n_; d_ = -d_; }        // el signo siempre en el numerador
        int g = gcd(n_ < 0 ? -n_ : n_, d_);
        if (g > 1) { n_ /= g; d_ /= g; }
    }
    int n_, d_;
};

int main() {
    Fraccion a{1, 2}, b{3, -6}, c{2, 4};
    cout << "a=" << a << " b=" << b << " c=" << c << "   (b = 3/-6 se normaliza a -1/2)\n";
    cout << "a+c=" << a + c << "  a-c=" << a - c << "  a*b=" << a * b << "  a/b=" << a / b << '\n';
    cout << boolalpha << "a==c: " << (a == c) << "  b<a: " << (b < a) << '\n';
    set<Fraccion> s{{1, 2}, {2, 4}, {1, 3}, {-1, 2}};          // 1/2 y 2/4 son la misma: el set la guarda una vez
    cout << "set:";
    for (const auto& f : s) cout << ' ' << f;
```

```
    cout << '\n';
    Fraccion x{1, 2};
    Fraccion y = x++;                                          // y vale lo de antes; x ya incrementado
    cout << "x++ -> y=" << y << ", x=" << x << "   ++x -> " << ++x << '\n';
    try { Fraccion malo{1, 0}; } catch (const invalid_argument& e) { cout << "error: " << e.what() << '\n'; }
}
```

Salida

```
a=1/2 b=-1/2 c=1/2   (b = 3/-6 se normaliza a -1/2)
a+c=1  a-c=0  a*b=-1/4  a/b=-1
a==c: true  b<a: true
set: -1/2 1/3 1/2
x++ -> y=1/2, x=3/2   ++x -> 5/2
error: denominador 0
```

Plantilla: qué escribir y en qué orden

1. **Atributos privados** con sufijo _.
2. **Constructor** con lista de inicialización (en el **orden en que declaraste los atributos**, no en el que los escribes).
3. **Getters `const`** (todo método que no modifica el objeto lleva `const`).
4. **Operadores** como métodos `const` que devuelven un objeto nuevo.
5. **`operator<<` como `friend`** dentro de la clase, devolviendo `os`.
6. Si usas memoria dinámica: **regla de los cinco** (destructor, copia, movimiento, asignación por valor con `swap`).

Truco: qué operadores necesitas para cada contenedor

- `set`, `map` (como clave) y `sort`: solo **`operator<`**.
- `find`, `count` en vector, `==` entre objetos: **`operator==`**.
- Imprimir con `cout`: **`operator<<`**.
- `x++` y `++x`: el sufijo lleva un parámetro `int` que no se usa; devuelve una **copia del valor anterior**.

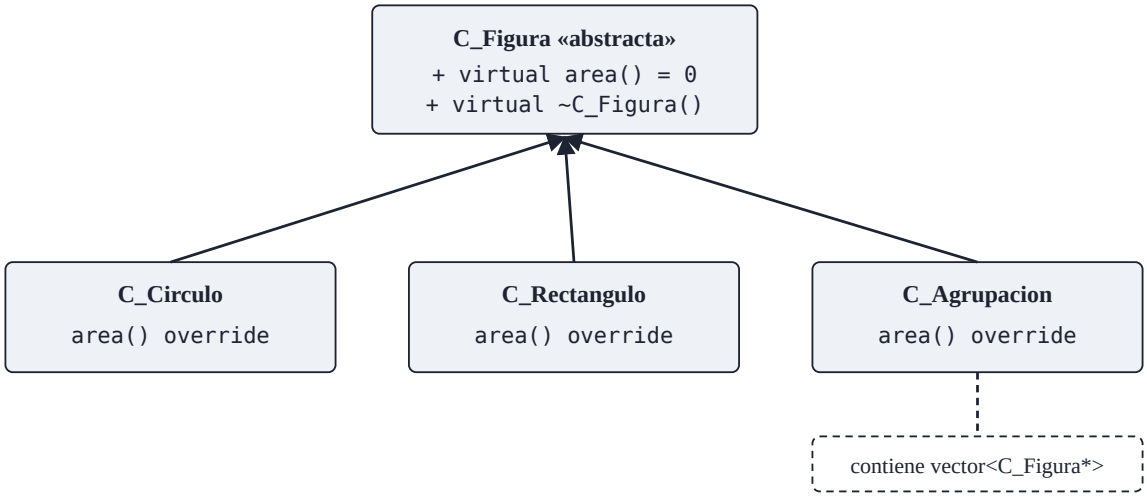
Qué suele fallar

- **Olvidar `const` en un getter** y que no compile al llamarlo desde una referencia `const&`.
- **`explicit` olvidado** en un constructor de un solo parámetro: permite conversiones silenciosas (`Fraccion f = 3;`). Es práctico aquí, pero en otras clases puede dar sorpresas.
- **`operator+` que modifica `*this`:** debe devolver un objeto nuevo; modificar es trabajo de `+=`.
- **Orden de inicialización:** `Caja(int a) : area_{ancho_ * alto_}, ancho_{a}` usa `ancho_` antes de darle valor, porque se inicializa en el orden de **declaración**.
- **Normalizar al construir** (aquí en `simplificar`) evita repetir lógica en cada operador y hace que `==` solo compare campos.

6 T3 · Herencia y polimorfismo

6.1 Herencia: la relación «es un»

Una **clase derivada** reutiliza y amplía una **clase base**. Un `C_Circulo` **es una** `C_Figura`: tiene todo lo de una figura (un `id`, un área, un perímetro) y añade lo suyo (un radio).



La flecha vacía significa «es un»: un C_Circulo ES una C_Figura (principio de Liskov).

CPP

```cpp
class C_Circulo : public C_Figura {      // herencia pública: «es un»
public:
    C_Circulo(string id, double r) : C_Figura{std::move(id)}, r_{r} {}   // construye primero la base
    ...
};
```

- El constructor de la base se llama **primero**; se invoca en la lista de inicialización.
- La destrucción va en orden inverso: primero la derivada, después la base.
- Con `protected`, los miembros son visibles para las derivadas pero no para el exterior.

Vídeo recomendado (en inglés, 8 min): herencia en C++ con ejemplos.

Vídeo recomendado

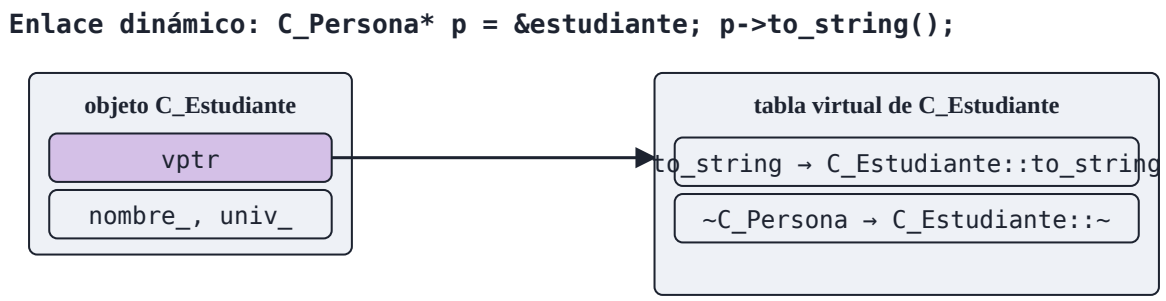
Inheritance in C++
The Cherno · 8:00
youtube.com/watch?v=X8nYM8wdNRE

6.2 Polimorfismo: un puntero a la base, muchos comportamientos

Imagina un vector de figuras. Quieres recorrerlo y llamar a `area()` sin preguntar de qué tipo es cada una. Eso es el **polimorfismo**: la misma llamada ejecuta código distinto según el **tipo real** del objeto.

Para que funcione necesitas **dos** cosas:

1. Que el método sea `virtual` en la base.
2. Que lo llames a través de un **puntero o una referencia** a la base (`C_Figura*`, `C_Figura&`).



Sin virtual no hay vptr: el compilador decide en compilación mirando el tipo del puntero (enlace estático).

Con virtual y puntero/referencia: se decide en ejecución mirando el objeto real (enlace dinámico).

Cada objeto con métodos virtuales lleva oculto un puntero a una **tabla de métodos virtuales (vtable)**. Al llamar a `p->area()`, el programa mira la tabla del objeto real y salta a la versión correcta. Es el **enlace dinámico**. Sin `virtual`, el compilador decide por el tipo declarado: **enlace estático**.

¿Qué método se ejecuta? Enlace estático frente a dinámico

método virtual | a través de Base* (apunta a Derivada)

Se ejecuta **Derivada::f()**.

La tabla de métodos virtuales del objeto real decide en tiempo de ejecución.

Captura del simulador interactivo (disponible en la versión web de la guía).

Qué suele fallar

- **Slicing**: `vector<C_Figura>` o `C_Figura f = circulo;` copia solo la parte base y pierde el polimorfismo. Usa punteros (`vector<C_Figura*>`, `vector<unique_ptr<C_Figura>>`).
- Olvidar `virtual` en el método de la base: la derivada lo «oculta» en lugar de redefinirlo.
- Firma ligeramente distinta (`const` o tipo del parámetro): no es una redefinición. **`override`** hace que el compilador te avise.
- Llamar a un método virtual desde el constructor: aún no es el tipo derivado.

Vídeos recomendados (en inglés): funciones virtuales y, después, cómo las implementa el compilador con la tabla virtual (la que dibuja la figura de arriba).

Vídeo recomendado

Virtual Functions in C++
The Cherno · 6:45
youtube.com/watch?v=oIV2KchSyGQ

Vídeo recomendado

How VIRTUAL functions work in C++
Lukasz Lipski · 4:41
youtube.com/watch?v=FncNE2vdtwQ

6.3 Clases abstractas y destructor virtual

Un método **virtual puro** (`virtual double area() const = 0;`) no tiene implementación en la base. Una clase con al menos uno es **abstracta**: no se puede instanciar, solo sirve como contrato. Las derivadas están obligadas a implementarlo (si no, también son abstractas).

La clase base de una jerarquía polimórfica debe tener **destructor virtual**: `virtual ~C_Figura() = default;`.

Si no, `delete p` con `p` de tipo `C_Figura*` solo ejecuta el destructor de la base y la parte derivada **no se destruye** (fuga o comportamiento indefinido).

6.4 Ejemplo completo

Esta es la jerarquía de figuras (con contenedor recursivo), compilada y ejecutada:

CPP

```cpp
#include <cmath>
#include <iostream>
#include <memory>
#include <set>
#include <sstream>
#include <string>
#include <typeinfo>
using namespace std;

class C_Figura {                                // clase abstracta
public:
    explicit C_Figura(string id) : id_{std::move(id)} {}
    virtual ~C_Figura() = default;              // destructor virtual
    virtual double area() const = 0;            // puras virtuales
    virtual double perimetro() const = 0;
    friend ostream& operator<<(ostream& os, const C_Figura& f) { return os << f.to_string(); }  // polimórfico
protected:
    virtual string to_string() const {
        ostringstream o; o << id_ << ": area=" << area() << " perimetro=" << perimetro(); return o.str();
    }
    string id_;
};

class C_Circulo : public C_Figura {
public:
    C_Circulo(string id, double r) : C_Figura{std::move(id)}, r_{r} {}
    double area() const override { return M_PI * r_ * r_; }
    double perimetro() const override { return 2 * M_PI * r_; }
protected:
    string to_string() const override { return "Circulo " + C_Figura::to_string(); }
private:
    double r_;
};

class C_Rectangulo : public C_Figura {
public:
    C_Rectangulo(string id, double b, double h) : C_Figura{std::move(id)}, b_{b}, h_{h} {}
    double area() const override { return b_ * h_; }
    double perimetro() const override { return 2 * (b_ + h_); }
protected:
    string to_string() const override { return "Rectangulo " + C_Figura::to_string(); }
private:
    double b_, h_;
};

class C_Agrupacion : public C_Figura {          // contenedor recursivo
public:
    explicit C_Agrupacion(string id) : C_Figura{std::move(id)} {}
    bool anyadir(C_Figura* f) { if (f == this) return false; return fs_.insert(f).second; }
    double area() const override { double s = 0; for (auto* f : fs_) s += f->area(); return s; }
    double perimetro() const override { double s = 0; for (auto* f : fs_) s += f->perimetro(); return s; }
protected:
    string to_string() const override {
        ostringstream o; o << "Agrupacion " << id_ << " {";
        for (auto* f : fs_) o << " [" << *f << "]";     // operator<< es público; f->to_string() no compilaría
        o << " }"; return o.str();
    }
private:
    set<C_Figura*> fs_;                         // punteros: sin ellos no hay polimorfismo
};

int main() {
    C_Circulo c{"c1", 1};
    C_Rectangulo r{"r1", 2, 3};
    C_Agrupacion g{"g1"};
    g.anyadir(&c); g.anyadir(&r);
```

```
    cout << "se anyade a si misma: " << boolalpha << g.anyadir(&g) << '\n';
    C_Figura* p = &r;                           // puntero a la base, objeto derivado
    cout << *p << '\n';
    cout << "area total g1 = " << g.area() << '\n';
    cout << "tipo de *p: " << typeid(*p).name() << " (nombre interno)\n";
    unique_ptr<C_Figura> u = make_unique<C_Circulo>("c2", 2);   // liberado automáticamente
    cout << u->area() << '\n';
}
```

Salida

```
se anyade a si misma: false
Rectangulo r1: area=6 perimetro=10
area total g1 = 9.14159
tipo de *p: 12C_Rectangulo (nombre interno)
12.5664
```

Puntos finos de este ejemplo:

- `operator<<` **no es virtual**: es una función libre. Delega en un método virtual `to_string()` (patrón *non-virtual interface*). Así `cout << *p` se comporta de forma polimórfica.
- `to_string` es `protected`. Una derivada la puede redefinir y llamar a la de su base (`C_Figura::to_string()`), pero **no** puedes llamar a `f->to_string()` a través de un `C_Figura*` desde otra clase: el compilador lo rechaza (accederías al miembro protegido de **otro** objeto de la base). Usa el `operator<<` público.
- `C_Agrupacion` guarda `set<C_Figura*>`: punteros para que haya polimorfismo, y comprueba `f == this` para no contenerse a sí misma.

Tipo en tiempo de ejecución y punteros inteligentes

`typeid(*p)` devuelve el tipo **real** del objeto si la clase es polimórfica (`typeid(*p) == typeid(C_Circulo)` sirve para contar objetos de cada tipo). El **nombre** que imprime depende del compilador. `dynamic_cast<C_Circulo*>(p)` devuelve `nullptr` si no lo es.

`unique_ptr<T>` es un puntero con **un solo dueño** que hace `delete` solo al salir. `shared_ptr<T>` cuenta propietarios. Con ellos no escribes `delete` a mano.

Más a fondo

Principio de sustitución de Liskov (SOLID). Si `B` hereda de `A`, debes poder usar un `B` en cualquier sitio donde se espera un `A` sin que el programa deje de ser correcto. Un `Cuadrado` que hereda de `Rectangulo` y hace que `setAncho` cambie también el alto rompe el principio: quien espera un rectángulo cualquiera se sorprende. Si no puedes decir «un B **es un** A» en todos los sentidos del contrato, usa composición.

Plantilla de jerarquía

CPP

```
class Base {
public:
    virtual ~Base() = default;
    virtual double f() const = 0;                       // pura
    friend ostream& operator<<(ostream& o, const Base& b) { return o << b.to_string(); }
protected:
    virtual string to_string() const { return "base"; }
};
class Der : public Base {
public:
    double f() const override { return 1; }
protected:
    string to_string() const override { return "der " + Base::to_string(); }
};
// uso: vector<Base*> v; for (auto* p : v) cout << *p;
```

6.5 Trampas del polimorfismo, con código

Cinco situaciones que se preguntan una y otra vez, todas en un programa. Ejecútalo mentalmente antes de mirar la salida.

CPP

```cpp
#include <iostream>
#include <memory>
#include <string>
#include <vector>
using namespace std;

class Animal {
public:
    Animal() { cout << "  Animal() presenta: " << nombre() << '\n'; }   // TRAMPA: en el constructor NO hay
    ↪ polimorfismo
    virtual ~Animal() = default;
    virtual string nombre() const { return "animal"; }
    virtual string sonido() const { return "..."; }
    string presentacion() const { return nombre() + " dice " + sonido(); }   // llama a lo virtual: usa el tipo REAL
};
class Perro : public Animal {
public:
    string nombre() const override { return "perro"; }
    string sonido() const override { return "guau"; }
};
class Gato : public Animal {
public:
    string nombre() const override { return "gato"; }
    string sonido() const override { return Animal::sonido() + "miau"; }   // llamar a la versión de la base:
    ↪ Base::metodo()
    void ronronear() const { cout << "  ...purrr\n"; }
};

int main() {
    cout << "1) Constructor de Perro (la base llama a nombre() antes de que exista la parte Perro):\n";
    Perro p;
    cout << "2) Slicing: guardar por VALOR corta el objeto\n";
    vector<Animal> por_valor;
    por_valor.push_back(Perro{});
    cout << "  " << por_valor[0].presentacion() << "   <- ya es un Animal, no un Perro\n";
    cout << "3) Por puntero inteligente se conserva el tipo real\n";
    vector<unique_ptr<Animal>> zoo;
    zoo.push_back(make_unique<Perro>());
    zoo.push_back(make_unique<Gato>());
    for (const auto& a : zoo) cout << "  " << a->presentacion() << '\n';
    cout << "4) dynamic_cast: pregunta '¿eres un Gato?' (devuelve nullptr si no)\n";
    for (const auto& a : zoo) {
        if (auto* g = dynamic_cast<Gato*>(a.get())) { cout << "  es un gato:"; g->ronronear(); }
        else cout << "  " << a->nombre() << " no es un gato\n";
    }
    cout << "5) Referencia a la base: tampoco hay slicing\n";
    const Animal& r = p;
    cout << "  " << r.presentacion() << '\n';
}
```

Salida

```
1) Constructor de Perro (la base llama a nombre() antes de que exista la parte Perro):
  Animal() presenta: animal
2) Slicing: guardar por VALOR corta el objeto
  Animal() presenta: animal
  animal dice ...   <- ya es un Animal, no un Perro
3) Por puntero inteligente se conserva el tipo real
  Animal() presenta: animal
  Animal() presenta: animal
  perro dice guau
  gato dice ...miau
4) dynamic_cast: pregunta '¿eres un Gato?' (devuelve nullptr si no)
  perro no es un gato
  es un gato:  ...purrr
5) Referencia a la base: tampoco hay slicing
  perro dice guau
```

Qué enseña cada salida

1. **Constructor:** dentro del constructor de `Animal` se llama a `nombre()` y sale `animal`, no `perro`: la parte `Perro` **aún no existe**. Regla: no llames a métodos virtuales desde constructores ni destructores.
2. **Slicing:** `vector<Animal>` guarda copias de la parte `Animal`. `push_back(Perro{})` **recorta** el objeto y pierde los métodos de `Perro`. Para polimorfismo necesitas **punteros** (`unique_ptr<Animal>`) o

referencias.

3. **Con `unique_ptr`** se conserva el tipo real: `perro dice guau`. Además, `Gato::sonido()` llama a `Animal::sonido()` para reutilizar la versión de la base: por eso sale `gato dice ...miau`.
4. **`dynamic_cast<Gato*>`** devuelve `nullptr` si el objeto no es un `Gato`. Úsalo solo cuando necesites un método que **no** está en la base (`ronronear`). Si puedes poner el método en la base como virtual, mejor.
5. **Referencia a la base** (`const Animal& r = p;`): tampoco hay slicing, `r.presentacion()` sigue dando `perro dice guau`.

Qué suele fallar

- **Destructor no virtual:** `unique_ptr<Animal> p = make_unique<Perro>()` destruiría solo la parte `Animal` y la clase derivada no liberaría sus recursos. Con jerarquías, el destructor de la base es **siempre** `virtual`.
- **Firma distinta sin `override`:** si escribes `string nombre()` sin `const` en la hija, **no** sobrescribe: crea otro método. `override` hace que el compilador te avise.
- **Clase abstracta instanciada:** si queda un método `= 0` sin implementar en la hija, la hija sigue siendo abstracta y `make_unique<Hija>()` no compila.
- **`typeid(p)` frente a `typeid(*p)`:** el primero da el tipo del puntero, el segundo el del objeto apuntado (la base debe tener algún método virtual).

Truco para el `operator<<` polimórfico

Una sola función `friend` en la base llama a un método virtual protegido (`imprimir`). Cada hija lo redefine y empieza con `Base::imprimir(os)` para no repetir. Es la solución que más puntos da y la más corta.

7 T4 · Hilos y exclusión mutua

7.1 Crear hilos con `std::thread`

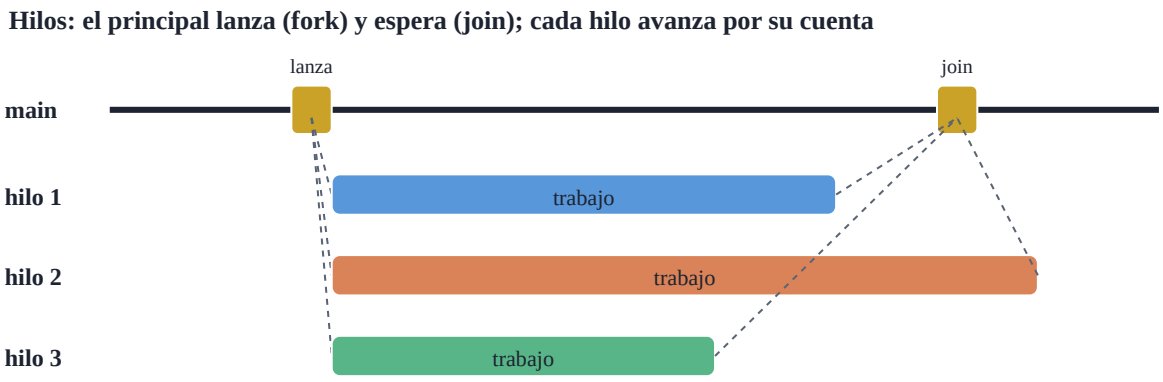
Un `std::thread` ejecuta una función **a la vez** que el resto del programa. El hilo empieza a correr en cuanto lo construyes.

CPP

```
void saluda(string quien, int veces) { for (int i = 0; i < veces; ++i) cout << quien << '\n'; }
int main() {
    thread t{saluda, "hilo", 3};     // se copian los argumentos
    saluda("main", 3);                // main sigue trabajando
    t.join();                          // main espera a que t termine
}
```

Reglas que debes saber de memoria:

- Antes de destruir un `thread` hay que hacer **`join()`** (esperar) o `detach()` (soltarlo). Si no, el programa termina con `std::terminate`.
- Los argumentos se **copian** al hilo. Para pasar por referencia, usa **`std::ref(x)`**: `thread t{f, std::ref(contador)};`.
- Para un método de objeto: `thread t{&Clase::metodo, &objeto, args...}`. Con una lambda es más cómodo.
- Compila con `-pthread` en Linux.
- Un vector de hilos: `vector<thread> v; v.emplace_back(f, i); for (auto& h : v) h.join();`.



Sin join (ni detach) al destruir el thread: std::terminate. Datos compartidos: mutex o atomic.

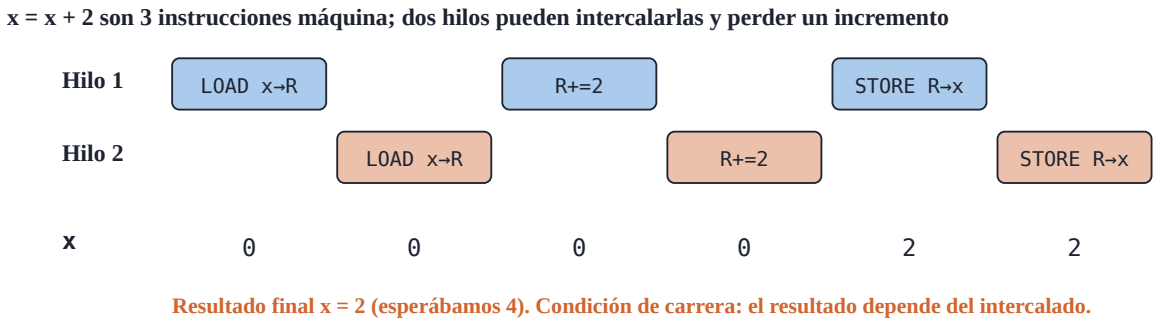
Vídeo recomendado (en inglés, 11 min): crear y esperar hilos con `std::thread`.

Vídeo recomendado

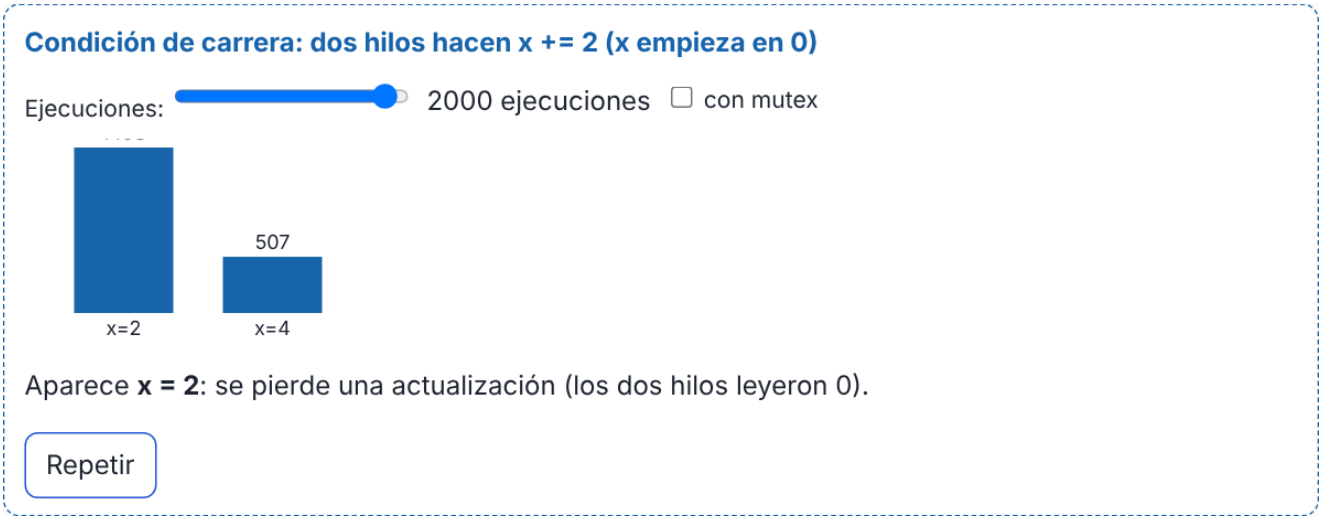
Threads in C++
The Cherno · 11:35
youtube.com/watch?v=wXBcwHwIt_I

7.2 La condición de carrera

Dos hilos suben un contador compartido: `++x`. Parece una sola instrucción, pero la CPU hace **tres**: leer `x` a un registro, sumar, escribir. Si los dos leen el mismo valor antes de que ninguno escriba, uno de los incrementos se **pierde**.



Resultado final x = 2 (esperábamos 4). Condición de carrera: el resultado depende del intercalado.



Captura del simulador interactivo (disponible en la versión web de la guía).

La zona de código que accede al dato compartido es la **sección crítica**. Debe ejecutarla **un solo hilo a la vez** (**exclusión mutua**).

7.3 Mutex y bloqueos

Un **`mutex`** es una puerta con cerrojo: un hilo hace `lock()` y entra; los demás esperan hasta el `unlock()`. No los uses a pelo: si olvidas el `unlock` (por un `return` o una excepción) el programa se queda bloqueado. Usa **bloqueos RAII**, que desbloquean solos al salir del ámbito:

Herramienta	Cuándo
`lock_guard<mutex> lk{m};`	Un `mutex`, sección crítica simple
`scoped_lock lk{m1, m2};`	**Varios** `mutex` a la vez, sin interbloqueo
`unique_lock<mutex> lk{m};`	Cuando puedes necesitar desbloquear, o con `condition_variable` (T5)
`atomic<int> x;`	Un único contador o bandera: operación indivisible sin `mutex`

CPP

```cpp
#include <atomic>
#include <iostream>
#include <mutex>
#include <thread>
#include <vector>
using namespace std;

int main() {
    const int N = 100000;
    int sin_proteger = 0;           // carrera de datos: resultado impredecible
    int con_mutex = 0;
    atomic<int> atomico{0};
    mutex m;
```

```
    auto trabajo = [&] {
        for (int i = 0; i < N; ++i) {
            ++sin_proteger;                             // LOAD, ADD, STORE: otro hilo puede colarse
            { lock_guard<mutex> lk{m}; ++con_mutex; }   // sección crítica
            ++atomico;                                  // operación indivisible
        }
    };
    vector<thread> hilos;
    for (int i = 0; i < 4; ++i) hilos.emplace_back(trabajo);   // los hilos arrancan ya
    for (auto& h : hilos) h.join();                            // esperar a que acaben
    cout << "esperado:  " << 4 * N << '\n';
    cout << "con mutex: " << con_mutex << '\n';
    cout << "atomico:   " << atomico << '\n';
    cout << "sin proteger acierta: " << boolalpha << (sin_proteger == 4 * N) << " (casi nunca)\n";
}
```

Salida

```
esperado:  400000
con mutex: 400000
atomico:   400000
sin proteger acierta: false (casi nunca)
```

Cómo piensa un buen diseño

Un `mutex` **protege datos**, no código. Para cada dato compartido, deberías poder decir «lo protege el `mutex` X». Si no puedes, hay una carrera.

Vídeo recomendado (en inglés, 12 min): qué es un mutex y cómo evita las carreras.

Vídeo recomendado

What is Mutex in C++ (Multithreading for Beginners)
CodeBeauty · 12:29
youtube.com/watch?v=-Qa1RqmXKG0

Opcional y avanzado (en inglés, 1 h 14 min): variables atómicas (`std::atomic`) y sus órdenes de memoria. Para el examen basta con saber que una operación atómica sobre una variable no necesita mutex.

Vídeo recomendado

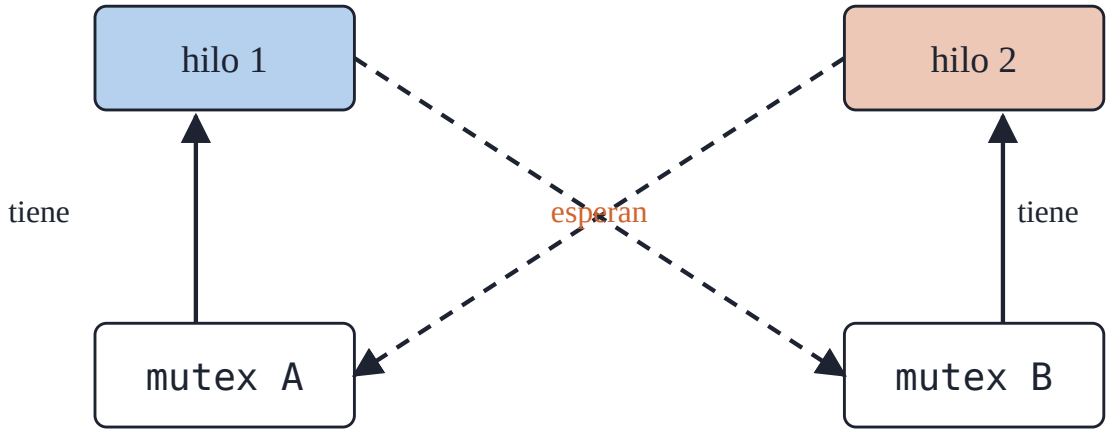
C++ atomics, from basic to advanced. What do they really do?
CppCon (Fedor Pikus) · 1:14:22
youtube.com/watch?v=ZQFzMfHIxng

7.4 Interbloqueo (deadlock)

Hilo A tiene `m1` y espera `m2`. Hilo B tiene `m2` y espera `m1`. Ninguno avanza nunca. Hay interbloqueo cuando se dan a la vez cuatro condiciones: exclusión mutua, retener y esperar, no expulsión y **espera circular**. La forma sencilla de romperlo es **eliminar la espera circular**:

- Tomar siempre los `mutex` **en el mismo orden**, o
- Tomarlos **todos a la vez** con `scoped_lock` (o `std::lock`), que usa un algoritmo que evita el interbloqueo.

Interbloqueo: cada hilo tiene un mutex y espera el del otro



Solución: coger SIEMPRE en el mismo orden, o std::lock / std::scoped_lock con todo

CPP

```cpp
#include <iostream>
#include <mutex>
#include <thread>
using namespace std;

mutex m1, m2, consola;
void ab() { for (int i = 0; i < 1000; ++i) { scoped_lock lk{m1, m2}; } }   // los dos a la vez
void ba() { for (int i = 0; i < 1000; ++i) { scoped_lock lk{m2, m1}; } }   // orden distinto: da igual
int main() {
    thread a{ab}, b{ba};
    a.join(); b.join();
    cout << "termina sin interbloqueo\n";
}
```

Salida

```
termina sin interbloqueo
```

Cena de los filósofos (3 cubiertos por filósofo, n+1 cubiertos)

Filósofos 3 Ciclos 20 Semilla 1 lock() de cada mutex uno a uno

Simular

INTERBLOQUEO. Comidas por filósofo: 0, 1, 0. Cubiertos tomados por: C1:F1 · C2:F2 · C3:F3 · C4:F3

En 50 semillas distintas hay interbloqueo en **50**.

Captura del simulador interactivo (disponible en la versión web de la guía).

7.5 Tareas periódicas: `sleep_until` frente a `sleep_for`

Para una tarea con periodo T:

- `sleep_for(T)` espera T **después** de acabar el trabajo, así que el periodo real es $T+C$ y el error se **acumula** (deriva).
- `sleep_until(siguiente); siguiente += T;` espera a un **instante absoluto**: el periodo no se desvía.

CPP

```cpp
#include <chrono>
#include <iostream>
#include <thread>
using namespace std;
using namespace std::chrono;

int main() {
    const auto periodo = 20ms;
    const auto inicio = steady_clock::now();
    auto siguiente = inicio + periodo;                  // instante absoluto
    for (int i = 1; i <= 5; ++i) {
        this_thread::sleep_for(3ms);                    // «trabajo» de la tarea
        cout << "tarea " << i << '\n';
        this_thread::sleep_until(siguiente);            // espera hasta el instante absoluto
        siguiente += periodo;                           // sin deriva acumulada
    }
    auto total = duration_cast<milliseconds>(steady_clock::now() - inicio).count();
    cout << "duracion ~ 100 ms: " << boolalpha << (total >= 100 && total < 115) << '\n';
}
```

Salida

```
tarea 1
tarea 2
tarea 3
tarea 4
tarea 5
duracion ~ 100 ms: true
```

Qué suele fallar

- Pasar una variable por valor al hilo esperando que se modifique: usa `std::ref`.
- Olvidar `join()`: el programa aborta.
- Un `mutex` distinto por hilo (cada uno protege lo suyo, no hay exclusión). El `mutex` debe ser **compartido**.
- Imprimir por `cout` desde varios hilos: la salida se mezcla. Protege la consola con su propio `mutex`.
- Dejar un `mutex` bloqueado al lanzar una excepción o hacer `return` (usa `lock_guard`).
- Dos `mutex` en orden distinto en dos hilos (interbloqueo).

Más a fondo

Modelo de memoria. Una carrera de datos (acceso concurrente a un dato, uno de ellos escritura, sin sincronización) es **comportamiento indefinido** en C++: el compilador puede reordenar o eliminar accesos. `mutex` y `atomic` imponen un orden que sí es correcto. Referencia: A. Williams, *C++ Concurrency in Action* (2.ª ed.).

Plantillas de concurrencia

- Lanzar y esperar: `vector<thread> v; for(...) v.emplace_back(f, args); for (auto& h : v) h.join();`
- Sección crítica: `{ lock_guard<mutex> lk{m}; /* tocar datos protegidos */ }`
- Dos recursos: `scoped_lock lk{m1, m2};`
- Periódica: `auto sig = steady_clock::now() + T; loop { trabajo(); sleep_until(sig); sig += T; }`

7.6 Plantillas y trucos de hilos

Cinco trucos con su salida. Todos usan resultados que no dependen del orden entre hilos, así que la salida es siempre la misma.

CPP

```cpp
#include <atomic>
#include <functional>
```

```
#include <iostream>
#include <mutex>
#include <numeric>
#include <thread>
#include <vector>
using namespace std;

void sumar_rango(const vector<int>& v, size_t a, size_t b, long& resultado) {
    resultado = accumulate(v.begin() + a, v.begin() + b, 0L);
}
void incrementar_copia(int x) { ++x; }              // recibe COPIA: el original no cambia
void incrementar_ref(int& x) { ++x; }

int main() {
    // TRUCO 1: thread copia los argumentos; para pasar una referencia hay que usar ref()
    int a = 0, b = 0;
    thread t1{incrementar_copia, a}; t1.join();
    thread t2{incrementar_ref, ref(b)}; t2.join();
    cout << "1) sin ref: a=" << a << "   con ref: b=" << b << '\n';

    // TRUCO 2: cada hilo escribe en SU hueco del vector: no hace falta mutex
    vector<int> datos(1000);
    iota(datos.begin(), datos.end(), 1);                // 1..1000
    vector<long> parcial(4);
    vector<thread> hilos;
    for (size_t i = 0; i < 4; ++i)
        hilos.emplace_back(sumar_rango, cref(datos), i * 250, (i + 1) * 250, ref(parcial[i]));
    for (auto& h : hilos) h.join();
    cout << "2) sumas parciales:";
    for (long p : parcial) cout << ' ' << p;
    cout << "  total=" << accumulate(parcial.begin(), parcial.end(), 0L) << " (esperado 500500)\n";

    // TRUCO 3: en un bucle, la lambda captura el índice POR VALOR ([i]); por referencia ([&]) el bucle lo cambia
    vector<int> orden(4, -1);
    hilos.clear();
    for (int i = 0; i < 4; ++i)
        hilos.emplace_back([i, &orden] { orden[i] = i * i; });
    for (auto& h : hilos) h.join();
    cout << "3) cuadrados:";
    for (int x : orden) cout << ' ' << x;
    cout << '\n';

    // TRUCO 4: mutex para varias operaciones a la vez; atomic para UNA operación sobre UNA variable
    long con_mutex = 0; mutex m; atomic<int> con_atomic{0};
    hilos.clear();
    for (int i = 0; i < 4; ++i)
        hilos.emplace_back([&] {
            for (int k = 0; k < 10000; ++k) { lock_guard<mutex> lk{m}; ++con_mutex; }
            for (int k = 0; k < 10000; ++k) ++con_atomic;
        });
    for (auto& h : hilos) h.join();
    cout << "4) mutex=" << con_mutex << " atomic=" << con_atomic << " (esperado 40000 los dos)\n";

    // TRUCO 5: SIEMPRE join (o detach) antes de que el objeto thread se destruya, o el programa aborta
    thread t{[] {}};
    cout << "5) joinable antes de join: " << boolalpha << t.joinable();
    t.join();
    cout << ", despues: " << t.joinable() << '\n';
}
```

Salida

```
1) sin ref: a=0   con ref: b=1
2) sumas parciales: 31375 93875 156375 218875  total=500500 (esperado 500500)
3) cuadrados: 0 1 4 9
4) mutex=40000 atomic=40000 (esperado 40000 los dos)
5) joinable antes de join: true, despues: false
```

Qué enseña cada salida

1. **thread copia los argumentos.** `thread{f, a}` pasa una copia; si `f` recibe `int&`, hay que escribir `ref(a)` o no compila (o no cambia el original).
2. **Reparte el trabajo en rangos y deja un hueco por hilo** (`parcial[i]`). Cada hilo escribe solo en su posición: **no necesita mutex**. Suma al final en el hilo principal.

3. **Captura el índice por valor** (`[i, &orden]`). Con `[&]` el bucle cambia `i` mientras el hilo lo lee: carrera. Es la trampa de captura más frecuente.
4. **`mutex` o `atomic`.** `atomic` sirve para una operación sobre **una** variable (`++`, `+=`). Si hay que hacer dos cosas a la vez (mover dinero de una cuenta a otra), necesitas `mutex`.
5. **`join` siempre.** Un `thread` que se destruye sin `join` ni `detach` aborta el programa (`std::terminate`).

Receta para «lanza N hilos y espera»

```
vector<thread> hilos;
for (int i = 0; i < N; ++i) hilos.emplace_back([i, &datos] { /* trabajo */ });
for (auto& h : hilos) h.join();
```

Usa `emplace_back` (construye el hilo en su sitio, no se puede copiar un `thread`) y haz **todos** los `join` en un segundo bucle.

Qué suele fallar

- **Mutex local:** si el `mutex` se declara dentro de la función del hilo, cada hilo tiene el suyo y no protege nada. Debe ser **compartido**.
- **`lock_guard` con alcance demasiado grande:** si lo declaras antes de un `sleep_for`, bloqueas a todos los demás mientras duermes. Pon llaves `{ lock_guard … }` alrededor de lo estrictamente crítico.
- **Imprimir con `cout` desde varios hilos** mezcla líneas. Protege la salida con su propio `mutex`.
- **Dos mutex en orden distinto** en dos hilos = interbloqueo. Usa `scoped_lock(m1, m2)`: toma los dos a la vez.

8 T5 · Monitores y variables condición

8.1 El problema: esperar a que se cumpla una condición

El `mutex` resuelve «solo uno a la vez». No resuelve «espera hasta que haya algo». Ejemplos típicos:

- Un consumidor debe esperar a que el productor haya puesto un dato.
- Un productor no puede escribir si el almacén está lleno.
- Varios hilos deben esperar en una barrera hasta que otro la abra.

Esperar dando vueltas en un `while` (espera activa) gasta CPU y es incorrecto. Lo correcto es **dormir** hasta que alguien avise: una **variable condición** (`condition_variable`).

8.2 `condition_variable`: cómo se usa siempre

Se usa **siempre junto con un `mutex` y un predicado**. Hay un patrón fijo:

CPP

```cpp
// Hilo que espera
unique_lock<mutex> lk{m};
cv.wait(lk, [&]{ return condicion; });   // 1) suelta el mutex y duerme  2) al despertar lo recupera  3) vuelve a
 ↪  comprobar el predicado
// aquí la condición es cierta y tienes el mutex

// Hilo que avisa
{ lock_guard<mutex> lk{m}; condicion = true; }   // cambia el estado protegido por el mutex
cv.notify_one();                                   // o notify_all()
```

Por qué **con predicado**:

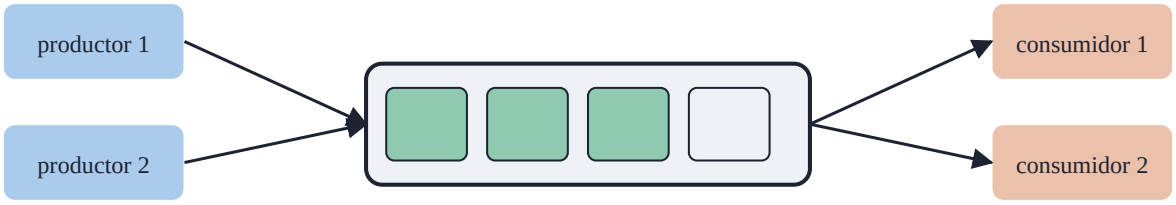
- Puede haber **despertares espurios**: `wait` se despierta sin que nadie avise.
- Puede que otro hilo se haya adelantado y haya consumido el dato: al despertar hay que **volver a comprobar**.
- Si avisas **antes** de que el otro empiece a esperar, el aviso se pierde; con predicado no importa, porque la condición ya es cierta y `wait` ni se duerme.

Método	Qué hace
`wait(lk, pred)`	Duerme hasta que `pred()` sea cierto (equivale a `while(!pred()) wait(lk);`)
`notify_one()`	Despierta a **un** hilo que espera
`notify_all()`	Despierta a **todos**; cada uno recomprueba su predicado
`wait_for / wait_until`	Con plazo máximo (útil en tiempo real)

Qué suele fallar

- Usar `wait` **sin predicado** y con un `if`: despertar espurio y se rompe.
- Cambiar la variable de la condición **sin** tener el `mutex`: carrera con el predicado.
- `notify_one` cuando hay varias condiciones distintas esperando en la misma variable: puede despertar al hilo equivocado. Usa `notify_all` o una variable por condición.
- Usar `lock_guard` con `wait`: necesitas `unique_lock` (porque `wait` lo desbloquea y bloquea).
- Olvidar la condición de **cierre**: los hilos quedan dormidos para siempre y el `join` no termina.

Productor–consumidor con cola acotada (monitor): quién espera y por qué



cola de capacidad 4 (3 ocupadas)

`poner: wait(hay_hueco) si está llena → notify(hay_dato)`
`sacar: wait(hay_dato) si está vacía → notify(hay_hueco)`

cerrar: bandera con el mutex + notify_all para que nadie se quede dormido

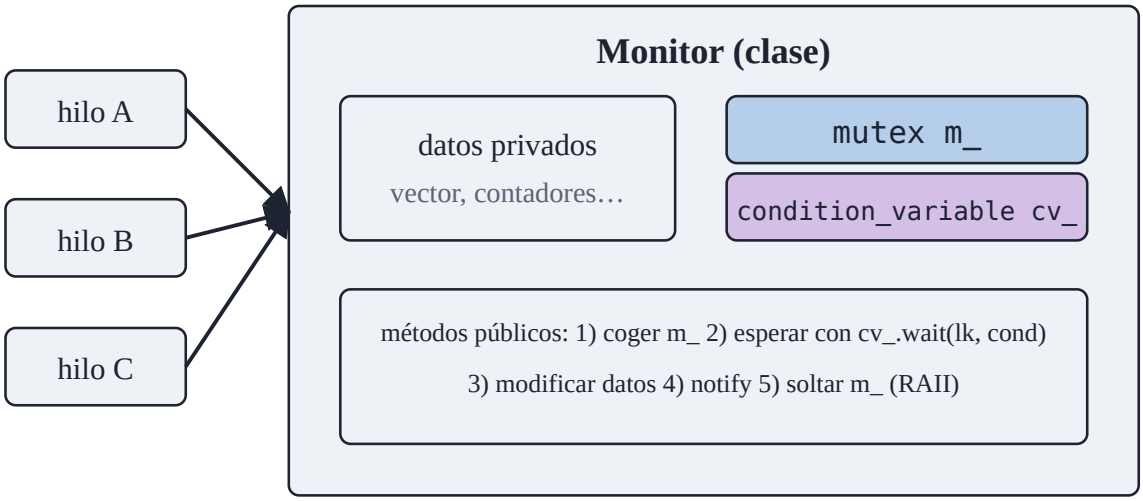
Vídeo recomendado (en inglés, 16 min): el problema del productor y el consumidor con `condition_variable`.

Vídeo recomendado

Producer Consumer Problem
CppNuts · 15:42
youtube.com/watch?v=rKGq9AlfPR4

8.3 Monitor

Un **monitor** es una clase que **encapsula**: los datos compartidos, el `mutex` que los protege y las variables condición. Los hilos solo usan los métodos públicos y cada uno hace internamente su `lock`. Es la forma de dar la respuesta de «la clase `Almacén`/`Puente`/`Barrera`» del examen.



Un buffer acotado completo (productor–consumidor), compilado y probado con varios consumidores:

CPP

```cpp
#include <condition_variable>
#include <iostream>
#include <mutex>
#include <queue>
#include <thread>
#include <vector>
using namespace std;

// Monitor: datos + mutex + condiciones de sincronización, todo dentro de la clase
```

```
class Buffer {
public:
    explicit Buffer(size_t cap) : cap_{cap} {}
    void poner(int v) {
        unique_lock<mutex> lk{m_};
        no_lleno_.wait(lk, [&] { return q_.size() < cap_; });      // espera mientras esté lleno
        q_.push(v);
        no_vacio_.notify_one();
    }
    bool sacar(int& v) {                                            // false = cerrado y vacío
        unique_lock<mutex> lk{m_};
        no_vacio_.wait(lk, [&] { return !q_.empty() || cerrado_; });
        if (q_.empty()) return false;
        v = q_.front(); q_.pop();
        no_lleno_.notify_one();
        return true;
    }
    void cerrar() { { lock_guard<mutex> lk{m_}; cerrado_ = true; } no_vacio_.notify_all(); }
private:
    mutex m_;
    condition_variable no_lleno_, no_vacio_;
    queue<int> q_;
    size_t cap_;
    bool cerrado_ = false;
};

int main() {
    Buffer b{3};
    long suma[2] = {0, 0};
    auto consumidor = [&](int id) { int v; while (b.sacar(v)) suma[id] += v; };
    thread c0{consumidor, 0}, c1{consumidor, 1};
    thread p{[&] { for (int i = 1; i <= 100; ++i) b.poner(i); b.cerrar(); }};
    p.join(); c0.join(); c1.join();
    cout << "suma total consumida = " << suma[0] + suma[1] << " (esperado 5050)\n";
}
```

Salida

```
suma total consumida = 5050 (esperado 5050)
```

Observa:

- Dos variables condición (`no_lleno_` y `no_vacio_`), una para cada condición de espera distinta.
- `sacar` devuelve `false` cuando el buffer está **cerrado y vacío**; así los consumidores acaban sin esperar para siempre.
- El estado del cierre se cambia **con el `mutex`** y se avisa con `notify_all`.

Vídeo recomendado (en inglés, 6 min): por qué la espera usa `unique_lock` y no `lock_guard`.

Vídeo recomendado

Why C++ Producer-Consumer Pattern Requires std::unique_lock
Suboptimal Engineer · 6:25
youtube.com/watch?v=uxMRTDxO1bQ

8.4 Cómo resolver un ejercicio de monitor en el examen

1. **Lista los atributos** que describen el estado (contadores, banderas, colas) y escribe qué `mutex` los protege (normalmente **uno**).
2. **Para cada operación bloqueante**, escribe en una frase la condición para poder seguir: «puedo entrar si…». Eso es el predicado.
3. **Escribe la operación**: `unique_lock` → `wait(lk, predicado)` → actualiza el estado → `notify_*`.
4. **Pregúntate quién despierta a quién**: cada cambio de estado que pueda hacer cierto un predicado debe terminar en un `notify`.
5. **Añade una forma de terminar** (`bool terminar_` + `notify_all`) si el enunciado lo pide.
6. **Comprueba el interbloqueo**: ¿puede haber dos hilos esperando que el otro avance? Si hay prioridades o turnos, define una regla **asimétrica** (un desempate fijo).

Más a fondo

Por qué no alcanza con un `mutex`. Un `mutex` garantiza exclusión, pero no ordena eventos. La variable condición es el mecanismo que expresa **precedencia** («esto solo ocurre después de aquello»). En teoría de la concurrencia, los monitores de Hoare/Mesa implementan exactamente este esquema; `std::condition_variable` es de estilo **Mesa**: al despertar no hay garantía de que la condición siga siendo cierta, de ahí el `while`/predicado. Referencia: A. Silberschatz et al., *Operating System Concepts*.

Plantilla de monitor

CPP

```cpp
class Monitor {
public:
    void operacion() {
        unique_lock<mutex> lk{m_};
        cv_.wait(lk, [this]{ return puedo_; });
        /* actualizar estado */
        cv_.notify_all();
    }
private:
    mutex m_;
    condition_variable cv_;
    /* estado protegido por m_ */
};
```

8.5 Plantilla de monitor genérica

Esta cola acotada sirve de **esqueleto** para casi cualquier ejercicio de monitor. Fíjate en los cuatro pasos comentados en `poner`: son siempre los mismos.

CPP

```cpp
#include <chrono>
#include <condition_variable>
#include <iostream>
#include <mutex>
#include <queue>
#include <thread>
#include <vector>
using namespace std;
using namespace chrono_literals;

// PLANTILLA DE MONITOR: copia este esqueleto y cambia solo el estado y los predicados.
template <typename T>
class Cola {
public:
    explicit Cola(size_t capacidad) : cap_{capacidad} {}
    bool poner(T x) {                                       // false si se cerró
        unique_lock<mutex> lk{m_};                          // 1) tomar el mutex
        hay_hueco_.wait(lk, [this] { return q_.size() < cap_ || cerrada_; });   // 2) esperar con PREDICADO (incluye
        ↪  el cierre)
        if (cerrada_) return false;
        q_.push(move(x));                                   // 3) cambiar el estado
        hay_dato_.notify_one();                             // 4) avisar a quien espera lo contrario
        return true;
    }
    bool sacar(T& x) {                                      // false si cerrada y vacía
        unique_lock<mutex> lk{m_};
        hay_dato_.wait(lk, [this] { return !q_.empty() || cerrada_; });
        if (q_.empty()) return false;
        x = move(q_.front()); q_.pop();
        hay_hueco_.notify_one();
        return true;
    }
    void cerrar() {
        { lock_guard<mutex> lk{m_}; cerrada_ = true; }
        hay_hueco_.notify_all();                            // el cierre interesa a TODOS
        hay_dato_.notify_all();
    }
    bool sacar_con_limite(T& x, chrono::milliseconds t) {   // espera como mucho t
        unique_lock<mutex> lk{m_};
        if (!hay_dato_.wait_for(lk, t, [this] { return !q_.empty(); })) return false;
        x = move(q_.front()); q_.pop();
```

```
        hay_hueco_.notify_one();
        return true;
    }
private:
    mutex m_;
    condition_variable hay_hueco_, hay_dato_;
    queue<T> q_;
    size_t cap_;
    bool cerrada_ = false;
};

int main() {
    Cola<int> cola{3};
    vector<thread> prod, cons;
    vector<long> suma(2, 0);
    for (int p = 0; p < 2; ++p)
        prod.emplace_back([&, p] { for (int i = 1; i <= 50; ++i) cola.poner(p * 100 + i); });
    for (int c = 0; c < 2; ++c)
        cons.emplace_back([&, c] { int x; while (cola.sacar(x)) suma[c] += x; });
    for (auto& t : prod) t.join();
    cola.cerrar();                                          // los consumidores terminan al vaciar
    for (auto& t : cons) t.join();
    cout << "suma total = " << suma[0] + suma[1] << " (esperado " << 2 * 1275 + 50 * 100 << ")\n";
    int x;
    Cola<int> vacia{1};
    cout << boolalpha << "sacar_con_limite en cola vacia: " << vacia.sacar_con_limite(x, 50ms) << '\n';
}
```

Salida

```
suma total = 7550 (esperado 7550)
sacar_con_limite en cola vacia: false
```

Receta de monitor en 5 pasos

1. Decide el **estado compartido** (aquí `q_`, `cap_`, `cerrada_`) y ponlo `private`, junto a **un** `mutex`.
2. Escribe las **condiciones de espera** como frases: «puedo poner si hay hueco **o** se cerró», «puedo sacar si hay dato **o** se cerró».
3. Cada método público hace: `unique_lock` → `wait(lk, predicado)` → comprobar cierre → **cambiar estado** → `notify`.
4. Avisa a **quien espera lo contrario**: tras `poner` avisas a los que sacan; tras `sacar`, a los que ponen.
5. El **cierre** pone la bandera con el mutex tomado y hace `notify_all` a todas las variables de condición.

Los cuatro errores clásicos

- **`if` en lugar de `while` / predicado.** Otro hilo puede haber cambiado el estado entre el aviso y tu despertar (y existen despertares espurios). El `wait` con predicado ya contiene el bucle.
- **Cambiar el estado fuera del mutex.** Toda lectura y escritura del estado, también la de la bandera de cierre, va con el mutex tomado.
- **Olvidar la condición de cierre en el predicado.** Sin `|| cerrada_`, los hilos dormidos nunca despiertan al terminar.
- **`notify_one` donde hace falta `notify_all`.** Si varios hilos esperan condiciones **distintas** sobre la misma variable de condición, un solo aviso puede despertar al que no toca y quedarse todos dormidos. Ante la duda: `notify_all`.

Truco: esperar con tiempo límite

`wait_for(lk, 50ms, predicado)` devuelve `false` si pasó el tiempo y el predicado sigue sin cumplirse. Útil cuando el enunciado pide «espera como mucho X ms».

9 T6 · Planificación

9.1 Qué hay que decidir: quién ejecuta y cuándo

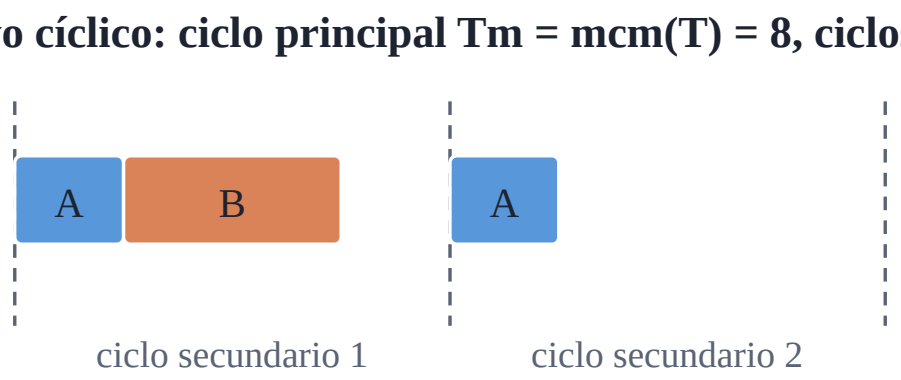
Con una sola CPU y varias tareas con plazos, hace falta una **política de planificación**. Después hay que **demostrar** (no probar) que todas cumplen sus plazos incluso en el peor caso. Hay dos familias:

- **Planificación estática** (ejecutivo cíclico): el plan se calcula **antes** de ejecutar y se repite.
- **Planificación por prioridades con expulsión**: cada tarea tiene una prioridad; si llega una más prioritaria, **expulsa** a la que está ejecutando.

Notación (la misma de T0): cada tarea i tiene cómputo C_i, periodo T_i y plazo $D_i \leq T_i$.

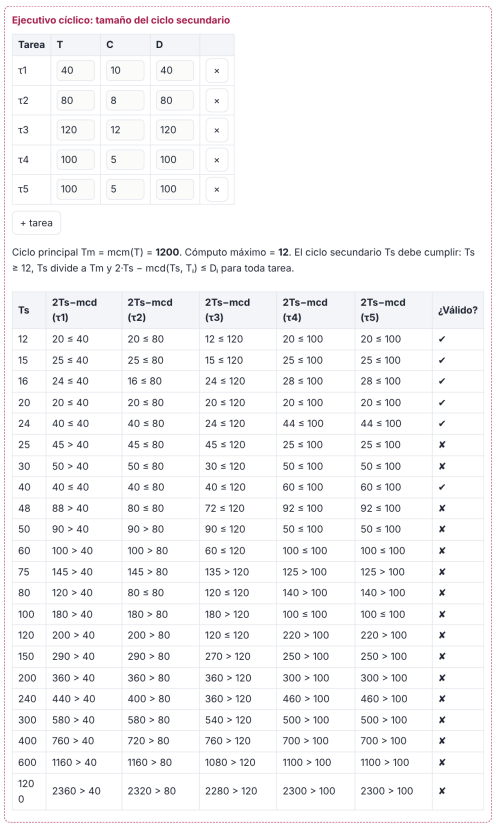
9.2 Ejecutivo cíclico

Si todas las tareas son periódicas, el comportamiento se repite cada **ciclo principal** $T_M = \text{mcm}(T_i)$ (el **hiperperiodo**). Se divide en **ciclos secundarios** de duración T_S y en cada uno se ejecutan unas tareas, sin interrupciones. Entre ciclos se espera con `sleep_until`.



Condiciones para elegir T_S:

1. T_S **divide** a T_M (el ciclo principal es un número entero de secundarios).
2. $T_S \geq \max C_i$ (así ninguna tarea se parte; si no, hay que trocear tareas).
3. Para cada tarea, entre su activación y su plazo debe caber un ciclo secundario completo: $2T_S - \text{gcd}(T_S, T_i) \leq D_i$.



Ejecutivo cíclico: tamaño del ciclo secundario

Tarea	T	C	D	
τ1	40	10	40	×
τ2	80	8	80	×
τ3	120	12	120	×
τ4	100	5	100	×
τ5	100	5	100	×

+ tarea

Ciclo principal Tm = mcm(T) = **1200**. Cómputo máximo = **12**. El ciclo secundario Ts debe cumplir: Ts ≥ 12, Ts divide a Tm y 2·Ts − mcd(Ts, T_i) ≤ D_i para toda tarea.

Ts	2Ts−mcd (τ1)	2Ts−mcd (τ2)	2Ts−mcd (τ3)	2Ts−mcd (τ4)	2Ts−mcd (τ5)	¿Válido?
12	20 ≤ 40	20 ≤ 80	12 ≤ 120	20 ≤ 100	20 ≤ 100	✔
15	25 ≤ 40	25 ≤ 80	15 ≤ 120	25 ≤ 100	25 ≤ 100	✔
16	24 ≤ 40	16 ≤ 80	24 ≤ 120	28 ≤ 100	28 ≤ 100	✔
20	20 ≤ 40	20 ≤ 80	20 ≤ 120	20 ≤ 100	20 ≤ 100	✔
24	40 ≤ 40	40 ≤ 80	24 ≤ 120	44 ≤ 100	44 ≤ 100	✔
25	45 > 40	45 ≤ 80	45 ≤ 120	25 ≤ 100	25 ≤ 100	✘
30	50 > 40	50 ≤ 80	30 ≤ 120	50 ≤ 100	50 ≤ 100	✘
40	40 ≤ 40	40 ≤ 80	40 ≤ 120	60 ≤ 100	60 ≤ 100	✔
48	88 > 40	80 ≤ 80	72 ≤ 120	92 ≤ 100	92 ≤ 100	✘
50	90 > 40	90 > 80	90 ≤ 120	50 ≤ 100	50 ≤ 100	✘
60	100 > 40	100 > 80	60 ≤ 120	100 ≤ 100	100 ≤ 100	✘
75	145 > 40	145 > 80	135 > 120	125 > 100	125 > 100	✘
80	120 > 40	80 ≤ 80	120 ≤ 120	140 > 100	140 > 100	✘
100	180 > 40	180 > 80	180 > 120	100 ≤ 100	100 ≤ 100	✘
120	200 > 40	200 > 80	120 ≤ 120	220 > 100	220 > 100	✘
150	290 > 40	290 > 80	270 > 120	250 > 100	250 > 100	✘
200	360 > 40	360 > 80	360 > 120	300 > 100	300 > 100	✘
240	440 > 40	400 > 80	360 > 120	460 > 100	460 > 100	✘
300	580 > 40	580 > 80	540 > 120	500 > 100	500 > 100	✘
400	760 > 40	720 > 80	760 > 120	700 > 100	700 > 100	✘
600	1160 > 40	1160 > 80	1080 > 120	1100 > 100	1100 > 100	✘
1200	2360 > 40	2320 > 80	2280 > 120	2300 > 100	2300 > 100	✘

Captura del simulador interactivo (disponible en la versión web de la guía).

El código es un bucle con `switch (ciclo)` y `sleep_until`:

CPP

```cpp
auto siguiente = steady_clock::now() + ts;
int ciclo = 0;
while (true) {
    switch (ciclo) {                      // plan estático del ciclo principal
        case 0: A(); B(); C(); break;
        case 1: A(); break;
        /* ... */
    }
    ciclo = (ciclo + 1) % N_CICLOS;
    this_thread::sleep_until(siguiente);   // instante absoluto: sin deriva
    siguiente += ts;
}
```

Ventajas: simple y predecible. Inconvenientes: rígido, difícil con tareas esporádicas y con periodos «incómodos» (hiperperiodo enorme).

Vídeo recomendado (en inglés, 28 min, curso universitario): ejecutivos cíclicos, ciclo principal y ciclos secundarios.

Vídeo recomendado

Lección 3: Ejecutivos cíclicos
Real Time Operating System - IITKGP · 27:45
youtube.com/watch?v=arF8CJzJxa4

9.3 Prioridades fijas: RM y DM

- **RM (Rate Monotonic):** menor **periodo** ⇒ mayor prioridad. Óptimo cuando $D_i = T_i$.
- **DM (Deadline Monotonic):** menor **plazo** ⇒ mayor prioridad. Óptimo cuando $D_i \leq T_i$ (incluye RM).

Dos formas de comprobar la planificabilidad:

1. Condición suficiente (rápida, pero pesimista). Para n tareas con $D = T$ (Liu y Layland):

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i} \leq n\left(2^{1/n} - 1\right)$$

La cota baja de 1 (n=1) a $\ln 2 \approx 0{,}693$ cuando $n \to \infty$. Si U la cumple, el conjunto es planificable. Si no, **no se puede concluir** nada.

2. Tiempo de respuesta (exacta). El tiempo de respuesta R_i es el máximo desde que i se activa hasta que termina. Se calcula iterando:

$$R_i^{(0)} = C_i + B_i, \qquad R_i^{(k+1)} = C_i + B_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i^{(k)}}{T_j} \right\rceil C_j$$

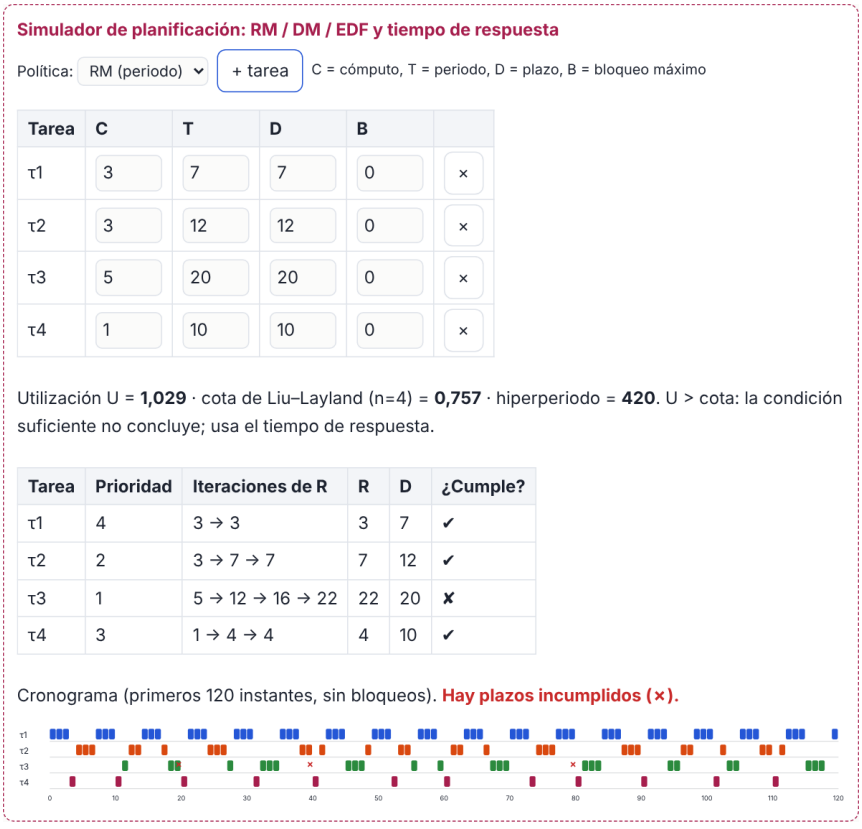
hasta que $R_i^{(k+1)} = R_i^{(k)}$ (converge) o se pasa de D_i (no planificable). $hp(i)$ son las tareas de **mayor prioridad** y B_i es el **bloqueo** (más abajo, vale 0 sin recursos compartidos). La tarea cumple si $R_i \leq D_i$.

Intuición

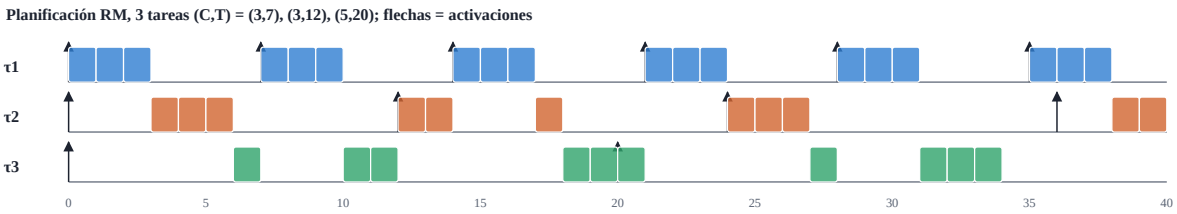
El tiempo de respuesta es «mi trabajo + el trabajo de los más prioritarios que llegan mientras espero». Cada vez que sumo trabajo ajeno, tardo más, y puede entrar **otra** activación de una tarea más prioritaria. La iteración busca el punto fijo.

Ejemplo: $(C, T) = (3, 7), (3, 12), (5, 20)$ con $D = T$. Es $U = 0{,}929 > 0{,}780$, así que la cota no sirve, pero el tiempo de respuesta sí concluye:

Tarea	Iteraciones de R	R	D	¿Cumple?
A (3,7)	3	3	7	Sí
B (3,12)	3 → 6 → 6	6	12	Sí
C (5,20)	5 → 11 → 14 → 17 → 20 → 20	20	20	Sí (justo)



Captura del simulador interactivo (disponible en la versión web de la guía).



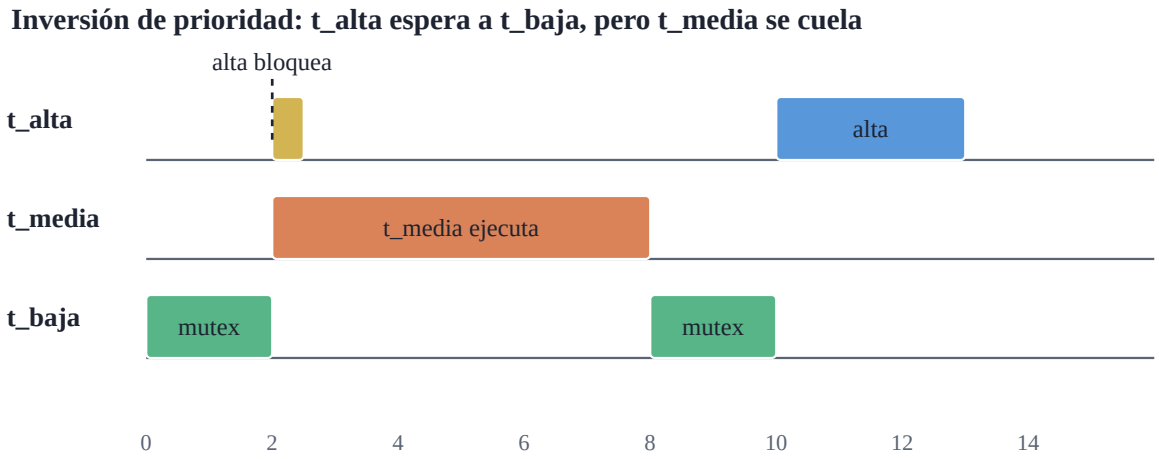
Vídeo recomendado (explicación breve de la planificación monotónica por tasa).

Vídeo recomendado

Planificación monotónica por tasa
Manoj MVR · 8:51
youtube.com/watch?v=xgW8VhEOpFg

9.4 Recursos compartidos: inversión de prioridad

Una tarea baja (L) toma un `mutex`. Llega una alta (H) que lo necesita: se **bloquea**. Hasta aquí, normal. Pero si ahora llega una **media** (M) que no usa el recurso, M expulsa a L (tiene más prioridad que L) y H sigue esperando **a M**, que tiene menos prioridad que H. Eso es la **inversión de prioridad**: la espera de H no está acotada por la sección crítica de L, sino por todo lo que haga M.



Con herencia de prioridad, t_baja ejecutaría a prioridad de t_alta y t_media no la interrumpiría (caso Mars Patl

Inversión de prioridad: L (baja), M (media), H (alta) y un mutex

Protocolo: sin protocolo



H termina en **t = 12** (llega en t = 3). Tiempo de respuesta de H = **9**; su cómputo propio es 4, luego pierde **5** unidades esperando.

L: 1 + sección crítica de 3 + 1. M llega en 2 y dura 4. H llega en 3: 1 + sección crítica de 2 + 1.

Captura del simulador interactivo (disponible en la versión web de la guía).

Dos protocolos que la resuelven:

Protocolo	Regla	Bloqueo
Herencia de prioridad	Si H se bloquea por L, L **hereda** la prioridad de H mientras tenga el recurso	Puede haber bloqueo en cadena; interbloqueo posible
Techo de prioridad inmediato (ICPP)	Cada recurso tiene un **techo** = prioridad máxima de las tareas que lo usan. Al **tomar** el recurso, la tarea sube a ese techo	Una tarea se bloquea **como mucho una vez** y como mucho una sección crítica; sin interbloqueos

Cálculo con ICPP

1. **Prioridades** (DM o RM).
2. **Techo** de cada recurso: máximo de las prioridades de las tareas que lo usan.
3. **Bloqueo** de la tarea i:

$$B_i = \max\{\, c_{k,r} \;:\; k \text{ de menor prioridad que } i,\ r \text{ con techo}(r) \geq P_i \,\}$$

donde $c_{k,r}$ es la duración máxima de la sección crítica de k sobre r. La tarea de **menor** prioridad tiene $B = 0$ y nunca se bloquea por una de **menor** prioridad.

1. **Tiempo de respuesta** con B_i en la recurrencia.

Qué suele fallar

- Tomar secciones críticas de tareas de **mayor o igual** prioridad en B_i: solo cuentan las de **menor** prioridad.
- Incluir recursos cuyo **techo es menor** que la prioridad de la tarea: no la bloquean (el techo es lo que lo decide, no quién usa el recurso).
- Olvidar que el plazo de una tarea esporádica es D y su periodo mínimo T: se analiza igual, con T como separación mínima.
- Parar la iteración en la primera vuelta: hay que repetir hasta que el valor **no cambie**.
- Mezclar «prioridad más alta» con «número más grande»: fija la convención (aquí, mayor número = mayor prioridad).

Vídeos recomendados sobre inversión de prioridad (en inglés) y protocolo de techo de prioridad (curso universitario, en inglés).

Vídeo recomendado

Introduction to RTOS Part 11 - Priority Inversion
DigiKey · 12:35
youtube.com/watch?v=C2xKhxROmhA

Vídeo recomendado

Lecture 16 : Priority Ceiling Protocol
Real Time Operating System - IITKGP · 30:47
youtube.com/watch?v=cI5wjYkShMI

9.5 EDF y utilización

EDF (Earliest Deadline First) da la prioridad a la tarea cuyo plazo absoluto es más próximo. Con $D = T$ y una CPU, el conjunto es planificable si y solo si $U \leq 1$ (Liu y Layland, 1973). Es más eficiente que RM, pero las prioridades cambian en tiempo de ejecución y el análisis con recursos es más complejo.

Más a fondo

La cota $n(2^{1/n} - 1)$ sale de buscar el peor reparto de periodos para el que RM aún funciona: para $n = 2$ es $0{,}828$, para $n = 3$ es $0{,}780$. Es solo suficiente: el conjunto $(3, 7), (3, 12), (5, 20)$ tiene $U = 0{,}929$ y es planificable. Referencias: C. L. Liu y J. W. Layland, «Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment», *J. ACM*, 1973; A. Burns y A. Wellings, *Real-Time Systems and Programming Languages*.

Planificación: fórmulas clave

- Utilización: $U = \sum C_i/T_i$.
- Cota RM (D=T): $U \leq n(2^{1/n} - 1)$ — $n = 1$: 1; $n = 2$: $0{,}828$; $n = 3$: $0{,}780$.
- EDF (D=T): $U \leq 1$.
- Hiperperiodo: $T_M = \text{mcm}(T_i)$.
- Ciclo secundario: $T_S \mid T_M$, $T_S \geq \max C_i$, $2T_S - \gcd(T_S, T_i) \leq D_i$.
- Tiempo de respuesta: $R_i = C_i + B_i + \sum_{j \in hp(i)} \lceil R_i/T_j \rceil C_j$.
- ICPP: techo$(r) = \max P_k$ de quienes usan r; $B_i = \max c_{k,r}$ con $P_k < P_i$ y techo$(r) \geq P_i$.

Vídeo recomendado (en inglés, 6 min): EDF paso a paso.

Vídeo recomendado

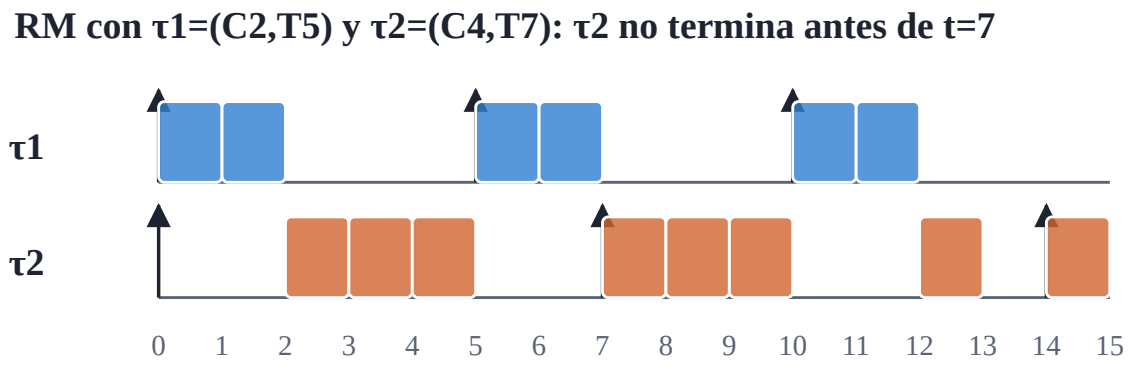
OS22 - Earliest Deadline First (EDF) Scheduling
EZCSE · 6:27
youtube.com/watch?v=lZdS9offd5s

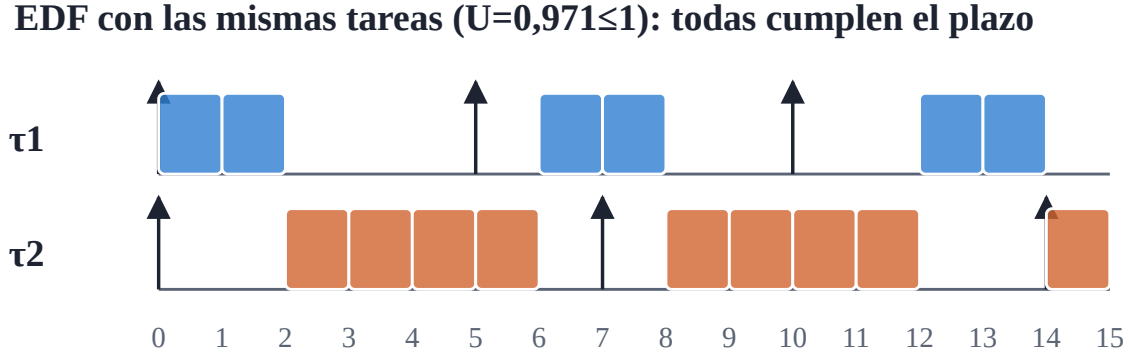
9.6 Cuando RM falla y EDF no: ejemplo explicado

Dos tareas, $\tau_1 = (C{=}2, T{=}5)$ y $\tau_2 = (C{=}4, T{=}7)$. La utilización es $U = 0{,}4 + 0{,}571 = 0{,}971$.

- **Test de utilización RM:** $0{,}971 > 0{,}828$. El test no decide.
- **Tiempo de respuesta de τ_2:** $R = 4 \to 4 + \lceil 4/5 \rceil 2 = 6 \to 4 + \lceil 6/5 \rceil 2 = 8 > 7$. **No es planificable con RM.**
- **EDF:** $U \leq 1 \Rightarrow$ **sí es planificable**.

El programa de abajo lo confirma simulando un hiperperiodo (35 unidades de tiempo). Con RM, τ_2 no acaba antes de su siguiente liberación en $t = 7$.





CPP

```
#include <algorithm>
#include <iostream>
```

```cpp
#include <numeric>
#include <vector>
using namespace std;

struct Tarea { int C, T; };

// Simulación por unidades de tiempo. politica: 0 = RM (menor periodo primero), 1 = EDF (plazo más cercano primero)
int simular(const vector<Tarea>& ts, int politica, int horizonte, vector<string>& log) {
    int n = ts.size(), fallos = 0;
    vector<int> resta(n, 0), plazo(n, 0);
    for (int t = 0; t < horizonte; ++t) {
        for (int i = 0; i < n; ++i) {
            if (t % ts[i].T == 0) {                       // nueva liberación
                if (resta[i] > 0) { ++fallos; log.push_back("t=" + to_string(t) + ": tau" + to_string(i + 1) + " NO
                 ↪  terminó antes de su nuevo periodo"); }
                resta[i] = ts[i].C; plazo[i] = t + ts[i].T;
            }
        }
        int mejor = -1;
        for (int i = 0; i < n; ++i) {
            if (resta[i] == 0) continue;
            if (mejor < 0 || (politica == 0 ? ts[i].T < ts[mejor].T : plazo[i] < plazo[mejor])) mejor = i;
        }
        if (mejor >= 0) --resta[mejor];
    }
    return fallos;
}

int main() {
    vector<Tarea> ts{{2, 5}, {4, 7}};
    double U = 0;
    for (auto& t : ts) U += double(t.C) / t.T;
    int H = lcm(5, 7);
    cout << "U = " << U << " (<= 1: EDF puede) ; cota RM n=2 = 0.828 (U la supera: RM no es seguro)\n";
    // Tiempo de respuesta de tau2 con RM: R = 4 + ceil(R/5)*2
    int R = 4;
    cout << "R2 (RM):";
    for (;;) { cout << ' ' << R; int nuevo = 4 + ((R + 4) / 5) * 2; if (nuevo == R || nuevo > 7) { if (nuevo != R)
     ↪  cout << ' ' << nuevo << " > D=7 -> NO planificable con RM"; break; } R = nuevo; }
    cout << '\n';
    vector<string> log;
    cout << "Simulación de " << H << " unidades:\n";
    cout << " RM : fallos = " << simular(ts, 0, H, log) << '\n';
    for (auto& l : log) cout << "      " << l << '\n';
    log.clear();
    cout << " EDF: fallos = " << simular(ts, 1, H, log) << '\n';
}
```

Salida

```
U = 0.971429 (<= 1: EDF puede) ; cota RM n=2 = 0.828 (U la supera: RM no es seguro)
R2 (RM): 4 6 8 > D=7 -> NO planificable con RM
Simulación de 35 unidades:
 RM : fallos = 1
      t=7: tau2 NO terminó antes de su nuevo periodo
 EDF: fallos = 0
```

Receta: ¿es planificable? (el orden que da más puntos)

1. **Calcula U.** Si $U > 1$: **no** es planificable con ningún algoritmo. Fin.
2. **RM, test de la cota** $n(2^{1/n} - 1)$. Si $U \leq$ cota: **sí**. Si no, no concluye: sigue.
3. **Tiempo de respuesta**, tarea por tarea, de **mayor a menor prioridad**. Si todas cumplen $R_i \leq D_i$: **sí**. Si una no: **no** con RM.
4. **EDF** (con $D = T$): $U \leq 1 \Rightarrow$ sí.
5. **Con recursos:** calcula B_i con los techos y repite el paso 3 sumando B_i.

Trucos de cálculo a mano

- **Techo rápido:** $\lceil R/T \rceil$ es «cuántas veces cabe T en R, redondeando **hacia arriba**». Si $R = T$ exacto, sale 1, no 2.
- **Arranca la iteración** en $R^{(0)} = C_i + B_i$ y **para** cuando dos valores seguidos coinciden o cuando pasas de D_i.

- **Prioridades RM:** ordena por periodo (menor = más prioridad). Si dos coinciden, el enunciado decide el desempate.
- **Hiperperiodo:** mcm(T_i). Con periodos 5 y 7 es 35; si son muchos, factoriza.

Qué suele fallar

- **Aplicar el test de utilización de RM a $D < T$.** Solo vale con plazo igual a periodo.
- **Usar $\lfloor \cdot \rfloor$ en vez de $\lceil \cdot \rceil$** en la recurrencia.
- **Olvidar el bloqueo B_i de las tareas intermedias** con ICPP: aunque τ_2 no use el recurso, sufre bloqueo si su techo es $\geq P_2$.
- **Declarar «no planificable»** solo porque U supera la cota. La cota es suficiente, no necesaria: hay que mirar el tiempo de respuesta.

10 T7 · Coma flotante y casos reales

10.1 Números reales en el ordenador

Un `double` guarda un número real con **52 bits** de mantisa (fracción), 11 de exponente y 1 de signo. Un `float`, con 23, 8 y 1. Con un número finito de bits solo se pueden representar **algunos** valores: el resto se **aproxima** al más cercano.

La clave: igual que $1/3 = 0{,}333\ldots$ no cabe en decimal, $0{,}1$ **no cabe en binario**:

$$0{,}1_{10} = 0{,}0001\,1001\,1001\,1001\,1001\,1001\ldots_2$$

Es una fracción periódica infinita. Cualquier cantidad finita de bits la corta y comete un pequeño error.

CPP

```cpp
#include <cmath>
#include <cstdio>
#include <iostream>
using namespace std;

int main() {
    cout.precision(17);
    cout << "0.1 + 0.2 == 0.3 ? " << boolalpha << (0.1 + 0.2 == 0.3) << '\n';
    cout << "0.1 + 0.2 = " << 0.1 + 0.2 << '\n';
    cout << "|0.1+0.2-0.3| < 1e-9 ? " << (fabs(0.1 + 0.2 - 0.3) < 1e-9) << '\n';
    double s = 0;
    for (int i = 0; i < 10; ++i) s += 0.1;
    cout << "suma de diez 0.1 = " << s << " (== 1 ? " << (s == 1.0) << ")\n";

    // Reloj del Patriot: décimas de segundo como entero, 0,1 truncado a 23 bits tras la coma
    const double truncado = floor(0.1 * (1 << 23)) / (1 << 23);
    const double error_tic = 0.1 - truncado;
    const long tics = 100L * 3600 * 10;                  // 100 horas
    const double deriva = tics * error_tic;
    printf("error por tic   = %.3e s\n", error_tic);
    printf("deriva en 100 h = %.4f s\n", deriva);
    printf("desplazamiento  = %.0f m (Scud a 1676 m/s)\n", deriva * 1676);
}
```

Salida

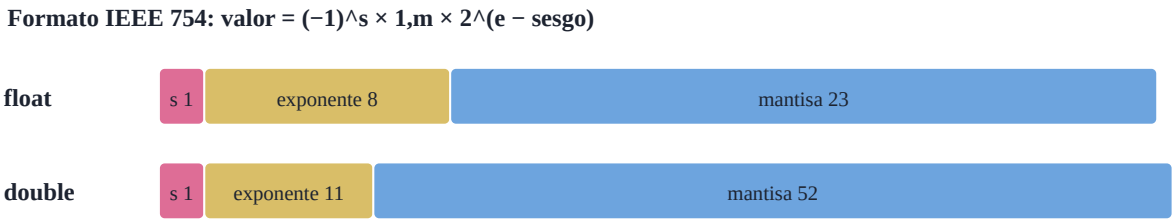
```
0.1 + 0.2 == 0.3 ? false
0.1 + 0.2 = 0.30000000000000004
|0.1+0.2-0.3| < 1e-9 ? true
suma de diez 0.1 = 0.99999999999999989 (== 1 ? false)
error por tic   = 9.537e-08 s
deriva en 100 h = 0.3433 s
desplazamiento  = 575 m (Scud a 1676 m/s)
```

Consecuencias prácticas:

- **Nunca compares `double` con `==`.** Compara con tolerancia: `fabs(a - b) < 1e-9`.
- Los errores **se acumulan**: sumar 0,1 diez veces no da exactamente 1.
- Sumar un número muy pequeño a uno muy grande **lo pierde** (los dígitos no caben).
- Para dinero o tiempo, usa **enteros** (céntimos, microsegundos) o tipos de tiempo como `std::chrono`.

Qué suele fallar

- Condiciones de bucle con `double` (`while (x != 1.0)`): puede no terminar nunca.
- Convertir `double` a `int` **trunca** hacia cero (3,99 → 3), no redondea.
- Dividir enteros: `1/2` es `0`; escribe `1.0/2`.
- Restar dos números casi iguales: se pierden cifras significativas (cancelación).



sesgo: 127 (float) · 1023 (double) · ≈7 cifras decimales (float) · ≈15–16 (double)

0,1 en binario es periódico (0,0001100110011…): se redondea → 0,1+0,2 ≠ 0,3

```
Épsilon de máquina: float ≈ 1,19e-7 · double ≈ 2,22e-16
```

Vídeos recomendados: uno en español sobre cómo se representan los reales y otro (en inglés) de Computerphile.

Vídeo recomendado

Cómo se representan los números de punto flotante
Spanning Tree · 9:56
youtube.com/watch?v=bbkcEiUjehk

Vídeo recomendado

Floating Point Numbers - Computerphile
Computerphile · 9:16
youtube.com/watch?v=PZRI1IfStY0

10.2 Caso 1: el misil Patriot (Dhahran, 1991)

El reloj interno del Patriot contaba el tiempo en **décimas de segundo** como un entero. Para convertirlo a segundos multiplicaba por $1/10$, almacenado con 24 bits y por tanto **truncado**. El error de cada tic era

$$e = 0{,}1 - \frac{\lfloor 0{,}1 \cdot 2^{23} \rfloor}{2^{23}} \approx 9{,}5 \cdot 10^{-8} \text{ s.}$$

Parece diminuto, pero la batería llevaba unas **100 horas** encendida, es decir, $100 \cdot 3600 \cdot 10 = 3{,}6 \cdot 10^{6}$ tics:

$$\text{deriva} = 3{,}6 \cdot 10^{6} \cdot 9{,}5 \cdot 10^{-8} \approx 0{,}34 \text{ s.}$$

Un Scud viaja a unos 1676 m/s, así que en 0,34 s se mueve más de **medio kilómetro**: el sistema miraba donde el misil ya no estaba y lo descartó como falsa alarma. (Con $e = 9{,}5 \cdot 10^{-8}$ exactos salen 573 m; con el valor sin redondear, 575 m: el orden de magnitud es lo importante.)

Error de coma flotante: el reloj del Patriot

Horas encendido:

0,1 s en binario: `0001100110011001100110011001100110011001…` (infinito periódico). Truncado a 23 bits tras la coma: error por tic = **9,537e-8** s.

Tras **120 h** hay 4.320.000 tics → deriva = **0,4120 s** → un Scud a 1676 m/s se «desplaza» **690 m** respecto a donde el sistema lo busca.

Captura del simulador interactivo (disponible en la versión web de la guía).

Lección

Un error **pequeño y sistemático** se vuelve grande con el tiempo. En tiempo real, el tiempo es lo que más se acumula. Por eso el tiempo se mide con enteros, o con tipos precisos, y los relojes se **recalibran**. Fuente de referencia: U.S. General Accounting Office, *Patriot Missile Defense: Software Problem Led to System Failure at Dhahran, Saudi Arabia* (IMTEC-92-26, 1992).

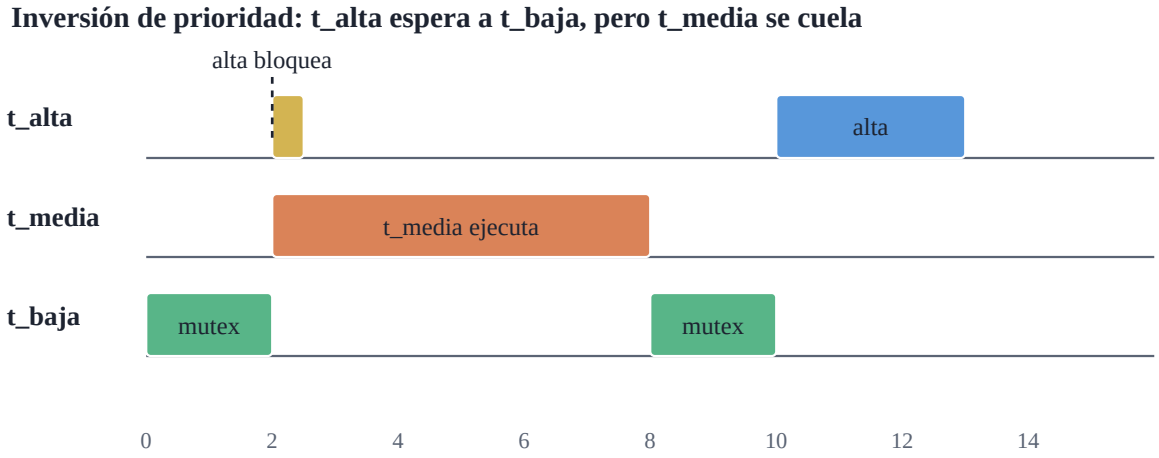
10.3 Caso 2: Mars Pathfinder (1997)

Pocos días después de aterrizar en Marte, el ordenador de la sonda empezó a **reiniciarse** solo. El sistema operativo era VxWorks, con planificación expulsiva por prioridades fijas. Las tareas que importan, de mayor a menor prioridad:

1. `bc_sched`: planifica el bus de datos y comprueba con un *watchdog* que `bc_dist` terminó en el ciclo anterior.
2. `bc_dist`: distribuye datos por el bus.
3. Varias tareas de prioridad media (comunicaciones, etc.).
4. `ASI/MET`: tarea científica de **baja** prioridad.

`bc_dist` y `ASI/MET` compartían una estructura protegida por un `mutex`. La secuencia fatal fue la **inversión de prioridad** que ya conoces:

1. `ASI/MET` toma el `mutex`.
2. `bc_dist` (alta) se bloquea esperándolo.
3. Tareas **medias** expulsan a `ASI/MET`, así que `bc_dist` espera cuanto ellas quieran.
4. `bc_sched` detecta que `bc_dist` no ha terminado y **reinicia** el sistema.



Con herencia de prioridad, t_baja ejecutaría a prioridad de t_alta y t_media no la interrumpiría (caso Patl

La solución fue activar la **herencia de prioridad** en el `mutex` (estaba desactivada por rendimiento). Se pudo hacer **desde la Tierra**, actualizando parámetros, porque el sistema conservaba sus herramientas de trazas. Puedes reproducir el efecto en el simulador de T6 (sin protocolo frente a herencia o techo).

Más a fondo

El protocolo de herencia y el de techo de prioridad se deben a L. Sha, R. Rajkumar y J. Lehoczky, «Priority Inheritance Protocols: An Approach to Real-Time Synchronization», *IEEE Trans. on Computers*, 1990. La explicación de lo ocurrido en Pathfinder circuló en el foro *RISKS Digest* (vol. 19, n.º 49, 1997). Sobre aritmética de coma flotante, la referencia clásica es D. Goldberg, «What Every Computer Scientist Should Know About Floating-Point Arithmetic», *ACM Computing Surveys*, 1991.

Coma flotante

- `double`: 1 bit de signo + 11 de exponente + 52 de mantisa (≈ 15-16 cifras decimales).
- `float`: 1 + 8 + 23 (≈ 7 cifras decimales).
- Comparar: `fabs(a - b) < tolerancia`.

- Error acumulado ≈ (error por operación) × (número de operaciones).
- Patriot: $e = 0{,}1 - \lfloor 0{,}1 \cdot 2^{23} \rfloor / 2^{23} \approx 9{,}5 \cdot 10^{-8}$ s; en 100 h: 0,34 s.

10.4 Experimentos con coma flotante

Cuatro experimentos que explican por qué falla == con reales y cómo evitarlo.

CPP

```cpp
#include <cmath>
#include <iomanip>
#include <iostream>
#include <vector>
using namespace std;

bool casi_igual(double a, double b, double rel = 1e-9) {          // tolerancia RELATIVA: funciona con números grandes
 ↪  y pequeños
    return fabs(a - b) <= rel * max(fabs(a), fabs(b));
}

int main() {
    cout << setprecision(10);
    const int N = 10'000'000;
    float sf = 0; double sd = 0;
    for (int i = 0; i < N; ++i) { sf += 0.1f; sd += 0.1; }
    // Kahan: guarda en 'c' lo que se pierde en cada suma y lo recupera en la siguiente
    double s = 0, c = 0;
    for (int i = 0; i < N; ++i) { double y = 0.1 - c; double t = s + y; c = (t - s) - y; s = t; }
    cout << "suma de 0,1 diez millones de veces (exacto: 1000000)\n";
    cout << "  float : " << sf << "\n  double: " << sd << "\n  Kahan : " << s << '\n';

    cout << "\nUn bucle con paso 0,1 y == nunca para exactamente en 1:\n";
    int vueltas = 0;
    for (double x = 0; x < 1.0 - 1e-9 && vueltas < 20; x += 0.1) ++vueltas;
    cout << "  con tolerancia: " << vueltas << " vueltas\n";
    double x = 0; for (int i = 0; i < 10; ++i) x += 0.1;
    cout << boolalpha << "  tras 10 sumas, x == 1.0 ? " << (x == 1.0) << " ; x = " << setprecision(17) << x << '\n';
    cout << setprecision(10);

    cout << "\nComparar con tolerancia absoluta frente a relativa:\n";
    double a = 1e12, b = 1e12 + 0.0001;
    cout << "  fabs(a-b) < 1e-9 : " << (fabs(a - b) < 1e-9) << " (falla: a y b son casi iguales)\n";
    cout << "  casi_igual(a,b)  : " << casi_igual(a, b) << '\n';

    cout << "\nOrden de suma: sumar de menor a mayor pierde menos\n";
    vector<float> v;
    v.push_back(1e8f);
    for (int i = 0; i < 100; ++i) v.push_back(1.0f);
    float mayor_primero = 0, menor_primero = 0;
    for (float t : v) mayor_primero += t;
    for (auto it = v.rbegin(); it != v.rend(); ++it) menor_primero += *it;
    cout << "  1e8 + cien unos, mayor primero: " << mayor_primero << " ; menor primero: " << menor_primero << "
     ↪  (exacto 100000100)\n";
}
```

Salida

```
suma de 0,1 diez millones de veces (exacto: 1000000)
  float : 1087937
  double: 999999.9998
  Kahan : 1000000

Un bucle con paso 0,1 y == nunca para exactamente en 1:
  con tolerancia: 10 vueltas
  tras 10 sumas, x == 1.0 ? false ; x = 0.99999999999999989

Comparar con tolerancia absoluta frente a relativa:
  fabs(a-b) < 1e-9 : false (falla: a y b son casi iguales)
  casi_igual(a,b)  : true

Orden de suma: sumar de menor a mayor pierde menos
  1e8 + cien unos, mayor primero: 100000000 ; menor primero: 100000096 (exacto 100000100)
```

Qué enseña cada salida

1. **Sumar 0,1 diez millones de veces:** `float` da 1 087 937 (error del 9 %), `double` da 999 999,9998 y **Kahan** da 1 000 000 exacto. Kahan guarda en `c` lo que se pierde en cada suma y lo recupera en la siguiente.
2. **Bucle con paso 0,1:** tras 10 sumas, `x` vale 0,99999999999999989, no 1; con tolerancia el bucle da exactamente 10 vueltas. Por eso `x == 1.0` es falso y un bucle `while (x != 1.0)` **no termina**. Usa `<` con margen o cuenta con un entero.
3. **Tolerancia absoluta frente a relativa:** con números grandes, `fabs(a-b) < 1e-9` da `false` aunque `a` y `b` sean prácticamente iguales; `casi_igual` compara la diferencia con **una fracción del valor** (`rel * max(|a|,|b|)`) y da `true`.
4. **Orden de suma:** 10^8 más cien unos da 100 000 000 si sumas primero el grande (los unos se pierden) y 100 000 096 si sumas primero los pequeños (exacto: 100 000 100). De menor a mayor se pierde menos.

Qué suele fallar

- `if (x == 0.3)` o `while (x != 1.0)` con reales: casi siempre un error.
- **Mezclar `int` y `double`:** `1/2` vale 0; escribe `1.0/2`.
- **Restar números casi iguales** (cancelación): se pierden cifras significativas. Reformula la fórmula si puedes.
- **Esperar que `float` tenga más de 7 cifras:** a partir de ahí los números no son exactos.

Regla práctica

Si vas a comparar reales: **tolerancia relativa**. Si vas a sumar muchos: de pequeño a grande, o con Kahan. Si necesitas contar: **entero**.

11 Prácticas

Las tres prácticas del curso, desarrolladas paso a paso. Cada una tiene el código completo (también disponible como archivos independientes), la salida esperada y la interpretación que se espera que escribas. Compila siempre con:

```
g++ -std=c++17 -Wall -Wextra -pthread fichero.cpp -o programa
```

11.1 Práctica 1 · Programación basada en objetos: strings y colecciones

Usar objetos de la biblioteca estándar sin definir clases propias: `string`, `vector`, `map`, flujos.

1.1 a 1.3 · Palabras, ordenación y ficheros

Qué se pide. Leer una línea, imprimir cada palabra en su propia línea sin signos de puntuación, guardarlas en un `vector<string>`, ordenarlas (de menor a mayor y de mayor a menor) y pasar a leer/escribir ficheros comprobando que entrada y salida son distintos.

Cómo se piensa.

1. Para separar palabras, un `istringstream` y `in >> p` (corta por espacios). Para quitar puntuación, `remove_if` + `erase` con `ispunct`.
2. `sort(v.begin(), v.end())` ordena por **código de carácter**: las mayúsculas (65-90) van **antes** que las minúsculas (97-122). Para ignorar mayúsculas, pasa un comparador (lambda) que compare las versiones en minúsculas.
3. Con ficheros, `ifstream` y `ofstream` se usan igual que `cin` y `cout`. Comprueba siempre `if (!in)`.

CPP

```cpp
// Práctica 1.1-1.3: palabras de una línea, ordenar y usar ficheros
#include <algorithm>
#include <cctype>
#include <fstream>
#include <iostream>
#include <sstream>
#include <string>
#include <vector>
using namespace std;

string sin_puntuacion(string p) {
    p.erase(remove_if(p.begin(), p.end(), [](unsigned char c) { return ispunct(c); }), p.end());
    return p;
}
vector<string> palabras(const string& linea) {            // 1.1: separa por espacios con un stringstream
    istringstream in{linea};
    vector<string> v; string p;
    while (in >> p) { p = sin_puntuacion(p); if (!p.empty()) v.push_back(p); }
    return v;
}
string minusculas(string s) { transform(s.begin(), s.end(), s.begin(), [](unsigned char c) { return tolower(c); });
 ↪  return s; }

int main() {
    string linea = "Programación de Tiempo Real o programación de tiempo real, ¿vale?";
    auto v = palabras(linea);
    cout << "1.1) una por línea:\n"; for (auto& p : v) cout << p << '\n';

    sort(v.begin(), v.end());                              // 1.2: orden por código de carácter
    cout << "1.2) orden por defecto (las mayúsculas van antes que las minúsculas):\n"; for (auto& p : v) cout << p << '
     ↪  '; cout << '\n';
    sort(v.begin(), v.end(), [](const string& a, const string& b) { return minusculas(a) < minusculas(b); });
    cout << "   ignorando mayúsculas:\n"; for (auto& p : v) cout << p << ' '; cout << '\n';
    sort(v.begin(), v.end(), [](const string& a, const string& b) { return minusculas(a) > minusculas(b); });
    cout << "   de mayor a menor:\n"; for (auto& p : v) cout << p << ' '; cout << '\n';

    // 1.3: ficheros (comprobando que entrada y salida son distintos)
    string entrada = "entrada.txt", salida = "salida.txt";
    if (entrada == salida) { cerr << "los ficheros deben ser distintos\n"; return 1; }
    { ofstream f{entrada}; f << linea << '\n'; }           // preparo un fichero de prueba
    ifstream in{entrada}; ofstream out{salida};
    if (!in || !out) { cerr << "no se pudo abrir\n"; return 1; }
```

```
    string l;
    while (getline(in, l)) for (auto& p : palabras(l)) out << p << '\n';
    cout << "1.3) escrito " << salida << '\n';
}
```

Salida

```
1.1) una por línea:
Programación
de
Tiempo
Real
o
programación
de
tiempo
real
¿vale
1.2) orden por defecto (las mayúsculas van antes que las minúsculas):
Programación Real Tiempo de de o programación real tiempo ¿vale
   ignorando mayúsculas:
de de o Programación programación Real real Tiempo tiempo ¿vale
   de mayor a menor:
¿vale Tiempo tiempo Real real Programación programación o de de
1.3) escrito salida.txt
```

Interpretación que debes escribir. «Sin comparador, `Programación` aparece antes que `de` porque la `P` (80) es menor que la `d` (100). Se soluciona comparando en minúsculas. `ispunct` solo reconoce signos ASCII: ¿ (UTF-8, dos bytes) no se elimina.»

1.4 y 1.5 · Histograma de palabras

Qué se pide. Un `map<string,int>` con el número de apariciones de cada palabra; mostrar las dos más repetidas y qué palabras acumulan el 20 % de las apariciones; después, un histograma por longitud con `vector<map<string,int>>`.

Cómo se piensa.

- `++hist[palabra]` hace todo: si no existe la crea con 0 y suma 1.
- Un `map` está ordenado por **clave**, no por valor. Para ordenar por frecuencia, copia a un `vector<pair<string, int>>` y ordénalo con un comparador por `second`.
- El 20 %: recorre ordenado de más a menos frecuente acumulando hasta llegar a `total/5`.
- Por longitud: `por_longitud[palabra.size()][palabra]`.

CPP

```
// Práctica 1: histograma de palabras de un fichero (map<string,int>) y por longitud (vector<map>)
#include <algorithm>
#include <cctype>
#include <fstream>
#include <iostream>
#include <map>
#include <sstream>
#include <string>
#include <vector>
using namespace std;

string normalizar(string p) {
    p.erase(remove_if(p.begin(), p.end(), [](unsigned char c) { return ispunct(c); }), p.end());   // sin puntuación
    transform(p.begin(), p.end(), p.begin(), [](unsigned char c) { return tolower(c); });         // sin mayúsculas
     ↪  (solo ASCII)
    return p;
}

int main(int argc, char* argv[]) {
    if (argc != 2) { cerr << "uso: " << argv[0] << " fichero.txt\n"; return 1; }
    ifstream in{argv[1]};
    if (!in) { cerr << "no se puede abrir " << argv[1] << '\n'; return 1; }

    map<string, int> hist;
    vector<map<string, int>> por_longitud(16);         // índice = nº de letras (suponemos <= 15)
    long total = 0;
```

```
    string palabra;
    while (in >> palabra) {                          // >> separa por espacios
        palabra = normalizar(palabra);
        if (palabra.empty()) continue;
        ++hist[palabra]; ++total;
        if (palabra.size() < por_longitud.size()) ++por_longitud[palabra.size()][palabra];
    }
    // ordenar por frecuencia: un map está ordenado por clave, así que pasamos a un vector de pares
    vector<pair<string, int>> v(hist.begin(), hist.end());
    sort(v.begin(), v.end(), [](const auto& a, const auto& b) { return a.second > b.second || (a.second == b.second &&
    ↪ a.first < b.first); });

    cout << "palabras: " << total << ", distintas: " << hist.size() << '\n';
    cout << "las dos mas repetidas:\n";
    for (size_t i = 0; i < 2 && i < v.size(); ++i) cout << "  " << v[i].first << ": " << v[i].second << '\n';
    long acum = 0; cout << "palabras que acumulan el 20% de las apariciones:";
    for (auto& [p, n] : v) { cout << ' ' << p << '(' << n << ')'; acum += n; if (acum * 5 >= total) break; }
    cout << "\nla mas repetida de cada longitud (1-6 letras):\n";
    for (size_t l = 1; l <= 6; ++l) {
        if (por_longitud[l].empty()) continue;
        auto it = max_element(por_longitud[l].begin(), por_longitud[l].end(), [](auto& a, auto& b) { return a.second <
        ↪ b.second; });
        cout << "  " << l << " letras: " << it->first << " (" << it->second << ")\n";
    }
}
```

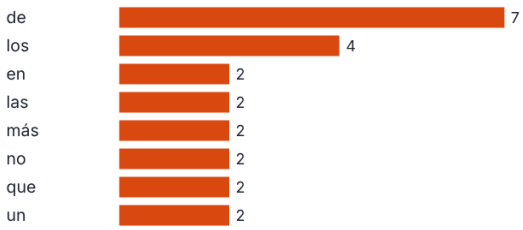
Salida

```
palabras: 1882, distintas: 716
las dos mas repetidas:
  de: 123
  y: 104
palabras que acumulan el 20% de las apariciones: de(123) y(104) que(88) a(44) el(40)
la mas repetida de cada longitud (1-6 letras):
  1 letras: y (104)
  2 letras: de (123)
  3 letras: que (88)
  4 letras: como (13)
  5 letras: mismo (6)
  6 letras: había (14)
```

Práctica 1: histograma de palabras (map<string,int>)

En un lugar de la Mancha, de cuyo nombre no quiero acordarme, no ha mucho tiempo que vivía un hidalgo de los de lanza en astillero, adarga antigua, rocín flaco y galgo corredor. Una olla de algo más vaca que carnero, salpicón las más noches, duelos y quebrantos los sábados, lentejas los viernes, algún palomino de añadidura los domingos, consumían las tres partes de

☑ ignorar mayúsculas




```
Palabras totales: 66
Palabras distintas: 50
Las dos más repetidas:
de: 7
los: 4
```

```
map<string,int> hist;
while (in >> palabra) {
    for (char& c : palabra) c = tolower((unsigned char)c);   // activo
    palabra.erase(remove_if(palabra.begin(), palabra.end(), [](unsigned char c){ return ispunc
    if (!palabra.empty()) ++hist[palabra];
}
```

Captura del simulador interactivo (disponible en la versión web de la guía).

Interpretación. Con el fragmento de 1882 palabras, `de` (123) e `y` (104) suman casi el 12 % del texto: las palabras funcionales dominan. Cinco palabras (`de`, `y`, `que`, `a`, `el`) acumulan el 20 %. Es la **ley de Zipf**: la frecuencia cae muy deprisa con el puesto en el ranking.

Qué suele fallar

- `hist[palabra]` para **consultar** si existe: la crea. Usa `find`.
- Olvidar convertir a minúsculas: `El` y `el` cuentan como palabras distintas.
- No comprobar el fichero abierto, o dejar `argv[1]` sin comprobar `argc`.
- Los acentos: `tolower` del C solo funciona con ASCII. É no se convierte.

11.2 Práctica 2 · Programación orientada a objetos

2.1 · Clase número complejo

Qué se pide. Atributos real e imaginario; constructor con dos valores y un constructor **delegado** sin parámetros (0+0i); `get`; módulo y argumento como `pair` con `hypot` y `atan2`; `operator<<`, `operator>>`, `+`, `-`.

Decisiones de diseño. Los operadores y `modulo_argumento` pueden ser funciones **libres** (fuera de la clase) porque solo necesitan los `get`: la clase queda más pequeña y estable. `operator>>` construye un temporal y hace `swap`: si la lectura falla, el objeto original no queda a medias.

CPP

```cpp
// Práctica 2.1: número complejo
#include <cmath>
#include <iostream>
#include <utility>
using namespace std;

class C_Complejo {
public:
    C_Complejo(double re, double im) : re_{re}, im_{im} {}
    C_Complejo() : C_Complejo(0, 0) {}                    // constructor delegado: 0 + 0i
    double real() const { return re_; }
    double imag() const { return im_; }
private:
    double re_, im_;
};

// Fuera de la clase: solo necesita los get. Reduce lo que la clase tiene que mantener.
pair<double, double> modulo_argumento(const C_Complejo& c) { return {hypot(c.real(), c.imag()), atan2(c.imag(),
 ↪  c.real())}; }

C_Complejo operator+(const C_Complejo& l, const C_Complejo& r) { return {l.real() + r.real(), l.imag() + r.imag()}; }
C_Complejo operator-(const C_Complejo& l, const C_Complejo& r) { return {l.real() - r.real(), l.imag() - r.imag()}; }
ostream& operator<<(ostream& os, const C_Complejo& c) { return os << c.real() << (c.imag() < 0 ? "-" : "+") <<
 ↪  fabs(c.imag()) << "i"; }
istream& operator>>(istream& is, C_Complejo& c) {            // formato: re im
    double re, im;
    if (is >> re >> im) { C_Complejo tmp{re, im}; swap(c, tmp); }   // construir temporal y swap
    return is;
}

int main() {
    C_Complejo a{3, 4}, b{1, -2}, z;
    cout << "a=" << a << " b=" << b << " z=" << z << '\n';
    cout << "a+b=" << a + b << "  a-b=" << a - b << '\n';
    auto [m, arg] = modulo_argumento(a);
    cout << "|a|=" << m << " arg(a)=" << arg << " rad\n";
}
```

Salida

```
a=3+4i b=1-2i z=0+0i
a+b=4+2i  a-b=2+6i
|a|=5 arg(a)=0.927295 rad
```

Práctica 2.1: número complejo (C_Complejo)

a = 3 + 4 i b = 1 + -2 i

```
C_Complejo a{3, 4}, b{1, -2};
cout << "a+b=" << a + b << "  a-b=" << a - b << '\n';
auto [m, arg] = modulo_argumento(a);
cout << "|a|=" << m << " arg(a)=" << arg << " rad\n";
```

```
a=3+4i b=1-2i
a+b=4+2i  a-b=2+6i
|a|=5 arg(a)=0.927295 rad
```

Captura del simulador interactivo (disponible en la versión web de la guía).

2.2 y 2.3 · Mensajes secretos y su registro

Qué se pide. `Mensaje_Secreto` con texto, clave y pista; `bloquear` (sin y con clave nueva, esta última solo si está desbloqueado), `desbloquear(clave)`, `esta_bloqueado`, `borrar`; `operator<<` que muestra la pista si está bloqueado. `Registro_Mensajes` con un `map<string, vector<Mensaje_Secreto>>` y métodos para añadir, consultar, leer el primero/último (borrándolo), listar y borrar con claves.

Cómo se piensa.

- `total_mensajes_persona` usa `find` (no `[]`) para no crear personas fantasma y convierte `size()` (sin signo) a `int`.
- `primer_mensaje` y `ultimo_mensaje` se parecen mucho: un método **privado** `leer(persona, clave, primero)` evita repetir código.
- Borrar del vector mientras se recorre exige iteradores: `m = erase(m)` o `++m`.

CPP

```cpp
// Prácticas 2.2 y 2.3: Mensaje_Secreto y Registro_Mensajes
#include <iostream>
#include <map>
#include <string>
#include <vector>
using namespace std;

class Mensaje_Secreto {
public:
    Mensaje_Secreto(string texto, int clave, string pista = "") : texto_{std::move(texto)}, clave_{clave},
     ↪  pista_{std::move(pista)} {}
    void bloquear() { bloqueado_ = true; }
    bool bloquear(int clave_nueva) { if (bloqueado_) return false; clave_ = clave_nueva; bloqueado_ = true; return
     ↪  true; }
    bool desbloquear(int clave) { if (clave == clave_) bloqueado_ = false; return !bloqueado_; }
    bool esta_bloqueado() const { return bloqueado_; }
    void borrar() { texto_.clear(); }
    const string& pista() const { return pista_; }
    friend ostream& operator<<(ostream& os, const Mensaje_Secreto& m) {
        return os << (m.bloqueado_ ? "[bloqueado] pista: " + m.pista_ : m.texto_);
    }
private:
    string texto_; int clave_; string pista_; bool bloqueado_ = true;
};

class Registro_Mensajes {
public:
    void nuevo_mensaje(const Mensaje_Secreto& m, const string& persona) { reg_[persona].push_back(m); }
    bool existe_persona(const string& p) const { return reg_.find(p) != reg_.end(); }
    int total_mensajes_persona(const string& p) const { auto it = reg_.find(p); return it == reg_.end() ? 0 :
     ↪  static_cast<int>(it->second.size()); }
    vector<string> lista_personas() const { vector<string> v; for (const auto& [p, _] : reg_) v.push_back(p); return
     ↪  v; }
    bool primer_mensaje(const string& p, int clave) { return leer(p, clave, true); }
    bool ultimo_mensaje(const string& p, int clave) { return leer(p, clave, false); }
    int borrar_todos(const string& p, const vector<int>& claves) {      // claves probadas una por mensaje
```

```
        auto it = reg_.find(p); if (it == reg_.end()) return 0;
        int borrados = 0; size_t k = 0;
        for (auto m = it->second.begin(); m != it->second.end(); ++k) {
            if (k < claves.size() && m->desbloquear(claves[k])) { m->borrar(); m = it->second.erase(m); ++borrados; }
            else ++m;                                                    // clave incorrecta: se queda
        }
        return borrados;
    }
private:
    bool leer(const string& p, int clave, bool primero) {               // método privado común
        auto it = reg_.find(p);
        if (it == reg_.end() || it->second.empty()) { cout << "sin mensajes para " << p << '\n'; return false; }
        auto m = primero ? it->second.begin() : prev(it->second.end());
        cout << "pista: " << m->pista() << '\n';
        if (!m->desbloquear(clave)) { cout << "clave incorrecta\n"; return false; }
        cout << *m << '\n'; m->borrar(); it->second.erase(m);
        return true;
    }
    map<string, vector<Mensaje_Secreto>> reg_;
};

int main() {
    Registro_Mensajes r;
    r.nuevo_mensaje({"hola Ana", 1234, "mi PIN"}, "Ana");
    r.nuevo_mensaje({"adios Ana", 99, "dos cifras"}, "Ana");
    cout << "mensajes de Ana: " << r.total_mensajes_persona("Ana") << ", de Luis: " <<
    ↪  r.total_mensajes_persona("Luis") << '\n';
    r.primer_mensaje("Ana", 1);        // clave incorrecta
    r.primer_mensaje("Ana", 1234);     // lo muestra y lo borra
    r.ultimo_mensaje("Ana", 99);
    r.primer_mensaje("Luis", 0);
    cout << "quedan " << r.total_mensajes_persona("Ana") << " mensajes\n";
}
```

Salida

```
mensajes de Ana: 2, de Luis: 0
pista: mi PIN
clave incorrecta
pista: mi PIN
hola Ana
pista: dos cifras
adios Ana
sin mensajes para Luis
quedan 0 mensajes
```

2.4 · Figuras y herencia

Clase abstracta `Figura` con `area()` y `perimetro()` virtuales puros, `operator<<` polimórfico apoyado en un `to_string()` protegido y virtual, destructor virtual, `Circulo`, `Rectangulo` y `Agrupacion_Figuras` (contenedor recursivo con `set<Figura*>`). La solución completa, compilada y ejecutada, está en el tema T3 («Ejemplo completo»).

¿Qué método se ejecuta? Enlace estático frente a dinámico

método virtual | a través de Base* (apunta a Derivada)

Se ejecuta **Derivada::f()**.

La tabla de métodos virtuales del objeto real decide en tiempo de ejecución.

Captura del simulador interactivo (disponible en la versión web de la guía).

2.5 · Actividades de un proyecto

Misma idea con un contenedor recursivo, ahora con **dos** niveles de abstracción: `Tarea_Compuesta` es abstracta (no implementa `duracion`) y sus dos hijas deciden cómo combinarla. Paralelas: el **máximo** de las duraciones. Secuenciales: la **suma**.

CPP

```cpp
// Práctica 2.5: actividades de un proyecto (contenedor recursivo con herencia)
#include <algorithm>
#include <iostream>
#include <sstream>
#include <string>
#include <vector>
using namespace std;

class Actividad {
public:
    Actividad(string id, string desc) : id_{std::move(id)}, desc_{std::move(desc)} {}
    virtual ~Actividad() = default;
    virtual int duracion() const = 0;
    friend ostream& operator<<(ostream& os, const Actividad& a) { return os << a.to_string(); }
protected:
    virtual string to_string() const { ostringstream o; o << id_ << " (" << desc_ << ") dura " << duracion(); return
    ↪  o.str(); }
    string id_, desc_;
};

class Tarea : public Actividad {
public:
    Tarea(string id, string desc, int d) : Actividad{std::move(id), std::move(desc)}, d_{d} {}
    int duracion() const override { return d_; }
private:
    int d_;
};

class Tarea_Compuesta : public Actividad {        // abstracta: no implementa duracion()
public:
    Tarea_Compuesta(string id, string desc) : Actividad{std::move(id), std::move(desc)} {}
    bool anyadir(Actividad* a) { if (a == this) return false; v_.push_back(a); return true; }
protected:
    string to_string() const override {
        ostringstream o; o << Actividad::to_string() << " {";
        for (auto* a : v_) o << " [" << *a << "]";
        o << " }"; return o.str();
    }
    vector<Actividad*> v_;
};

class Tareas_Paralelas : public Tarea_Compuesta {
public:
    using Tarea_Compuesta::Tarea_Compuesta;       // hereda constructores
    int duracion() const override { int m = 0; for (auto* a : v_) m = max(m, a->duracion()); return m; }   // el máximo
};
class Tareas_Secuenciales : public Tarea_Compuesta {
public:
    using Tarea_Compuesta::Tarea_Compuesta;
    int duracion() const override { int s = 0; for (auto* a : v_) s += a->duracion(); return s; }          // la suma
};

int main() {
    Tarea t1{"T1", "cavar", 3}, t2{"T2", "cimentar", 5}, t3{"T3", "pintar", 2}, t4{"T4", "limpiar", 1};
    Tareas_Paralelas par{"P", "a la vez"};
    par.anyadir(&t1); par.anyadir(&t2);
    Tareas_Secuenciales seq{"S", "en orden"};
    seq.anyadir(&par); seq.anyadir(&t3); seq.anyadir(&t4);
    cout << seq << '\n';
    cout << "duracion total = " << seq.duracion() << " (max(3,5) + 2 + 1)\n";
}
```

Salida

```
S (en orden) dura 8 { [P (a la vez) dura 5 { [T1 (cavar) dura 3] [T2 (cimentar) dura 5] }] [T3 (pintar) dura 2] [T4
↪  (limpiar) dura 1] }
duracion total = 8 (max(3,5) + 2 + 1)
```

Interpretación. La duración de `S` es 8 = max(3, 5) + 2 + 1: la tarea paralela interna aporta 5 (la más larga), no 3 + 5. El polimorfismo hace que `S.duracion()` llame a `P.duracion()` sin saber qué tipo de tarea es.

11.3 Práctica 3 · Programación concurrente

3.1 y 3.3 · Suma de matrices con hilos y con thread pool

Qué se pide. Sumar dos matrices `vector<vector<int>>` con n hilos de forma que no haya carreras **sin mutex**: cada hilo suma filas **disjuntas**. Después, un *thread pool*: los hilos piden al «director» la siguiente fila.

Cómo se piensa.

- 3.1: el hilo id suma las filas $id, id+n, id+2n, \ldots$. Como cada hilo **escribe** en filas distintas de `c`, no hay carrera y no hace falta `mutex`. Las matrices de entrada solo se leen: se pasan con `cref`.
- 3.3: ahora el reparto es dinámico. El contador de «siguiente fila» **sí** es un dato compartido: lo protege un `mutex` dentro de la clase `Director`.

CPP

```cpp
// Prácticas 3.1 y 3.3: suma de matrices por filas con n hilos y con thread pool
#include <chrono>
#include <iostream>
#include <mutex>
#include <thread>
#include <vector>
using namespace std;
using Matriz = vector<vector<int>>;

Matriz crear(int tam) {                                  // valores 0..tam-1 en cada fila
    Matriz m(tam, vector<int>(tam));
    for (auto& fila : m) for (int j = 0; j < tam; ++j) fila[j] = j;
    return m;
}
ostream& operator<<(ostream& os, const Matriz& m) {
    for (const auto& f : m) { for (int x : f) os << x << ' '; os << '\n'; }
    return os;
}
// 3.1: cada hilo suma las filas inicio, inicio+salto, inicio+2*salto... No comparten escritura: no hace falta mutex.
void suma_filas(const Matriz& a, const Matriz& b, Matriz& c, size_t inicio, size_t salto) {
    for (size_t i = inicio; i < a.size(); i += salto)
        for (size_t j = 0; j < a[i].size(); ++j) c[i][j] = a[i][j] + b[i][j];
}
Matriz suma_paralelo(const Matriz& a, const Matriz& b, int n_hilos) {
    Matriz c(a.size(), vector<int>(a.size()));
    vector<thread> hilos;
    for (int id = 0; id < n_hilos; ++id) hilos.emplace_back(suma_filas, cref(a), cref(b), ref(c), id, n_hilos);
    for (auto& h : hilos) h.join();
    return c;
}
// 3.3: el «director» reparte la siguiente fila a quien la pida. Su contador sí es compartido: mutex.
class Director {
public:
    explicit Director(size_t n_filas) : n_{n_filas} {}
    bool siguiente(size_t& fila) { lock_guard<mutex> lk{m_}; if (sig_ >= n_) return false; fila = sig_++; return true;
    ↪ }
private:
    mutex m_; size_t sig_ = 0, n_;
};
Matriz suma_pool(const Matriz& a, const Matriz& b, int n_hilos) {
    Matriz c(a.size(), vector<int>(a.size()));
    Director dir{a.size()};
    auto trabajador = [&] {
        size_t f;
        while (dir.siguiente(f))
            for (size_t j = 0; j < a[f].size(); ++j) c[f][j] = a[f][j] + b[f][j];
    };
    vector<thread> hilos;
    for (int i = 0; i < n_hilos; ++i) hilos.emplace_back(trabajador);
    for (auto& h : hilos) h.join();
    return c;
}

int main() {
    Matriz a = crear(4), b = crear(4);
    cout << "a + b con 3 hilos:\n" << suma_paralelo(a, b, 3);
    cout << "mismo resultado con thread pool: " << boolalpha << (suma_pool(a, b, 3) == suma_paralelo(a, b, 3)) << '\n';

    const int N = 1500;
    Matriz A = crear(N), B = crear(N);
    for (int nh : {1, 2, 4}) {
        auto t0 = chrono::steady_clock::now();
```

```
        Matriz C = suma_paralelo(A, B, nh);
        auto t1 = chrono::steady_clock::now();
        cout << nh << " hilos: " << chrono::duration_cast<chrono::microseconds>(t1 - t0).count() << " us (C[7][7]=" <<
        ↪  C[7][7] << ")\n";
    }
}
```

Salida de una ejecución (los tiempos cambian en cada máquina y ejecución):

OUT

```
a + b con 3 hilos:
0 2 4 6
0 2 4 6
0 2 4 6
0 2 4 6
mismo resultado con thread pool: true
1 hilos: 7713 us (C[7][7]=14)
2 hilos: 7088 us (C[7][7]=14)
4 hilos: 6739 us (C[7][7]=14)
```

Práctica 3.1: suma de matrices por filas con n hilos

Tamaño (tam×tam) 6 Hilos 3

Hilo	Filas que suma (inicial = id, incremento = n)	nº filas
0	0, 3	2
1	1, 4	2
2	2, 5	2

```
tiempo modelo = filas del hilo más cargado × tam = 2 × 6 = 12 sumas
tiempo con 1 hilo = 36 sumas
mejora teórica = 3,00×
```

```
vector<thread> v;
for (int id = 0; id < 3; ++id)
    v.emplace_back(suma_fila, ref(a), ref(b), ref(c), id, 3);   // fila inicial id, salto 3
for (auto& h : v) h.join();
```

Captura del simulador interactivo (disponible en la versión web de la guía).

Interpretación. La mejora real es **modesta**: sumar es muy barato y el tiempo lo domina el acceso a memoria y la reserva de la matriz resultado. Más hilos que núcleos no ayuda. Para un cálculo más costoso por elemento, la mejora sí se acerca al número de núcleos.

3.2 · La cena de los filósofos

Qué se pide. n filósofos, $n+1$ cubiertos (un `mutex` cada uno). Para comer, el filósofo i necesita **tres** cubiertos: i, $i+1$, $i+2$ (módulo $n+1$). Ciclo: coge cubiertos, come 10 ms, los suelta, piensa 10 ms. Usar `std::lock` o `scoped_lock`; probar después con `lock()` individuales y observar el interbloqueo.

Cómo se piensa. Con `lock()` uno a uno, cada filósofo se queda con su primer cubierto y espera el segundo, que tiene otro: **espera circular** ⇒ interbloqueo. `scoped_lock lk{c1, c2, c3}` los coge **todos a la vez** con un algoritmo seguro.

CPP

```
// Práctica 3.2: cena de los filósofos con 3 cubiertos por filósofo (n filósofos, n+1 cubiertos)
#include <chrono>
```

```
#include <iostream>
#include <mutex>
#include <thread>
#include <vector>
using namespace std;
using namespace std::chrono_literals;

mutex consola;
void imprimir(const string& s) { lock_guard<mutex> lk{consola}; cout << s << '\n'; }

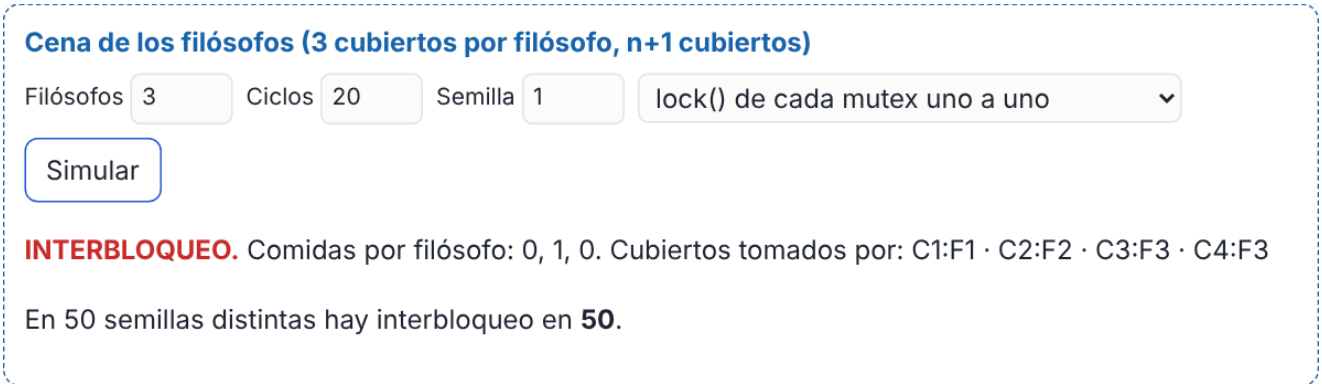
void filosofo(int id, mutex& c1, mutex& c2, mutex& c3, int ciclos) {
    for (int i = 0; i < ciclos; ++i) {
#ifdef INTERBLOQUEO
        c1.lock(); this_thread::sleep_for(1ms); c2.lock(); c3.lock();     // uno a uno: cada hilo retiene y espera
#else
        scoped_lock lk{c1, c2, c3};                                       // los tres a la vez, sin interbloqueo
#endif
        imprimir("F" + to_string(id) + " come");
        this_thread::sleep_for(10ms);
#ifdef INTERBLOQUEO
        c3.unlock(); c2.unlock(); c1.unlock();
#endif
        imprimir("F" + to_string(id) + " piensa");
        this_thread::sleep_for(10ms);
    }
}

int main(int argc, char* argv[]) {
    int n = argc > 1 ? stoi(argv[1]) : 3, ciclos = argc > 2 ? stoi(argv[2]) : 2;
    vector<mutex> cubiertos(n + 1);                     // un mutex por cubierto
    vector<thread> hilos;
    for (int i = 0; i < n; ++i)                          // F_i usa los cubiertos i, i+1, i+2 (módulo n+1)
        hilos.emplace_back(filosofo, i + 1, ref(cubiertos[i]), ref(cubiertos[(i + 1) % (n + 1)]), ref(cubiertos[(i +
        ↪  2) % (n + 1)]), ciclos);
    for (auto& h : hilos) h.join();
    imprimir("todos han terminado");
}
```

Con `-DINTERBLOQUEO` (cada `lock()` por separado y una pausa de 1 ms entre ellos) el programa se bloquea: en cinco ejecuciones de prueba con 3 filósofos y 50 ciclos se cortó por tiempo en las cinco. Sin esa definición, termina siempre:

OUT

```
F1 come
F2 piensa
...
todos han terminado
```



Captura del simulador interactivo (disponible en la versión web de la guía).

3.4 · Almacén (monitor)

Qué se pide. Un generador que escribe los valores 0 a 6 y varios lectores. Cada lector debe leer cada valor **una vez** y el generador **no puede escribir** el siguiente hasta que **todos** hayan leído el actual. Clase `Almacen` como monitor, con cierre ordenado.

Cómo se piensa. Estado: el valor y un `map<thread::id,bool>` que dice quién ya lo leyó. Dos condiciones de espera, ambas sobre el mismo `mutex`:

- El **generador** espera hasta que `todos_han_leido()`.
- Un **lector** espera hasta que él **no** ha leído el valor actual.

Cada cambio de estado (`anyadir`, `leer`) hace `notify_all`: así se despierta quien corresponda y los demás recomprueban su predicado. `cerrar()` pone una bandera y despierta a todos para que ninguno se quede dormido para siempre.

CPP

```cpp
// Práctica 3.4: Almacen (monitor). Un generador y varios lectores: cada lector lee cada valor UNA vez,
// y el generador no escribe el siguiente hasta que todos los lectores han leído el actual.
#include <atomic>
#include <chrono>
#include <condition_variable>
#include <iostream>
#include <map>
#include <mutex>
#include <thread>
#include <vector>
using namespace std;
using namespace std::chrono_literals;

class Almacen {
public:
    void anyadir_lector(thread::id id) { lock_guard<mutex> lk{m_}; leido_[id] = true; }     // empieza «sin pendiente»
    void anyadir(int v) {
        unique_lock<mutex> lk{m_};
        cv_.wait(lk, [&] { return todos_han_leido() || cerrado_; });                        // espera si alguien no
         ↪  ha leído
        if (cerrado_) return;
        valor_ = v;
        for (auto& [id, ok] : leido_) ok = false;                                           // ahora todos tienen
         ↪  pendiente
        cv_.notify_all();
    }
    bool leer(int& v) {                                                                     // false = almacén cerrado
        unique_lock<mutex> lk{m_};
        auto yo = this_thread::get_id();
        cv_.wait(lk, [&] { return !leido_[yo] || cerrado_; });                              // espera un valor nuevo
        if (leido_[yo]) return false;                                                       // cerrado y sin pendiente
        v = valor_; leido_[yo] = true;
        cv_.notify_all();                                                                   // quizá era el último
         ↪  lector
        return true;
    }
    void cerrar() { { lock_guard<mutex> lk{m_}; cerrado_ = true; } cv_.notify_all(); }
private:
    bool todos_han_leido() const { for (const auto& [id, ok] : leido_) if (!ok) return false; return true; }
    mutex m_;
    condition_variable cv_;
    map<thread::id, bool> leido_;
    int valor_ = 0;
    bool cerrado_ = false;
};

int main() {
    Almacen a;
    atomic<bool> seguir{true};
    vector<long> suma(3, 0), cuenta(3, 0);
    vector<thread> lectores;
    for (int i = 0; i < 3; ++i)
        lectores.emplace_back([&, i] {
            a.anyadir_lector(this_thread::get_id());
            int v;
            while (seguir && a.leer(v)) { suma[i] += v; ++cuenta[i]; this_thread::sleep_for(chrono::milliseconds(2 + 3
             ↪  * i)); }
        });
    this_thread::sleep_for(50ms);                                   // deja que se registren los lectores
    thread generador{[&] { for (int v = 0; v <= 6; ++v) a.anyadir(v); }};
    generador.join();
    this_thread::sleep_for(60ms);                                   // los lectores terminan de leer el último valor
    seguir = false; a.cerrar();
    for (auto& h : lectores) h.join();
```

```
    for (int i = 0; i < 3; ++i) cout << "lector " << i << ": " << cuenta[i] << " valores, suma " << suma[i] << '\n';
    cout << "(cada lector debe leer 7 valores y sumar 21)\n";
}
```

Salida

```
lector 0: 7 valores, suma 21
lector 1: 7 valores, suma 21
lector 2: 7 valores, suma 21
(cada lector debe leer 7 valores y sumar 21)
```

Práctica 3.4: almacén con un generador y varios lectores (monitor)

Lectores: 5 lectores semilla: 1

✔ Cada lector leyó los 7 valores **una vez y en orden**; el generador nunca escribió antes de que todos leyeran.

Suma que ve cada lector: **21, 21, 21, 21, 21** (esperado 21).

Paso	Hilo	Acción
1	**Generador**	escribe 0
2	Lector 0	lee 0
3	Lector 3	lee 0
4	Lector 4	lee 0
5	Lector 2	lee 0
6	Lector 1	lee 0

El orden de los lectores cambia con la semilla; el resultado, no.

Captura del simulador interactivo (disponible en la versión web de la guía).

Interpretación. Cada lector lee exactamente 7 valores y suma 21 (0+1+…+6), sea cual sea la velocidad de cada uno: la sincronización fuerza el ritmo del más lento. Es el esquema de «productor con varios consumidores que necesitan todos el mismo dato».

Resumen de prácticas

- P1: `++hist[p]`, `find` para consultar, `sort` con lambda para ordenar por valor.
- P2: constructor delegado, `operator<<` con `to_string()` virtual, destructor virtual, contenedor recursivo con punteros.
- P3: filas disjuntas ⇒ sin mutex; contador compartido ⇒ mutex; varios recursos ⇒ `scoped_lock`; esperar condición ⇒ `condition_variable` con predicado.

12 Exámenes resueltos

12.1 Empieza por aquí

Los más recientes y los que mejor reflejan el examen actual (POO 70 %, concurrencia 30 %). Empieza por el primero y ve hacia atrás.

Mayo 2026

- Mayo 2026 · Ejercicio 1: suscripciones de una plataforma (POO) — *Herencia y polimorfismo*
- Mayo 2026 · Ejercicio 2: robots y cintas transportadoras (concurrencia) — *Hilos y exclusión mutua*

Junio 2026

- Junio 2026 · Ejercicio 1: central de eventos (POO) — *Herencia y polimorfismo*
- Junio 2026 · Ejercicio 2: barreras (monitores) — *Monitores y variables condición*

Mayo 2025

- Mayo 2025 · Ejercicio 1: paquetería (POO) — *Herencia y polimorfismo*
- Mayo 2025 · Ejercicio 2: puente de vía única (monitor) — *Monitores y variables condición*

Junio 2025

- Junio 2025 · Ejercicio 1: nóminas de empleados (POO) — *Herencia y polimorfismo*
- Junio 2025 · Ejercicio 2: fábrica de patinetes (monitor) — *Monitores y variables condición*

12.2 Practicar por tema

Ejercicios resueltos paso a paso, agrupados por tema. Los exámenes sin resolución completa están en la lista de enunciados.

Herencia y polimorfismo

- Mayo 2026 · Ejercicio 1: suscripciones de una plataforma (POO) (Mayo 2026)
- Junio 2026 · Ejercicio 1: central de eventos (POO) (Junio 2026)
- Mayo 2025 · Ejercicio 1: paquetería (POO) (Mayo 2025)
- Junio 2025 · Ejercicio 1: nóminas de empleados (POO) (Junio 2025)

Hilos y exclusión mutua

- Mayo 2026 · Ejercicio 2: robots y cintas transportadoras (concurrencia) (Mayo 2026)

Monitores y variables condición

- Junio 2026 · Ejercicio 2: barreras (monitores) (Junio 2026)
- Mayo 2025 · Ejercicio 2: puente de vía única (monitor) (Mayo 2025)
- Junio 2025 · Ejercicio 2: fábrica de patinetes (monitor) (Junio 2025)

Planificación de tiempo real

- Febrero 2017 · Planificación con techo de prioridad inmediato (Febrero 2017)
- Junio 2016 · Planificación con 4 tareas y dos recursos (Junio 2016)
- Septiembre 2016 · Planificación: un sistema que NO es planificable (Septiembre 2016)
- Septiembre 2016 · Ejecutivo cíclico: ciclo secundario y código (Septiembre 2016)

12.3 Buscar un examen: enunciados por curso

Todos los exámenes del material, agrupados por curso. Los marcados con «resuelto» tienen resolución paso a paso más abajo.

Curso 2025-26

Junio 2026 – 2 ejercicios (resuelto)

Examen "Programación de Sistemas en Tiempo Real" – Junio 2026. Dispone de 2h:30m. Si el programa no compila, la puntuación se divide por 2.

Resuelto en la guía: Junio 2026 · Ejercicio 1: central de eventos (POO) · Junio 2026 · Ejercicio 2: barreras (monitores)

Ejercicio 1 (Herencia y polimorfismo)

```
EJERCICIO 1: POO (70%)
Se desea implementar un sistema para gestionar los eventos de un sistema. Para ello, implemente la
siguiente funcionalidad:
C_RECIBIDOR (10%)
   •   Clase abstracta, sin implementación, con dos métodos virtuales abstractos:
          o recibir_valor(): recibe una cadena de texto y no retorna ningún valor.
          o get_nombre(): no recibe ningún parámetro y retorna una cadena de texto.
C_PANTALLA (20%): Hereda de C_RECIBIDOR
   •   Atributos: 'nombre' (string) y 'ancho' de la pantalla (entero).
   •   Constructores:
          o Constructor que inicializa ambos atributos.
          o Constructor delegado, que recibe el 'nombre' y asume el ancho de 'pantalla' es 80.
   •   Métodos:
          o Sobrescribir el método recibir_valor(), imprimiendo por consola la cadena de
             texto que recibe, ajustando la longitud de cada línea al valor del atributo 'ancho'.
          o Sobrescribir el método get_nombre(), retornando el atributo 'nombre'.
C_ALARMA (20%): Hereda de C_RECIBIDOR
   •   Atributos: 'nombre' de la alarma (string).
   •   Constructor: Recibe el 'nombre' de la alarma.
   •   Métodos:
          o Sobrescribir el método recibir_valor(), imprimiendo por consola la cadena de
             texto solo si contiene un valor numérico. Utilice el método int isdigit (char c),
             de la librería <cctype>, que retorna un valor distinto de cero si el carácter que se le
             pasa es un número.
          o Sobrescribir el método get_nombre(), retornando el atributo 'nombre'.
C_CENTRAL (50%):
   •   Atributos: 'nombre' de la central (string) y un mapa de temas a vector de punteros a
       C_Recibidor, llamado 'suscritos'.
   •   Constructor: recibe el 'nombre' de la central de eventos.
   •   Métodos:
          o add_tema(): recibe un string t, que representa el tema de un evento, y lo añade al
             mapa si no está ya añadido. Utilice suscritos[t] para inicializar el vector asociado.
          o suscribirse(): recibe un string, que representa el tema del evento, y un objeto de
             tipo C_Recibidor. El método añade el objeto al vector asociado al tema, pero solo si
             el tema ha sido previamente registrado. El método devuelve un booleano que indica si
             el objeto se pudo añadir.
          o suscribirse_a_todos(): recibe un objeto de tipo C_Recibidor. El método añade el
             objeto a los vectores asociados a todos los temas asociados al mapa.
          o enviar_valor(): recibe un string, el tema del evento, y otro string, el mensaje que se
             va a enviar. Este método invoca, a su vez, el método recibir_valor() de todos los
             objetos almacenados en el vector asociado al tema, pasándole el mensaje recibido. El
             método devuelve un booleano que indica si el tema existe.

   •   Operador <<: imprime el número de temas registrados en la central de eventos, así como el
       nombre de todos los objetos del tipo C_Recibidor asociados a cada tema. Los nombres que
       muestra el siguiente ejemplo son "|Pantalla| : De 15" y "|Alarma| : Beep". Un ejemplo:
Central 'Central eventos' con 3 temas:
        humedad:
                |Pantalla| : De 15,                        |Alarma| : Beep,
        presion:
                |Alarma| : Beep,
        temperatura:
                |Pantalla| : De 80,                        |Alarma| : Beep,
Añada todas las funciones que considere necesarias. Solo puede utilizar la funcionalidad
proporcionada por la librería estándar de C++. Haga también una función main() de ejemplo.
```

Ejercicio 2 (Monitores y variables condición)

```
EJERCICIO 2: CONCURRENCIA (30%)

Haga un programa en C++ que implemente las siguientes clases de utilidad concurrente y haga una
función main() para mostrar un ejemplo de uso. Añada todas las funciones que considere necesarias.
Solo puede utilizar la funcionalidad proporcionada por la librería estándar de C++.
```

```
Diseñe una clase Barrera, que se comporte como un monitor, con dos métodos públicos: void
entrar() y void abrir(). La clase implementa esta funcionalidad: los hilos invocan el método
entrar() y quedan bloqueados, hasta que otro hilo “abre” la barrera invocando el método abrir().
A partir de ese momento, todos los hilos se van desbloqueando y van saliendo del método entrar().
Una vez la barrera está abierta, ningún hilo, ni siquiera los que invoquen entrar() tras la apertura,
deberán quedar bloqueados.
Diseñe una clase Barrera_Num, que se comporte como un monitor, que tiene un constructor que
recibe el número máximo de hilos (max_hilos) que pueden quedar bloqueados y tiene un método
void entrar(). El comportamiento en este caso es que, hasta que max_hilos no hayan invocado
el método entrar(), todos ellos deben quedar bloqueados. Cuando hay max_hilos,
automáticamente se abre la barrera y se van desbloqueando todos los hilos que estaban esperando.
Una vez la barrera está abierta, ningún hilo, ni siquiera los que invoquen entrar() tras la apertura,
deberán quedar bloqueados.
Haga una función main() para mostrar un ejemplo de uso de ambas clases. La salida por consola
debe estar protegida frente acceso concurrente. Como pista, coja el mutex en el constructor de las
clases, y no se olvide de que los hilos lo vayan cogiendo y soltando.
Una posible salida por consola podría ser:
 Hilo Barrera #1 entrando                          Hilo Barrera_Num #1 entrando
 Hilo Barrera #2 entrando                          Hilo Barrera_Num #2 entrando
 Hilo Barrera #4 entrando                          Hilo Barrera_Num #3 entrando
 Hilo Barrera #5 entrando                          Hilo Barrera_Num #3 saliendo
 Hilo Barrera #3 entrando                          Hilo Barrera_Num #1 saliendo
 Hilo Barrera #1 saliendo                          Hilo Barrera_Num #2 saliendo
 Hilo Barrera #2 saliendo                          Hilo Barrera_Num #4 entrando
 Hilo Barrera #5 saliendo                          Hilo Barrera_Num #4 saliendo
 Hilo Barrera #3 saliendo                          Hilo Barrera_Num #5 entrando
 Hilo Barrera #4 saliendo                          Hilo Barrera_Num #5 saliendo
```

Mayo 2026 – 2 ejercicios (resuelto)

Examen “Programación de Sistemas en Tiempo Real” – Mayo 2026. Dispone de 2h:30m. Si el programa no compila, la puntuación se divide por 2.

Resuelto en la guía: Mayo 2026 · Ejercicio 1: suscripciones de una plataforma (POO) · Mayo 2026 · Ejercicio 2: robots y cintas transportadoras (concurrencia)

Ejercicio 1 (Herencia y polimorfismo)

```
EJERCICIO 1: POO (70%)
Se desea implementar un sistema para gestionar las suscripciones de una plataforma de contenidos
digitales. Para ello, debe diseñar e implementar las siguientes tres clases:
C_SUSCRIPCION (30%)
   •   Atributos: ID (entero), precio base mensual (real) y nombre del titular (string).
   •   Constructores:
          o Constructor que inicialice todos los atributos con valores suministrados.
          o Constructor delegado que reciba el ID y el nombre, inicializando el precio base a 0€.
   •   Métodos:
          o Métodos getter y setter para el precio base, método getter para el ID.
          o buscar_titular(): recibe una parte del texto del titular de la suscripción y devuelve
             true si el texto proporcionado está contenido en el nombre del titular.
          o factura_total(): método virtual que retorna el coste mensual de la suscripción.
   •   Operador <<: imprime los datos del objeto. Utilice un método virtual to_string() para facilitar
       la implementación en toda la jerarquía de herencia. Un ejemplo de salida por consola:
       Suscripcion #101 (15.5€), Titular: Juan Perez
C_SUSCRIPCION_PREMIUM (25%)
   •   Hereda de C_SUSCRIPCION.
   •   Atributos adicionales: Servicio de soporte (string) y suplemento premium (real).
   •   Constructores:
          o Constructor que inicialice todos los atributos.
          o Constructor delegado que reciba ID, titular y servicio de soporte, inicializando todos
             los precios a 0.
          o Constructor que reciba un objeto C_SUSCRIPCION (para copiar los valores de sus
             atributos) y el nombre del servicio de soporte, inicializando el suplemento a 0.
   •   Métodos:
          o Getter y setter para el suplemento premium.
          o Redefinir factura_total() para sumar el precio base y el suplemento premium.
          o Redefinir el método to_string(). Ejemplo: Suscripcion #202 (20€), Titular: Ana
               Lopez (Premium: Soporte 24h + 5.5€)
C_PLATAFORMA (45%)
   •   Atributos: Nombre de la plataforma (string) y un vector de punteros a C_SUSCRIPCION.
   •   Constructor: Recibe el nombre de la plataforma. El vector se inicializa vacío.
   •   Métodos:
          o contar_estandar() y contar_premium(): Devuelven el número de suscripciones de
             cada tipo (utilice typeid).
          o buscar_por_nombre(): tiene un parámetro, texto del nombre del titular. El método
             devuelve cuántas suscripciones tienen ese nombre de titular.
```

```
          o add_suscripcion(): recibe un objeto del tipo c_suscripción, y añade el puntero al
             vector solo si el ID no existe ya. Retorna un booleano.

           o   cancelar_suscripcion(): recibe un ID de suscripción. Busca, libera la memoria y
             elimina del vector la suscripción con dicho ID. Recuerde que necesita utilizar un
             iterador para borrar un elemento de un vector. El método retorna un booleano.
   •   Operador <<: Imprime el resumen de la plataforma y el listado de suscripciones. Un ejemplo:
StreamMaster: 3 usuarios
     Suscripcion #101 (15.5€), Titular: Juan Perez
     Suscripcion #102 (12€), Titular: Ana Garcia
     Suscripcion #202 (20€), Titular: Luis Lopez (Premium: Soporte 24h - España + 5.5€)
Añada todas las funciones que considere necesarias. Solo puede utilizar la funcionalidad
proporcionada por la librería estándar de C++. Haga también una función main() de ejemplo.
```

Ejercicio 2 (Hilos y exclusión mutua)

```
EJERCICIO 2: CONCURRENCIA (30%)

Haga un programa en C++ que implemente la siguiente simulación. Añada todas las funciones que
considere necesarias. Solo puede utilizar la funcionalidad proporcionada por la librería estándar de
C++. Haga una función main() para ejecutar el código.
Se pide simular el comportamiento de una fábrica de
ensamblaje de piezas, según muestra la figura. Hay tres
robots, que se modelan como hilos, que montan una
pieza a partir de dos partes. Los parámetros que recibe la
función del hilo se describen más abajo. Estas partes les
llegan a través de dos cintas transportadoras (CT en la
figura). Por ejemplo, la pieza que monta el robot 1 utiliza
una parte que llega por la CT1 y otra parte que llega por
la CT3. Antes de montar la pieza, para asegurar la
integridad física de los equipos, el robot tiene que
conseguir acceso exclusivo a las cintas que llevan las
partes que necesita. Este acceso se va a modelar
mediante mutex. Una vez que tiene asegurado el acceso exclusivo, el hilo (1) duerme un tiempo
(simula el ensamblaje; su valor es configurable mediante parámetro), (2) imprime un mensaje por
consola, (3) libera recursos, (4) duerme otro tiempo (simula preparación para ensamblar la siguiente
pieza). Los parámetros del hilo son: su nombre, el número de piezas que tiene que fabricar, referencias
a los mutex de los recursos compartidos (incluyendo uno para usar la consola), y el tiempo que debe
dormir (milisegundos para una simulación rápida). Un ejemplo de ejecución sería:

 [Robot 1] ha procesado pieza 0                    [Robot 3] ha procesado pieza 4
 [Robot 3] ha procesado pieza 0                    [Robot 3] ha terminado
 [Robot 1] ha procesado pieza 1                    [Robot 2] ha procesado pieza 3
 [Robot 3] ha procesado pieza 1                    [Robot 1] ha procesado pieza 2
 [Robot 2] ha procesado pieza 0                    [Robot 2] ha procesado pieza 4
 [Robot 3] ha procesado pieza 2                    [Robot 1] ha procesado pieza 3
 [Robot 2] ha procesado pieza 1                    [Robot 2] ha terminado
 [Robot 3] ha procesado pieza 3                    [Robot 1] ha procesado pieza 4
 [Robot 2] ha procesado pieza 2                    [Robot 1] ha terminado
```

Curso 2024-25

Junio 2025 – 2 ejercicios (resuelto)

Examen “Programación de Sistemas en Tiempo Real” – junio 2025. Dispone de 2h:30m

Resuelto en la guía: Junio 2025 · Ejercicio 1: nóminas de empleados (POO) · Junio 2025 · Ejercicio 2: fábrica de patinetes (monitor)

Ejercicio 1 (Herencia y polimorfismo)

```
EJERCICIO 1: POO (70%)

Se pide que implemente un programa para gestionar las nóminas de los empleados de una empresa.
Para ello, se pide que diseñe las siguientes cuatro clases (la primera es abstracta y las dos siguientes
guardan una relación de herencia con la primera):
C_EMPLEADO (20%)
   •   Esta clase es la base de la jerarquía de herencia y se va a crear como una clase abstracta.
   •   Tiene como atributo el nombre y la dirección del empleado (ambos de tipo string).
   •   Tiene un constructor que recibe valores para inicializar todos los atributos de la clase, y un
       constructor delegado que recibe el nombre del empleado (la dirección se inicializa con “”).
   •   Tiene un método para obtener el nombre. Tiene dos métodos para obtener y cambiar la
       dirección del empleado.
   •   Tiene un método puro virtual get_sueldo(), que en esta clase no hace nada.
   •   Tiene el operator<< para imprimir en un flujo los atributos del objeto. Debe implementarse
```

```
       de forma que las clases hijas puedan también utilizarlo.

C_EMPLEADO_SUELDO_FIJO (15%)
   •   Esta clase hereda de C_Empleado.
   •   Añade como atributo el sueldo fijo del empleado (double).
   •   Tiene un constructor que recibe valores para inicializar todos los atributos de la clase, y un
       constructor delegado que recibe únicamente el nombre del empleado (el resto de atributos se
       inicializan con valores por defecto: “” y 0).
   •   Tiene un método para cambiar el valor del sueldo fijo.
   •   Sobrescribe el método virtual get_sueldo(), que retorna el valor del sueldo fijo del empleado.
   •   El operator<< imprime en un flujo los atributos del objeto. Por ejemplo:

C_EMPLEADO_SUELDO_VARIABLE (20%)
   •   Hereda de la clase C_Empleado.
   •   Añade como atributos el sueldo base y el complemento (ambos valores reales).
   •   Tiene un constructor que recibe valores para inicializar todos los atributos de la clase, y un
       constructor delegado que recibe únicamente el nombre del empleado (el resto de atributos se
       inicializan con valores por defecto: “” y 0).
   •   Tiene dos métodos para cambiar el valor del sueldo base y el complemento.
   •   Sobrescribe el método virtual get_sueldo(), que retorna el sueldo base más el complemento.
   •   El operator<< imprime en un flujo los atributos del objeto como en el caso anterior.

C_EMPRESA (45%)
   •   Tiene los siguientes atributos: nombre de la empresa (de tipo string) y vector de punteros a
       C_Empleado.
   •   Tiene un constructor, al que se le pasa el nombre de la empresa. El vector se inicializa por
       defecto.

   •   Tiene un método para obtener el número total de empleados con sueldo fijo, el número total de
       empleados con sueldo variables (ambos tipos están mezclados en el vector), y un tercer método
       para obtener un puntero al empleado con el mayor sueldo.
   •   Tiene un método add_empleado(), que recibe un empleado (de cualquiera de los dos tipos) y
       lo añade al vector.
   •   Tiene un método borrar_empleado(), que recibe el nombre de un empleado y lo elimina del
       vector. El método retorna un valor booleano para indicar si ha podido realizar esta acción.
   •   Tiene el operator<< para imprimir en un flujo los atributos del objeto. Por ejemplo:
               Empresa: UPCT, lista de empleados:
Añada todas las funciones que considere necesarias. Solo puede utilizar la funcionalidad
proporcionada por la librería estándar de C++. Haga también una función ‘main’ en la que muestre
cómo utilizar las clases diseñadas.
```

Ejercicio 2 (Monitores y variables condición)

```
EJERCICIO 2: CONCURRENCIA (30%)

Se pide desarrollar un programa que simula una fábrica de patinetes eléctricos. Para ello, programe tres
tipos de hilos: uno fabrica los cuerpos del patinete (tarda 500ms por cuerpo), otro fabrica las ruedas
(tarda 200ms por rueda) y otro termina la construcción del patinete ensamblando un cuerpo con dos
ruedas (tarda 300ms por patinete). Diseñe también una clase monitor C_Almacen, que esté libre de
condiciones de carrera y pueda utilizarse en una aplicación concurrente, y que almacene los dos tipos de
pieza que se requieren para fabricar los patinetes. Esta clase tiene un vector para almacenar cuerpos de
patinete y otro para las ruedas (para no complicar la implementación, estas partes se van a implementar
mediante strings). Añada los mutex y variables de condición necesarias para asegurar la exclusión mutua
en el acceso a los vectores y para evitar que se saquen piezas de un vector que está vacío.
Haga una función main() en la que se creen 2 instancias de cada hilo. El programa tiene que terminar
limpiamente (es decir, los 6 hilos creados tienen que terminar de forma normal, llegando al final de su
función) 2 segundos después de haber comenzado. Utilice una variable booleana en C_Almacen para
controlar el momento en que tienen que terminar todos los hilos. Una posible ejecución:

 [P1] rueda 1                                          [P1] rueda 4
 [P2] rueda 1                                          [P2] rueda 4
 [P1] rueda 2                                          [P1] cuerpo 2
 [P2] rueda 2                                          [P2] cuerpos termina
 [P1] cuerpo 1                                         [P1] cuerpos termina
 [P2] cuerpo 1                                         [P1] rueda 5
 [P1] rueda 3                                          [P1] ruedas termina
 [P2] rueda 3                                          [P2] rueda 5
 [E1] ensambla patinete 1 usando [P2]                  [P2] ruedas termina
    cuerpo 1, [P1] rueda 1, [P2] rueda 1               [E1] termina
 [E2] ensambla patinete 1 usando [P1]                  [E2] termina
    cuerpo 1, [P2] rueda 2, [P1] rueda 2               Simulación terminada.
```

Mayo 2025 – 2 ejercicios (resuelto)

Examen “Programación de Sistemas en Tiempo Real” – Mayo 2025. Dispone de 2h:30m

Resuelto en la guía: Mayo 2025 · Ejercicio 1: paquetería (POO) · Mayo 2025 · Ejercicio 2: puente de vía única (monitor)

Ejercicio 1 (Herencia y polimorfismo)

> EJERCICIO 1: POO (70%)
>
> Se pide que implemente un programa para gestionar una empresa de paquetería. Para ello, se pide que diseñe las siguientes tres clases (las dos primeras guardan una relación de herencia):
> C_ENVIO (30%)
> - Tiene los siguientes atributos: código (valor entero), precio (precio del servicio, real), y dirección completa del destinatario (string).
> - Tiene un constructor que inicializa todos los atributos mediante los valores suministrados al mismos. Tiene otro constructor, delegado, que recibe solo el código y el destinatario, inicializando el precio a 0€.
> - Tiene métodos para obtener y cambiar el valor del precio de un envío. Tiene un método para obtener el código de un envío.
> - Tiene un método buscar_destinatario(), que recibe como parámetro una parte de la dirección del destinatario y comprueba si, efectivamente, este valor está presente en la dirección del objeto.
> - Tiene un método virtual coste_total(), que retorna el coste del envío.
> - Tiene el operator<< para imprimir en un flujo los atributos del objeto. Por ejemplo:
>
> Envio #12345 (90€), destinatario: Mario, UPCT, 30202, Cartagena
>
> C_ ENVIO_INTERNACIONAL (25%)
> - Hereda de la clase C_Envio.
> - Añade como atributos el país de destino (de tipo string) y los costes de aduanas (valor real).
> - Tiene un constructor que inicializa todos los atributos mediante los valores suministrados al mismo. Tiene otro constructor, delegado, que recibe solo el código del envío, el destinatario y el país de destino, inicializando a 0 los valores del precio y el coste de la aduana. Tiene otro constructor que recibe un objeto del tipo C_Envio (para copiar sus valores) y el país de destino, y que inicializa a 0 el coste de las aduanas.
> - Tiene métodos para obtener y cambiar el coste de la aduana.
> - Actualiza el método polimórfico coste_total() para incluir también el coste de las aduanas.
> - Tiene el operator<< para imprimir en un flujo los atributos del objeto. Puede utilizar el texto que se genera para un C_Envio. Por ejemplo:
>
> Envio #54321 (50€), destinatario: Xi Ling, Gran Muralla (China + 15€)
>
> C_CENTRO_LOGISTICO (45%)
> - Tiene los siguientes atributos: ciudad en la que se encuentra el centro (de tipo string) y vector de punteros a C_Envio.
> - Tiene un constructor, al que se le pasa la ciudad. El vector se inicializa por defecto.
> - Tiene un método para obtener el número total de envíos normales, otro para obtener el número de envíos internacionales (están mezclados en el vector), y un tercero para obtener la cantidad de envíos cuya dirección de destinatario contiene el texto que se le pasa a este método como parámetro.
>
> - Tiene un método add_envio(), que recibe un envío (de cualquiera de los dos tipos) y lo añade al vector solo si no existe ya un envío con el mismo código. El método retorna un valor booleano para indicar si ha podido realizar esta acción.
> - Tiene un método borrar_envio(), que recibe el código del envío (de cualquiera de los dos tipos) que debe eliminar del vector. El método retorna un valor booleano para indicar si ha podido realizar esta acción.
> - Tiene el operator<< para imprimir en un flujo los atributos del objeto. Por ejemplo:
>
> Madrid: 3 envios
> Envio #12345 (90€), destinatario: Mario, UPCT, 30202, Cartagena
> Envio #12345 (90€), destinatario: María, UPCT, 30202, Cartagena
> Envio #54321 (50€), destinatario: Xi Ling, Gran Muralla (China + 15€)
>
> Añada todas las funciones que considere necesarias. Solo puede utilizar la funcionalidad proporcionada por la librería estándar de C++. Haga también una función ‘main’ en la que muestre cómo utilizar las clases diseñadas.

Ejercicio 2 (Monitores y variables condición)

> EJERCICIO 2: CONCURRENCIA (30%)
>
> Se pide diseñar una clase monitor C_Puente para el siguiente problema, junto con un ejemplo de uso. La implementación de la clase tiene que estar libre de condiciones de carrera y ser segura para su utilización en la siguiente aplicación concurrente. En una región montañosa, existe un puente de vía única que conecta el norte con el sur. Por razones de seguridad, solo un tren puede cruzar el puente a la vez, y se debe evitar que haya un desequilibrio excesivo en el flujo de trenes desde una sola dirección. Para ello, la clase C_Puente implementa el siguiente comportamiento: un tren cruza solo si no hay otro cruzando y si la cantidad de trenes esperando del sentido opuesto no supera las 3 unidades. La clase C_Puente:
> - Tiene como atributos un mutex y una variable de condición, contadores de trenes esperando a cruzar en cada sentido y sentido en el que está actualmente cruzando el tren (si es que algún tren

```
        está cruzando el puente).
    • Método que un tren invoca para solicitar cruzar el puente de norte_a_sur() y de sur_a_norte().
        Estos métodos reciben como parámetros un identificador de hilo y un valor que indica el tiempo
        en milisegundos que tarda el tren en cruzar, e implementan la sincronización requerida para
        asegurar la condición de paso por el puente (solo 1 tren a la vez) y de prioridad (si hay más de 3
        trenes esperando a cruzar en el otro sentido, se les tiene que dar prioridad). Los métodos también
        se encargan de la impresión por consola y de esperar el tiempo que tarda el tren en cruzar.
Haga una función main() en la que se creen 10 trenes en cada sentido. Cada tren será un hilo, que recibirá
como parámetros un objeto del tipo C_Puente, un número de identificación y el tiempo que tarda en
cruzar el puente. A continuación, se muestra un ejemplo de salida por consola con 2 trenes en cada
sentido (leyenda: o llegada al puente, | cruzando puente, x salida del puente):

 Tren [NS1] para cruzar: o                              Tren [NS2] cruzando:           |
 Tren [NS1] cruzando:    |                              Tren [NS2] ha cruzado:         x
 Tren [NS2] para cruzar: o                              Tren [SN2] cruzando:           |
 Tren [SN2] para cruzar: o                              Tren [SN2] ha cruzado:         x
 Tren [SN1] para cruzar: o                              Tren [SN1] cruzando:           |
 Tren [NS1] ha cruzado: x                               Tren [SN1] ha cruzado:         x
```

Curso 2023-24

Abril 2024 – 2 ejercicios

Examen “Programación de Sistemas en Tiempo Real” – Abril 2024. Dispone de 2h:30m

Ejercicio 1 (Clases y objetos)

```
EJERCICIO 1: POO (70%)

Se pide que programe una aplicación para gestionar sistema de control de acceso a una empresa. Para
ello, desarrolle las siguientes clases:

C_Entrada_Salida (35%)

  • Tiene como atributos el código de empleado (número entero), la fecha (string, suponga que en
      formato “dd/mm/aaaa”), un vector con las horas de entrada y salida del empleado (vector del tipo
      std::chrono::time_point<system_clock>) y el máximo número de horas que puede trabajar el
      empleado en esa fecha (número entero).
  •   Tiene un constructor que recibe como parámetros el código de empleado, un string que
      representa la fecha y el número máximo de horas que puede trabajar el empleado en esa fecha.
      Tiene un segundo constructor delegado que asume que el máximo de horas trabajadas es 8.
  •   Tiene dos métodos que retornan la fecha y el código de empleado.
  •   Método constante horas_trabajadas(), que no tiene parámetros y retorna el número de horas que
      ha trabajado el empleado o 0 si no puede calcularlo. Para poder calcular el número de horas
      trabajadas es necesario que el número de valores almacenados en el vector sea par, ya que primero
      se almacena en el vector la hora de entrada y después la de salida. Recuerde que la resta de dos
      valores de tipo time_point retorna un valor de tipo duration, que este valor se tiene que convertir a
      horas mediante la función genérica duration_cast<>, y que el valor de un duration se obtiene
      utilizado el método count().
  •   Método add_entrada() que no recibe parámetros y retorna un valor booleano que indica si se ha
      podido añadir la nueva hora de entrada. Se puede añadir una nueva hora de entrada al vector si la
      hora almacenada anteriormente en el vector es de salida y si el trabajador no ha trabajado más del
      número máximo de horas que puede trabajar ese día. Si se puede añadir, la hora se toma como la
      hora actual (std::chrono::system_clock::now()).
  •   Método add_salida() que no recibe parámetros y retorna un valor booleano que indica si se ha
      podido añadir la nueva hora de salida. Se puede añadir una nueva hora de salida al vector si la hora
      almacenada anteriormente en el vector es de entrada (es decir, el vector almacena un número
      impar de valores). Si se puede añadir, la hora se toma como la hora actual
      (std::chrono::system_clock::now()).
  •   Define el operator<<() para imprimir en un flujo los atributos del objeto en el formato que estime
      oportuno, menos los valores del vector.

C_Maquina_Control_Horario (65%)

  •  Tiene como atributos la fecha actual (string, suponga que en formato “dd/mm/aaaa”) y un mapa
     de fechas (string) a un vector de objetos de tipo C_Entrada_Salida.
  • La clase no tiene constructor.
  • Método fijar_fecha () que recibe como valor la fecha que se va a configurar como fecha actual para
     realizar la gestión horaria y retorna un valor booleano. Si la fecha ya existe en el mapa (use método
     find()), este método retorna falso. Si no existe, se cambia la fecha actual y se retorna verdadero.
   • Método add_entrada() que recibe como valor el código de usuario y retorna un valor booleano
     que indica si se ha podido añadir la nueva hora de entrada. Este método busca el código de usuario

     en el mapa, según indica la fecha actual, e intenta añadir una nueva hora de entrada mediante el
     método add_entrada(), retornando después el código de retorno de este método. Si no se
     encontrara el código de empleado, añade un nuevo objeto al vector antes de invocar el método.
```

- Método add_salida() que recibe como valor el código de usuario y retorna un valor booleano que indica si se ha podido añadir la nueva hora de salida. Este método busca el código de usuario en el mapa, según indica la fecha actual, e intenta añadir una nueva hora de salida mediante el método add_salida(), retornando después el código de retorno de este método. Si no se encontrara el código de empleado, retorna falso.
- Método get_empleado() que recibe como valor el código de usuario y retorna un mapa de fecha a C_Entrada_Salida con todos los valores de dicho empleado. Si el código de empleado no existiera, retornará un mapa vacío. Para añadir el objeto de tipo C_Entrada_Salida al mapa de resultados utilice el método insert().

Ejercicio 2 (Monitores y variables condición)

EJERCICIO 2: CONCURRENCIA (30%)

Se pide diseñar una clase monitor C_Monitor para un esquema productor-consumidor. Tiene que estar libre de condiciones de carrera y ser seguro para su utilización en una aplicación concurrente. Esta clase:

- Tiene como atributos dos vectores para almacenar valores enteros, dos mutex (para protegerlos frente al acceso concurrente) y el tamaño máximo permitido para ambos vectores (que es el mismo).
- Un constructor que recibe la cantidad máxima de valores que pueden almacenar los vectores e inicializa al valor 0 todas las posiciones de los dos vectores.
- Métodos tam_v1() y tam_v2() que retornan la cantidad de valores que almacena el primer y el segundo vector, respectivamente.
- Métodos leer_v1(), leer_v2() y leer_v1y2(), que tienen como parámetro la posición del vector que se quiere leer (serán dos valores en el caso del tercer método) y retorna el valor almacenado en el vector de valores o -1 si la posición suministrada se encuentra fuera del rango del vector. En el caso del método leer_v1y2(), los dos valores se leen tras haber cogido los dos mutex. Este método retorna dos valores (utilice la clase Pair).
- Métodos escribir_v1(), escribir_v2() y escribir_v1y2(), que tienen como parámetros la posición y el valor que se quiere escribir en dicha posición y que retornan un valor booleano que indica si dicha posición existe en el vector (recuerde que el tamaño de los vectores está limitado) y si ha podido guardar el valor en ella. En el caso del método escribir_v1y2(), los dos valores se escriben tras haber cogido los dos mutex. Este método recibirá 4 valores.

La aplicación está formada por hilos del tipo productor y del tipo consumidor, que comparten un objeto del tipo C_Monitor. Las funciones que definen el comportamiento de los hilos reciben cuatro parámetros: un string que identifica al hilo en la salida por consola, un entero que representa la cantidad de milisegundos que debe dormir el hilo entre usos consecutivos del monitor, un número que indica la posición del vector sobre la que trabaja el hilo y el objeto monitor. Los hilos repiten el siguiente comportamiento 10 veces: almacenan/leen (puede utilizar cualquier versión leer_v1() o leer_v2(), escribir_v1() o escribir_v2()) un valor en la posición del vector indicada, lo muestran por consola y duermen el tiempo indicado hasta la siguiente ejecución. Haga una función main en la que se creen 5 hilos de cada tipo.

Mayo 2024 – 2 ejercicios

Examen “Programación de Sistemas en Tiempo Real” – Mayo 2024. Dispone de 2h:30m

Ejercicio 1 (Clases y objetos)

EJERCICIO 1: POO (70%)

Se pide que programe una aplicación para gestionar un sistema de almacenamiento y gestión de información de redes sociales. Para ello, desarrolle las siguientes clases:

C_Post (25%)

- Tiene como atributos la fecha del post (string), el nombre del usuario que lo escribió (string), el texto del post (string) y el número de ‘likes’ (entero).
- Tiene un constructor que recibe como parámetros valores para todos los atributos.
- Tiene un método que retorna el nombre de usuario y otro para el texto del post.
- Método constante encontrar(), que recibe una palabra clave y retorna un valor booleano que indica si ha encontrado la palabra clave en el texto del post.
- Define el operator<<() para imprimir en un flujo los atributos del objeto en el formato que estime.

Añada el siguiente código dentro de la clase:

```
friend bool operator< (const C_Post &r, const C_Post &l) { return r.nombre_usuario<l.nombre_usuario; }
```

C_Procesador_Posts (75%)

- Tiene como atributos un conjunto (set<>) de temáticas (string de una sola palabra), el nombre que se le va a dar a la categoría “otras” (string) y un mapa que asocia a cada temática un conjunto (set<> de nuevo) de Posts que contienen dicha temática en su texto.

- La clase tiene un constructor que recibe el conjunto de temáticas que van a ser gestionadas por el objeto y el nombre que se le va a dar a la categoría “otras”. Haga también un constructor delegado que asuma que el valor para el segundo parámetro es “otras”. El constructor tiene que inicializar correctamente el mapa.
- Método add_post () que recibe como valor un Post y lo añade al conjunto de posts del mapa, dependiendo de la temática (o palabra clave) que contenga. Es caso de que el post no contenga ninguna de las temáticas, se añadirá a la categoría “otras” (o al nombre que haya fijado el usuario en la construcción del objeto). Es posible añadir un post en más de una entrada del mapa si tiene la temática correspondiente. Es decir, esta clase funciona como un clasificador de posts.
- Método constante num_posts_tematicas() que no recibe parámetros y retorna, utilizando una estructura de tipo mapa, la cantidad de posts que se almacenan de cada palabra clave, incluyendo la categoría “otras”.
- Método constante total_posts () que no recibe parámetros y retorna la cantidad total de posts almacenados (ignorando el hecho de que puedan estar repetidos).
- Método posts_usuario_tematica() que recibe como valor la temática y retorna el conjunto de posts de dicho usuario. Si el usuario no tiene posts en dicha temática o la temática no existe o el mapa no tiene posts en la temática, se retornará el conjunto vacío.
- Método usuario_mas_posts_tematica() que recibe como valor la temática y retorna el nombre del usuario que tiene más posts en dicha temática. Si la temática no existe o el mapa no tiene posts en la temática, se retorna el string vacío.
- Define el operator<<() para imprimir en un flujo los atributos del objeto en el formato que estime.

Ejercicio 2 (Monitores y variables condición)

```
EJERCICIO 2: CONCURRENCIA (30%)

Se pide que diseñe el prototipo de un sistema de procesamiento de información con el siguiente
funcionamiento: un hilo inserta valores enteros en un vector (Repositorio1) cada 50ms y otro hilo procesa
ese vector cada segundo, calculando el valor medio de todos los números enteros almacenados en el
vector y guardando dicho valor medio en otro vector (Repositorio2). Este segundo hilo vacía el vector del
Repositorio1 tras procesar sus valores. Emplee espera absoluta en el comportamiento de los hilos o
espera relativa si no pudiera. En este prototipo, el hilo que inserta valores en el Repositorio1 puede
insertar siempre el mismo valor, el 7 por ejemplo.

 El Repositorio1 tiene un método para insertar valores en su vector y otro método para obtener la media
de los valores almacenados y vaciar el vector (utilice la clase plantilla optional<> para modelar el hecho
de que el vector pudiera estar vacío y, por tanto, no se podría retornar ningún valor). El Repositorio2 tiene
un método para añadir un valor a su vector y otro método para retornar el siguiente valor al último que
se haya leído del vector (utilice optional<> como valor de retorno para representar el hecho de que no
hay ningún valor nuevo).

El código desarrollado tiene que estar libre de condiciones de carrera y ser seguro para su utilización en
una aplicación concurrente. Diseñe Repositorio1 y Repositorio2 para que se comporten como clases de
tipo monitor. Haga una función main en C++ en la que se ejecute la aplicación.
```

Curso 2022-23

Junio 2023 – 2 ejercicios

Ejercicio 1 (Clases y objetos)

```
EJERCICIO 1: POO

Se pide que programe una aplicación para gestionar la toma de fotografías aéreas. Para ello se
pide que utilice el siguiente código y comience por la clase C_Imagen.
       struct RGB { // para modelar colores, valor entre 0 y 255 por campo
         int R, G, B;
       };
       ostream& operator<< (ostream& os, const RGB &c) {
         os << "("<<c.R<<", "<<c.G<<", "<<c.B<<")";
         return os;
       }
       enum Tipo_Terreno { tierra, agua, vegetal, urbano, mixto };

C_Imagen

       • Tiene como atributos las coordenadas GPS donde se tomó la imagen (tres valores de
         tipo double), el valor umbral (un entero) que se va a utilizar para establecer que un
         píxel es gris y una matriz (vector de vectores) que almacena valores del tipo RGB.
       • Tiene un constructor que recibe como parámetros las coordenadas GPS del punto en
         que se tomó la imagen y el valor que se va a utilizar como umbral de gris en la foto.
         Este último parámetro tiene como valor por defecto 20.
       • Método tam_imagen() que devuelve, como un pair<>, el número de filas y columnas
         que forman la imagen. Si la matriz está vacía se devuelve la pareja de valores -1,-1.
       • Método nueva_fila(), que tiene como parámetro un vector de valores de tipo RGB
```

(una fila de la imagen) y que retorna un valor de tipo bool que indica si la fila se ha podido añadir correctamente a la imagen. Esto es debido a que el sensor fotográfico que se utiliza captura únicamente una fila con cada foto que hace. La fila se puede añadir si la imagen está vacía o si no está vacía y la longitud del vector es la misma que la de las filas añadidas anteriormente.
- Método n_pixels_tipo(), que tiene como parámetro el tipo de terreno (del tipo enumerado Tipo_Terreno) del que se quiere contar cuántos píxeles hay en la matriz que almacena la información de la imagen y retorna dicha cantidad de píxeles. Un píxel se clasifica de tipo 'tierra' si la cantidad de color rojo (campo R en la estructura RGB) es superior a la suma de los otros dos colores. De forma similar, será 'agua' si B es superior a la suma de los otros dos colores y 'vegetal' si G es superior a la suma de los otros dos colores. Será 'urbano' si el valor absoluto de la diferencia entre cada dos colores es menor que el umbral establecido en la imagen, y 'mixto' en caso de que no pertenezca a ninguna de las categorías anteriores.
- Tiene el operator<< para imprimir en un flujo los atributos del objeto. Por ejemplo: "Imagen en 2, 3, 4; umbral 20" y posteriormente el contenido de la matriz.

Además de implementar la clase C_Imagen, se pide que programe una función fotos_max_tipo(), que recibe como parámetros un vector de objetos del tipo C_Imagen y un tipo de terreno, y

devuelva la posición dentro del vector de la imagen que tiene un mayor número de píxeles de ese tipo de terreno.

Haga una función main() en la que pruebe la clase desarrollada de la siguiente forma: cree un objeto, imprima su tamaño, añada una fila con dos valores (de tipo RGB), imprima la imagen, añada una nueva fila con dos valores (de tipo RGB), imprima la imagen, añada una fila con tres valores (de tipo RGB), imprima la imagen.

Ejercicio 2 (Monitores y variables condición)

EJERCICIO 2: CONCURRENCIA

Se pide diseñar una clase monitor C_Monitor para gestionar un esquema productor-consumidor doble. El diseño del monitor tiene que estar libre de condiciones de carrera y ser seguro para su utilización en una aplicación concurrente. Esta clase:

- Tiene como atributos un mutex y un vector de enteros.
- Método anyadir(), que tiene como parámetro el valor que se debe añadir al final del vector y no retorna nada.
- Método obtener(), sin parámetros, y que retorna el primer valor que almacena el vector de enteros. Puesto que es posible que el vector esté vacío, se va a utilizar como tipo de retorno el tipo parametrizado optional. Si el vector tiene algún valor que retornar, éste se borrará para no ser retornado de nuevo en la siguiente invocación.

La aplicación está formada por tres hilos y dos objetos del tipo C_Monitor:
- El primer hilo, "productor" en la salida de pantalla que se muestra más abajo, repite 10 veces lo siguiente: genera un valor, lo guarda en el primer monitor (utilizando el método anyadir())y se duerme un tiempo. Para simplificar, este hilo puede producir los números del 1 al 10.
- El segundo hilo, llamado "intermedio", repite 10 veces lo siguiente: obtiene un valor del primer monitor (utilizando el método obtener()), hace ciertas operaciones con él, guarda el nuevo valor en el segundo monitor y se duerme un tiempo. Para simplificar, la operación puede ser sencillamente multiplicar por 10. Tenga en cuenta que el resultado de obtener() puede ser un valor vacío. En ese caso, el hilo debe dormir 50ms y ese intento no debe contabilizar para las 10 repeticiones.
- El tercer hilo, llamado "consumidor", repite 10 veces lo siguiente: obtiene un valor del segundo monitor, va sumando los valores que recupera y se duerme un tiempo. Tenga en cuenta que el resultado de obtener() puede ser un valor vacío. En ese caso, el hilo debe dormir 50ms y ese intento no debe contabilizar para las 10 repeticiones. Una vez que el hilo termine, debe imprimir por consola la media de valor acumulado.

Las funciones de los hilos reciben como parámetros, además de los monitores necesarios, el tiempo que deben dormir al final de cada ciclo. Para facilitar el desarrollo del programa, se

recomienda imprimir por consola las acciones que va realizando cada hilo (teniendo en cuenta que la consola es un recurso compartido).

Haga una función main en la que se cree un hilo de cada tipo y se muestre el comportamiento del programa. A continuación se muestra una salida por consola (simplificada a 3 valores):

```
    [Consumidor] durmiendo ...
    [Intermedio] durmiendo ...
    [Consumidor] durmiendo ...
[Productor] genera 1
[Intermedio] obtiene 1 y produce 10
[Productor] genera 2
```

```
[Consumidor] obtiene 10
[Intermedio] obtiene 2 y produce 20
[Productor] genera 3
[Consumidor] obtiene 20
[Intermedio] obtiene 3 y produce 30
[Productor] ha terminado
[Consumidor] obtiene 30
[Intermedio] ha terminado
[Consumidor] la media es 20 y termina
```

Mayo 2023 – 6 ejercicios

Ejercicio 1 (Clases y objetos)

```
EJERCICIO 1: M-PBO

Una empresa guarda los datos de hora de entrada y salida de sus empleados de cada mes en un
fichero con el siguiente formato: <nombre>, E/S, <hora>. Los datos de cada trabajador están
separados por ';'. Por ejemplo, "Mario Pérez, E, 8" representa que el trabajador entró a las 8h, y
"Maria Lopez, S, 13" indica que salió a las 13h. Se quiere realizar un programa que almacene
todos los datos de entrada y salida de los trabajadores en una estructura de datos compuesta
(un mapa que almacena vectores), de forma que luego se pueda calcular el total de horas
trabajadas por mes. Se pide un prototipo suponiendo que los datos están en string msg {"Mario
Pérez, E, 8; Maria Lopez, E, 9; Mario Pérez, S, 15; Maria Lopez, S, 13; Mario Pérez, E, 8;Mario
Pérez, S, 15"}, que hay el mismo número de entradas que de salidas, que se alternan y que la
hora de salida siempre es posterior a la de entrada. Para estos datos de entrada, la salida sería:

       Mario Pérez ha trabajado 14 horas

       Maria Lopez ha trabajado 4 horas

Pista: puesto que se tienen que guardar dos datos por cada trabajador (tipo de marcaje (E/S) y
hora), se propone definir una estructura o tipo similar para hacerlo. Utilice la clase stringstream
para procesar los datos de entrada más fácilmente.
```

Ejercicio 2 (Clases y objetos)

```
EJERCICIO 2: M-POO

Se pide que programe una aplicación para gestionar las cuentas de los clientes en un banco. Para
ello se pide que comience por la clase C_Cuenta.
C_Cuenta

       • Tiene como atributos el nombre del titular, el número de cuenta y el saldo actual.
       • Tiene un constructor que recibe como parámetros el nombre del titular y el número
         de cuenta. El saldo de una cuenta recién creada se pone a 0.
       • Tiene el operator<< para imprimir en un flujo los atributos del objeto. Por ejemplo:
         "Mario (ccc 12345678): 123.45 euros".
       • Métodos para leer el número de cuenta y el saldo actual.
       • Método ingresar(), que tiene como parámetro la cuantía de dinero y retorna si el
         ingreso se ha podido realizar correctamente (valor de tipo bool). El ingreso se puede
         realizar si el valor del parámetro es positivo.
       • Método reintegrar(), que tiene como parámetro la cuantía de dinero y retorna si se
         ha podido retirar el dinero de la cuenta correctamente. El reintegro se puede realizar
         si el valor del parámetro es positivo y si el saldo final de la cuenta sigue siendo
         positivo.
       • Método transferencia(), que tiene como parámetros la cuantía de dinero y un objeto
         que representa la cuenta destino (que se pasa por referencia), y retorna si la

         transferencia se ha podido realizar correctamente. La transferencia se puede realizar
         si la cantidad monetaria es positiva y si el saldo de la cuenta origen no se queda en
         negativo.

Haga una función main() en la que pruebe la clase desarrollada, creando al menos dos objetos,
imprimiendo su contenido por consola, haciendo un ingreso en cada una de las cuentas y una
transferencia entre ellas.
```

Ejercicio 3 (Monitores y variables condición)

```
EJERCICIO 3: M-CONCURRENCIA

Se pide diseñar una clase monitor C_Monitor para gestionar las modificaciones concurrentes
sobre un vector de enteros.
```

- Tiene como atributos un mutex, el vector y un booleano que controla si los hilos que acceden al monitor deben continuar su ejecución.
- Tiene un constructor que inicializa el valor booleano a true y un método que retorna el valor de este booleano.
- Método anyadir(), que tiene como parámetro el valor que se debe añadir al final del vector y no retorna nada. Una vez que el vector tenga más de 10 elementos, el atributo booleano que controla la ejecución de los hilos debe ponerse a false.
- Método cambiar(), que tiene como parámetros una posición del vector y el valor que debe poner en dicha posición, y que no retorna nada. Solo si dicha posición existe debe ser cambiado el valor almacenado en ella.
- Método para retornar el vector con los valores almacenados en el monitor.

Defina las funciones de dos hilos, uno que utilizará el método cambiar() y otro que utilizará el método anyadir(). Mientras el vector del monitor no tenga 10 elementos, los hilos deberán utilizar el método asignado y dormir 100ms entre cada invocación. Los valores insertados en el vector y la posición utilizada (es decir, los valores suministrados a los métodos) pueden ser fijos. Haga una función main en la que se cree un hilo de cada tipo y se imprima finalmente el contenido del vector almacenado en el monitor.

Ejercicio 4 (Clases y objetos)

EJERCICIO 1: T-PBO

Una empresa de mediciones topográficas le ha pedido que realice un programa para procesar la información que recibe de su software de procesamiento de formas geométricas. Este programa retorna una cadena de texto con el siguiente formato: “<forma> : <valores>”, que debe procesar. Hay tres valores posibles para forma: T/R/C, triángulo, rectángulo, círculo. Después de la forma aparecen uno o dos valores, separados por ‘:’, los necesarios para poder estimar el área de la forma. Un ejemplo de esta cadena es string msg {"T : 1.1 : 8\n T : 2 : 2.8\n C : 1.1 \n R : 2.3 : 4.5\n C : 3.3"}. El carácter ‘\n’ se utiliza para separar las formas. En la librería <cmath> se define la constante M_PI, con el valor de la constante π. Se pide que guarde en vectores diferentes los valores de área para cada tipo de forma y que finalmente calcule y muestre por pantalla la suma total de las áreas de cada uno de los tipos de forma.

Pista: utilice la clase stringstream para procesar los datos de entrada más fácilmente.

Ejercicio 5 (Clases y objetos)

EJERCICIO 2: T-POO

Se pide que diseñe la clase C_Trabajador para almacenar los datos de los trabajadores de una empresa, de forma que a final de mes se pueda calcular su nómina.
C_Trabajador

- Tiene como atributos el nombre del trabajador, su salario base, salario por hora trabajada y un vector en el que se van guardando las horas que trabaja cada día. Los dos primeros atributos no pueden cambiarse una vez creado el objeto.
- Tiene un constructor que recibe valores para cada atributo.
- Tiene el operator<< para imprimir en un flujo los atributos del objeto. Por ejemplo: " Mario Pérez, base: 700 | por hora: 3, total horas: 0”.
- Un método para cambiar el valor del salario por hora. Debe comprobar que el nuevo salario es positivo antes de realizar el cambio y retorna un valor booleano que indica si ha podido hacerlo.
- Un método para anyadir_horas() para añadir al vector las horas trabajadas en un día. Este método retorna true si el valor de horas es positivo y menor de 10, y si en total no se han añadido más de 30 entradas al vector.
- Un método para calcular la nómina del trabajador de la siguiente forma: salario base + horas trabajadas * salario por hora.

Haga una función main() en la que pruebe la clase desarrollada, creando al menos un objeto, imprimiendo su contenido por consola, añadiendo horas trabajadas e imprimiendo la nómina.

Ejercicio 6 (Monitores y variables condición)

EJERCICIO 3: T-CONCURRENCIA

Se pide diseñar una clase monitor C_Repositorio para gestionar un esquema concurrente productor/consumidor. El programa tendrá un hilo que genera valores enteros positivos que se almacenan en un vector dentro de la clase, y dos hilos consumidores que sacarán dichos valores en el mismo orden en que se han insertado. Puesto que los hilos llevan sus propias velocidades, la clase monitor tiene también un mapa para recordar el valor del último índice del vector que leyó cada hilo consumidor. Para identificar un hilo consumidor y poder saber por dónde se ha

quedado, utilice el tipo de dato thread::id y el método thread::id this_thread::get_id().

- Tiene como atributos un mutex, el vector de enteros donde se almacenan los valores y el mapa que lleva la cuenta de la posición de lectura en que se ha quedado cada hilo consumidor.
- Método guardar_valor(), que tiene como parámetro el valor que se debe añadir al final del vector y no retorna nada.
- Método recuperar_valor(), que no tiene parámetros y retorna el siguiente valor que debe leer el hilo consumidor, según el orden en que el hilo productor ha insertado los valores en el vector. Si el hilo consumidor ya ha leído todos los valores disponibles, el método retorna un valor negativo. Por ejemplo, el vector puede almacenar 10 elementos, pero el hilo consumidor 1 puede ir por el índice 3 y el consumidor 2 por el índice 8. Si el hilo consumidor no estaba registrado en el mapa, se añade al registro y se retorna como resultado el primer valor que se almacenó en el vector.

Defina las funciones para los dos hilos. El hilo productor utilizará el método guardar_valor() de la clase C_Repositorio, mientras que los hilos consumidores utilizarán el método recuperar_valor(). Después de invocar el método correspondiente, cada hilo dormirá un tiempo especificado como parámetro de la función.
Para controlar la finalización del programa, puede declarar una variable booleana global, bool continuar = true, que utiliza la función del hilo consumidor para saber si debe terminar o no. El hilo consumidor la pondrá a false después de insertar, por ejemplo, 10 valores.

Curso 2021-22

Julio 2022 – 3 ejercicios

Ejercicio 1 (Herencia y polimorfismo)

EJERCICIO 1
Se pide que diseñe parte de la infraestructura de una aplicación de alquiler de habitaciones y emplazamientos partiendo del siguiente código:

```
struct Alojamiento{
    virtual double calcular_precio() const = 0;
    friend ostream& operator<< (ostream &os, const Alojamiento &a);
    Alojamiento (string direccion) : direccion_{direccion} {}
    string get_dir() const { return direccion_; }
protected :
    virtual void print(ostream &os) const = 0;
private:
    string direccion_;
};
ostream& operator<< (ostream &os, const Alojamiento &a) {
    a.print(os);
    return os;
};
```

Se pide que implemente las siguientes clases, que comparten una jerarquía de herencia:
Hotel

- Hereda de Alojamiento.
- Tiene como atributos el número de estrellas, el precio por noche (se supone que es siempre el mismo), número máximo de huéspedes que se pueden aceptar por reserva, y el mínimo y máximo de días que se puede realizar una reserva.
- Tiene un constructor que recibe como parámetros valores para inicializar todos los atributos, y otro constructor (delegado), que asume que el valor mínimo de días es 1 y el máximo 7.
- Implementa los dos métodos virtuales de Alojamiento. Para calcular el precio, el método tiene que preguntar el número de huéspedes y el número de días que se quedarán, asumiendo que las habitaciones son individuales. Si algún valor es negativo o está fuera del rango que establecen los atributos (número máximo de huéspedes que se pueden aceptar por reserva, y el mínimo y máximo de días que se puede realizar una reserva), el método retorna el código de error -1. Una posible implementación del método print podría generar: “Hotel en Cartagena de 4 estrellas, precio por noche: 85”.

Vivienda

- Hereda de Alojamiento.
- Tiene como atributos el número mínimo de días que se puede alquilar (no hay máximo), la cantidad de huéspedes que caben en una vivienda (se supone que son todas iguales) y el precio de alquiler de la vivienda por día.
- Tiene un constructor que recibe como parámetros valores para inicializar todos los atributos.
- Implementa los dos métodos virtuales de Alojamiento. Para calcular el precio, el método tiene que preguntar el número de huéspedes y el número de días que se

quedarán. A partir del número de huéspedes y el valor del atributo cantidad-de-huéspedes-que-caben-en-una-vivienda el método calcula la cantidad de viviendas que hay que reservar. Si algún valor es negativo o está fuera del rango que establecen los atributos, el método retorna el código de error -1. Una posible implementación del método print podría generar: “Vivienda en La Manga para 5 personas, precio por dia: 150”.

Castillo

- Hereda de Alojamiento.
- Tiene como atributos el número máximo de días que se puede alquilar (el mínimo es 1), el precio de alquiler del castillo, y un mapa de string a double que establece el precio de cada estancia del castillo que se quiera alquilar (van por separado del alquiler del propio castillo).
- Tiene un constructor que recibe como parámetros valores para inicializar todos los atributos.
- Implementa los dos métodos virtuales de Alojamiento. Para calcular el precio, el método tiene que preguntar si se quieren alquilar también las estancias especiales del castillo y el número de días que se alquilarán. Un ejemplo de definición de estas estancias es {{"Trono", 1000}, {"Salon Banquetes", 2000}, {"Habitaciones reales", 3000}}. Si algún valor es negativo o está fuera del rango que establecen los atributos, el método retorna el código de error -1. Una posible implementación del método print podría generar: “Castillo en Madrid con 3 estancias, precio minimo por hacer reserva: 4000”.

Puede ampliar el número de funciones y métodos como estime necesario. Escriba una función main de prueba corta para comprobar el funcionamiento de las clases desarrolladas y que el polimorfismo funciona adecuadamente. Para ello, cree un objeto de cada clase, lo asigna a un puntero de tipo Alojamiento y luego lo imprime por consola a través del puntero. Solo puede utilizar la funcionalidad que proporciona la librería estándar de C++.

Ejercicio 2 (Clases y objetos)

EJERCICIO 2

Se pide programar la infraestructura básica para gestionar los datos recogidos por sensores en una planta industrial. Para ello, se pide implementar la clase Centro_Datos que contenga los atributos necesarios para cumplir los siguientes requisitos:

1. Se deben poder almacenar todos los valores, de tipo double, enviados por los sensores instalados en la planta. Es decir, se guarda el histórico de valores, no únicamente el último que se ha recibido. Cada sensor está identificado por un string, que no contiene espacios en blanco.
2. Para evitar problemas de sensores con códigos erróneos, la clase debe programarse para aceptar solo los códigos de los sensores que se suministren al construir el objeto. De esta forma, si llega un valor de un sensor cuyo código de identificación no se ha suministrado al construir el objeto, debe ser ignorado.

En cuanto a la funcionalidad de la clase, se piden los siguientes métodos:

- Constructor que recibe una lista de códigos de identificación de sensores, para implementar el segundo requisito. En una posible implementación, puede inicializar el vector de valores asociado al código del sensor invocando, por ejemplo, el método reserve().
- Método actualizar_datos(), que recibe como parámetro un string con los datos a procesar y no retorna nada. Otra parte del código, que no se pide en este ejercicio, es la encargada de recibir los datos de los sensores por un canal de comunicación, crear un string con todos ellos, e invocar este método para añadir los nuevos datos a Centro_Datos. La estructura básica del string es “código-sensor valor-double”, repetida tantas veces como haga falta. Es decir, la cantidad de valores que contiene el parámetro de este método es variable. Un ejemplo puede ser: string datos_2 {"s1 11 s2 11 s3 1 s2 21 s1 21 s1 31"}. La implementación de este método tiene que tener en cuenta el segundo requisito.
- Método datos_sensor(), que recibe como parámetro el código de identificación de un sensor y retorna todos los valores que se han almacenado para dicho sensor. Si el código no estaba registrado, retorna un vector vacío.
- Método últimos_valores(), que retorna el último valor que se ha almacenado para cada sensor. Tenga cuidado con los sensores registrados, pero para los que todavía no se ha recibido ningún valor.
- El operator<< para imprimir el contenido de Centro_Datos por consola. Por ejemplo, el siguiente código mostraría la salida por consola que aparece en el cajetín (es un ejemplo de código, la implementación de su clase no tiene que coincidir al 100%):

```
Centro_Datos cd {{"s1", "s3"}};
string datos_1 {"s1 1 s2 1 s2 2 s1 2 s1 3"};
cd.actualizar_datos(datos_1);
cout << cd << '\n7;
string datos_2 {"s1 11 s2 11 s3 1 s2 21 s1 21 s1 31"};
```

```
                cd.actualizar_datos(datos_2);
                cout << cd << '\n7;
          s1 -> {1, 2, 3, }
          s3 -> {}

          s1 -> {1, 2, 3, 11, 21, 31, }
          s3 -> {1, }

Escriba una función main de prueba corta para comprobar el funcionamiento de la clase
desarrollada. Solo puede utilizar la funcionalidad que proporciona la librería estándar de C++.
```

Ejercicio 3 (Monitores y variables condición)

```
EJERCICIO 3: CONCURRENCIA

Se quiere realizar un simulador de un cruce a nivel, que se va a implementar mediante una clase
que se comporta como un monitor, Cruce_Nivel. El paso a nivel es cruzado por coches y trenes,
que se van a implementar como hilos en el simulador. El comportamiento es el siguiente.
Mientras haya un coche o un tren cruzando el paso a nivel, es posible que otro coche o tren cruce
el paso a nivel sin bloquear. Si un coche o tren intenta cruzar cuando un tren o un coche está
cruzando, debe bloquear hasta que el cruce a nivel no esté siendo cruzado por ningún vehículo.
En el simulador, el coche tarda 150 milisegundos en atravesar el paso a nivel, mientras que un
tren tarda 300ms. Los trenes utilizan el método cruza_tren() del monitor, y los coches usan el
método cruza_coche() para interactuar con el cruce a nivel. Cada vez que un vehículo (coche o
tren) cruza el paso a nivel, imprime un mensaje por consola indicando cuántos vehículos lo están
cruzando a la vez. Un ejemplo de salida por consola puede ser el siguiente (suponiendo que todos
los hilos intentan acceder al paso a nivel a la vez):

       Pasando tren 1, total cruzando 1
       Pasando tren 0, total cruzando 2
       Pasando tren 2, total cruzando 3
       Pasando tren 3, total cruzando 4
       Pasando tren 4, total cruzando 5
       Tren 1 ha pasado, total cruzando 4
       Tren 2 ha pasado, total cruzando 3
       Tren 0 ha pasado, total cruzando 2
       Tren 4 ha pasado, total cruzando 1
       Tren 3 ha pasado, total cruzando 0
       Pasando coche 3, total cruzando 1
       Pasando coche 1, total cruzando 2
       Pasando coche 2, total cruzando 3
       Pasando coche 4, total cruzando 4
       Pasando coche 0, total cruzando 5
       Coche 3 ha pasado, total cruzando 4
       Coche 1 ha pasado, total cruzando 3
       Coche 2 ha pasado, total cruzando 2
       Coche 4 ha pasado, total cruzando 1
       Coche 0 ha pasado, total cruzando 0

Haga una función main que cree un objeto del tipo Cruce_Nivel, 10 hilos que se comportan como
coche y otros 10 como trenes. Solo puede utilizar la funcionalidad que proporciona la librería
estándar de C++.
```

Junio 2022 – 3 ejercicios

Ejercicio 1 (Herencia y polimorfismo)

```
EJERCICIO 1
Se pide que diseñe parte de la infraestructura de una aplicación de dibujo vectorial partiendo
del siguiente código:
            struct I_Forma {
                virtual double calcular_perimetro() const = 0;
                virtual double calcular_area() const = 0;
                friend ostream& operator<< (ostream &os, const I_Forma &f);
                I_Forma (int id) : id_{id} {}
                int get_id() const { return id_; }
            protected :
                virtual void print(ostream &os) const = 0;
            private:
                int id_;
            };
            ostream& operator<< (ostream &os, const I_Forma &f) {
                f.print(os);
                return os;
            }
```

Se pide que implemente las siguientes clases, que comparten una jerarquía de herencia:
Triangulo

- Hereda de I_Forma.
- Tiene como atributos las longitudes de los tres lados que lo forman.
- Tiene un constructor que recibe como parámetros el identificador y las longitudes de los lados que lo forman.
- Implementa los tres métodos virtuales de I_Forma. Para el cálculo del área, se utiliza la fórmula de Herón: $\sqrt{s \cdot (s - l_1) \cdot (s - l_2) \cdot (s - l_3)}$, donde 's' es el semiperímetro (la mitad del perímetro del triángulo). Una posible implementación del método print podría generar: "Triangulo #1, l1=3, l2=4, l3=6, area: 5.33268, perimetro: 13".

Rectángulo

- Hereda de I_Forma.
- Tiene como atributos las longitudes de los dos lados que lo forman.
- Tiene un constructor que recibe como parámetros el identificador y las longitudes de los lados que lo forman.
- Implementa un método get_lado1.
- Implementa los tres métodos virtuales de I_Forma. Una posible implementación del método print podría generar: "Rectangulo #2, l1=3, l2=4, area: 12, perimetro: 14".

Cuadrado

- Hereda de Rectangulo.
- Tiene como atributo la longitud del lado que lo forma.
- Tiene un constructor que recibe como parámetros el identificador y la longitud del lado.

- Implementa solo el método virtual print, que podría generar: "Cuadrado #3 de lado=3, area: 9, perimetro: 12".

Escriba una función main de prueba corta para comprobar el funcionamiento de las clases desarrolladas y que el polimorfismo funciona adecuadamente. Para ello, cree un objeto de cada clase, lo asigna a un puntero de tipo I_Forma y luego lo imprime por consola a través del puntero. Solo puede utilizar la funcionalidad que proporciona la librería estándar de C++.

Ejercicio 2 (Clases y objetos)

EJERCICIO 2

Se pide programar la infraestructura básica para gestionar grafos no dirigidos. Un grafo está definido por tener un conjunto de nodos y un conjunto de arcos, que conectan entre sí dos nodos. Estos arcos pueden definir una dirección para la conexión o ser bidireccionales (no dirigidos). Se pide que defina una estructura Nodo, que almacena el identificador (un número entero) del nodo, y una clase Grafo, con el siguiente contenido:

- Como atributos, tiene un vector de Nodos y un mapa que asocia cada Nodo con el conjunto de nodos con que está conectado. Para evitar replicar nodos, el mapa es de punteros a Nodo. Tiene también el número de arcos del grafo.
- Método add_nodo(), que recibe como parámetro un nodo y la añade al vector de nodos si no existe ya un nodo con el mismo identificador. Retorna un valor booleano para indicar si ha añadido el nodo o no.
- Método add_arco(), que recibe como parámetros los códigos de identificación de dos nodos y retorna un valor lógico que indica si ha podido crear el arco entre ambos nodos. El arco se puede crear si los códigos de los nodos son distintos, si existen los nodos y si no existía ya un arco entre ambos nodos. Si se cumplen estas restricciones, se añade el arco en ambos nodos del mapa (por ejemplo, add_arco(1,2) añade el nodo 2 al conjunto de nodos con que está conectado el nodo 1, y añade el nodo 1 al conjunto de nodos con que está conectado el nodo 2).
- Método max_num_arcos_nodo(), que retorna cuál es el número máximo de arcos que tiene un nodo del grafo.
- El operator<< para imprimir el contenido del grafo por consola. Por ejemplo:

```
Grafo con 6 nodos y 7 arcos
Nodo 1
          2      3       4      6
Nodo 2
          1      4       6
... el resto de nodos
```

Escriba una función main de prueba corta para comprobar el funcionamiento de las clases desarrolladas. Solo puede utilizar la funcionalidad que proporciona la librería estándar de C++.

Ejercicio 3 (Hilos y exclusión mutua)

EJERCICIO 3: CONCURRENCIA

Se quiere realizar un programa que realice el cómputo del valor de PI mediante el método de Monte Carlo. Este método utiliza valores aleatorios para resolver algunos problemas, sobre todo de simulación. En este caso, el cálculo de PI se realiza de la siguiente manera: se construye un cuadrado de lado 1 y se inscribe un cuarto de una circunferencia de lado 1, centrada en el vértice inferior izquierdo. Se genera un conjunto de pares x-y de forma aleatoria, dentro del cuadrado, que representan la coordenada de un punto. Se cuentan cuantos puntos se generan dentro de la circunferencia. Se multiplica este número por 4 y se divide por el total de pares generados y se obtiene la estimación de PI.

Se quiere comprobar si la longitud del lado del cuadrado o el número de experimentos tiene alguna influencia sobre el resultado obtenido, y para hacer mejor uso de la CPU del sistema se pide desarrollar un programa concurrente que permita lanzar múltiples hilos con múltiples pruebas a la vez. Para ello, se pide que desarrolle una clase Monte_Carlo_Pi con la siguiente estructura:

- Como atributos, el valor del lado del cuadrado y el número de experimentos a realizar. Estos valores serán inicializados por el constructor. Es posible que necesite atributos adicionales.
- Un método realizar_estimacion(), que ejecuta la simulación de Monte Carlo descrita para obtener el valor de PI. Utilice la clase uniform_real_distribution<> (a, b) para obtener un generador de números aleatorios reales entre los valores a y b.
- El método operator<< que imprime los datos del experimento y el resultado por consola. Por ejemplo: “1000 experimentos con lado 19. Aprox PI: 3.192”

Haga una función main que cree al menos 10 objetos de tipo Monte_Carlo_Pi con 1000 simulaciones y longitud de lado variable, y 10 objetos con longitud de lado 2 y número de simulaciones variable. Cree posteriormente 20 hilos, que ejecutarán el método realizar_estimacion(). Por último, imprima por consola los resultados. Solo puede utilizar la funcionalidad que proporciona la librería estándar de C++.

Mayo 2022 – 3 ejercicios

Ejercicio 1 (Herencia y polimorfismo)

EJERCICIO 1

Se pide que diseñe parte de la infraestructura de una red social. Concretamente, las clases que modelan los posts en la red. Para ello, se piden las siguientes dos clases, que comparten una jerarquía de herencia:
POST

- Tiene como atributos el texto del post, el número de likes y vistas, el autor, la fecha de creación y la de modificación (ambas del tipo time_point).
- Tiene un constructor que recibe como parámetros el texto del post y el autor. El número de likes y vistas se inicializan a cero y las fechas de creación y modificación a la fecha en que se crea el objeto.
- Tiene el operator<< para imprimir en un flujo los atributos del objeto. Por ejemplo: "texto del post" – Mario, 0 likes, 0 vistas. Este operador tiene que implementarse de forma polimórfica para que pueda ser utilizado por la siguiente clase que se pide.
- Método para leer el post, que incrementa el número de vistas del mismo y retorna el texto del post.
- Método virtual modificar_texto(), que recibe el nuevo texto para el post y no retorna ningún valor. El método cambia el texto almacenado en el objeto por el nuevo valor suministrado y cambia el valor del atributo fecha de modificación.

POST_HISTORIA

- Hereda de la clase anterior Post.
- Como atributo extra tiene un vector de string, en el que se van a almacenar todas las versiones del texto del post.
- Tiene un constructor que recibe como parámetros el texto del post y el autor.
- Debe poder ser utilizada por el operator<< implementado en la clase Post. Por ejemplo: "texto del post" – Mario, 0 likes, 0 vistas, 2 versiones.
- Implementa el método modificar_texto() guardando en el vector de string la versión del texto del post antes de cambiarla por el valor suministrado y cambiar el valor del atributo fecha de modificación.
- Tiene un método recuperar_version(), que recibe un entero que representa el índice de la versión del texto del post que debe reemplazar el texto actual. Este método actualiza también la fecha de modificación. Retorna true (y realiza las modificaciones) si existe el índice en el vector, false en caso contrario.

```
Escriba una función main de prueba corta para comprobar el funcionamiento de las clases
desarrolladas. Solo puede utilizar la funcionalidad que proporciona la librería estándar de C++.
```

Ejercicio 2 (Clases y objetos)

```
EJERCICIO 2

Se pide programar un sencillo gestor de películas, UPCT_Flix, que permita gestionar las películas
a las que tienen acceso los usuarios de la plataforma. Implemente la estructura Pelicula, que tiene
los siguientes atributos (todos públicos) y sin métodos: titulo, año en que se estrenó y duración
en segundos (utilice un tipo de dato de la librería <chrono>). Añada también una función para
imprimir los detalles de una película por consola.
A continuación, implemente la clase UPCT_Flix, que tiene como atributos un vector de películas
y un mapa de string a vector de punteros a películas. La idea es que los datos de las películas
están almacenadas en el vector, mientras que el mapa representa las películas que un
determinado usuario (identificado por un string) tiene en su subscripción. El segundo campo del
mapa, vector de punteros a películas, es necesario porque múltiples usuarios pueden tener
acceso a la misma película, y no se quiere almacenar múltiples copias de la misma película
(ocupan más espacio y dificultan el mantenimiento). Esta clase tiene los siguientes métodos:
    • Método add_pelicula(), que recibe como parámetro una película y la añade al vector
        de películas si no existe ya una película con el mismo nombre. Retorna un valor booleano
        para indicar si ha añadido la película o no.
    • Método add_pelicula_usuario(), que recibe como parámetros el título de una
        película y el identificador de un usuario, y añade a la subscripción de dicho usuario un
        puntero a la película que tiene dicho título (buscándola en el vector de películas). Si la
        película no existe o ya estaba añadida al usuario, el método retorna un valor false; true
        en caso contrario.
    • Método peliculas_comunes_usuarios(), que recibe como parámetros los
        identificadores de dos usuarios y retorna un vector de las películas (no punteros, los
        objetos) que están en la subscripción de ambos usuarios.

Escriba una función main de prueba corta para comprobar el funcionamiento de UPCT_Flix. Solo
puede utilizar la funcionalidad que proporciona la librería estándar de C++.
```

Ejercicio 3 (Monitores y variables condición)

```
EJERCICIO 3: CONCURRENCIA

Se quiere realizar un programa que simule un juego en el que dos equipos de hilos compiten por
conseguir que todos los valores de un vector de enteros (variable compartida) tengan un valor
par o impar. Cada hilo solo puede cambiar un valor del vector cada vez y, tras hacerlo, se duerme
un tiempo (que se especifica al crear el hilo). Implemente un monitor para gestionar el acceso al
recurso compartido, para gestionar correctamente el juego y gestionar la terminación de los
hilos (que se ejecutan hasta que uno de los equipos gane). Este monitor puede tener la siguiente
estructura:
    • Un constructor que recibe como parámetros un vector de enteros que contiene los
        valores para el juego.
    • Dos métodos, invocados por los hilos de cada equipo, que cambian el primer valor
        par/impar que encuentren en el vector por un valor impar/par, suministrado por el hilo
        que lo invoca. Estos métodos, además, se encargan de comprobar si el juego ya ha
        terminado (es decir, si todos los valores del vector son pares o impares) y, por tanto,
        indican a los hilos si deben terminar ya (mediante el valor de retorno).
    • Un método para determinar qué equipo ha ganado el juego.
Haga una función main para el juego que tenga 3 hilos que intentan que todos los valores del
vector sean pares (duermen durante 200ms) y 3 hilos que intentan que sean impares (duermen
durante 100ms). Solo puede utilizar la funcionalidad que proporciona la librería estándar de
C++. Un ejemplo de salida por consola puede ser:

       ...
       (terminan todos los hilos)
       Contenido monitor: 0 4 2 4 4 6 2 8 0 22 4 0 4 2 12
       Ganador: El equipo que invoca 'cambia_par'
```

Curso 2020-21

Parcial 1 abril 2021 – 7 ejercicios

Ejercicio 1 (Otros)

```
ENUNCIADOS DE EJERCICIOS DE PROGRAMACIÓN BASADA EN OBJETOS
```

Ejercicio 2 (C++ básico y STL (strings, vector, map, set, flujos))

> EJERCICIO 1
>
> La empresa que gestiona un parking público le encarga el diseño preliminar de una aplicación para gestionar las tres plantas de aparcamiento que tiene, con cien (100) plazas por planta. Para ello, le pide que desarrolle un prototipo con un menú (10%) que permita seleccionar entre las siguientes opciones.
>
> - (55%) Solicitar una plaza de aparcamiento. El programa pedirá el número de planta en que quiere aparcar y la matrícula del vehículo (un string). El programa comprobará que el número de planta está comprendido entre 0-2 y que la matrícula tiene el formato NNNNLLL (4 números y 3 letras). Utilice las funciones de la librería <cctype> para realizar estas comprobaciones. Si todos los datos son correctos y hay una plaza libre en la planta elegida, el programa debe reservar la plaza para la matrícula aportada y comunicar al usuario qué plaza le ha asignado.
> - (35%) Liberar una plaza de aparcamiento. El programa pedirá el número de planta en que se encuentra estacionado el vehículo y la matrícula del mismo. El programa comprobará que el número de planta está comprendido entre 0-2 y que la matrícula tiene el formato NNNNLLL (4 números y 3 letras). Utilice las funciones de la librería <cctype> para realizar estas comprobaciones. Si los datos son correctos, y en la planta del parking hay un vehículo con la matrícula suministrada, el sistema liberará la plaza y comunicará al usuario que todo está bien. Si no, le informará del error y volverá al menú.
>
> Se recomienda organizar la funcionalidad solicitada en varias funciones para facilitar tanto el desarrollo del código, como para comprobar que funciona como se espera. Para almacenar los datos de los vehículos puede utilizar tanto <array> como <vector>. Tendrá que modelar y gestionar las situaciones “especiales” siguientes: “no quedan plazas libres”, “matrícula incorrecta” y “matrícula inexistente”. Puede hacer uso de valores concretos (por ejemplo, el string vacío), <optional>, devolver una pareja/tupla, o cualquier otra opción que considere para gestionar estas situaciones. Intente marcar como constante todos los parámetros que se pasan a una función, si solo necesita acceso de lectura a los mismos. Pero si eso genera errores de compilación, no dude en eliminar la restricción constante. Para resolver el ejercicio solo puede utilizar la funcionalidad proporcionada por la librería estándar de C++.

Ejercicio 3 (C++ básico y STL (strings, vector, map, set, flujos))

> EJERCICIO 2
>
> La empresa que gestiona tres muelles de carga/descarga de mercancías en un puerto le encarga el diseño preliminar de una aplicación para gestionarlos. Cada muelle cuenta con veinte (20) puntos de amarre para cargueros. Para ello, le pide que desarrolle un prototipo con un menú (10%) que permita seleccionar entre las siguientes opciones.
>
> - (55%) Solicitar un punto de amarre. El programa pedirá el número de muelle en que quiere atracar y la matrícula del carguero (un string). El programa comprobará que el número de muelle está comprendido entre 0-2 y que el carguero no está ya atracado en dicho muelle (es decir, que su matrícula no está ya almacenada). Si todos los datos son correctos y hay un punto de amarre libre en el muelle elegido, el programa debe reservar el punto de amarre a la matrícula del carguero aportada y comunicar al usuario qué punto le ha asignado.
> - (30%) Liberar un punto de amarre. El programa pedirá el número de muelle en que se encuentra atracado el carguero y la matrícula del mismo. El programa comprobará que el número de planta está comprendido entre 0-2. Si los datos son correctos, y en el muelle elegido hay un carguero con dicha matrícula, el sistema liberará el punto de amarre y comunicará al usuario que todo está bien. Si no, le informará del error y volverá al menú.
>
> Se recomienda organizar la funcionalidad solicitada en varias funciones para facilitar tanto el desarrollo del código, como para comprobar que funciona como se espera. Para almacenar los datos de los cargueros puede utilizar tanto <array> como <vector>. No utilice <set> porque el orden (la posición) en que se almacenan los valores es importante, y <set> mantiene la colección siempre ordenada, por lo que puede “recolocar” cargueros (no es lo que se quiere). Tendrá que modelar y gestionar las situaciones “especiales” siguientes: “no quedan puntos de amarre libres en el muelle”, “matrícula repetida” y “matrícula inexistente”. Puede hacer uso de valores concretos (por ejemplo, el string vacío), <optional>, devolver una pareja/tupla, o cualquier otra opción que considere para gestionar estas situaciones. Intente marcar como constante todos los parámetros que se pasan a una función, si solo necesita acceso de lectura a los mismos. Pero si eso genera errores de compilación, no dude en eliminar la restricción constante. Para resolver el ejercicio solo puede utilizar la funcionalidad proporcionada por la librería estándar de C++.

Ejercicio 4 (C++ básico y STL (strings, vector, map, set, flujos))

> EJERCICIO 3
>
> La empresa que gestiona un cine con tres salas de proyección le encarga el diseño preliminar de una aplicación para gestionarlas. Cada sala cuenta con cincuenta (50) asientos para espectadores. Para ello, le pide que desarrolle un prototipo con un menú (10%) que permita seleccionar entre las siguientes opciones.

- (60%) Reservar un asiento. El programa pedirá el número de sala, el DNI del espectador y el número de asiento, aproximado, desde el que el espectador quiere ver la película. El programa comprobará que el número de sala está comprendido entre 0-2 y que el número de asiento preferido está entre 0-49. Si todos los datos son correctos, buscará el asiento más cercano al preferido del espectador (por encima y por debajo), y si hay algún asiento libre, el programa

  debe reservar el asiento al DNI aportado y comunicar al usuario qué asiento le ha asignado. Por ejemplo, si el usuario quiere el asiento 23, pero está ocupado, el programa debe buscar cuál es el asiento libre más cercano: el 23, el 24 o el 22, el 25 o el 21, el 26 o el 20, etc.
- (30%) Cancelar una reserva. El programa pedirá el número de sala y el DNI con que se hizo la reserva. El programa comprobará que el número de sala está comprendido entre 0-2. Si los datos son correctos, y en la sala hay un asiento reservado al DNI suministrado, el sistema liberará el asiento y comunicará al usuario que todo está bien. Si no, le informará del error y volverá al menú.

Se recomienda organizar la funcionalidad solicitada en varias funciones para facilitar tanto el desarrollo del código, como para comprobar que funciona como se espera. Para almacenar los datos de los espectadores puede utilizar tanto <array> como <vector>. Tendrá que modelar y gestionar las situaciones “especiales” siguientes: “no quedan asientos libres en la sala” y “reserva inexistente”. Puede hacer uso de valores concretos (por ejemplo, el string vacío), <optional>, devolver una pareja/tupla, o cualquier otra opción que considere para gestionar estas situaciones. Intente marcar como constante todos los parámetros que se pasan a una función, si solo necesita acceso de lectura a los mismos. Pero si eso genera errores de compilación, no dude en eliminar la restricción constante. Para resolver el ejercicio solo puede utilizar la funcionalidad proporcionada por la librería estándar de C++.

Ejercicio 5 (Clases y objetos)

EJERCICIO 4

Una empresa fabrica triángulos y rectángulos de goma, de distintos tamaños, para cubrir superficies, y guarda el stock que tiene disponible en un fichero de texto que tiene la siguiente estructura: “t␣l1␣l2\n”. En el fichero se repite esta estructura tantas veces como triángulos y rectángulos fabrica. ‘t’ es el tipo de figura geométrica (un carácter: ‘r’ -> rectángulo, ‘t’ -> triángulo), ␣ es un espacio en blanco y l1 y l2 son las longitudes necesarias para calcular el área de la figura (valores enteros). Para facilitar la tarea, se simulará la entrada de fichero con un string. Por ejemplo: string datos {"t 3 4\nr 2 2\nr 3 2\nt 3 2\nr 4 4\nt 4 4\nr 1 2\nt 5 2\n₂};

Dados estos datos de prueba, el programa debe solicitar al usuario cuál es la superficie que quiere cubrir (valor positivo) y calculará e imprimirá por consola cuál es la superficie máxima que se puede cubrir (sin sobrepasar el valor suministrado por el usuario) utilizando el mayor número de triángulos y cuadrados disponibles. Para simplificar el algoritmo, se supone que solo hay una unidad de cada tamaño. Si utiliza <set>, recuerde que tiene que recorrerlo con iteradores o mediante el bucle for de colección, ya que no define operator[].

Por ejemplo, para el string datos anterior:

Superficie triangulos: 3, 5, 6, 8,
Superficie rectangulos: 2, 4, 6, 16,

Ejecución 1:
Superficie a cubrir? >> 12
Triangulos: 8, cuadrados: 12

Ejecución 2:
Superficie a cubrir? >> 27
Triangulos: 22, cuadrados: 12

Ejecución 3:
Superficie a cubrir? >> 33
Triangulos: 22, cuadrados: 28

Intente marcar como constante todos los parámetros que se pasan a una función, si solo necesita acceso de lectura a los mismos. Pero si eso genera errores de compilación, no dude en eliminar la restricción constante. Para resolver el ejercicio solo puede utilizar la funcionalidad proporcionada por la librería estándar de C++.

Ejercicio 6 (C++ básico y STL (strings, vector, map, set, flujos))

EJERCICIO 5

Se pide que desarrolle dos funciones que implementan la siguiente funcionalidad sobre string: una función que invierte el orden de las palabras de una frase y otra que implementa buscar/reemplazar con más de una palabra. Se supone que el texto no tiene signos de

```
puntuación. Ejemplos:

    1. Si el texto de entrada es “en un lugar de la mancha de cuyo nombre no quiero acordarme”,
        el texto de salida es “acordarme quiero no nombre cuyo de mancha la de lugar un en”.
    2. Si el texto de entrada es “la parte contratante de la primera parte sera considerada como
        la parte contratante de la primera parte", el texto buscado es “parte contratante de la”, y
        el texto de sustitución es “vigésimo”, el texto de salida será “la vigesimo primera parte
        sera considerada como la vigesimo primera parte”.
Las funciones tienen que recibir los valores como parámetros, pero desde la función main()
puede pasar como parámetros los valores de ejemplo que se proporcionan en el enunciado para
que el desarrollo del código sea más rápido. Intente marcar como constante todos los parámetros
que se pasan a una función, si solo necesita acceso de lectura a los mismos. Pero si eso genera errores
de compilación, no dude en eliminar la restricción constante. Para resolver el ejercicio solo puede
utilizar la funcionalidad proporcionada por la librería estándar de C++.
```

Ejercicio 7 (C++ básico y STL (strings, vector, map, set, flujos))

```
EJERCICIO 6

Se pide que haga un programa que agrupe las palabras de un texto largo según la letra por la que
comienzan, para luego poder obtener todas las palabras que empiezan por una determinada
letra. Se supone que el texto no tiene signos de puntuación.

Para ello, se recomienda utilizar <map> que relaciona una letra con un string (las palabras que
comienzan por dicha letra), e ir ampliando el string conforme se encuentra una nueva palabra
que comienza por dicha letra (por ejemplo, concatenando la nueva palabra). Para facilitar la
resolución del ejercicio, se permite que haya palabras repetidas. Tras tener organizado los datos,
se piden dos funciones:

   1. Una función que retorna un string formado por todas las palabras que empiezan por una
      vocal.
   2. Una función que retorna un string formado por todas las palabras que se encuentran
      comprendidas entre dos letras, que se pasan como parámetros a la propia función. En
      caso de que la letra final sea menor, alfabéticamente, que la inicial, la función debe
      retornar el string vacío. Se recuerda que las letras están ordenadas alfabéticamente y de
      forma consecutiva en el código ASCII.

Por ejemplo, si el texto largo fuera:
    const string texto_largo {"en un lugar de la mancha de cuyo nombre no quiero acordarme no
ha mucho tiempo que vivia un hidalgo de los de lanza en astillero adarga antiguo rocin flaco y
galgo corredor una olla de algo mas vaca que carnero salpican las mas noches duelos y
quebrantos los sabados lentejas los viernes algan palomino de anadidura los domingos
consumian las tres partes de su hacienda"};

SOLO VOCALES:
acordarme | astillero | adarga | antiguo | algo | algan | anadidura |
en | en |

olla |
un | un | una |

ENTRE 'a₇ y 'd₇:
acordarme | astillero | adarga | antiguo | algo | algan | anadidura |

cuyo | corredor | carnero | consumian |
de | de | de | de | de | duelos | de | domingos | de |

ENTRE 'd₇ y 'a₇:

Las funciones tienen que recibir los valores como parámetros, pero desde la función main()
puede pasar como parámetros los valores de ejemplo que se proporcionan en el enunciado para
que el desarrollo del código sea más rápido. Intente marcar como constante todos los parámetros
que se pasan a una función, si solo necesita acceso de lectura a los mismos. Pero si eso genera errores
de compilación, no dude en eliminar la restricción constante. Para resolver el ejercicio solo puede
utilizar la funcionalidad proporcionada por la librería estándar de C++.
```

Parcial 2 junio 2021 – 5 ejercicios

Ejercicio 1 (Otros)

```
ENUNCIADOS DE EJERCICIOS DE PROGRAMACIÓN BASADA EN OBJETOS
```

Ejercicio 2 (Clases y objetos)

EJERCICIO 1

Se pide que diseñe una aplicación para gestionar los préstamos de libros de una biblioteca. Para ello, se pide que diseñe las siguientes dos clases:
LIBRO

- Tiene como atributos el título, autor, año de publicación del libro, un código interno de inventariado (id), y un booleano para saber si el ejemplar está prestado.
- Tiene un constructor para inicializar todos los atributos anteriores mediante parámetros, salvo el booleano, que se crea con valor false.
- Tiene métodos para obtener el valor de cada uno de los atributos (getters)
- Tiene el operator<< para imprimir en un flujo los atributos del objeto. Por ejemplo: 'Programacion en C++', Ceballos, 2019. [123abc, D], donde la 'D$_7$ significa 'disponible'.
- Tiene un método prestar() y otro retornar(), que modifican el valor booleano para indicar que se ha prestado o se ha devuelto, respectivamente. Estos métodos devuelven un valor lógico para indicar si la acción se ha realizado correctamente. Por ejemplo, si el libro estaba prestado, la invocación del método retornar() devuelve true, mientras que la del método prestar() devuelve false.

BIBLIOTECA

- Como atributos tiene un vector de objetos de la clase Libro
- Tiene un método nuevo_libro(), que acepta como parámetro un Libro y retorna si ha podido añadir el Libro al vector. Un Libro se puede añadir a la colección si su código de inventariado (id) no está repetido en otro Libro de la colección.
- Tiene un método buscar_por_titulo(), que recibe el título del libro que se está buscando, y que retorna un vector de Libro con todos los ejemplares cuyo título coincide con el valor suministrado. En caso de que no se encuentre ningún título, retorna el vector vacío.
- Tiene un método prestar(), que recibe el código de inventariado de un libro y retorna, utilizando un <optional>, el libro prestado. Hace uso del método prestar() de Libro.
- Tiene un método retornar(), que recibe un Libro y retorna un valor booleano para indicar si se ha podido devolver. Es decir, si el Libro se encuentra en la colección de la Biblioteca (comparando los códigos de inventariado) y si el Libro estaba efectivamente prestado. Hace uso del método retornar() de Libro.

Haga una función 'main' en la que cree algunos Libros y los añada a una Biblioteca. Muestre también en funcionamiento los métodos implementados.

Ejercicio 3 (Hilos y exclusión mutua)

EJERCICIO 2

Se pide que diseñe una aplicación para jugar a una versión simplificada del juego de los barquitos. Para ello, se pide que diseñe las siguientes dos clases:
TABLERO

- Tiene como atributos un vector con el estado de cada casilla de una línea del juego de los barquitos (recuerde, es una versión simplificada del juego). Una casilla puede tener tres estados: 'agua', 'barquito' y 'hundido'. En esta versión, los barquitos solo ocupan una casilla. También tiene como atributo el número de barquitos que almacena el vector y que quedan por hundir.
- Tiene un constructor para inicializar el tamaño del tablero (una línea realmente, no una matriz como en el juego real) y el número de barquitos que habrá en el tablero. Se supone que ambos valores son positivos y que el tamaño del tablero es mayor que el número de barquitos que se quiere posicionar en él. El constructor debe situar, en posiciones aleatorias, cada uno de los barquitos e inicializar a ‘agua’ el resto del vector.
- Tiene un constructor que inicializa el tablero con valores por defecto: tamaño 50 y 5 barquitos.
- Tiene un método get_tam() que retorna el tamaño del tablero
- Tiene el operator<<() para imprimir, en un flujo, el estado del tablero (una línea, realmente). Por ejemplo, puede representar el estado 'agua' como _, 'barquito' como B y 'hundido con *.
- Tiene un método quedan_barcos() que retorna la cantidad de barcos que faltan por hundir.
- Tiene un método disparo() que recibe la coordenada a la que se dispara y retorna el estado en que se encuentra dicha coordenada después del disparo. Los estados 'agua' y 'hundido' no se ven afectados por el disparo, únicamente 'barquito' pasa a 'hundido' y decrementa el número de barquitos disponibles.

JUGADOR

- Como atributos tiene el nombre del Jugador y un vector de objetos de la clase Tablero
- Tiene un constructor que acepta el nombre del jugador, la cantidad de tableros que quiere generar, el tamaño de los tableros y el número de barquitos. Estos dos últimos valores son los mismos para todos los tableros que se vayan a crear.
- Tiene un constructor que acepta el nombre del jugador y la cantidad de tableros que quiere generar. Cada tablero se inicializa utilizando el constructor por defecto de la clase Tablero
- Tiene el operator<<() para imprimir, en un flujo, el nombre del jugador y el estado de todos sus tableros.
- Tiene un método jugar_ha_perdido(), que recibe como parámetros el número de tablero en que se hace la jugado y la coordenada a la que se dispara. Si los valores son erróneos (son negativos, el jugador no tiene tantos tableros, se dispara a una coordenada que no existe), el método devuelve false. Si los valores son correctos, el método actualiza el estado de los tableros del jugador y retorna si ha perdido ya el juego. El jugador pierde el juego si le han hundido todos los barquitos de todos sus tableros.

Haga una función 'main' en la que cree un Jugador y muestre en funcionamiento los métodos implementados.

ENUNCIADOS DE EJERCICIOS DE CONCURRENCIA

Ejercicio 4 (Hilos y exclusión mutua)

EJERCICIO 3

Haga un programa que implemente la siguiente funcionalidad concurrente. Se tienen dos variables compartidas de tipo double, v1 y v2, y dos tipos de hilo (M, D) que reciben dos valores cuando se crean: un número ‘n’ y el tiempo ‘t’, en milisegundos, que deben esperar entre dos ejecuciones consecutiva de una operación. Los hilos de tipo ‘M’ cambian el valor de v1 o v2 multiplicándola/s por el número ‘n’ (sobrescriben v1 o v2), mientras que los de tipo ‘D’ cambian el valor de v1 o v2 dividiéndola/s por el número ‘n’ (sobrescriben v1 o v2). Después de actualizar el valor, ambos tipos de hilo duermen un tiempo ‘t’ antes de volver a realizar la operación. Cada hilo decide, en cada iteración y de forma aleatoria, sobre qué variable actuar: solo sobre v1, solo sobre v2 o sobre ambas a la vez (en este caso, debe asegurarse acceso exclusivo a ambas variables mientras realiza las operaciones). Pueden tomar esta decisión, por ejemplo, generando un número aleatorio entre 1 y 3.
Cada hilo imprime por consola qué operación realizó sobre qué variable y cuál es el valor que finalmente almacenó en la/s variable/s. Esta impresión por consola debe hacerse de forma segura frente a la concurrencia.
Implemente una función main en la que se inicialicen las variables v1 y v2 al valor 10 y se lancen dos hilos de cada tipo. El main esperará unos 5 segundos antes de indicar a los hilos que deben terminar de ejecutarse. Para que se haga una idea, creando 2 hilos de tipo ‘M’ y de tipo ‘D’, que multiplican y dividen por 2 y 3, una posible salida por consola del programa podría ser:

```
MULTIPLICAR: v2 -> 30
DIVIDIR: v1 -> 5
DIVIDIR: v1 -> 1.66667 v2 -> 10
MULTIPLICAR: v1 -> 3.33333
MULTIPLICAR: v1 -> 10 v2 -> 30
DIVIDIR: v1 -> 5 v2 -> 15
DIVIDIR: v1 -> 1.66667 v2 -> 5
MULTIPLICAR: v1 -> 3.33333
MULTIPLICAR: v1 -> 10 v2 -> 15
DIVIDIR: v1 -> 5
DIVIDIR: v1 -> 1.66667
MULTIPLICAR: v2 -> 30
MULTIPLICAR: v1 -> 5 v2 -> 90
DIVIDIR: v2 -> 45
FIN MULTIPLICAR
FIN DIVIDIR
FIN DIVIDIR
FIN MULTIPLICAR
[MAIN] finalizado :-)
```

Ejercicio 5 (Monitores y variables condición)

EJERCICIO 4

Implemente una clase que haga de barrera para un grupo de hilos. Una barrera es un objeto que se utiliza para bloquear el avance de un número cualquiera de hilos y conseguir sincronización entre ellos. Es similar a la barrera de un paso a nivel, por ejemplo, que bloquea el paso de personas y coches hasta que se levanta. Se pide que implemente una clase Barrera_Secreta, que se crea con un “número secreto” y proporciona un método probar_suerte(), que recibe un valor. Los hilos del sistema utilizan el método probar_suerte() para entrar en la barrera e intentar abrirla. Si el valor que un hilo suministra al método probar_suerte() coincide con el número secreto que se utilizó al crear la barrera, ésta se abre

```
y deja salir, progresivamente, todos los hilos que estuvieran bloqueados. Si no coincide, el hilo se
queda bloqueado. El método probar_suerte() retorna un valor booleano que solo es true para el hilo
que consiguió adivinar el valor secreto. Para el resto de hilos, al terminar la ejecución del método,
retorna false. Implemente la clase Barrera_Secreta utilizando mutex, variables de condición y todos
los atributos que pueda necesitar.
Haga una función main de prueba en la que demostrar el uso de la clase Barrera_Secreta. En la
implementación de esta función debe asegurar que los mensajes que se imprimen por consola están
correctamente sincronizados para evitar el solapamiento.
Por ejemplo, inicialice la Barrera_Secreta a un valor conocido (20), defina una distribución de
números aleatorios entre 1 y 100, y luego vaya creando hilos que invocan el método probar_suerte()
con un valor aleatorio, hasta que uno de ellos adivine el valor. Una vez adivinado el valor, la función
main debe dejar de crear hilos y terminar limpiamente. Imprima qué hilo ha adivinado el valor y cuál
era el valor secreto. Para que se haga una idea, una posible salida por consola del programa podría ser:

[HILO 1] probando con 21
[HILO 7] probando con 17
[HILO 8] probando con 18
[HILO 3] probando con 16
[HILO 5] probando con 17
[HILO 6] probando con 15
[HILO 2] probando con 17
[HILO 4] probando con 19
[HILO 9] probando con 20
[HILO 9] ha descubierto el numero secreto: 20
[MAIN] finalizado :-)
```

Julio 2021 – 3 ejercicios

Ejercicio 1 (Otros)

```
JULIO 2020-2021

ENUNCIADOS DE EJERCICIOS DE PROGRAMACIÓN BASADA EN OBJETOS
```

Ejercicio 2 (C++ básico y STL (strings, vector, map, set, flujos))

```
EJERCICIO 1

Se pide que implemente la siguiente funcionalidad para la gestión de textos. Dado un texto (el primer
párrafo del capítulo 1 de El Quijote, por ejemplo) y una lista de letras, implemente un programa que
extraiga del texto todas las palabras que contienen todas las letras de la lista proporcionada. Ignore los
signos de puntuación (coma o punto) en el texto. Utilice el siguiente texto para probar su programa:

string str {"En un lugar de la Mancha, de cuyo nombre no quiero acordarme, no ha
mucho tiempo que vivia un hidalgo de los de lanza en astillero, adarga antigua,
rocin flaco y galgo corredor. Una olla de algo mas vaca que carnero, salpicon las
mas noches, duelos y quebrantos los sabados, lentejas los viernes, algun palomino
de añadidura los domingos, consumian las tres partes de su hacienda."};

Con este texto, las palabras que tienen las letras “aoc” (en cualquier orden) son: |acordarme,|
|flaco|    |carnero,|      |salpicon|      |consumian|

Añada todas las funciones que considere necesarias. Solo puede utilizar la funcionalidad
proporcionada por la librería estándar de C++. Se recomienda utilizar <sstream> para procesar más
fácilmente el texto de entrada.
```

Ejercicio 3 (C++ básico y STL (strings, vector, map, set, flujos))

```
EJERCICIO 2

Se pide que implemente la siguiente funcionalidad para la gestión de textos. Dado un texto (el primer
párrafo del capítulo 1 de El Quijote, por ejemplo) y un tamaño de palabra, se pide que parta la frase
original cuando encuentre una palabra de dicha longitud, y almacene el texto posiciones sucesivas de
un vector. Ignore los signos de puntuación (coma o punto) en el texto. Utilice el siguiente texto para
probar su programa:

string str {"En un lugar de la Mancha, de cuyo nombre no quiero acordarme, no ha
mucho tiempo que vivia un hidalgo de los de lanza en astillero, adarga antigua,
rocin flaco y galgo corredor. Una olla de algo mas vaca que carnero, salpicon las
mas noches, duelos y quebrantos los sabados, lentejas los viernes, algun palomino
de añadidura los domingos, consumian las tres partes de su hacienda."};
```

```
Con este texto, los primeros fragmentos de texto con tamaño de palabra 5 son los siguientes (se
calculan más, que no se muestra). Cada fragmento de texto de la siguiente lista está almacenado en
una posición de un vector:
    - En un lugar
    - de la Mancha, de cuyo nombre no quiero acordarme, no ha mucho
    - tiempo que vivia

Añada todas las funciones que considere necesarias. Solo puede utilizar la funcionalidad
proporcionada por la librería estándar de C++. Se recomienda utilizar <sstream> para procesar más
fácilmente el texto de entrada.
```

Junio 2021 – 7 ejercicios

Ejercicio 1 (Otros)

```
JUNIO 2020-2021

ENUNCIADOS DE EJERCICIOS DE PROGRAMACIÓN BASADA EN OBJETOS
```

Ejercicio 2 (C++ básico y STL (strings, vector, map, set, flujos))

```
EJERCICIO 1

(70%) Una empresa le pide que realice un programa para buscar la secuencia de valores consecutivos,
de tamaño ‘k’, cuya suma es máxima y menor que otro valor ‘v_max’ en un vector de tamaño ‘t’. Por
ejemplo, si el vector es {1,2,3,4,5,6}, k=3 y v_max=10, la suma máxima de secuencias de tamaño 3
con un valor menor que 10 es 9, dado por la secuencia {2,3,4} (las secuencias posibles son {1,2,3},
{2,3,4}, {3,4,5}, {4,5,6}). Los valores almacenados en el vector pueden ser positivos y negativos.
Debe desarrollar una función que acepta el vector, el tamaño de los subconjuntos y el valor máximo
de la suma, y que retorna el valor de la suma, la posición del vector donde comienza el subconjunto
que genera la suma máxima, y el valor de ‘k’. Si el tamaño de los subconjuntos es mayor que el tamaño
del vector, la función debe retornar un error.

(30%) Para poder probar dicha función, se debe solicitar al usuario del programa los siguientes datos:
tamaño del vector de prueba, tamaño de los subconjuntos y valor máximo de la suma. En la entrada de
valores de usuario, tiene que comprobar que se introduce efectivamente un número y que dicho número
cumple la restricción de que es mayor que 7 (tamaño del vector), es mayor que 2 (tamaño del
subconjunto) y es mayor que 0 (suma máxima). El vector de prueba se genera simplemente
almacenando valores consecutivos, comenzando por 0 (es decir, {0,1,2,3, ...}).

Un ejemplo de ejecución es el siguiente (‘>>’ indica que el usuario debe introducir un valor):
       Tamaño del vector? >> 100
       Tamaño de los subconjuntos? >> 7
       Valor máximo de la suma? >> 500
       La suma máxima, menor que 500, de 7 elementos es 497, posición 68
       (68+69+70+71+72+73+74)

Añada todas las funciones que considere necesarias. Solo puede utilizar la funcionalidad
proporcionada por la librería estándar de C++. Recuerde que puede retornar varios valores utilizando
<tuple> o declarando una estructura, y que <optional> se puede utilizar para retornar, opcionalmente,
un valor. Tenga cuidado de no acceder a posiciones que se encuentren fuera de los límites del vector.
```

Ejercicio 3 (C++ básico y STL (strings, vector, map, set, flujos))

```
EJERCICIO 2

(70%) Una empresa le pide que realice un programa para buscar el máximo valor que puede generar
una función sobre un subconjunto de valores almacenados en un vector. El programa solicita al usuario
la secuencia de operaciones que quiere aplicar sobre un subconjunto de los valores del vector, y retorna
el valor máximo alcanzado y la posición inicial del subconjunto que lo genera. Las únicas operaciones
permitidas sobre los valores son +, -, *, /. Las operaciones comienzan a aplicarse sobre el valor 1, que
es fijo, y el resultado se va acumulando entre operaciones sucesivas. Solo hay que tener precaución al
aplicar la división, que debe comprobar que no se realiza una división por cero.

Por ejemplo, si el vector inicial es {1,2,3,4,5}, y el usuario introduce como operaciones “+*-” (un
string), el valor máximo es 11, alcanzado en la posición de índice 2. Estos son los tres subconjuntos
que se van generando. El valor 1 inicial es siempre fijo, según se ha descrito en el enunciado:

   -   ((1+1)*2)-3 = 1
   -   ((1+2)*3)-4 = 5
```

- ((1+3)*4)-5 = 11

Debe desarrollar una función que acepta el vector y el conjunto de operaciones a realizar, y que retorna el valor máximo generado y la posición del vector donde comienza el subconjunto que lo genera. Si la cantidad de operaciones a realizar es mayor que el tamaño del vector, la función debe retornar un error.

(30%) Para poder probar dicha función, se debe solicitar al usuario del programa los siguientes datos: tamaño del vector de prueba y lista de operaciones (un string). En la entrada de valores de usuario tiene que comprobar que se introduce efectivamente un número mayor que 7 (tamaño del vector) y que el conjunto de operaciones solo contiene los caracteres de las operaciones permitidas (+ - * /). El vector de prueba se genera simplemente almacenando valores consecutivos, comenzando por 0 (es decir, {0,1,2,3, ...}).
Un ejemplo de ejecución es el siguiente (‘>>’ indica que el usuario debe introducir un valor):

```
Tamaño del vector? >> 10
Introduce operaciones (+ - * /) sin espacio: -+*p
Introduce operaciones (+ - * /) sin espacio: -+*/
El valor máximo es 1.77778, posición 6
```

Añada todas las funciones que considere necesarias. Solo puede utilizar la funcionalidad proporcionada por la librería estándar de C++. Recuerde que puede retornar varios valores utilizando <tuple> o declarando una estructura, y que <optional> se puede utilizar para retornar, opcionalmente, un valor. Tenga cuidado de no acceder a posiciones que se encuentren fuera de los límites del vector. Como valor más pequeño, puede utilizar el valor -INFINITY.

ENUNCIADOS DE EJERCICIOS DE PROGRAMACIÓN ORIENTADA A OBJETOS

Ejercicio 4 (Hilos y exclusión mutua)

EJERCICIO 3

Se pide que implemente un programa para gestionar planificadores de tareas. Para ello, se pide que diseñe las siguientes dos clases:
TAREA

- Tiene los siguientes atributos: título y descripción (de tipo string), minutos restantes y prioridad de la tarea (valores enteros).
- Tiene tres constructores:
  - Uno para inicializar todos los atributos mediante parámetros.
  - Uno para inicializar todos los atributos menos la prioridad, que se inicializa a 5.
  - Uno por defecto, que inicializa los atributos de tipo strings al valor string vacío, los minutos a 100 y la prioridad a 5.
- Tiene un método para obtener el valor del atributo prioridad y para saber si una tarea ha terminado ya (es decir, si el atributo minutos restantes tiene un valor menor o igual a 0).
- Tiene métodos para fijar los valores del título, la descripción y la prioridad. Son métodos que retornan un valor booleano para indicar si se ha podido realizar el cambio o no. El título y la descripción solo se pueden cambiar si no tenía valor, y la prioridad de una tarea solo puede cambiarse hacia un valor menor del que tiene (la prioridad mínima es 1).
- Tiene un método descontar_t_trabajado(), que recibe la cantidad de minutos que se ha trabajado en la tarea y que descuenta ese valor del atributo minutos restantes, siempre que éste sea positivo (es decir, que todavía quedara tiempo por trabajar). El método devuelve un valor booleano para indicar si ha podido llevar a cabo esta tarea.
- Tiene el operator<< para imprimir en un flujo los atributos del objeto. Por ejemplo:

```
[Preparar examen]: PBO + POO + Concurrencia.
        Prioridad: 3, tiempo restante: 500
```

PLANIFICADOR

- Tiene los siguientes atributos: título del planificador (de tipo string) y una lista de nombres de personas y tareas asignadas (un mapa).
- Tiene un constructor, al que se le pasa el título del planificador que se va a crear.
- Tiene un método anyadir_persona_tarea(), que recibe como valores el nombre de una persona y la tarea que se le quiere asignar, y que, si esa persona no tiene actualmente ninguna tarea asignada, registra la asignación en el mapa y retorna true. Si la persona ya tenía una tarea asignada, retorna false.
- Tiene un método personas_tareas_prioridad (), que recibe como valor una prioridad y retorna una lista de personas (un vector) que tienen asignadas una tarea con dicho nivel de prioridad.
- Tiene un método tareas_terminadas (), que retorna una lista (un vector) con las tareas que ya están terminadas.

- Tiene el operator<< para imprimir en un flujo el título del planificador y, por cada tarea, nombre de la persona que la realiza y campos de la tarea.

Añada todas las funciones que considere necesarias. Solo puede utilizar la funcionalidad

proporcionada por la librería estándar de C++. Haga también una función ‘main’ en la que muestre cómo utilizar las clases diseñadas.

Ejercicio 5 (Hilos y exclusión mutua)

EJERCICIO 4

Se pide que implemente un programa para gestionar la carga de una flota de camiones. Para ello, se pide que diseñe las siguientes dos clases:
PALET

- Tiene los siguientes atributos: identificador y descripción (de tipo string), peso actual y peso máximo (valores enteros).
- Tiene un constructor que inicializa todos los atributos mediante los valores suministrados al mismos, salvo el peso actual, que inicialmente es 0.
- Tiene métodos para obtener el valor del identificador del palet, su descripción, el peso actual y la capacidad que tiene disponible el palet (peso máximo menos el actual).
- Tiene un método set_descripcion() para cambiar el contenido del atributo.
- Tiene un método anyadir_peso(), que tiene como parámetro el peso que se va a añadir al palet, y que devuelve un valor booleano para indicar si el palet tenía capacidad para almacenar dicho peso y si, efectivamente, se ha podido añadir al palet.
- Tiene el operator<< para imprimir en un flujo los atributos del objeto. Por ejemplo:

[123-abc] Libros de C++, 80·kg de un máximo de 100·kg

CAMION

- Tiene los siguientes atributos: matrícula del camión (de tipo string), peso actual y peso máximo autorizado (valores enteros) y una lista de ciudades y palets que tiene que llevar (un mapa).
- Tiene un constructor, al que se le pasa la matrícula del camión y el peso máximo autorizado.
- Tiene métodos para obtener la matrícula del camión, el peso actual y la capacidad de carga que tiene disponible.
- Tiene un método cargar() que recibe como parámetros un palet y un destino, y retorna un valor booleano que indica si se puede almacenar el palet en el camión. Un palet se puede añadir a un camión si el peso del palet no es cero, si el camión no supera su peso máximo autorizado al cargarlo, si no existe ya un palet con el mismo identificador y si el destino no ha sido añadido ya.
- Tiene un método palet_mas_pesado(), que retorna el palet con mayor peso almacenado en el camión.
- Tiene un método lista_destinos (), que retorna una lista (un vector) con los destinos que tiene que visitar el camión.
- Tiene un método lista_palets_destinos (), que tiene un parámetro que es una lista de destinos (un vector de string) y que retorna un vector con los palets que tienen que descargarse en los destinos proporcionados.

- Tiene el operator<< para imprimir en un flujo la matrícula del camión, peso actual y peso máximo autorizado y, por cada ciudad, los datos del palet que tiene que llevar.

Añada todas las funciones que considere necesarias. Solo puede utilizar la funcionalidad proporcionada por la librería estándar de C++. Haga también una función ‘main’ en la que muestre cómo utilizar las clases diseñadas.

ENUNCIADOS DE EJERCICIOS DE PROGRAMACIÓN CONCURRENTE

Ejercicio 6 (Monitores y variables condición)

EJERCICIO 5

Desarrolle un monitor Mapa_Compartido para que dos hilos puedan compartir el acceso concurrente a un mapa de string a entero sin que se pierdan valores. Es decir, asegurando que un hilo no escribe dos veces o lee dos veces la misma clave sin que ésta haya sido borrada entre lecturas o escrituras consecutivas (más detalles a continuación). El monitor proporciona tres métodos:
- Leer(): recibe una clave (string) y retorna el valor (entero) almacenado en dicha clave en el mapa. Este método debe bloquear al hilo que lo invoca si la clave no está almacenada en el mapa. Si está, debe borrarla antes de retornar el valor.
- Escribir(): recibe una clave (string) y el valor (entero) que deben ser almacenados en el mapa. Este método debe bloquear al hilo que lo invoca si la clave ya existe, para de esta forma evitar sobrescribir valores que todavía no hayan sido leídos.
- Terminar(): permite liberar correctamente todos los hilos que pudieran estar bloqueados en el monitor para que haya una finalización limpia del programa.

Realice un programa para probar el correcto funcionamiento del monitor, creando un hilo que utilice el método Leer() y otro que utilice el método Escribir() de Mapa_Compartido para comprobar que:

```
    1. El monitor bloquea al hilo que lee si la clave no está almacenada en el mapa.
    2. El monitor bloquea al hilo que escribe si la clave todavía está presente en el mapa.

En la implementación de esta función debe asegurar que los mensajes que se imprimen por
consola están correctamente sincronizados para evitar el solapamiento. Solo puede utilizar la
funcionalidad proporcionada por la librería estándar del lenguaje C++.
```

Ejercicio 7 (Monitores y variables condición)

```
EJERCICIO 6

Desarrolle un monitor Buffer_Compartido para que dos hilos puedan compartir el acceso
concurrente a un vector de valores enteros de un tamaño fijado al construir el objeto.
Implemente el siguiente comportamiento: no se pueden escribir o leer valores hasta que se
hayan leído o escrito todos los valores que caben en el buffer (más detalles a continuación). La
escritura y lectura de valores comienza de nuevo desde el principio del vector (índice 0). El
comportamiento es circular: cuando se llega al final del vector (su tamaño no cambia), se vuelve
al principio. El monitor proporciona tres métodos:
    - Leer_siguiente(): retorna el siguiente valor (entero) almacenado en el buffer. No se
       retornan todos los valores, sino que, en cada invocación del método, se devuelve el
       siguiente valor almacenado. Este método debe bloquear al hilo que lo invoca si no se han
       escrito todos los valores en el buffer.

   -   Escribir(): recibe un valor (entero), que debe ser almacenados en la siguiente posición
       del vector. Este método debe bloquear al hilo que lo invoca si ya se han escrito todos los
       valores en el buffer, pero todavía no han sido leídos.
   -   Terminar(): permite liberar correctamente todos los hilos que pudieran estar
       bloqueados en el monitor para que haya una finalización limpia del programa.

Realice un programa para probar el correcto funcionamiento del monitor, creando un hilo que
utilice el método Leer() y otro que utilice el método Escribir() para comprobar que funciona
correctamente. Por ejemplo, cree el Buffer_Compartido con tamaño 5, y luego haga que los hilos
que leen y escriben en el buffer invoquen los métodos 15 veces. Tenga cuidado al inicializar el
vector: vector<int> v(5) crea un vector de 5 elementos incializados a 0, mientras que vector<int>
v{5} crea un vector de 1 elemento inicializado al valor 5.

En la implementación de esta función debe asegurar que los mensajes que se imprimen por
consola están correctamente sincronizados para evitar el solapamiento. Solo puede utilizar la
funcionalidad proporcionada por la librería estándar del lenguaje C++.
```

Curso 2019-20

Ejercicios parcial PBO 2019-20 – 25 ejercicios

Ejercicio 1 (C++ básico y STL (strings, vector, map, set, flujos))

```
ENUNCIADOS DE EJERCICIOS DE PROGRAMACIÓN BASADA EN OBJETOS
```

Ejercicio 2 (C++ básico y STL (strings, vector, map, set, flujos))

```
EJERCICIO 1

Haga una función que calcule y retorne el resultado de aplicar la función matemática unión (un nuevo
conjunto que contiene todos los elementos no repetidos de los dos parámetros de entrada). La función
recibe como parámetros los dos conjuntos de entrada (modelados como set<int>). Supondremos que
los parámetros de entrada NO están vacíos. Haga también una función main con, al menos, un ejemplo
de utilización, donde además se impriman los conjuntos de entrada y salida. Añada todas las funciones
auxiliares que pudiera necesitar.
```

Ejercicio 3 (C++ básico y STL (strings, vector, map, set, flujos))

```
EJERCICIO 2

Haga una función que calcule y retorne el resultado de aplicar la función matemática intersección (un
nuevo conjunto que contiene solo los elementos que aparecen en ambos parámetros de entrada). La
función recibe como parámetros los dos conjuntos de entrada (modelados como set<int>).
Supondremos que los parámetros de entrada NO están vacíos. Haga también una función main con, al
menos, un ejemplo de utilización, donde además se impriman los conjuntos de entrada y salida. Añada
todas las funciones auxiliares que pudiera necesitar.
```

Ejercicio 4 (C++ básico y STL (strings, vector, map, set, flujos))

> EJERCICIO 3
>
> Haga una función que calcule y retorne el resultado de aplicar la función matemática diferencia (un nuevo conjunto que contiene solo los elementos que aparecen en un conjunto, pero no en el otro). La función recibe como parámetros los dos conjuntos de entrada (modelados como set<int>). Supondremos que los parámetros de entrada NO están vacíos. Haga también una función main con, al menos, un ejemplo de utilización, donde además se impriman los conjuntos de entrada y salida. Añada todas las funciones auxiliares que pudiera necesitar.

Ejercicio 5 (C++ básico y STL (strings, vector, map, set, flujos))

> EJERCICIO 4
>
> Haga una función que compruebe si dos conjuntos son subconjuntos (si todos los elementos de uno están también contenidos en el otro). La función recibe como parámetros los dos conjuntos de entrada (modelados como set<int>) y retorna un valor bool. Haga también una función main con dos ejemplos de utilización (uno con respuesta true y otro con respuesta false), donde además se impriman los conjuntos de entrada y salida. Añada todas las funciones auxiliares que pudiera necesitar.

Ejercicio 6 (C++ básico y STL (strings, vector, map, set, flujos))

> EJERCICIO 5
>
> Haga una función que reciba un valor de tipo string y devuelva un valor bool que indique si el parámetro contiene caracteres repetidos. Si el parámetro está vacío, debe retornar false. Obligatoriamente, debe utilizar la colección set<T> para implementar la solución. Haga una función main con, al menos, tres ejemplos de utilización: parámetro vacío, parámetro con caracteres repetidos y parámetro sin caracteres repetidos. Añada todas las funciones auxiliares que pudiera necesitar.

Ejercicio 7 (C++ básico y STL (strings, vector, map, set, flujos))

> EJERCICIO 6
>
> Haga una función que reciba un valor de tipo string y devuelva un valor bool que indique si el parámetro contiene caracteres repetidos. Si el parámetro está vacío, debe retornar false. Obligatoriamente, debe utilizar la colección vector<T> para implementar la solución. Haga una función main con, al menos, tres ejemplos de utilización: parámetro vacío, parámetro con caracteres repetidos y parámetro sin caracteres repetidos. Añada todas las funciones auxiliares que pudiera necesitar.

Ejercicio 8 (C++ básico y STL (strings, vector, map, set, flujos))

> EJERCICIO 7
>
> Haga una función que reciba un valor de tipo string y devuelva un valor bool que indique si el parámetro contiene caracteres repetidos. Si el parámetro está vacío, debe retornar false. Obligatoriamente, debe utilizar la colección map<T> para implementar la solución. Haga una función main con, al menos, tres ejemplos de utilización: parámetro vacío, parámetro con caracteres repetidos y parámetro sin caracteres repetidos. Añada todas las funciones auxiliares que pudiera necesitar.

Ejercicio 9 (C++ básico y STL (strings, vector, map, set, flujos))

> EJERCICIO 8
>
> Haga una función que reciba un valor de tipo string y devuelva un valor bool que indique si el parámetro contiene caracteres repetidos. Si el parámetro está vacío, debe retornar false. No puede utilizar ninguna colección (vector<T>, set<T>, map<T>, array<T,N>, ni array de C []) para implementar la solución. Haga una función main con, al menos, tres ejemplos de utilización: parámetro vacío, parámetro con caracteres repetidos y parámetro sin caracteres repetidos. Añada todas las funciones auxiliares que pudiera necesitar.

Ejercicio 10 (C++ básico y STL (strings, vector, map, set, flujos))

> EJERCICIO 9
>
> Se pide que haga una función que acepte un string y retorne un nuevo string que contenga la secuencia más larga de vocales consecutivas contenidas en el string. Por ejemplo, si la entrada fuera "abcdeaiefghija", la función debe retornar "eaie". Haga una función main con un ejemplo de utilización. Añada todas las funciones auxiliares que pudiera necesitar.

Ejercicio 11 (C++ básico y STL (strings, vector, map, set, flujos))

> EJERCICIO 10
>
> Se pide que haga una función que acepte un vector de enteros y retorne un nuevo vector que contenga la secuencia más larga de números pares consecutivos. Por ejemplo, si la entrada fuera {1,2,4,2,8,3,4,8,5,4,2,7}, la función debe retornar {2,4,2,8}. Haga una función main con un ejemplo de utilización. Añada todas las funciones auxiliares que pudiera necesitar.

Ejercicio 12 (C++ básico y STL (strings, vector, map, set, flujos))

> EJERCICIO 11
>
> Se pide que haga una función que acepte un string y un número entero, y que retorne un string que contenga únicamente las palabras cuya longitud es superior al indicado por el parámetro numérico. Se supone que las palabras están separadas por un espacio en blanco. No es necesario eliminar los signos de puntuación (punto, coma, etc.) que puedan aparecer. Por ejemplo, si la entrada fuera "uno dos tiempo. Real " y el número fuera 4, la función debe retornar "tiempo. Real ". Haga una función main con un ejemplo de utilización. Añada todas las funciones auxiliares que pudiera necesitar. Puede utilizar otras librerías de C++, como <sstream>, para procesar el string de entrada.

Ejercicio 13 (C++ básico y STL (strings, vector, map, set, flujos))

> EJERCICIO 12
>
> Se pide que haga una función que acepte un vector de enteros y retorne un nuevo vector que contenga únicamente los números primos (aquellos que son únicamente divisibles por sí mismos y por la unidad) almacenados en el parámetro. Haga una función main con un ejemplo de utilización en el que genere el vector de entrada de forma aleatoria, mediante una distribución uniforme (entre 0 y 200) de la librería <random>. Añada todas las funciones auxiliares que pudiera necesitar.
> Se deja como ejercicio.

Ejercicio 14 (C++ básico y STL (strings, vector, map, set, flujos))

> EJERCICIO 13
>
> Se pide que haga una función que acepte un string y un vector de enteros, ambos del mismo tamaño. La función debe retornar un nuevo string formado por tantas repeticiones de cada uno de los caracteres como indica los valores almacenados en el vector. Si no tienen el mismo tamaño, retornará el string vacío. Por ejemplo, si la entrada fuera "abcd" y {1,2,3,2}, la salida será "abbcccdd". Haga una función main con un ejemplo de utilización. Añada todas las funciones auxiliares que pudiera necesitar.

Ejercicio 15 (C++ básico y STL (strings, vector, map, set, flujos))

> EJERCICIO 14
>
> Se pide que haga una función que acepte un string y un vector de enteros (de tamaño menor que el string; si no, retorne string vacío), y que retorne un nuevo string resultado de eliminar del string que se pasa como parámetro las posiciones indicadas en el vector de enteros. Haga una función main con un ejemplo de utilización. Añada todas las funciones auxiliares que pudiera necesitar.

Ejercicio 16 (C++ básico y STL (strings, vector, map, set, flujos))

> EJERCICIO 15
>
> Se pide que haga una función que acepte un vector de enteros y retorne un nuevo vector que contenga únicamente aquellas secuencias de tres números tales que el tercero es la suma de los dos anteriores. Por ejemplo, si el vector de entrada es {1,2,3,2,2,3,5,8,5,4,2,7}, el de salida tiene que ser {1,2,3,2,3,5,3,5,8}. Haga una función main con un ejemplo de utilización. Añada todas las funciones auxiliares que pudiera necesitar.

Ejercicio 17 (C++ básico y STL (strings, vector, map, set, flujos))

> EJERCICIO 16
>
> Se pide que haga una función que acepte un vector de string y retorne otro vector que contiene solo los strings que comienzan o acaban en una letra mayúscula. Haga una función main con un ejemplo de utilización. Añada todas las funciones auxiliares que pudiera necesitar.

Ejercicio 18 (C++ básico y STL (strings, vector, map, set, flujos))

> EJERCICIO 17
>
> El sensor de distancia de la marca ACME genera una cadena de texto que tiene 181 valores de tipo double, separados por comas y con el siguiente formato: "v1,v2,v3,...v181\n₂. El sensor cubre un ángulo de 180o y mide, en cada punto de la cadena de texto, la distancia hasta el objeto que se encuentra a dicho ángulo (empezando por -90o). Haga una función que reciba un string como el descrito y retorne un valor que indique el ángulo y la distancia a que se encuentra el obstáculo más cercano (utilizando un struct). Haga una función main con un ejemplo de utilización en el que genere el string de entrada de forma aleatoria, mediante una distribución uniforme (entre 0 y 100) de la librería <random>. Añada todas las funciones auxiliares que pudiera necesitar. Puede utilizar otras librerías de C++, como <sstream>, para procesar el string de entrada.

Ejercicio 19 (C++ básico y STL (strings, vector, map, set, flujos))

> EJERCICIO 18
>
> El sensor de potencia de la marca ACME devuelve una cadena de texto con el siguiente formato "Amp= 'valorA' Volt= 'valorV'\n₂, donde los valores son de tipo double. Se pide que haga un programa que acepte un string formado por varias cadenas como la anterior (siempre acabadas en '\n₇), y devuelva un vector<double> que contenga el valor de la potencia (calculada como V*A). Un ejemplo del string de entrada es string pot {"Amp= 1.1 Volt= 220.1\n Amp= 1.7 Volt= 220.2\n₂. Haga una función main con un ejemplo de utilización. Añada todas las funciones auxiliares que pudiera necesitar. Puede utilizar otras librerías de C++, como <sstream>, para procesar el string de entrada.

Ejercicio 20 (C++ básico y STL (strings, vector, map, set, flujos))

> EJERCICIO 19
>
> Haga una función que reciba una string y un caracter de separación, y devuelva un vector<string> en el que se ha partido el string suministrado según el caracter de separación. El caracter de separación puede dejarlo o no, según le convenga. No olvide añadir el resto del string que pueda quedar, aunque no esté delimitado por el caracter de separación. Por ejemplo, si la entrada es "hola. ADIOS. Tiempo
>
> 33 2342@#¢..LALA", y el caracter de separación es el '.', el vector devuelto es: {"hola", " ADIOS", " Tiempo 33 2342@#¢", "", "LALA"}. Haga una función main con un ejemplo de utilización. Añada todas las funciones auxiliares que pudiera necesitar. Puede utilizar otras librerías de C++, como <sstream>, para procesar el string de entrada.

Ejercicio 21 (C++ básico y STL (strings, vector, map, set, flujos))

> EJERCICIO 20
>
> Haga una función que reciba una string y un numero entero positivo, y devuelva un vector<string> en el que se ha partido el string suministrado según la cantidad de palabras especificada por el número entero. Se supone que las palabras están separadas por un espacio en blanco. No es necesario eliminar los signos de puntuación (punto, coma, etc.) que puedan aparecer. No olvide añadir el resto del string que pueda quedar, aunque no esté delimitado por el caracter de separación. Por ejemplo, si la entrada es "hola. ADIOS. Tiempo 33 2342@#¢..LALA", y el número es 2, el vector devuelto es: {"hola. ADIOS.", " Tiempo 33", " 2342@#¢..LALA"}. Haga una función main con un ejemplo de utilización. Añada todas las funciones auxiliares que pudiera necesitar. Puede utilizar otras librerías de C++, como <sstream>, para procesar el string de entrada.

Ejercicio 22 (C++ básico y STL (strings, vector, map, set, flujos))

> EJERCICIO 21
>
> El sensor GPS de la marca ACME devuelve una cadena de texto con el siguiente formato "X: 'valorX' Y: 'valorY' Z: 'valorZ'\n₂. Se pide que haga un programa que acepte un string formado por varias cadenas como la anterior (siempre acabadas en '\n₇), y devuelva un vector de 3 valores de tipo double, que contenga la media de cada una de las tres coordenadas. Por ejemplo, si el string de entrada es string gps {"X: 1 Y: 2 Z: 3\n X: 10 Y: 20 Z: 30\nX: 100 Y: 200 Z: 300\n₂, la función debe devolver el vector con los valores {37 , 74, 111}. Haga una función main con un ejemplo de utilización. Añada todas las funciones auxiliares que pudiera necesitar. Puede utilizar otras librerías de C++, como <sstream>, para procesar el string de entrada.

Ejercicio 23 (C++ básico y STL (strings, vector, map, set, flujos))

> EJERCICIO 22
>
> El sistema de control de acceso de un colegio almacena el nombre del alumno y el retraso que ha tenido al llegar en una base de datos. Al final del mes, se genera un string con el formato (separado por comas)

> nombre y apellidos, retraso en minutos\n, siendo el retraso un valor entero. Una fila por alumno. El mismo alumno puede aparecer más de una vez. Un ejemplo de datos de entrada puede ser string alus {"Aitor Menta, 12, \nArmando Casitas, 20\n Aitor Menta, 5\n$_2$}. Se pide que haga una función que retorne un mapa de nombres y total de minutos de retraso acumulados durante el mes. Haga una función main con un ejemplo de utilización. Añada todas las funciones auxiliares que pudiera necesitar. Puede utilizar otras librerías de C++, como <sstream>, para procesar el string de entrada.

Ejercicio 24 (C++ básico y STL (strings, vector, map, set, flujos))

> EJERCICIO 23
>
> El sistema de pedidos de un supermercado recibe un string con la compra que solicita un cliente, con el formato (separado por comas) producto, precio unitario, cantidad\n. Un producto por fila. Se pide
>
> que haga una función que reciba este string y retorne el precio total de la compra. Haga una función main con un ejemplo de utilización. Añada todas las funciones auxiliares que pudiera necesitar. Puede utilizar otras librerías de C++, como <sstream>, para procesar el string de entrada.

Ejercicio 25 (C++ básico y STL (strings, vector, map, set, flujos))

> EJERCICIO 24
>
> La base de datos de un colegio retorna una cadena de texto con la siguiente información por cada alumno (formato separado por comas): "'nombre', 'apellido', 'dia', 'mes', 'año nacimiento' \n$_2$. Se pide que haga un programa que acepte un string formado por varias cadenas como la anterior (siempre acabadas en '\n$_7$) y que devuelva un vector que contenga la información de cada alumno en un struct. Suponga que los dos primeros campos de la cadena de entrada son de tipo string, y los tres siguiente son enteros. No se pide comprobar si la fecha de nacimiento es correcta, tan solo procesar la entrada. Un ejemplo de datos de entrada es string alus {"Aitor, Menta Menta, 1, 12, 2010\nArmando, Casitas, 2,3, 2210\n$_2$};. Haga una función main con un ejemplo de utilización. Añada todas las funciones auxiliares que pudiera necesitar. Puede utilizar otras librerías de C++, como <sstream>, para procesar el string de entrada.

Junio 2020 – 18 ejercicios

Ejercicio 1 (Herencia y polimorfismo)

> ENUNCIADOS DE EJERCICIOS DE PROGRAMACIÓN ORIENTADA A OBJETOS

Ejercicio 2 (Clases y objetos)

> EJERCICIO 1
>
> (30%) Haga una clase Asignatura con los siguientes atributos (elija los tipos de datos adecuados): título asignatura, título del grado en que se imparte, créditos teóricos y créditos prácticos. Complete la clase con las siguientes funciones:
>
> - Contructor para todos los atributos.
> - Constructor con todos los parámetros salvo los créditos, para los que se asume 3T+3P.
> - operator<< para imprimir por consola la información de la asignatura.
> - Funciones miembro para consultar todos los atributos.
>
> Añada la siguiente función, necesaria para que funcione la clase Asignatura:
>
> ```
> bool operator< (const Asignatura &lhs, const Asignatura &rhs) {
>   return lhs.get_titulo() < rhs.get_titulo();
> }
> ```
>
> (70%) Haga una clase Alumno con los siguientes atributos (elija los tipos de datos adecuados): nombre, apellidos, año de nacimiento, y un mapa Asignatura-nota, en el que se va a guardar el expediente del estudiante. Complete la clase con las siguientes funciones:
>
> - Contructor para todos los atributos, salvo el mapa.
> - operator<< para imprimir por consola la información personal del alumno y el número de créditos que ha aprobado.
> - Función para matricular al alumno de una asignatura. Si dicha asignatura ya estaba en el mapa, la función retorna 'false'. Si no, la añade con nota 0, y retorna 'true'.
> - Función para cambiar la nota de un alumno. Retorna 'false' si el alumno no estaba matriculado o si ya la había aprobado (nota mayor o igual que 5). En caso contrario, actualiza la nota y retorna 'true'.
>
> No es necesario que añada una función main.

Ejercicio 3 (Clases y objetos)

> EJERCICIO 2
>
> (30%) Haga una clase Asignatura con los siguientes atributos (elija los tipos de datos adecuados): título asignatura, código del grado en que se imparte, curso, total de créditos de la asignatura. Complete la clase con las siguientes funciones:
>
> - Contructor para todos los atributos.
> - Constructor con todos los parámetros salvo los créditos, el que se asume el valor 6.
> - operator<< para imprimir por consola la información de la asignatura.
> - Funciones miembro para consultar todos los atributos.
>
> Añada la siguiente función, necesaria para que funcione la clase Asignatura:
>
> ```
> bool operator< (const Asignatura &lhs, const Asignatura &rhs) {
>   return lhs.get_titulo() < rhs.get_titulo();
> }
> ```
>
> (70%) Haga una clase Alumno con los siguientes atributos (elija los tipos de datos adecuados): nombre, apellidos, código del grado en que se ha matriculado, y un mapa Asignatura-nota, en el que se va a guardar el expediente del estudiante. Complete la clase con las siguientes funciones:
>
> - Contructor para todos los atributos, salvo el mapa.
> - Función para matricular al alumno de una asignatura. Si dicha asignatura ya estaba en el mapa, o si el alumno no está matriculado en el grado en que se imparte, la función retorna 'false'. Si no, la añade con nota 0, y retorna 'true'.
> - Función para aprobar una asignatura. Se le pasa la asignatura y la nota. Si la nota es menor que cinco, el alumno ya había aprobado la asignatura, o no estaba matriculado, la función retorna 'false'. Si no, modifica la nota de la asignatura y retorna 'true.
> - Función que retorna la nota media del alumno (considerando únicamente las asignaturas aprobadas).
>
> No es necesario que añada una función main.

Ejercicio 4 (Clases y objetos)

> EJERCICIO 3
>
> (40%) Haga una clase Avion con los siguientes atributos (elija los tipos de datos adecuados cuando no se proporciona ninguno): tipo de avión (atributo de un tipo enumerado con tres valores {civil, militar, comercial}), coordenadas x, y , z del avión, código de identificación del avión. Complete la clase con las siguientes funciones:
>
> - Contructor para todos los atributos.
> - operator<< para imprimir por consola la información del Avión.
> - Funciones miembro para el valor tipo de avión y el identificador del Avión.
> - Función miembro que calcula y retorna la distancia entre el Avión y otro Avión que se pasa como parámetro (en el espacio tridimensional, la distancia sigue calculándose como la raíz cuadrada de los cuadrados de las diferencias de posición en los tres ejes).
>
> Añada la siguiente función, necesaria para que funcione la clase Avión:
>
> ```
> bool operator< (const Avion &lhs, const Avion &rhs) {
>     return lhs.get_id() < rhs.get_id();
> }
> ```
>
> (60%) Haga una clase Radar con tres colecciones para almacenar Aviones cuyo tipo es 'civil', Aviones cuyo tipo es 'militar', y Aviones cuyo tipo es 'comercial', sabiendo que no se puede almacenar dos veces el mismo Avión. Complete la clase con las siguientes funciones:
>
> - Función para añadir un Avión a la colección que gestiona el Radar. Si el Avión ya estaba, la función retorna 'false'. Si no, lo añade y retorna 'true'. Debe añadir el Avión a la colección correcta.
> - Función para eliminar un Avión de la colección que gestiona el Radar. Si el Avión está, lo elimina y retorna 'true'. Si no está, retorna 'false'.
> - Función que acepta como parámetros un Avión y una distancia, y retorna una colección con todos los Aviones que se encuentran a distancia menor que la suministrada, del Avión que se pasa como parámetro.
>
> Haga una pequeña demostración de la utilización de las clases en la función main.

Ejercicio 5 (Clases y objetos)

> EJERCICIO 4

(40%) Haga una clase Avion con los siguientes atributos (elija los tipos de datos adecuados cuando no se proporciona ninguno): amigo_enemigo (atributo de un tipo enumerado con tres valores {amigo, enemigo, desconocido}), coordenadas x, y , z del avión, código de identificación del avión. Complete la clase con las siguientes funciones:

- Contructor para todos los atributos.
- operator<< para imprimir por consola la información del Avión.
- Funciones miembro para el valor amigo_enemigo y el identificador del Avión.
- Función miembro que calcula y retorna la distancia entre el Avión y otro Avión que se pasa como parámetro (en el espacio tridimensional, la distancia sigue calculándose como la raíz cuadrada de los cuadrados de las diferencias de posición en los tres ejes).

Añada la siguiente función, necesaria para que funcione la clase Avión:

```
bool operator< (const Avion &lhs, const Avion &rhs) {
    return lhs.get_id() < rhs.get_id();
}
```

(60%) Haga una clase Radar con una colección para almacenar objetos del tipo Avion, sabiendo que no se puede almacenar dos veces el mismo Avión. Complete la clase con las siguientes funciones:

- Función para añadir un Avión a la colección que gestiona el Radar. Si el Avión ya estaba, la función retorna 'false'. Si no, lo añade y retorna 'true'.
- Función que retorna una colección con todos los Aviones del tipo 'amigo' que tiene el Radar. Si no hay ninguno, la función debe retornar una colección vacía.
- Función que acepta como parámetros un Avión y una distancia, y retorna una colección con todos los Aviones que se encuentran a distancia menor que la suministrada, del Avión que se pasa como parámetro.
- Función para eliminar un Avión de la colección que gestiona el Radar. Si el Avión está, lo elimina y retorna 'true'. Si no está, retorna 'false'.

Haga una pequeña demostración de la utilización de las clases en la función main.

Ejercicio 6 (Herencia y polimorfismo)

EJERCICIO 5

La empresa ACME le ha pedido que haga un programa para gestionar sus empleados y realizar el cálculo de sus nóminas a final de mes. De momento quiere solo un prototipo de la aplicación, en la que habrá una clase Empleado, que tendrá la información general sobre la persona, y una clase Empleado_Fijo, que hereda de Empleado, que tiene un sueldo fijo más las horas extra trabajadas. (40%) La clase Empleado tiene las siguientes características:

- Sus atributos son nombre, apellidos y DNI del Empleado. Haga un constructor que permita inicializar estos atributos.
- Implementa operator<< de forma polimórfica para imprimir la información del Empleado.
- Tiene un método virtual para que las clases hijas puedan calcular y retornar la nómina.

(60%) La clase Empleado_Fijo tiene las siguientes características:

- Hereda de la clase Empleado y añade como atributos el sueldo base del empleado, el precio al que se pagan las horas extra, y el número de horas extra realizadas en el mes.
- Haga un constructor para crear objetos de esta clase. Además de los valores para el Empleado, se deben suministrar valores para el sueldo base y el precio por hora extra realizada. Al crearse el objeto, las horas extra se inicializan a cero.
- Implementa la/s funcion/es necesarias para que operator<< funcione de forma polimórfica para imprimir la información del Empleado_Fijo.
- Implementa el método para calcular y retornar la nómina.
- Tiene un método para añadir horas extra. Este método comprueba que el número de horas extra que se pasa como parámetro es positivo, y limita el máximo de horas extras trabajadas a 15.

Haga una pequeña demostración de la utilización de las clases en la función main.

Ejercicio 7 (Herencia y polimorfismo)

EJERCICIO 6

La empresa ACME le ha pedido que haga un programa para gestionar sus empleados y realizar el cálculo de sus nóminas a final de mes. De momento quiere solo un prototipo de la aplicación, en la que habrá una clase Empleado, que tendrá la información general sobre la persona, y una clase Empleado_Horas, que hereda de Empleado, cuya nómina depende de la cantidad total de horas que ha trabajado ese mes. (40%) La clase Empleado tiene las siguientes características:

- Sus atributos son nombre, apellidos y DNI del Empleado. Haga un constructor que permita inicializar estos atributos.
- Implementa operator<< de forma polimórfica para imprimir la información del Empleado.

- Tiene un método virtual para que las clases hijas puedan calcular y retornar la nómina.

(60%) La clase Empleado_Horas tiene las siguientes características:

- Hereda de la clase Empleado y añade como atributos el número de horas trabajadas y el precio al que se pagan las horas.

- Haga un constructor para crear objetos de esta clase. Además de los valores para el Empleado, se deben suministrar valor para el precio por hora trabajada. Al crearse el objeto, las horas trabajadas se inicializan a cero.
- Implementa la/s funcion/es necesarias para que operator<< funcione de forma polimórfica para imprimir la información del Empleado_Horas.
- Implementa el método para calcular y retornar la nómina.
- Tiene un método para añadir horas trabajadas. Este método solo suma las horas que se pasan como parámetro si están comprendidas entre 1 y 8. También limita el número máximo de horas trabajadas a 160.

Haga una pequeña demostración de la utilización de las clases en la función main.

Ejercicio 8 (Herencia y polimorfismo)

EJERCICIO 7

La empresa ACME le ha pedido que haga un programa para gestionar sus empleados y realizar el cálculo de sus nóminas a final de mes. De momento quiere solo un prototipo de la aplicación, en la que habrá una clase Empleado, que tendrá la información general sobre la persona, y una clase Empleado_Comision, que hereda de Empleado, que tiene un sueldo fijo más una comisión sobre el total de las ventas realizadas durante el mes. (40%) La clase Empleado tiene las siguientes características:

- Sus atributos son nombre, apellidos y DNI del Empleado. Haga un constructor que permita inicializar estos atributos.
- Implementa operator<< de forma polimórfica para imprimir la información del Empleado.
- Tiene un método virtual para que las clases hijas puedan calcular y retornar la nómina.

(60%) La clase Empleado_Comision tiene las siguientes características:

- Hereda de la clase Empleado y añade como atributos el sueldo base, el valor total de ventas que ha realizado, y el porcentaje de comisión que cobra a fin de mes.
- Haga un constructor para crear objetos de esta clase. Además de los valores para el Empleado, se deben suministrar valores para el sueldo base y la comisión de ventas. Al crearse el objeto, el total de ventas se inicializa a cero.
- Implementa la/s funcion/es necesarias para que operator<< funcione de forma polimórfica para imprimir la información del Empleado_Comision.
- Implementa el método para calcular y retornar la nómina.
- Tiene un método para añadir el valor de ventas realizada. Este método comprueba que dicho valor es positivo antes de incrementar el total, y después retorna 'true'. Si es negativo, retorna 'false' inmediatamente.

Haga una pequeña demostración de la utilización de las clases en la función main.

Ejercicio 9 (Monitores y variables condición)

EJERCICIO 8

La empresa ACME le ha pedido que haga un programa para gestionar sus empleados y realizar el cálculo de sus nóminas a final de mes. De momento quiere solo un prototipo de la aplicación, en la

que habrá una clase Empleado, que tendrá la información general sobre la persona, y una clase Empleado_Turnos, que hereda de Empleado, el precio de cada turno de trabajo, mañana, tarde y noche, es distinto.

(40%) La clase Empleado tiene las siguientes características:

- Sus atributos son nombre, apellidos y DNI del Empleado. Haga un constructor que permita inicializar estos atributos.
- Implementa operator<< de forma polimórfica para imprimir la información del Empleado.
- Tiene un método virtual para que las clases hijas puedan calcular y retornar la nómina.

(60%) La clase Empleado_Turnos tiene las siguientes características:

- Hereda de la clase Empleado y añade como atributos el número de veces que ha trabajado en cada turno (mañana, tarde y noche), y el precio al que se paga la jornada trabajada en cada uno de ellos.
- Haga un constructor para crear objetos de esta clase. Además de los valores para el Empleado,

```
        se deben suministrar el precio al que se paga la jornada en cada turno. Al crearse el objeto, las
        horas trabajadas en cada turno se inicializan a cero.
    •   Implementa la/s funcion/es necesarias para que operator<< funcione de forma polimórfica
        para imprimir la información del Empleado_Horas.
    •   Implementa el método para calcular y retornar la nómina.
    •   Tiene un método para añadir el turno en que ha trabajado.

Haga una pequeña demostración de la utilización de las clases en la función main.

ENUNCIADOS DE EJERCICIOS DE PROGRAMACIÓN CONCURRENTE
```

Ejercicio 10 (Monitores y variables condición)

```
EJERCICIO 1

Implemente una clase que haga de barrera para un grupo de hilos. Una barrera es un objeto que se
utiliza para bloquear el avance de un número de hilos y conseguir sincronización entre ellos. Es similar
a la barrera de un paso a nivel, por ejemplo, que bloquea el paso de personas y coches hasta que se
levanta. Se pide que implemente una clase Barrera_Manual, que proporciona un método para 'abrir'
la barrera y otro para que los hilos esperen su 'apertura' (queden bloqueados). Implemente la barrera
utilizando mutex y variables de condición. Haga una función main de prueba, en la que cree varios
hilos con los que demostrar el uso de la clase Barrera_Manual. Por ejemplo, cree 5 hilos que quedan
bloqueados, mientras que en el main espera a que el usuario introduzca un número cualquiera para
'abrir' la barrera. Para que se haga una idea, una posible ejecución podría ser:

Pulsa una tecla para abrir B1 ...
[Hilo 4] entrando en b1
[Hilo 3] entrando en b1
[Hilo 2] entrando en b1
[Hilo 0] entrando en b1
[Hilo 1] entrando en b1
a // valor introducido por teclado, con cin >> XX, para que, después de leerlo, se
'abra' la barrera
[Hilo 4] terminando
[Hilo 3] terminando
[Hilo 2] terminando
[Hilo 0] terminando
[Hilo 1] terminando
Main Manual ha terminado
```

Ejercicio 11 (Monitores y variables condición)

```
EJERCICIO 2

Implemente una clase que haga de barrera para un grupo de hilos. Una barrera es un objeto que se
utiliza para bloquear el avance de un número de hilos y conseguir sincronización entre ellos. Es similar
a la barrera de un paso a nivel, por ejemplo, que bloquea el paso de personas y coches hasta que se
levanta. Se pide que implemente una clase Barrera_Grupo, que proporciona un método que deja a los
hilos bloqueados, a la espera de su 'apertura'. La clase Barrera_Grupo proporciona un constructor que
acepta como parámetro el número de hilos que deben estar bloqueados en la barrera antes de que ésta
se 'abra' automáticamente. Implemente la barrera utilizando mutex y variables de condición. Haga una
función main de prueba, en la que cree varios hilos con los que demostrar el uso de la clase
Barrera_Grupo. Por ejemplo, cree una Barrera_Grupo con parámetro 5, y 5 hilos, que irán bloqueando
en la barrera hasta que estén los 5, momento en que se 'abrirá' automáticamente. Para que se haga una
idea, una posible ejecución podría ser:

[Hilo 4] entrando en b1

[Hilo 3] entrando en b1
[Hilo 2] entrando en b1
[Hilo 1] entrando en b1
[Hilo 0] entrando en b1

[Hilo 0] terminando
[Hilo 4] terminando
[Hilo 3] terminando
[Hilo 2] terminando
[Hilo 1] terminando
Main Grupo ha terminado
```

Ejercicio 12 (Monitores y variables condición)

```
EJERCICIO 3

Implemente una clase que haga de barrera para un grupo de hilos. Una barrera es un objeto que se
utiliza para bloquear el avance de un número de hilos y conseguir sincronización entre ellos. Es similar
a la barrera de un paso a nivel, por ejemplo, que bloquea el paso de personas y coches hasta que se
levanta. Se pide que implemente una clase Barrera_Temporizada, que proporciona un método que
deja a los hilos bloqueados, a la espera de su 'apertura'. La clase Barrera_Temporizada proporciona un
constructor que acepta como parámetro el número de segundos que debe permanecer 'cerrada', esto es,
bloqueando a cualquier hilo que invoque dicho método. Una vez pase el tiempo especificado en el
constructor, la barrera se abre automáticamente. Implemente la barrera utilizando mutex y variables
de condición. También necesitará que la clase tenga como atributo un hilo, que será el que realiza la
espera y posteriormente es encarga de 'abrir' la barrera. Implemente la función que ejecuta el hilo como
una función miembro de la clase, y utilice el constructor de hilo al que se le pasa el método y el objeto
(this) que debe ejecutar. Haga una función main de prueba, en la que cree varios hilos con los que
demostrar el uso de la clase Barrera_Temporizada. Por ejemplo, cree una Barrera_Temporizada con
parámetro 3, y 5 hilos, que irán bloqueando en la barrera hasta que pasen 3 segundos, momento en que
se abrirá automáticamente. Para que se haga una idea, una posible ejecución podría ser:

[Hilo 4] entrando en b1
[Hilo 3] entrando en b1
[Hilo 2] entrando en b1
[Hilo 1] entrando en b1
[Hilo 0] entrando en b1

[Hilo 0] terminando
[Hilo 4] terminando
[Hilo 3] terminando
[Hilo 2] terminando
[Hilo 1] terminando
Main Grupo ha terminado
```

Ejercicio 13 (C++ básico y STL (strings, vector, map, set, flujos))

```
EJERCICIOS PARCIAL PBO

EJERCICIO 1

Una biblioteca pública le ha pedido que haga un programa para mostrar a los usuarios algunas
estadísticas de los libros que tienen en sus fondos: libro con mayor y menor tiempo de espera, y tiempo
de espera medio para recibir en préstamo un libro. El punto de inicio es una cadena de texto que genera
el sistema, formada por múltiples líneas, cada una de ellas con el siguiente formato: "'id libro' 'días
prestado'\n₂ (valores separados por un espacio en blanco y con un retorno de carro al final). Un mismo
libro puede aparecer más de una vez si tiene préstamos pendientes. Un ejemplo de la cadena de texto
puede ser string ejemplo {"1 5 2 4 3 3 1 4 2 2 3 2 "};, y la salida en este caso sería
Max espera: 1(9 dias)
Min espera: 3(5 dias)
Espera media: 6.66667
Cree todas las funciones que considere necesarias y una función main para probar la aplicación.
```

Ejercicio 14 (Monitores y variables condición)

```
EJERCICIO 2

Una empresa de transporte recibe, al final de cada día, una cadena de texto con los envíos que tiene
que realizar al día siguiente, de manera que tenga tiempo para planificar su flota de vehículos. Esta
cadena está formada por múltiples líneas, que tienen el siguiente formato: "'ciudad' 'kgs'\n₂ (valores
separados por un espacio en blanco y con un retorno de carro al final), donde ciudad es un string y kgs
el número de kgs que tiene que llevar a dicha ciudad. Una misma ciudad puede aparecer varias veces,
ya que la empresa de transporte proporciona sus servicios a varias empresas productoras de bienes. Un
ejemplo de la cadena de texto puede ser string mercancias {"Murcia 100 Cartagena 200
Alicante 300 Murcia 400 Murcia 300 Cartagena 400 "};. Por otro lado, la empresa de transporte
tiene dos tipos de vehículos: uno que puede llevar hasta 200kg y otro hasta 600kg. Se pide que haga
una función main, y todas las funciones auxiliares que pueda necesitar, que calcule cuántos vehículos
de cada tipo se necesitan para cada ciudad, suponiendo que la empresa quiere minimizar el número
total de vehículos que utiliza y que un mismo vehículo no puede repartir mercancías a más de una
ciudad. Para el ejemplo, la salida sería:
Ciudad Alicante -> 300 kgs
Ciudad Cartagena -> 600 kgs
Ciudad Murcia -> 800 kgs
Alicante
Vehículos 600kg -> 0
Vehículos 200kg -> 2
Cartagena
Vehículos 600kg -> 1
Vehículos 200kg -> 0
```

```
Murcia
Vehículos 600kg -> 1
Vehículos 200kg -> 1
```

Ejercicio 15 (C++ básico y STL (strings, vector, map, set, flujos))

```
EJERCICIO 3

La empresa de electrónica ACME recibe, al principio de cada día, un listado con los componentes
electrónicos que le han enviado para fabricar ordenadores. Esta cadena está formada por múltiples
líneas, que tienen el siguiente formato: "'componente' 'unidades'\n₂ (valores separados por un espacio

en blanco y con un retorno de carro al final), donde componente es un string (con valores posibles
"CPU", "RAM", "PCB") y unidades es un número entero que representa la cantidad de unidades
disponibles. Un mismo componente puede aparecer varias veces, ya que puede ser suministrado por
distintos proveedores. Un ejemplo de la cadena de texto puede ser string piezas_str {"CPU 1 RAM
3 PCB 2 CPU 3 RAM 2 PCB 5 "};. La empresa está interesada en conocer cuántos ordenadores puede
construir con dichos componentes, y le pide que haga una función que reciba como parámetros la
cantidad de CPUs, módulos de RAM, y tarjetas PCB que tiene cada ordenador, e imprima por consola
cuántos ordenadores con dichas características se podrían construir. Por ejemplo, la salida de la
llamada a la función cuantos_pcs(1,1,1) sería:
Componente: CPU -> 4 uds
Componente: PCB -> 7 uds
Componente: RAM -> 5 uds
Se pueden construir 4 equipos con las piezas disponibles
Y la salida por consola de la llamada a la función cuantos_pcs(1,2,1) sería:
Componente: CPU -> 4 uds
Componente: PCB -> 7 uds
Componente: RAM -> 5 uds
Se pueden construir 2 equipos con las piezas disponibles
Haga una función main, y todas las funciones auxiliares que pueda necesitar para resolver este
ejercicio.
```

Ejercicio 16 (C++ básico y STL (strings, vector, map, set, flujos))

```
EJERCICIO 4

La empresa de inversiones ACME le ha pedido que haga un programa para detectar empresas no
rentables en su cartera de inversiones, de forma que puedan vender sus acciones tras las crisis del
COVID-19. Para ello, los analistas quieren ser capaces de analizar las secuencias de valores de las
acciones de diferentes empresas que van generando, buscando aquellas que están experimentado un
decrecimiento sostenido en el tiempo. Suponga que el punto de entrada es un mapa, con la secuencia
de cotizaciones de múltiples empresas, una por clave. Suponga también que estos datos ya están
disponibles. El siguiente es un ejemplo, aunque podría tener cualquier número de entidades (en lugar
de solo 3) y la secuencia de cotizaciones podría ser distinta (7 valores por entidad):
map<string, vector<double>> cotizaciones_acciones {
{"Daily Bugle", {11,22,33,44,55,66,77}},
{"Cyberdyne systems", {41,52,62,73,82,15,36}},
{"Industrias Stark", {5,16,17,51,33,31,14}}
};
(30%) Le piden una función comprobar_datos, que retorna un valor 'true' si todas las empresas
tienen el mismo número de elementos, y 'false' en caso contrario. Esto es imprescindible para que los
resultados del análisis sean significativos. En los datos de ejemplo, las 3 empresas tienen el mismo
número de valores: 7.
(70%) Le piden una función mayor_secuencia_decreciente, que retorna el nombre de la empresa
que tiene la secuencia de valores estrictamente decreciente más larga de todas las empresas analizadas.
Una secuencia es estrictamente decreciente si cada valor de la misma es mayor que el siguiente.
Aplicada a los datos de ejemplo, esta función debe retornar "Industrias Stark", puesto que es la empresa
que tiene la secuencia de valores decrecientes más larga.
Haga una pequeña demostración de la utilización de estas funciones en la función main. Puede, por
supuesto, incluir nuevas funciones auxiliares si así lo considera necesario.
```

Ejercicio 17 (C++ básico y STL (strings, vector, map, set, flujos))

```
EJERCICIO 5

La empresa de inversiones ACME le ha pedido que haga un programa para detectar empresas no
rentables en su cartera de inversiones, de forma que puedan vender sus acciones tras las crisis del
COVID-19. Para ello, los analistas quieren ser capaces de analizar las secuencias de valores de las
acciones de diferentes empresas que van generando, buscando aquellas que están experimentado un
decrecimiento sostenido en el tiempo. Suponga que el punto de entrada es un vector, con la secuencia
de cotizaciones de múltiples empresas, una por fila. Suponga también que estos vectores ya están
disponibles. El siguiente es un ejemplo, aunque podría tener cualquier número de entidades (en lugar
```

> de solo 3) y la secuencia de cotizaciones podría ser distinta (7 valores por entidad):
> vector<vector<double>> cotizaciones_acciones {
> {1,2,3,4,5,6,7},
> {4,5,6,7,8,5,6},
> {5,6,7,4,3,2,1}
> };
> (30%) Le piden una función comprobar_datos, que retorna un valor 'true' si todas las filas del vector tienen el mismo número de elementos, y 'false' en caso contrario. Esto es imprescindible para que los resultados del análisis sean significativos. En los datos de ejemplo, las 3 empresas tienen el mismo número de valores: 7.
> (70%) Le piden una función mayor_secuencia_decreciente, que retorna el índice (posición) de la empresa que tiene la secuencia de valores estrictamente decreciente más larga de todas las empresas analizadas. Una secuencia es estrictamente decreciente si cada valor de la misma es mayor que el siguiente. Aplicada a los datos de ejemplo, esta función debe retornar 2.
> Haga una pequeña demostración de la utilización de estas funciones en la función main. Puede, por supuesto, incluir nuevas funciones auxiliares si así lo considera necesario.

Ejercicio 18 (C++ básico y STL (strings, vector, map, set, flujos))

> EJERCICIO 6
>
> La empresa de inversiones ACME le ha pedido que haga un programa para detectar empresas rentables en las que invertir tras las crisis del COVID-19. Para ello, los analistas quieren ser capaces de analizar las secuencias de valores de las acciones de diferentes empresas que van generando, buscando aquellas que están experimentado un mayor crecimiento sostenido. Suponga que el punto de entrada es un mapa, con la secuencia de cotizaciones de múltiples empresas, una por clave. Suponga también que estos datos ya están disponibles. El siguiente es un ejemplo, aunque podría tener cualquier número de entidades (en lugar de solo 3) y la secuencia de cotizaciones podría ser distinta (7 valores por entidad):
> map<string, vector<double>> cotizaciones_acciones {
> {"Daily Bugle", {11,22,33,44,55,66,77}},
> {"Cyberdyne Systems", {41,52,62,73,82,15,36}},
> {"Industrias Stark", {5,16,17,11,23,31,14}}
> };
> (30%) Le piden una función comprobar_datos, que retorna un valor 'true' si todas las empresas tienen el mismo número de elementos, y 'false' en caso contrario. Esto es imprescindible para que los resultados del análisis sean significativos. En los datos de ejemplo, las 3 empresas tienen el mismo número de valores: 7.
> (70%) Le piden una función mayor_secuencia_creciente, que retorna el nombre de la empresa que tiene la secuencia de valores estrictamente creciente más larga de todas las empresas analizadas. Una secuencia es estrictamente creciente si cada valor de la misma es menor que el siguiente. Aplicada a los datos de ejemplo, esta función debe retornar "Daily Bugle", puesto que es la empresa que tiene la secuencia de valores crecientes más larga.
>
> Haga una pequeña demostración de la utilización de estas funciones en la función main. Puede, por supuesto, incluir nuevas funciones auxiliares si así lo considera necesario.
>
> EJERCICIO7
>
> La empresa de inversiones ACME le ha pedido que haga un programa para detectar empresas rentables en las que invertir tras las crisis del COVID-19. Para ello, los analistas quieren ser capaces de analizar las secuencias de valores de las acciones de diferentes empresas que van generando, buscando aquellas que están experimentado un mayor crecimiento sostenido. Suponga que el punto de entrada es un vector, con la secuencia de cotizaciones de múltiples empresas, una por fila. Suponga también que estos vectores ya están disponibles. El siguiente es un ejemplo, aunque podría tener cualquier número de entidades (en lugar de solo 3) y la secuencia de cotizaciones podría ser distinta (7 valores por entidad):
> vector<vector<double>> cotizaciones_acciones {
> {1,2,3,4,5,6,7},
> {4,5,6,7,8,5,6},
> {5,6,7,4,2,3,4}
> };
> (30%) Le piden una función comprobar_datos, que retorna un valor 'true' si todas las filas del vector tienen el mismo número de elementos, y 'false' en caso contrario. Esto es imprescindible para que los resultados del análisis sean significativos. En los datos de ejemplo, las 3 empresas tienen el mismo número de valores: 7.
> (70%) Le piden una función mayor_secuencia_creciente, que retorna el índice (posición) de la empresa que tiene la secuencia de valores estrictamente creciente más larga de todas las empresas analizadas. Una secuencia es estrictamente creciente si cada valor de la misma es menor que el siguiente. Aplicada a los datos de ejemplo, esta función debe retornar 0, puesto que la primera empresa tiene la secuencia de valores crecientes más larga.
> Haga una pequeña demostración de la utilización de estas funciones en la función main. Puede, por supuesto, incluir nuevas funciones auxiliares si así lo considera necesario.

Septiembre 2020 – 7 ejercicios

Ejercicio 1 (Herencia y polimorfismo)

> ENUNCIADOS DE EJERCICIOS DE PROGRAMACIÓN ORIENTADA A OBJETOS

Ejercicio 2 (Herencia y polimorfismo)

> EJERCICIO 1
>
> (20%) Haga una clase Figura, que será la base de una jerarquía de herencia, con un atributo 'id' de tipo string y las siguientes funciones miembro:
>
> - Contructor para el atributo.
> - Funciones virtuales para calcular y retornar el área y el perímetro de una figura. En esta clase, estas funciones pueden retornar el valor 0.
> - Función operator<< polimórfica para imprimir por consola la información de la figura y de todas las clases hija.
>
> (20%) Implemente la clase Circulo, que hereda de Figura, y tiene como atributo su radio. Implemente el constructor del círculo, las funciones para calcular área y perímetro y para poder imprimir correctamente un círculo. Utilice la constante M_PI definida en la librería <cmath>. Un ejemplo de salida por consola puede ser:
>
> ```
> ===== C1 =====
> Circulo de radio 2
> ```
>
> (20%) Implemente la clase Rectangulo, que hereda de Figura, y tiene como atributos su base y altura. Implemente el constructor del rectángulo, las funciones para calcular área y perímetro y para poder imprimir correctamente un rectángulo. Un ejemplo de salida por consola puede ser:
>
> ```
> ===== R1 =====
> Rectángulo de 2 x 3
> ```
>
> (40%) Implemente la clase Agrupacion, que hereda de Figura, y tiene como atributo un vector de punteros a figuras (para que el polimorfismo funcione correctamente), de forma que esta clase sea un contenedor recursivo. Implemente el constructor de la agrupación (solo debe inicializar el 'id'). En cuanto a las funciones para calcular área y perímetro, éstas calculan y retornan el sumatorio de las áreas o perímetros de todas las figuras que contiene la agrupación. La función que imprime la agrupación debe imprimir los datos de todas las figuras que contiene. Un ejemplo de salida por consola puede ser:
>
> ```
> ===== Ag1 =====
> Agrupación de 3 elementos:
> ===== C1 =====
> Circulo de radio 2
> ===== R1 =====
> Rectángulo de 2 x 3
> ===== C2 =====
> Circulo de radio 3
> ```
>
> Además, la agrupación define una función para añadir un objeto de cualquier tipo de la jerarquía de figuras al vector (comprobando que no se añade a sí misma) y una función que retorna un vector con todos los círculos que contiene la agrupación (añadiendo también, de forma recursiva, los círculos que pudieran tener las agrupaciones anidadas).
>
> No es necesario que añada una función main.

Ejercicio 3 (Monitores y variables condición)

> EJERCICIO 2
>
> (20%) Una empresa le ha pedido un prototipo de una aplicación para realizar la gestión de proyectos. Haga una clase Actividad, que será la base de una jerarquía de herencia, con dos atributos de tipo string para almacenar el 'id' y la 'descripcion' de la actividad y las siguientes funciones miembro:
>
> - Constructor para los atributos.
> - Función virtual para calcular y retornar la duración de las actividades (la duración será un valor entero). En esta clase, esta función puede retornar el valor 0.
> - Función operator<< polimórfica para imprimir por consola la información de la figura y de todas las clases hija.
>
> (10%) Implemente la clase Tarea, que hereda de Actividad, y tiene como atributo su duración (valor entero). Implemente el constructor de la tarea y las funciones polimórficas (para calcular y retornar la duración de la tarea y para poder imprimir correctamente una tarea). Un ejemplo de salida por consola puede ser:

```
=====[ t2 ]=====
"La segunda tarea que hay que hacer"
Duración estimada: 22

(20%) Implemente la clase Tarea_Compuesta, que hereda de Actividad, y tiene como atributo un
vector de punteros a Actividad (para que el polimorfismo funcione correctamente), de forma que esta
clase sea un contenedor recursivo. Implemente el constructor (solo debe inicializar el 'id' y 'desc' del
padre) y una función para añadir un objeto de cualquier tipo de la jerarquía de actividades al vector
(comprobando que no se añade a sí misma).

(25%) Implemente la clase Tareas_Paralelas, que hereda de Tarea_Compuesta, y que no tiene
atributos adicionales. Implemente el constructor (solo debe inicializar el 'id' y 'desc' del padre) y las
dos funciones polimórficas: para calcular y retornar la duración de esta tarea (el máximo de las
duraciones de todas las tareas que tiene el vector de actividades) y para realizar la impresión por
consola de todas las actividades que contiene esta tarea paralela. Un ejemplo de salida por consola
puede ser (en este caso, la duración de la tarea paralela sería 22, el máximo de las 2 tareas que la
integran):

=====[ par1 ]=====
"Primera tarea paralela"
2 tarea/s paralelas:
=====[ t1 ]=====
"La primera tarea que hay que hacer"
Duración estimada: 11
=====[ t2 ]=====
"La segunda tarea que hay que hacer"
Duración estimada: 22

(25%) Implemente la clase Tareas_Secuenciales, que hereda de Tarea_Compuesta, y que no tiene
atributos adicionales. Implemente el constructor (solo debe inicializar el 'id' y 'desc' del padre) y las
dos funciones polimórficas: para calcular y retornar la duración de esta tarea (el sumatorio de las
duraciones de todas las tareas que tiene el vector de actividades) y para realizar la impresión por
consola de todas las actividades que contiene esta tarea paralela. Un ejemplo de salida por consola
puede ser (en este caso, la duración de la tarea secuencial sería 33, la suma de las 2 tareas que la
integran):

=====[ seq1 ]=====
"Primera tarea paralela"
2 tarea/s secuenciales:
=====[ t1 ]=====
"La primera tarea que hay que hacer"
Duración estimada: 11
=====[ t2 ]=====
"La segunda tarea que hay que hacer"
Duración estimada: 22

No es necesario que añada una función main.

ENUNCIADOS DE EJERCICIOS DE PROGRAMACIÓN CONCURRENTE
```

Ejercicio 4 (Monitores y variables condición)

```
EJERCICIO 1

Se quiere realizar un programa concurrente en que exista un hilo que genera valores enteros y varios
hilos que los leen. El diseño del programa debe evitar que el hilo que genera los datos bloquee, y
asegurar que hasta que todos los hilos 'lectores' hayan leído un dato, no se proporcionan nuevos datos
aunque un hilo lector vuelva a solicitarlo (debe quedar bloqueado). Para ello, se pide crear una clase
Almacen que sirva como buffer (complejo). Un objeto de esta clase será compartido entre el hilo que
genera los datos y los hilos que los leen. Este Almacen debe tener los atributos y las funciones
necesarias para implementar la funcionalidad descrita, y ser desarrollado como un monitor (es decir,
conteniendo las primitivas de sincronización y exclusión mutua necesarias). Algunos consejos para su
implementación:

   • Como atributos, debe tener un vector<int> para guardar los valores generados, más un
     índice que indica qué posición del vector se está enviando a los hilos lectores. Para controlar
     qué hilos han leído los datos, y los que están pendiente de hacerlo, puede añadir un mapa
     std::thread::id, bool. thread::id es un tipo de dato que representa el identificador que
     tiene un hilo tras ser creado por el sistema operativo, y se puede obtener invocando la función
     std::this_thread::get_id().
   • Como funciones miembro, puede implementar una función para añadir un valor al vector (será
     utilizada por el hilo que genera valores), una función para añadir el id de un hilo lector al mapa
     de lectores, y una función para leer el siguiente valor (esta función podrá bloquear al hilo
     invocante dependiendo de si ya había leído el valor anteriormente o no, según se ha descrito).
```

```
    Estas funciones deben utilizar las primitivas de sincronización adecuadas para asegurar que no
    existen problemas en la concurrencia de uso del Almacen.
  • Las condiciones de sincronización, que provocan que algunos de los hilos lectores bloqueen,
    son: a) que un hilo intente leer un valor nuevo sin que el resto de hilos lectores hayan leído el
    valor actual, y b) que no haya valores nuevos en el Almacen.
  • Añada una función miembro para cerrar el Almacen. Esta función será invocada antes de que
    termine el programa y su objetivo es desbloquear todos los hilos que pudieran estar bloqueados
    en las primitivas de sincronización del Almacen y asegurar una finalización ordenada y
    correcta de todos los hilos del programa.

El hilo que genera valores puede implementarse con un simple bucle que inserta en el Almacen valores
del 0 al 6. Esta función recibe una referencia al objeto Almacen compartido entre todos los hilos.

La función que implementa los hilos lectores recibe como parámetros un identificador (string), el
tiempo que debe esperar entre lecturas sucesivas del Almacen, una referencia al objeto Almacen
compartido entre los hilos, y una referencia a una variable que controla cuándo debe terminar el hilo
lector.

Haga un ejemplo de utilización creando un Almacen, un hilo generador y tres hilos lectores. Una
posible ejecución podría ser la siguiente:

LECTOR 2 -[ comienza
LECTOR 1 -[ comienza
LECTOR 3 -[ comienza
LECTOR 1 -[ lee 0
LECTOR 2 -[ lee 0
LECTOR 3 -[ lee 0
LECTOR 1 -[ lee 1
LECTOR 2 -[ lee 1
LECTOR 3 -[ lee 1
LECTOR 1 -[ lee 2
LECTOR 2 -[ lee 2
LECTOR 3 -[ lee 2
LECTOR 1 -[ lee 3
LECTOR 2 -[ lee 3
LECTOR 3 -[ lee 3
LECTOR 1 -[ lee 4
LECTOR 2 -[ lee 4
LECTOR 3 -[ lee 4
LECTOR 1 -[ lee 5
LECTOR 2 -[ lee 5
LECTOR 3 -[ lee 5
LECTOR 1 -[ lee 6
LECTOR 2 -[ lee 6
LECTOR 3 -[ lee 6
LECTOR 1 -[ termina
LECTOR 2 -[ termina
LECTOR 3 -[ termina
```

Ejercicio 5 (Monitores y variables condición)

```
EJERCICIO 2

Se quiere realizar un programa concurrente en que exista un hilo que genera valores enteros y varios
hilos que los leen. El diseño del programa debe asegurar que el hilo que genera valores no los puede
insertar uno nuevo hasta que todos los hilos 'lectores' hayan leído el valor anterior, y que los hilos
lectores no pueden leer dos veces el mismo valor, aunque vuelvan a solicitarlo (debe quedar
bloqueado). Para ello, se pide crear una clase Almacen que implemente este comportamiento. Un
objeto de esta clase será compartido entre el hilo que genera los datos y los hilos que los leen. Este
Almacen debe tener los atributos y las funciones necesarias para implementar la funcionalidad descrita,
y ser desarrollado como un monitor (es decir, conteniendo las primitivas de sincronización y exclusión
mutua necesarias). Algunos consejos para su implementación:

  • Como atributos, debe tener un int para guardar el valor generado, más un conjunto de valores
    bool para gestionar su comportamiento. Para controlar qué hilos han leído los datos, y los que
    están pendiente de hacerlo, puede añadir un mapa std::thread::id, bool. thread::id es un
    tipo de dato que representa el identificador que tiene un hilo tras ser creado por el sistema
    operativo, y se puede obtener invocando la función std::this_thread::get_id().
  • Como funciones miembro, puede implementar una función para añadir un valor al Almacen
    (será utilizada por el hilo que genera valores, y deberá bloquear al hilo generador dependiendo
    de si todos los hilos lectores han leído el valor o no, según se ha descrito), una función para
    añadir el id de un hilo lector al mapa de lectores, y una función para leer el siguiente valor (esta
    función podrá bloquear al hilo invocante dependiendo de si ya había leído el valor

    anteriormente o no, según se ha descrito). Estas funciones deben utilizar las primitivas de
    sincronización adecuadas para asegurar que no existen problemas en la concurrencia de uso
```

del Almacen.
- Las condiciones de sincronización, que provocan que algunos de los hilos lectores bloqueen, son: a) que un hilo 'lector' intente leer un valor nuevo sin que el resto de hilos lectores hayan leído el valor actual, y b) que un hilo generador intente cambiar el valor del Almacen sin que todos los hilos lectores hayan leído el valor actual.
- Añada una función miembro para cerrar el Almacen. Esta función será invocada antes de que termine el programa y su objetivo es desbloquear todos los hilos que pudieran estar bloqueados en las primitivas de sincronización del Almacen y asegurar una finalización ordenada y correcta de todos los hilos del programa.

El hilo que genera valores puede implementarse con un simple bucle que inserta en el Almacen valores del 0 al 6. Esta función recibe una referencia al objeto Almacen compartido entre todos los hilos.

La función que implementa los hilos lectores recibe como parámetros un identificador (string), el tiempo que debe esperar entre lecturas sucesivas del Almacen, una referencia al objeto Almacen compartido entre los hilos, y una referencia a una variable que controla cuándo debe terminar el hilo lector.

Haga un ejemplo de utilización creando un Almacen, un hilo generador y tres hilos lectores. Una posible ejecución podría ser la siguiente:

```
LECTOR 2 -[ comienza
LECTOR 1 -[ comienza
LECTOR 3 -[ comienza
LECTOR 1 -[ lee 0
LECTOR 2 -[ lee 0
LECTOR 3 -[ lee 0
LECTOR 1 -[ lee 1
LECTOR 2 -[ lee 1
LECTOR 3 -[ lee 1
LECTOR 1 -[ lee 2
LECTOR 2 -[ lee 2
LECTOR 3 -[ lee 2
LECTOR 1 -[ lee 3
LECTOR 2 -[ lee 3
LECTOR 3 -[ lee 3
LECTOR 1 -[ lee 4
LECTOR 2 -[ lee 4
LECTOR 3 -[ lee 4
LECTOR 1 -[ lee 5
LECTOR 2 -[ lee 5
LECTOR 3 -[ lee 5
LECTOR 1 -[ lee 6
LECTOR 2 -[ lee 6
LECTOR 3 -[ lee 6
LECTOR 1 -[ termina
LECTOR 2 -[ termina
LECTOR 3 -[ termina
```

Ejercicio 6 (C++ básico y STL (strings, vector, map, set, flujos))

EJERCICIOS PARCIAL PBO

EJERCICIO 1

Se pide que haga un programa que ayude a realizar crucigramas. Concretamente, se pide que haga parte de la selección de palabras. Como entrada, se proporciona un string que contiene una lista de palabras, separadas por saltos de carro ('\n$_7$), entre las que hacer la selección. El usuario del programa debe poder pedir una nueva palabra de una longitud dada, y que contenga una letra en concreto (para poder hacer el crucigrama). Además, el programa debe asegurar que no retorna dos veces la misma palabra (se supone que el string de entrada no contiene palabras repetidas). Si ya no quedan palabras que cumplan, se retorna el string vacío (""). Haga una función main con un ejemplo de utilización del mismo. Un ejemplo de ejecución, con la siguiente definición de datos: const string datos = "hola\ntiempo\nreal";, sería:

palabra con 't$_7$ y tam 6 ('tiempo'): tiempo
palabra con 'l$_7$ y tam 4 ('hola'): hola
palabra con 'l$_7$ y tam 4 ('real'): real
palabra con 'a$_7$ y tam 2 (''):
palabra con 'l$_7$ y tam 4 (''):
palabra con 'z$_7$ y tam 4 (''):

Observe que la segunda vez que se pide tam=4 y letra 'l$_7$, se retorna otra palabra. Y en la penúltima petición, como ya se han retornado todas las palabras que tienen una 'l$_7$ y tam 4, se devuelve el string vacío. Este ejercicio tiene que compilar. Si no compila, se obtiene la mitad de la puntuación. Tiene

```
libertad para utilizar todas las librerías de C++ que estime, pero tiene que definir y utilizar, al menos,
dos funciones además de la función main.
```

Ejercicio 7 (Clases y objetos)

```
EJERCICIO 2

Se pide que haga un programa que ayude a realizar crucigramas. Concretamente, se pide que haga
parte de la selección de palabras. Como entrada, se proporciona un string que contiene una lista de
palabras, separadas por saltos de carro ('\n7), entre las que hacer la selección. El programa mostrará al
usuario un listado de las palabras que tienen el número de vocales o de consonantes que el usuario
especifique. Para comprobar si un carácter es alfanumérico, utilice la librería <cctype>. Si no hay
palabras que cumplan la condición, se imprimirá "no hay palabras que cumplan". Haga una función
main con un ejemplo de utilización del mismo. Un ejemplo de ejecución, con la siguiente definición
de datos: const string datos = "hola\ntiempo\nreal\nblanco";, sería:

palabras con 2 vocales:
        hola    real   blanco
palabras con 3 consonantes:
        tiempo
palabras con 13 consonantes:
        no hay palabras que cumplan

Este ejercicio tiene que compilar. Si no compila, se obtiene la mitad de la puntuación. Tiene libertad
para utilizar todas las librerías de C++ que estime, pero tiene que definir y utilizar, al menos, dos
funciones además de la función main. Si utiliza un mapa para resolver el ejercicio, tenga en cuenta que
tiene que existir una función bool operator< (tipo_t &lhs, tipo_t &rhs) que compara dos
elementos del primer tipo del mapa.
```

Curso 2016-17

Febrero 2018 – 5 ejercicios

EXAMEN DE "PROGRAMACIÓN DE SISTEMAS EN TIEMPO REAL", Dispone de 3 horas para hacer el examen. Lea bien las preguntas antes de responder. Durante el examen sólo se responderán preguntas relacionadas con el enunciado del mismo. Recuerde que, si quiere mantener la nota del parcial o de Moodle, no debe responder a los dos últimos apartados (ejercicio del parcial y cuestionario de Moodle).

Ejercicio 1 (C++ básico y STL (strings, vector, map, set, flujos))

```
EJERCICIOS DE PROGRAMACIÓN (3 PUNTOS)
Se pide que haga un programa para cifrar y descifrar mensajes secretos que están formados por la primera letra de cada
frase (delimitadas por un punto y seguido), mientras que el espacio en blanco del mensaje secreto se convierte en un
 ↪ salto
de línea (un '\n'). Suponga que siempre se van a utilizan minúsculas. Tenga en cuenta que después del '.' irá un
 ↪ espacio en
blanco o en retorno de línea (cuando se esté descifrando) o tendrá que insertarlo en caso de que esté cifrando el
 ↪ mensaje.
     • Para cifrar, suponga que existe un fichero "dict.txt" que contiene una frase que comienza por cada letra del
       alfabeto en cada línea (aunque pueden estar almacenadas en cualquier orden), que se van a utilizar para
        ↪ cifrar el
       mensaje. Utilice un mapa para almacenar la frase correspondiente a cada letra. La función de cifrado retorna
        ↪ un
       string con el mensaje entero (todas las frases juntas).
     • Para desarrollar las funciones de descifrado, suponga que el mensaje se tiene en un fichero, cuyo nombre se pasa
       como parámetro. Cuando haya descifrado el mensaje lo puede imprimir por consola.
Ejemplo:
     • Mensaje: "Toledo está cerca de Madrid. Integrada con el monte que la rodea. Empedradas tiene la mayoría de las
       calles. Más parece que no pasa el tiempo por ella. Podría decirse que está como siempre. O tal vez mejor."
     • Mensaje secreto: "Tiempo".
Las funciones involucradas en el cifrado y descifrado puntúan 1,5 puntos, cada una. Este método de cifrado es solo un
ejercicio. No lo utilice nunca.
```

Ejercicio 2 (Hilos y exclusión mutua)

```
EJERCICIO DE CONCURRENCIA (3 PUNTOS)
Se pide que diseñe una clase Futuro, una clase ejecutable Servidor con un método que retorna objetos de tipo Futuro, y
un conjunto de hilos que utilizan los Futuros retornados por el Servidor para obtener un valor.
Un Futuro almacena un valor, que se va a suponer de tipo entero, que estará disponible en algún momento del futuro, y
que es capaz de bloquear a cualquier hilo que pretenda leer el valor cuando todavía no está disponible. La clase Futuro
está formada por los siguientes elementos:
      • Atributos: un valor entero, un mutex para bloquear el acceso al valor si no está disponible, y todos aquellos
        otros atributos que estime necesarios para modelar correctamente el comportamiento de la clase.
```

> • Un constructor por defecto, que asume que no se fija el valor del Futuro.
> • Un constructor que acepta un valor entero con el que se fija el Futuro, y que desbloquea por tanto el acceso al valor.
> • Un método set_valor que fija el valor del futuro y que desbloquea el acceso al mismo. Solo puede cambiarse el valor del Futuro una vez.
> • Un método get_valor que retorna el valor almacenado en el Futuro o que bloquea al hilo invocante si no estuviera disponible todavía.
>
> La clase Servidor tiene los siguientes elementos:
> • Entre los atributos de la clase, tendrá que definir, como mínimo, un vector de Futuros y un mutex.
> • Un método servicio permite a otros hilos solicitar un nuevo Futuro al servidor. Este método crea y añade a un vector interno un nuevo Futuro, y retorna una referencia a dicho Futuro al hilo que invoca esta función.
> • Un método f_hilo que va fijando un valor en los Futuros almacenados en la clase tras la ejecución del método servicio. Para facilitar el código, se fijarán valores consecutivos en el Futuro, comenzando por 10. Esta función debe bloquear mientras no haya un nuevo Futuro en el que fijar el valor, y dormirá un tiempo de 1000 ms tras fijar el valor.
> • Un método terminar que permite que la función f_hilo termine limpiamente.
> Cree una función para simular hilos “cliente”, que invocan 5 veces el método servicio de un Servidor para obtener un valor Futuro, y que luego lo imprima por consola. Esta función recibe como parámetro un número para identificar el hilo que imprime el valor.
> Diseñe también la función ‘main’ que cree un Servidor, lance un hilo que ejecuta la función f_hilo del servidor, y posteriormente cree 5 hilos “cliente”. Cuando terminen todos los hilos debe hacer que el servidor termine limpiamente-

Ejercicio 3 (Planificación de tiempo real)

> PREGUNTA DE DESARROLLO (1,5 PUNTOS)
> Describa el funcionamiento del planificador del sistema operativo, gestión de tareas, eventos de planificación y cambio de contexto.

Ejercicio 4 (Clases y objetos)

> EJERCICIO DEL PARCIAL (1,5 PUNTO)
> Se pide que programe la clase Polinimio, para modelar dicha entidad matemática. La clase debe tener la siguiente funcionalidad:
> • Tiene un único atributo: un vector de double donde se almacenan los coeficientes del polinomio.
> • Añada un constructor que acepta como parámetro un vector de double que contiene los coeficientes con que se quiere inicializar el polinomio. Basta con copiarlos.
> • Añada un constructor al que se le pasan tres valores de tipo double para crear fácilmente un polinomio de segundo grado.
> • Añada la función operator<< para imprimir el polinomio por consola: coeficiente·x^j. Por ejemplo: +3.3·x^2 +2.2·x^1 + 1.1
> • Añada un método eval, que recibe el punto donde debe evaluarse el polinomio y que retorna dicho valor.
> • Añada un método grado que retorna el grado del polinomio.
> • Añada un método set, que recibe el valor y la posición cuyo valor debe cambiar. Retorna true si ha podido hacer el cambio y false si la posición no existe en el polinomio (por ejemplo, intentar cambiar el coeficiente x9 cuando el polinomio es de grado 5).
> • Añada un método get que retorna el valor de una posición del polinomio que se pasa como parámetro. Retorna 0 si la posición no existe (por ejemplo, intentar acceder a x9 cuando el polinomio es de grado 5).
> • Añada un método sumar que recibe un Polinomio como parámetro y que retorna otro Polinomio, resultado de sumar los dos polinomios (el objeto sobre el que se invoca el método y el parámetro). La suma se realiza coeficiente a coeficiente, completando con ceros en caso de que los polinomios no tengan el mismo grado. Por ejemplo: (x^2 + 3·x − 5) + (x^5 − 4·x^4 + x + 2) = x^5 -4·x^4 + x^2 +4·x - 3
>
> Haga también una función ‘main’ donde cree dos polinomio utilizando los dos constructores proporcionados, evalúe uno de ellos en un punto solicitado al usuario por teclado, y luego imprima el polinomio resultado de sumar los dos polinomios anteriormente creados.
>
> EXAMEN DE “PROGRAMACIÓN DE SISTEMAS EN TIEMPO REAL”,
> Nombre, apellidos y DNI:
>
> Cada respuesta mal contestada resta 1 3 de la puntuación total.

Ejercicio 5 (Clases y objetos)

> CUESTIONARIO DE MOODLE (1 PUNTO)
> P1. El montículo es una zona para que el usuario gestione libremente la memoria.
>
> Verdadero Falso

```
P2. Si está declarado el vector 'int v[100]', ¿cuál de las siguientes sentecias 'for' NO es correcta?

              for (const auto i : v) { ... }    for (int i : v) { ... }       for (int &i : v) { ... }    for (var i :
              ↪  v) { ... }

P3. ¿Cuántas veces se ejecuta el cuerpo del bucle siguiente?

               int i = 0;
               while (1) {
                   ++i;
                   if (i>=10) {
                        ++i;
                        if (i==10) {
                             break;
                        }
                   }
               }

                                                 10      Infinitas veces           110          0

P4. ¿Qué se imprimirá al ejecutar el siguiente código?

               int i = 0;
               while (i<5) {
                   if (i == 3) {
                       break;
                   }
                   std::cout << i;
               }

                                        0123         No compila            Infinitamente 0             01234

P5. ¿Cuál de los siguientes elementos NO está relacionado con el uso de flujos (stream) en C++?

                    Palabra reservada auto            operator<<              Librería <ifstream>         operator>>

P6. ¿Cuál de los siguientes prototipos se corresponde con el de una función que no acepta parámetros y retorna un
valor de tipo long?

                           long f1 (null);     long f1 (void);            long f1 (long a);        void f1 (long);

P7. ¿Cuál de las siguientes palabras reservadas de C++ está relacionada con el modelado de algoritmos?

                                                int     while    class      private

P8. ¿Cuál de las siguientes afirmaciones sobre el run-time storage es FALSA?:

      Se crea un stack frame cada vez que se crea un objeto                 El usuario debe gestionar el montículo
      Tiene una zona para constantes y variables globales y estáticas       Almacena una copia del código

P9. ¿Cuál de las siguientes expresiones calcula el valor 6,14?

                         3+3+((long)0.14)         3+3.14    ((long) 3.14) + 3     (long)(3+3.14)

P10. Una expresión en C++ es:

         Conjunto de identificadores o literales unidos por operadores que, tras ser evaluado, produce un valor
         Agrupación válida de elementos léxicos acabada en ';'
         Agrupación válida de sentencias simples o complejas, delimitada por un bloque '{' '}'
         Agrupación válida de sentencias simples o complejas que, tras ser evaluadas, produce un valor
```

Junio 2017 – 5 ejercicios

EXAMEN DE "PROGRAMACIÓN DE SISTEMAS EN TIEMPO REAL", Dispone de 3 horas para hacer el examen. Lea bien las preguntas antes de responder. Durante el examen sólo se responderán preguntas relacionadas con el enunciado del mismo. Recuerde que, si quiere mantener la nota del parcial o de Moodle, no debe responder a los dos últimos apartados (ejercicio del parcial y cuestionario de Moodle).

Ejercicio 1 (Clases y objetos)

```
EJERCICIOS DE PROGRAMACIÓN (3 PUNTOS)
Se pide programar un filtro anti-spam muy simplificado, basado en la versión original de filtro Bayesiano que propuso
 ↪  Paul
Graham en 2002 (http://www.paulgraham.com/spam.html). Para ello, se dispone de 1000 ficheros con el cuerpo de
```

> correos que no son spam y otros 1000 ficheros que sí lo son. Suponga que se llaman “no_spam_XXX.txt” y “spam_XXX.txt”, respectivamente. El algoritmo se basa en generar dos histogramas de las palabras que aparecen en ambos conjuntos de datos, y luego combinar dichos histogramas en un tercero, que contiene la probabilidad de que la aparición de una palabra
> indique que el correo es spam. Recuerde que un histograma almacena la cantidad de veces que se repite, en este caso, una
> determinada palabra. Para simplificar el código, no es necesario eliminar signos de puntuación, ni números ni otros caracteres extraños. Y se puede ignorar la capitalización de las palabras. Se pide implementar una función ‘main’ y las funciones necesarias para programar este filtro. Concretamente:
>
> 1. Implemente una o varias funciones para obtener el histograma de las palabras que aparecen en mensajes clasificados como spam, y en aquellos que no son spam (1,5 ptos).
> 2. Implemente una o varias funciones que generen el tercer histograma (según se describe a continuación, 2 ptos).
> 3. Implemente la función ‘main’ que haga uso de las funciones anteriores (0,5 ptos).
>
> El tercer histograma contiene la probabilidad de que la aparición de una palabra en un correo electrónico indique que es
> spam. Esta probabilidad se va a calcular solo para aquellas palabras que aparecen en el histograma de palabras spam, y solo cuando la palabra aparece más de 5 veces en total (tanto en correos spam como no spam). La fórmula que permite calcular la probabilidad de spam es: $P = \frac{m}{b+m}$, donde ‘m’ es la cantidad de veces que la palabra aparece en un correo
> spam, y ‘b’ la cantidad de veces que aparece en un correo no spam. Si una palabra no aparece en un mapa, su valor es 0. Antes de almacenar la probabilidad en el tercer histograma, limítela al rango 0.01 a 0.99.
> Para implementar los histogramas utilice la clase map, definida en el fichero de cabecera <map>. Un mapa es un conjunto de pares clave-valor, definido como template <class Key, class T> class map. Algunas funciones y consejos que podría necesitar:
>
> - T_6 operator[](Key &val): permite acceder al valor almacenado para una clave dada. Tanto para leer como para escribir. Por ejemplo, mapa[“hola”]++ incrementaría en 1 el valor almacenado en el mapa para la clave “hola”.
> - int count (Key &val): devuelve la cantidad de veces que aparece la clave dada en el mapa. Puede serle útil para saber si una palabra está o no en el mapa.
> - Si utiliza el siguiente bucle para recorrer todos los elementos de un mapa, for (auto &e : mapa), recuerde que la clave está en e.first, y el valor en e.second.
>
> Para terminar el filtro faltaría por implementar la función que, dado un nuevo correo electrónico, combina todas las probabilidades de las palabras que en él aparecen para determinar si el correo es spam o no. Pero esto se deja como ejercicio para casa.

Ejercicio 2 (Hilos y exclusión mutua)

> EJERCICIO DE CONCURRENCIA (3 PUNTOS)
> Se pide que diseñe un sistema concurrente de gestión de copias de seguridad de mensajes para una gran empresa. La empresa considera cuatro niveles de copia: solo al hdd1, a ambos discos duros (valor hdds), solo a la nube y a todos los
> dispositivos (valor completo). Estos dispositivos están representados por el tipo de dato log_t. Por otro lado, la empresa tiene dos discos duros (hdd1 y hdd2) conectados en local y ha contratado también almacenamiento en la nube (nube). Estos dispositivos están representados por el tipo de dato disp_t. Se pide que programe unos hilos que se
>
> integren con parte del software que ya está desarrollado. Concretamente, el punto de integración está definido por las siguientes funciones y tipos de datos:
>
> ```
> enum class log_t {hdd1, hdds, nube, completo};
>
> log_t obtener_mensaje (std::string &msg) { }
>
> enum class disp_t {hdd1, hdd2, nube};
>
> void copiar_mensaje (disp_t disp, const std::string &str) { }
> ```
>
> El hilo obtiene el siguiente mensaje a almacenar mediante la función obtener_mensaje, y utiliza la función copiar_mensaje para copiarlo en el dispositivo correspondiente, de acuerdo al valor retornado por obtener_mensaje.
> Haga un programa que cree tres hilos de este tipo. Puede suponer que los hilos se ejecutan indefinidamente. El programa debe asegurar que no se producen condiciones de carrera en el acceso a los recursos compartidos, que los mensajes se almacenan siempre en el mismo orden en todos los dispositivos seleccionados, y que además no sucede ninguna situación de interbloqueo entre los hilos que forman el programa.

Ejercicio 3 (Planificación de tiempo real)

> PREGUNTA DE DESARROLLO (1,5 PUNTOS)
> Describa el funcionamiento del planificador del sistema operativo, gestión de tareas, eventos de planificación y cambio de
> contexto.

Ejercicio 4 (Clases y objetos)

> EJERCICIO DEL PARCIAL (1,5 PUNTO)
> Se pide que programe las clases Nota y Asignatura para realizar un programa de gestión de alumnos de una universidad.

La clase Nota tiene la siguiente funcionalidad:
- Los atributos son el valor numérico, el porcentaje que supone sobre la nota final de la asignatura, y una descripción de la nota (un string, como por ejemplo “parcial”).
- Añada un constructor al que se le pase los valores para los tres atributos.
- Añada la función operator<< para imprimir una descripción de la Nota.
- Añada un método cambiar_nota que permite cambiar el valor numérico de la Nota siempre que el valor que se suministre se encuentre entre 0 y 10. Retorna un bool para indicar si se ha realizado el cambio o no.
- Añada un método calcular que retorna el valor real de la nota, una vez aplicado el porcentaje sobre el valor numérico de la Nota.

Por su parte, la clase Asignatura tiene la siguiente funcionalidad:
- Los atributos son el nombre de la asignatura, el curso en que se imparte, y un vector con las notas que conforman la nota de la Asignatura.
- Añada un constructor al que se le pase el nombre y el curso en que se imparte la Asignatura.
- Añada la función operator<< para imprimir una descripción de la Asignatura.
- Añada un método cambiar_nota que permite cambiar el valor numérico de una de las Notas de la Asignatura, cuyo índice se pasa como parámetro. El método debe comprobar que el índice está en rango y retorna un bool para indicar si se ha realizado el cambio o no.
- Añada un método calcular que retorna el valor numérico de la Asignatura, obtenido como la suma de los valores reales de todas las Notas que almacena. Utilice el método calcular definido en Nota.
- Añada un método anyadir_nota que añade una Nota al vector de Notas de la Asignatura.

Haga también una función ‘main’ donde utilice las clases anteriores para modelar la nota de la asignatura “Programación de sistemas en tiempo real”.

EXAMEN DE “PROGRAMACIÓN DE SISTEMAS EN TIEMPO REAL”,
Nombre, apellidos y DNI:

Cada respuesta mal contestada resta 1 3 de la puntuación total.

Ejercicio 5 (C++ básico y STL (strings, vector, map, set, flujos))

CUESTIONARIO DE MOODLE (1 PUNTO)
P1. ¿Cuál de los siguientes no es un elemento léxico del lenguaje C++?
Literales Funciones Identificadores Separadores

P2. El lenguaje de programación C++ distingue entre mayúsculas y minúsculas.
Verdadero Falso

P3. Toda sentencia 'switch' puede convertirse en un 'if/else' pero no viceversa.
Verdadero Falso

P4. Suponiendo la siguiente definición de funciones,

```
int esPar (int x) {
   printf("P2);
   return x%2==0;
}
```

```
int esImpar (int x) {
   printf("I2);
   return x%2==1;
}
```

indique qué se imprime al ejecutar:

```
int a = 10;
if (esPar(a) || esImpar(a)) {
        printf("X2);
}
```

IPX PX PIX IX

P5. ¿Qué valor se imprime por pantalla tras ejecutar el siguiente código?

```
int f1 (int n) {
    n++;
}
```

```
void main (void) {
     int n = 10;
     f1(n);
     printf("%d2, n);
}
```

12 Ni siquiera compila 11 10

P6. ¿Qué se imprimirá al ejecutar el siguiente código?

```
int i = 0;
for (i; i<5; ++i) {
    if (i == 3) {
          break;
    }
    std::cout << i;
}
```

Nada, no compila 01234 0123 012

P7. ¿Cuál de los siguientes NO es un tipo primitivos del lenguaje C++?

Número en coma real Vectores Número enteros Caracteres

```
P8. El tiempo de vida de una variable es una propiedad de tiempo de ejecución.
                                                Verdadero     Falso

P9. Las sentencias de selección utilizan operadores relacionales y lógicos.
                                                Verdadero       Falso

P10. ¿Es correcta la siguiente definición de la función para calcular el factorial de un número entero?
  int factorial (int n) {
      if (n==1) {
            return 1;
      } else {
            return p*factorial(--n);
      }
  }

          No, produce un bucle infinito   No, calcula otra función matemática distinta       Sí       No, no compila
```

Parcial 2016-17 – 1 ejercicios

Ejercicio 1 (Herencia y polimorfismo)

```
PARCIAL DE “PROGRAMACIÓN DE SISTEMAS EN TIEMPO REAL”,

Dispone de 2 horas para hacer el examen. Lea bien las preguntas antes de responder. Durante el examen sólo se
 ↪  responderán
preguntas relacionadas con el enunciado del mismo. El examen puntúa 1,5 puntos de la nota final de la asignatura. La
 ↪  nota
se guarda todo el curso y se puede recuperar en cualquier convocatoria. Tras acabar el código, tiene que:
    • Avisar en el aula para enseñarle que el código por lo menos compila y entregarle este enunciado con su nombre.
       Es imprescindible que el código compile; si no compila, la nota es un 2.
    • Subir en un único fichero ZIP (con su nombre) todo el código fuente del examen a la actividad que se habrá creado
       en el Aula Virtu@l. Ponga su nombre y DNI en un comentario en TODOS los ficheros.

ENUNCIADO

Se pide que diseñe una implementación para un juego de marcianitos. Para ello, se piden 3 clases y una función ‘main’
 ↪  para
probar su funcionamiento. Las clases son Bala, Marcianito y Pantalla. Defina las 3 clases en un fichero parcial.h y en
este orden para que no haya problemas de compilación:

          #ifndef PARC_PARCIAL_H
          #define PARC_PARCIAL_H
          #include <iostream>
          class Bala {
             ...
          };
          class Marcianito {
             ...
          };
          class Pantalla {
             ...
          };
          #endif //PARC_PARCIAL_H

En el Aul@ Virtual encontrará un fichero con la función ‘main.cpp’ de prueba. Este fichero está semi completo, con
comentarios en diversas partes para que las complete con el código correspondiente.
Por favor, respete esta estructura para que el código de pruebas y corrección funcione correctamente. Además,
 ↪  proporcione
una implementación de todos los métodos solicitados (aunque no sea del todo correcta) para que el programa compile.
 ↪  Cada
método se corrige por separado, pero es imprescindible que por lo menos todo el código compile.
Puede añadir más métodos y atributos si lo necesita en cualquiera de las clases o la función ‘main’, pero debe
 ↪  proporcionar
como mínimo el código solicitado en el enunciado.

CLASE BALA (2 PTOS)

Tiene 5 atributos de tipo entero: coordenadas ‘x’ e ‘y’, la velocidad a la que avanza la Bala en ambos ejes y el daño
 ↪  que hace
al impactar. Tiene un constructor para inicializar todos los atributos con los valores que se pasen; es decir, un
 ↪  constructor
con 5 parámetros. Define 3 métodos, para obtener las coordenadas de la Bala y el daño que hace. Define operator<< para
imprimir los valores de los atributos de la Bala con el formato: (x, y) @ [vx, vy] : daño. Por último, tiene un método
 ↪  de
tipo void llamada tick para simular el movimiento de la Bala: incrementa la posición de la Bala dependiendo de su
```

velocidad. Este método recibe dos parámetros, el alto y ancho de la pantalla del juego, de forma que, si la posición de la Bala

en algún momento se saliera de los límites, volvería a aparecer por “el otro lado”. Por ejemplo, si la pantalla midiera 200x200
y tras actualizar la posición de la Bala, ésta acabara en el punto (100, 230), la nueva posición de la Bala sería (100, 30). Y
lo mismo debería suceder si por ejemplo la Bala acabara en (-120, 150) à la nueva posición sería (80, 150).

CLASE MARCIANITO (3 PTOS)

Tiene 6 atributos de tipo entero: coordenadas y velocidades, número de ticks que han pasado y vida. Define un constructor
para inicializar las coordenadas de la nave y la vida; las velocidades en los ejes se inicializan siempre al valor 1 y los ticks a
0. Define 3 métodos, para obtener la vida y las coordenadas donde se encuentra el Marcianito. Define operator<< para imprimir los valores de los atributos del Marcianito con el formato: (x, y) @ [vx, vy] : vida : ticks. Tiene también una función void tick que simula el movimiento del Marcianito. Este método recibe dos parámetros, el alto y ancho de la pantalla del juego, de forma que, si la posición del Marcianito en algún momento se saliera de los límites, volvería a
aparecer por “el otro lado”, como en el caso de la Bala. Adicionalmente, el método debe llevar la cuenta del número de ticks
que el Marcianito permanece vivo, ya que cada 10 ticks se aumenta la velocidad en ambos ejes en 3 unidades, y cada 50 ticks se aumenta la vida en un 10% del valor actual. Este método no hace nada si el Marcianito está muerto (el valor del
atributo vida es menor o igual a cero). Por último, define el método colisionar, que retorna un bool y acepta un Bala. Este método comprueba si la Bala impacta con la posición del Marcianito comparando las coordenadas de ambos. En caso de que coincidan, reduce la vida del Marcianito según el daño que realiza la Bala y retorna true. Si el Marcianito está muerto o no coinciden las coordenadas, devuelve false.

CLASE PANTALLA (4 PTOS)

Tiene 4 atributos: dos enteros que almacenan el tamaño de la pantalla, un vector de Bala y un vector de Marcianito. Define
dos constructores: uno al que se le pasa el tamaño de la pantalla (alto y ancho) y otro al que además se le pasa un vector de
Bala. Define un método para añadir una Bala, y otro método para añadir un Marcianito. Tiene un método operator<< para imprimir el tamaño de la pantalla e imprimir todas las Balas y Marcianitos que almacena. Define un método marcianitos_muertos que calcula y retorna si la vida de todos los Marcianitos es menor o igual que cero. Por último, tiene un método void tick que invoca los métodos tick de las Balas y Marcianitos almacenadas en los vectores, y luego invoca el método colisionar de Marcianito para comprobar las colisiones de cada Bala con cada Marcianito.

FUNCIÓN MAIN (1 PTOS)

En el Aul@ Virtual encontrará un fichero ‘main.cpp’ semi-completo, con comentarios para que añada el código y código ya escrito. Por favor, respete el código ya proporcionado y sustituya los comentarios por el código solicitado.

Consejos:
   (1) Cree un proyecto C++ en CodeBlocks y fije el modo C++11 (menú Settings à Compiler, opción –std=c++11)
   (2) Cree un nuevo fichero (menú File à New à Empty File) en el mismo directorio en que se encuentra el fichero ‘main.cpp’ que se ha creado con el proyecto. Nombre del fichero: ‘parcial.h’
   (3) Copie el contenido que se muestra al principio del enunciado en el fichero ‘parcial.h’
   (4) Copie el contenido del fichero ‘main.cpp’ del Aula Virtu@l al fichero ‘main.cpp’ que se ha creado con el proyecto
   (5) Conforme va programando las clases Bala, Marcianito y Pantalla, vaya descomentando y completando el fichero ‘main.cpp’ y compile regularmente. No lo deje para el final, ya que el código debe por lo menos compilar

Fichero main.cpp:

```
#include "parcial.h₂

#include <iostream>
#include <iomanip>

int main () {
     std::cout << std::boolalpha;
     // Declarar una Bala b1
     std::cout << "b1: " << b1 << '\n₇;
     for (int i=0; i<100; ++i)
          // invocar el método tick de b1 (cualquier valor)
     std::cout << "b1 100: " << b1 << '\n₇;

     // Declarar un Marcianito m1
     std::cout << "m1: " << m1 << ", colision b1: " << m1.colisionar(b1)<< '\n₇;
     std::cout << "\tm1 tras colision: " << m1 << '\n₇;
```

```
     for (int i=0; i<100; ++i)
          // invocar el método tick de m1 (cualquier valor)

     // Declarar Pantalla p1
     // Añadir 3 Marcianitos a p1
     // Invocar tick de p1
     // Imprimir p1 y comprobar si todos los marcianitos están muertos

     // Añadir 3 Balas a p1
     // Invocar tick de p1
     // Imprimir p1 y comprobar si todos los marcianitos están muertos

     return 0;
}
```

Septiembre 2017 – 4 ejercicios

EXAMEN DE "PROGRAMACIÓN DE SISTEMAS EN TIEMPO REAL", Dispone de 3 horas para hacer el examen. Lea bien las preguntas antes de responder. Durante el examen sólo se responderán preguntas relacionadas con el enunciado del mismo. Recuerde que, si quiere mantener la nota del parcial o de Moodle, no debe responder a los dos últimos apartados (ejercicio del parcial y cuestionario de Moodle).

Ejercicio 1 (C++ básico y STL (strings, vector, map, set, flujos))

```
EJERCICIOS DE PROGRAMACIÓN (3 PUNTOS)
Se pide que haga un programa que lea un fichero de texto, formado por líneas (delimitadas por '\n') y páginas (marcadas
por un delimitador, por ejemplo ---PAG---), y haga un par de funciones que calculen cuál es la página que tiene más
líneas (función pagina_con_mas_lineas) y cuál es la página que tiene más caracteres (función
pagina_con_mas_caracteres), ya que ambas métricas no tienen por qué coincidir. Declare una estructura de datos,
basada en un vector, para almacenar el contenido de texto de las líneas que forman cada página del fichero. La función
 ↪  que
lee el fichero y rellena la estructura de datos recibe como parámetros el nombre del fichero y el delimitador que debe
utilizar para identificar el final de una página en el fichero de texto. Cada una de las tres funciones vale 1 punto.
```

Ejercicio 2 (Hilos y exclusión mutua)

```
EJERCICIO DE CONCURRENCIA (3 PUNTOS)
Se pide que diseñe un simulador de una línea de fabricación en serie, en la que cada trabajador completa la pieza que
 ↪  ha
fabricado el anterior. Y, obviamente, hasta que el trabajador previo no termina de fabricar la pieza, el siguiente
 ↪  trabajador
no puede comenzar a trabajar en ella. Cada trabajador va a ser modelado como un hilo que tarda un tiempo aleatorio en
terminar una pieza, que en el simulador se modela mediante una distribución normal de media 2000ms y desviación
200ms. El simulador acepta como parámetros el número de trabajadores de la línea de fabricación y el número de piezas
que deben fabricar. Todos los trabajadores (hilos) ejecutan la misma función, por lo que debe diseñar el programa de
forma que sea posible. Imprima una pequeña traza de lo que sucede, como la que se muestra a continuación:

Trabajador 1 construyendo pieza 1                              Trabajador 4 construyendo pieza 1
Trabajador 2 construyendo pieza 1                              Trabajador 1 construyendo pieza 4
Trabajador 1 construyendo pieza 2                              Trabajador 2 construyendo pieza 4
Trabajador 2 construyendo pieza 2                              Trabajador 5 construyendo pieza 1
Trabajador 3 construyendo pieza 1                              Trabajador 4 construyendo pieza 2
Trabajador 1 construyendo pieza 3                              Trabajador 3 construyendo pieza 3
Trabajador 2 construyendo pieza 3                              Trabajador 1 construyendo pieza 5
Trabajador 3 construyendo pieza 2                              Trabajador 2 construyendo pieza 5

Diseñe también la función 'main' de para crear una simulación con 5 trabajadores.
Consejos: utilice vectores para almacenar los hilos y mutex que tendrá que definir para asegurar el comportamiento de
línea de fabricación; si organiza bien los vectores y utiliza correctamente los parámetros de la función del hilo,
 ↪  podrá
organizar el código fácilmente; el primer y último trabajador se comportan de forma ligeramente distinta, ya que el
primero no tiene que esperar a que nadie le entregue una pieza, y el último no le pasa la pieza a nadie más;
 ↪  proporcione un
identificador a cada hilo para mejorar la impresión por consola y para organizar mejor el código.
```

Ejercicio 3 (Planificación de tiempo real)

```
PREGUNTA DE DESARROLLO (1,5 PUNTOS)
Describa el funcionamiento del planificador del sistema operativo, gestión de tareas, eventos de planificación y
 ↪  cambio de
contexto.
```

Ejercicio 4 (Clases y objetos)

```
EJERCICIO DEL PARCIAL (1,5 PUNTO)
Se pide que programe las clases Reserva y Restaurante para realizar un programa de gestión de las reservas para comer
o cenar en un restaurante. Se proporcionan las siguientes estructuras de ayuda:
struct Fecha {
    int _dia, _mes;
    friend std::ostream& operator<< (std::ostream &os, const Fecha& f) {
        os << f._dia << " del " << f._mes;
        return os;
    }
};
struct Hora {
    int _hora, _minutos;
    friend std::ostream& operator<< (std::ostream &os, const Hora& h) {
        os << '['<< h._hora << " : " << h._minutos << ']';
        return os;
    }
};
La clase Reserva tiene la siguiente funcionalidad:
    • Los atributos son el nombre de la persona que realiza la Reserva y el menú que ha encargado (ambas, de tipo
        string), la Hora de la reserva y el número de mesa.
    • Añada un constructor al que se le pasan el nombre y la Hora de la Reserva, y un segundo constructor al que
        además se le pasa el menú. Por defecto, el número de mesa es 0.
    • Añada la función operator<< para imprimir una descripción de la Reserva, que imprime solo los atributos que
        tienen un valor significativo (por ejemplo, el menú solo si el cliente lo ha encargado).
    • Añada un método get_hora (que retorna la Hora de la Reserva), set_mesa (que permite cambiar el número
        de la mesa por el que se pasa como parámetro) y get_nombre (que retorna el nombre al que se ha hecho la
        Reserva).
Por su parte, la clase Restaurante tiene la siguiente funcionalidad:
    • Los atributos son el número máximo de mesas que pueden estar reservadas en un Fecha dada, y una estructura
        de datos (libro de reservas) para almacenar las Reservas por Fecha.
    • Añada un constructor al que se le pase el número máximo de mesas del Restaurante.
    • Añada la función operator<< para imprimir todas las Reservas realizadas en el Restaurante, junto a la Fecha
        de la Reserva.
    • Añada un método anyadir_reserva, que recibe como parámetros una Reserva y una Fecha, y que retorna un
        valor bool para indicar si ha podido hacer o no la Reserva (es decir, si todavía quedaban mesas libres en el
        Restaurante en la Fecha especificada). Una vez comprobado que quedan mesas libres para la Fecha deseada, el
        método fija el número de mesa para la Reserva utilizando el método set_mesa.
    • Añada un método buscar_reservas, que recibe como parámetro el nombre de la persona que ha realizado una
        Reserva, y retorna un valor bool para indicar si existe al menos una Reserva a dicho nombre. El método debe
        imprimir también los datos de todas las Reservas que encuentre, o un mensaje de error en caso de que no haya
        encontrado ninguna Reserva.
Para implementar el libro de reservas utilice la clase map, definida en el fichero de cabecera <map>. Un mapa es un
conjunto de pares clave-valor, definido como template <class Key, class T> class map. La clave será del tipo
Fecha y el valor será el conjunto de Reservas. Algunas funciones y consejos que podría necesitar:
    • T₆ operator[](Key &val): permite acceder al valor almacenado para una clave dada, tanto para leer como
        para escribir. Puede invocar nuevos métodos sobre el valor retornado, siempre que éste los soporte claro.
    • Si utiliza el siguiente bucle para recorrer todos los elementos de un mapa, for (auto &e : mapa), recuerde que
        la clave está en e.first, y el valor en e.second.

Haga también una función 'main' donde utilice las clases anteriores para añadir tres Reservas a un Restaurante,
 ↪  imprimir
todas las Reservas del Restaurante, y luego buscar una de ellas.
```

Curso 2015-16

Febrero 2017 – 8 ejercicios (resuelto)

EXAMEN DE "PROGRAMACIÓN DE SISTEMAS EN TIEMPO REAL", Dispone de 3 horas para hacer el examen. Lea bien las preguntas antes de responder. Durante el examen sólo se responderán preguntas relacionadas con el enunciado del mismo.

Resuelto en la guía: Febrero 2017 · Planificación con techo de prioridad inmediato

Ejercicio 1 (Otros)

```
EJERCICIOS DE PROGRAMACIÓN (3 PUNTOS)
Se pide que diseñe un sistema de control de acceso con tres zonas de seguridad (zona1 a zona3; defínalas como un tipo
 ↪  enumerado). Se
pide crear 3 clases: Datos_Personales, Empleado y Seguridad con la funcionalidad que se describe en los siguientes
 ↪  apartados.
```

Ejercicio 2 (Clases y objetos)

```
EJERCICIO (1,5 PUNTOS)

La clase Datos_Personales:
     •    Tiene 3 atributos de tipo string para almacenar el nombre y los dos apellidos. Pueden tener visibilidad
     ↪  pública.
     •    Implemente un constructor al que se le pasan 3 strings.
     •    Implemente el operator<< para imprimir el nombre completo.
     •    Implemente el operator== para comprobar si dos objetos de este tipo son iguales. Dos objetos son iguales si
     ↪  ambos tienen los
          atributos con los mismos valores. La declaración de esta función: bool operator== (const Datos_Personales_T
          ↪  &a);
La clase Empleado:
     •    Tiene un atributo del tipo Datos_Personales, otro la clave de acceso (tipo int) y un vector con los permisos
     ↪  para cada zona (3
          valores de tipo bool en este caso). Visibilidad privada.
     •    Implemente un constructor al que se le pasen un objeto de tipo Datos_Personales y si tiene permiso para
     ↪  entrar en cada zona (3
          valores de tipo bool en este caso). El valor de clave se tiene que generar aleatoriamente dentro del
          ↪  constructor, utilizando una
          distribución uniforme entre 1000 y 1000000. Imprima por consola la clave generada y guárdela dentro del
          ↪  objeto creado.
     •    Implemente una función comprobar_credenciales, que recibe una clave y comprueba si coincide con la clave del
     ↪  empleado.
     •    Implemente el operator<< para imprimir los datos del empleado, salvo la clave que no se imprime.
Ejemplo de código que utiliza las clases anteriores:
                    Datos_Personales d {"Mario", "Pérez", "Ruiz"};
                    Empleado e {d, true, false, true};
                    std::cout << e << '\n7;
Salida por consola:
                   Clave: 784520                                          ß esta línea la imprime el constructor de
                    ↪  Empleado
                   Mario Pérez Ruiz [ +           -    + ]              ß esta línea la imprime operator<< de Empleado
La clase Seguridad:
     •    Tiene un atributo para almacenar un std::vector de objetos de la clase Empleado, y un entero que guarda la
     ↪  cantidad de
          empleados almacenados en el vector. Visibilidad privada.
     •    Implemente una función dar_alta, que recibe un objeto de tipo Empleado y lo registra en el vector. La
     ↪  posición que ocupe en el
          vector será el identificador del empleado. Antes de añadir el empleado debe comprobar que no está dado de
          ↪  alta previamente.
          Si lo está, la función retornará un valor negativo. En caso contrario, devolverá el identificador del
          ↪  empleado.
     •    Implemente una función acceder, que recibe como parámetros el identificador, clave y zona a la que se quiere
     ↪  acceder, y
          retorna si dicho acceso es posible o no.
```

Ejercicio 3 (Clases y objetos)

```
EJERCICIO DE PRÁCTICAS (1,5 PUNTOS)

Amplíe la funcionalidad de la clase Empleado con:
    •     Una función tiene_permiso, que recibe como parámetro la zona a la que se pretende entrar, y retorna si el
    ↪  empleado tiene
          permiso para acceder a dicha zona o no. No hace falta que vuelva a escribir todo el código de la clase
          ↪  Empleado, solo lo nuevo.
    •     Un nuevo constructor que reciba como parámetros el nombre y los apellidos (3 strings) y los permisos de
    ↪  zona, y cree un
          nuevo Empleado utilizando el constructor definido en el apartado anterior.
    •     Implemente dos versiones del operator==, una que acepte un objeto de tipo Empleado y la otra de tipo
    ↪  Datos_Personales.
    •     Implemente una función get_datos_personales que retorne el nombre completo del Empleado.
Amplíe la funcionalidad de la clase Seguridad con:
   •     Una nueva versión de la función dar_alta que reciba como parámetros el nombre y los apellidos (3 strings) y
   ↪  los permisos de
         zona, y dé de alta a dicho empleado utilizando la función dar_alta definida anteriormente.

    •      Implemente una función listar_zona que reciba como parámetro una zona de la base e imprima por consola los
    ↪  datos de los
           Empleados que pueden acceder a dicha zona.
    •      Implemente una función buscar que reciba un string y retorna un vector con todos los Empleados cuyo nombre o
    ↪  apellido
           coincide con el parámetro suministrado.
```

Ejercicio 4 (Hilos y exclusión mutua)

> EJERCICIOS DE CONCURRENCIA (4 PUNTOS)
> Se pide programar una aplicación de logging que permita a varios hilos distintos almacenar mensajes en un fichero de texto sin que haya condiciones de carrera y sin que los mensajes aparezcan mezclados. Para ello se pide:

Ejercicio 5 (Monitores y variables condición)

> EJERCICIO (2 PUNTOS)
>
> Implemente la clase Logger como un monitor (es decir, una clase que incluye la exclusión mutua y la sincronización en el acceso a sus atributos). Por razones de eficiencia, esta clase almacena temporalmente en un buffer (std::vector) los mensajes (strings) que recibe antes de volcarlos en disco. También limita el número máximo de mensajes del buffer. Diseñe la clase Logger:
>
> - Tiene como atributos el buffer donde almacenar temporalmente los mensajes, el fichero (std::ofstream) donde guardarlos finalmente, el tamaño máximo, y las variables de control de concurrencia que estime necesarias.
> - Implemente un constructor que recibe el nombre del fichero donde se quieren guardar los mensajes y el tamaño máximo (número de strings) que puede almacenar el buffer.
> - Implemente una función encolar que reciba un mensaje y lo almacene en el buffer, siempre que no esté lleno. En este caso, el hilo debe bloquear, a la espera de que haya espacio.
> - Implemente una función guardar_disco que reciba la cantidad de mensajes que debe volcar del buffer en disco, borrando dichos mensajes a continuación. La función retorna un valor bool para indicar el resultado de la operación. Si el número solicitado es negativo o mayor que el tamaño máximo del buffer, retornará false. Como en el caso anterior, si el buffer está vacío se debe bloquear al hilo invocante a la espera de que haya suficientes mensajes para realizar el volcado a disco.
> - Implemente una función guardar_todo_disco, similar a la anterior salvo que en este caso vuelca todo el contenido en disco.

Ejercicio 6 (Hilos y exclusión mutua)

> EJERCICIO DE PRÁCTICAS (2 PUNTOS)
>
> Amplíe la funcionalidad anterior creando una función main que utilice la clase Logger. Cree tres hilos que periódicamente utilicen la función encolar para almacenar un mensaje. El mensaje (string) debe ser pasado como parámetro a la función que invoca el hilo. Cree también un hilo que periódicamente utilice la función guardar_todo_disco para vaciar el buffer. Las esperas aleatorias en ambos casos se sitúan entre 10 y 1000 milisegundos. La función main debe crear y lanzar los 4 hilos y esperar a que terminen antes de finalizar.
> Amplíe la clase Logger con una función finalizar que cierre el fichero y se asegure de que ningún hilo queda bloqueado. Tendrá posiblemente que añadir nuevos atributos a la clase, así como modificar ligeramente el código de las funciones definidas anteriormente.

Ejercicio 7 (Planificación de tiempo real)

> EJERCICIO DE PLANIFICACIÓN (2 PUNTOS)
> Un sistema de tiempo real consta de 3 tareas con los siguientes parámetros temporales. Las tareas se ejecutan con planificación con prioridades fijas en plazo y desalojo, y los accesos a los objetos compartidos se efectúan mediante el protocolo del techo de prioridad inmediato. La duración máxima de las secciones críticas que ejecutan las tareas sobre los objetos compartidos es:
>
> | Tarea | Tipo | T | C | D | Tarea | M | N |
> |---|---|---|---|---|---|---|---|
> | A | Esporádica | 60 | 4 | 20 | A | | 2 |
> | B | Periódica | 10 | 4 | 10 | B | 2 | - |
> | C | Periódica | 50 | 8 | 50 | C | 1 | 2 |
>
> Se pide:
>
> 1) Calcular las prioridades de las tareas y los techos de prioridad de los objetos compartidos.
> 2) Calcular el tiempo de bloqueo máximo de cada tarea.
> 3) Calcular el tiempo de respuesta máximo de las todas las tareas. ¿Están garantizados los requisitos temporales del sistema?

Ejercicio 8 (Planificación de tiempo real)

```
PREGUNTA DE DESARROLLO (1 PUNTO)
Describa el funcionamiento del planificador del sistema operativo, gestión de tareas, eventos de planificación y
↪ cambio de contexto.
```

Junio 2016 – 6 ejercicios (resuelto)

EXAMEN DE "PROGRAMACIÓN DE SISTEMAS EN TIEMPO REAL", Dispone de 3 horas para hacer el examen. Lea bien las preguntas antes de responder. Durante el examen sólo se responderán preguntas relacionadas con el enunciado del mismo.

Resuelto en la guía: Junio 2016 · Planificación con 4 tareas y dos recursos

Ejercicio 1 (Clases y objetos)

```
EJERCICIOS DE PROGRAMACIÓN (3 PUNTOS)
EJERCICIO (1,5 PUNTOS)

Se pide que diseñe dos clases, clase Pasajero y clase Avion. La clase Pasajero tiene los siguientes atributos: nombre,
apellido y dni (string) y origen y destino del pasajero (string). Implemente las siguientes funciones:
    • Constructor con tres parámetros: nombre, apellido y dni (string). Estos atributos no pueden ser alterados una
      vez se ha creado la instancia del pasajero.
    • Constructor con cinco parámetros: nombre, apellido, dni, origen y destino (string). Los tres atributos
      primeros no pueden ser alterados una vez se ha creado la instancia del pasajero.
    • Operator<< para imprimir la información básica del pasajero (datos personales), así como el origen y destino de
      su viajes, si es que tales datos están disponibles.
    • Funciones cambiar_origen y cambiar_destino. Ambas funciones aceptan un parámetro de tipo string (el
      nuevo aeropuerto) y retornan un valor lógico. Las funciones devuelven false si el origen coincide con el
      ↪ destino o
      el destino con el origen, o si se le pasa el mismo valor que ya tenía el atributo, y true en caso contrario.
    • Función comparar_viaje, a la que se le pasan como parámetros un origen y un destino (string) y retorna true
      si dichos parámetros coinciden con los del Pasajero y false en caso contrario.

La clase Avion tiene los siguientes atributos: marca y modelo (string), número máximo de pasajeros que puede llevar el
avión, origen y destino del avión (string) y un vector con los pasajeros que lleva. Implemente las siguientes
↪ funciones:
    • Constructor con tres parámetros: marca, modelo y número máximo de pasajeros. Estos atributos no pueden ser
      alterados una vez se ha creado la instancia del avión.
    • Operator<< para imprimir la información básica del avión (marca, modelo y número máximo de pasajeros), así
      como el origen, destino, y número de pasajeros actuales, si es que tales datos están disponibles.
    • Función embarcar_pasajero a la que se le pasa un objeto de tipo Pasajero y retorna true si ha conseguido
      embarcarlo exitosamente. Retorna false si el avión está ya lleno o si los datos de viaje del pasajero (origen
      ↪ y
      destino) no coinciden con los del avión (para evitar que un pasajero viaje a un destino distinto al que
      ↪ compró).
      Esta función hace uso de la función comparar_viaje, definida en la clase Pasajero.
    • Función listar_pasajeros que ni acepta ni retorna ningún valor y que imprime por consola la lista de
      pasajeros embarcados.
    • Funciones cambiar_origen y cambiar_destino. Ambas funciones aceptan un parámetro de tipo string (el
      nuevo aeropuerto) y retornan un valor lógico. Las funciones devuelven false si el origen coincide con el
      ↪ destino o
      el destino con el origen, o si se le pasa el mismo valor que ya tenía el atributo, y true en caso contrario.
    • Función desembarcar_pasajeros, que no acepta ni retorna ningún valor, y que sencillamente borra la lista de
      pasajeros del avión.
```

Ejercicio 2 (Clases y objetos)

```
EJERCICIO DE PRÁCTICAS (1,5 PUNTOS)

Amplíe las clases Pasajero y Avion con la siguiente funcionalidad, pero antes defina una estructura Fecha con todos los
atributos públicos y ninguna función. La estructura Fecha tiene tres campos: día, mes y año (int). Amplíe Pasajero:
     • Añada un nuevo atributo para almacenar la fecha de nacimiento.

    •   Cree un nuevo constructor con seis parámetros: nombre, apellido y dni (string) y día, mes y año de
        nacimiento (int). Estos atributos no pueden ser alterados una vez se ha creado la instancia del pasajero.
    •   Función comparar_pasajero, a la que se le pasa un objeto de tipo Pasajero. La función retorna true si los datos
        personales del objeto y del parámetro coinciden, y false en caso contrario.
    •   Función diferencia_anyos que recibe una Fecha y retorna la cantidad de años de diferencia entre la fecha
        suministrada y la fecha de nacimiento del pasajero.

Amplíe la clase Avion:
   • Función buscar_pasajero a la que se le pasan como parámetros el nombre, apellido y dni (todos strings) de
     un pasajero y retorna un valor booleano que indica si el pasajero está embarcado o no. Esta función hace uso de
     la función comparar_pasajero, definida en la clase Pasajero.
   • Función listar_pasajeros_rango_edad a la que se le pasan como parámetros la edad mínima y máxima que
```

```
        se está buscando y una Fecha de referencia, y que retorna un vector de Pasajeros que cumple dicho criterio.
        ↪ Esta
        función hace uso de la función diferencia_anyos, definida en la clase Pasajero.
```

Ejercicio 3 (Hilos y exclusión mutua)

```
EJERCICIOS DE CONCURRENCIA (4 PUNTOS)

EJERCICIO (2 PUNTOS)

Se pide una aplicación concurrente donde una serie de aviones (hilos) intentan aterrizar en un aeropuerto. Defina una
clase Aeropuerto que gestione el procedimiento de aterrizaje, que funciona de la siguiente manera: en un momento dado,
solo puede haber un avión aterrizando en el aeropuerto, y hasta 2 aviones ejecutando la fase de aproximación. Un avión
tiene que pasar primero por la fase de aproximación antes de poder aterrizar. La clase Aeropuerto proporciona una
función aterrizar, que recibe como parámetro el nombre del avión (string), y que se encarga de gestionar el
procedimiento de aterrizaje. El constructor de la clase acepta dos parámetros: el tiempo medio y la desviación de los
aviones al ejecutar la fase de aproximación y aterrizaje (según una distribución normal). Utilice la función
std::this_thread::sleep_for para retrasar la ejecución del hilo.

La función que modela el comportamiento del avión recibe 3 parámetros: el nombre del avión y el tiempo medio y la
desviación del tiempo de viaje del avión (según una distribución normal). El avión tarda un tiempo en llegar al
↪ aeropuerto
de destino, y en cuanto lo consigue invoca la función aterrizar para comenzar el procedimiento de aterrizaje. La
secuencia completa se muestra de manera esquemática a continuación:

                                                        Aeropuerto

                          Aproximación                  Aterrizando
                          (max 2 aviones)               (1 avión)              Avión aterrizado J
                Vuelo     t_aproximación                t_aterrizaje
                t_vuelo

Por ejemplo, una ejecución del programa sería el siguiente:
Avion ‘vuelo 118’ comenzando a aterrizar ... (1)                Avion ‘vuelo 117’ aterrizando (3)
Avion ‘vuelo 116’ comenzando a aterrizar ... (1)                Avion ‘vuelo 118’ en aproximacion (2)
Avion ‘vuelo 113’ comenzando a aterrizar ... (1)                Avion ‘vuelo 117’ ha aterrizado (4)
Avion ‘vuelo 114’ comenzando a aterrizar ... (1)                Avion ‘vuelo 111’ aterrizando (3)
Avion ‘vuelo 115’ comenzando a aterrizar ... (1)                Avion ‘vuelo 116’ en aproximacion (2)
Avion ‘vuelo 112’ aterrizando (3)                               Avion ‘vuelo 111’ ha aterrizado (4)
Avion ‘vuelo 111’ en aproximacion (2)                           Avion ‘vuelo 118’ aterrizando (3)
Avion ‘vuelo 112’ ha aterrizado (4)                             Avion ‘vuelo 113’ en aproximacion (2)

   Avion ‘vuelo 118’ ha aterrizado (4)                          Avion ‘vuelo 113’ ha aterrizado (4)
   Avion ‘vuelo 116’ aterrizando (3)                            Avion ‘vuelo 114’ aterrizando (3)
   Avion ‘vuelo 114’ en aproximacion (2)                        Avion ‘vuelo 114’ ha aterrizado (4)
   Avion ‘vuelo 116’ ha aterrizado (4)                          Avion ‘vuelo 115’ aterrizando (3)
   Avion ‘vuelo 113’ aterrizando (3)                            Avion ‘vuelo 115’ ha aterrizado (4)
   Avion ‘vuelo 115’ en aproximacion (2)                        Fin
```

Ejercicio 4 (Hilos y exclusión mutua)

```
EJERCICIO DE PRÁCTICAS (2 PUNTOS)

Se pide que implemente una solución concurrente a la conjetura de Colatz. Básicamente, la conjetura de Colatz dice que
“para cualquier número natural ‘n’ realicemos los siguientes cálculos: si ‘n’ es par dividámoslo por 2; si ‘n’ es impar
multipliquémoslo por 3 y sumémosle 1 al resultado; y repitiendo el proceso con los números obtenidos, la secuencia
↪ siempre
acabará en 1”. Se pide un hilo que realice la operación sobre los números pares obtenidos en la secuencia, y otro para
↪ los
valores impares. Programe las funciones de los hilo para que continúen procesando hasta que generen un 1. Un ejemplo de
ejecución es el siguiente (donde P indica que el valor de la secuencia ha sido obtenido por el hilo que procesa valores
pares, e I el hilo que procesa los impares):

44, 22(P), 11(P), 34(I), 17(P), 52(I), 26(P), 13(P), 40(I), 20(P), 10(P), 5(P), 16(I),
8(P), 4(P), 2(P), 1(P), ... y 1!
```

Ejercicio 5 (Planificación de tiempo real)

```
EJERCICIO DE PLANIFICACIÓN (2 PUNTOS)
Un sistema de tiempo real consta de 4 tareas con los siguientes parámetros temporales. Las tareas se ejecutan con
planificación con prioridades fijas y desalojo, y los accesos a los objetos compartidos se efectúan mediante el
↪ protocolo del
techo de prioridad inmediato. La duración máxima de las secciones críticas que ejecutan las tareas sobre los objetos
```

```
compartidos es:

              Tarea       Tipo        T     C   D                                Tarea    M    N
                A      Esporádica    100    4   15                                 A      1    -
                B       Periódica     10    4   10                                 B      2    1
                C       Periódica     50    8   50                                 C      -    2
                D      Esporádica     40    6   25                                 D      -    -
Se pide:

    a) Calcular las prioridades de las tareas y los techos de prioridad de los objetos compartidos.
    b) Calcular el tiempo de bloqueo máximo de cada tarea.
    c) Calcular el tiempo de respuesta máximo de las todas las tareas. ¿Están garantizados los requisitos temporales
     ↪  del
       sistema?
```

Ejercicio 6 (Planificación de tiempo real)

```
PREGUNTA DE DESARROLLO (1 PUNTO)
Describa el funcionamiento del planificador del sistema operativo, gestión de tareas, eventos de planificación y
 ↪  cambio de
contexto.
```

Septiembre 2016 – 2 ejercicios (resuelto)

EXAMEN DE "PROGRAMACIÓN DE SISTEMAS EN TIEMPO REAL", Dispone de 3 horas para hacer el examen. Lea bien las preguntas antes de responder. Durante el examen sólo se responderán preguntas relacionadas con el enunciado del mismo.

Resuelto en la guía: Septiembre 2016 · Planificación: un sistema que NO es planificable · Septiembre 2016 · Ejecutivo cíclico: ciclo secundario y código

Ejercicio 1 (Clases y objetos)

```
EJERCICIOS DE PROGRAMACIÓN (3 PUNTOS)

EJERCICIO (1,5 PUNTOS)

Se pide que diseñe dos clases, clase Punto y clase Malla.
(0,5 puntos) La clase Punto tiene dos atributos: coordenadas 'x' e 'y' (double). Implemente además
las siguientes funciones:
    • Constructor con dos parámetros de tipo double
    • Operator<< para imprimir las coordenadas del Punto.
    • Una función distancia_punto que acepta un parámetro de tipo Punto y retorna la distancia a
       dicho punto: d = x4 − x6 ' − y4 − y6 '
   •   Una función desplazar, que no retorna valor, y que acepta una pareja de valores double que
       representan el incremento de posición del Punto
   •   Operator== que comprueba si dos Puntos son iguales, es decir, si se encuentran en la misma
       posición de memoria (por ejemplo, Punto p1; p1 == p1) o si sus coordenadas están
       "suficientemente cerca" (abs(x1-x2)<epsilon, ε < 10-. )

(1 punto) La clase Malla tiene dos atributos: un Punto que representa el centro y un vector de
Puntos que definen la malla en sí. Implemente además las siguientes funciones:
   • Constructor que acepta un Punto, que representa el centro de la Malla, y que tiene que
      añadirse además a la lista de puntos de la misma.
   • Operator<< para imprimir todos los Puntos de la Malla y la posición del centro.
   • Una función add que añade un nuevo Punto a la Malla siempre que no esté ya incluido. Utilice
      la función operator== definido para Punto. Si ha podido añadir el Punto la función retorna
      true, false en caso de que estuviera repetido.
   • Una función distancia_punto_mas_alejado que añade calcula y retorna la distancia al
      Punto que se encuentra más lejos del centro de la Malla.
   • Una función eliminar_puntos_cercanos, que no retorna valor, y que evita que en el vector
      de Puntos haya dos Puntos que se encuentren a una distancia menor del valor suministrado a
      la función. Cualquier pareja de Puntos, no solo Puntos almacenados consecutivamente.
```

Ejercicio 2 (Clases y objetos)

```
EJERCICIO DE PRÁCTICAS (1,5 PUNTOS)

Se pide que diseñe la clase Segmento, que esté formada por dos Puntos (inicial y final). Implemente
las siguientes funciones (es posible que necesite ampliar la funcionalidad de la clase Punto):
    • Constructor con cuatro parámetros de tipo double que representan las cuatro coordenadas
        de los puntos que definen el Segmento.
    • Constructor con dos parámetros de tipo Punto que representan las coordenadas de los
```

puntos que definen el Segmento.
- Operator<< para imprimir las coordenadas del Segmento.
- Una función longitud que calcula y retorna la longitud de un Segmento, como distancia entre sus extremos.
- Una función angulo, que calcula y retorna el ángulo que forma el segmento respecto a los ejes de coordenadas. Considere que el segmento está definido del Punto inicial al final.
- Una función desplazar que no retorna valor, y que acepta una pareja de valores double que representan el incremento de posición del Segmento
- Operator== que comprueba si dos Segmentos son iguales, es decir, si los Puntos que definen los extremos son iguales, tal y como se ha definido la igualdad en el apartado anterior.
- Una función intersecta_ejes que comprueba si el Segmento intersecta alguno de los ejes de coordenadas.

12.4 Resoluciones paso a paso

Mayo 2026 · Ejercicio 1: suscripciones de una plataforma (POO)

Enunciado

Enunciado

EJERCICIO 1: POO (70%)
Se desea implementar un sistema para gestionar las suscripciones de una plataforma de contenidos digitales. Para ello, debe diseñar e implementar las siguientes tres clases:
C_SUSCRIPCION (30%)
- Atributos: ID (entero), precio base mensual (real) y nombre del titular (string).
- Constructores:
  - Constructor que inicialice todos los atributos con valores suministrados.
  - Constructor delegado que reciba el ID y el nombre, inicializando el precio base a 0€.
- Métodos:
  - Métodos getter y setter para el precio base, método getter para el ID.
  - buscar_titular(): recibe una parte del texto del titular de la suscripción y devuelve true si el texto proporcionado está contenido en el nombre del titular.
  - factura_total(): método virtual que retorna el coste mensual de la suscripción.
- Operador <<: imprime los datos del objeto. Utilice un método virtual to_string() para facilitar la implementación en toda la jerarquía de herencia. Un ejemplo de salida por consola: Suscripcion #101 (15.5€), Titular: Juan Perez

C_SUSCRIPCION_PREMIUM (25%)
- Hereda de C_SUSCRIPCION.
- Atributos adicionales: Servicio de soporte (string) y suplemento premium (real).
- Constructores:
  - Constructor que inicialice todos los atributos.
  - Constructor delegado que reciba ID, titular y servicio de soporte, inicializando todos los precios a 0.
  - Constructor que reciba un objeto C_SUSCRIPCION (para copiar los valores de sus atributos) y el nombre del servicio de soporte, inicializando el suplemento a 0.
- Métodos:
  - Getter y setter para el suplemento premium.
  - Redefinir factura_total() para sumar el precio base y el suplemento premium.
  - Redefinir el método to_string(). Ejemplo: Suscripcion #202 (20€), Titular: Ana Lopez (Premium: Soporte 24h + 5.5€)

C_PLATAFORMA (45%)
- Atributos: Nombre de la plataforma (string) y un vector de punteros a C_SUSCRIPCION.
- Constructor: Recibe el nombre de la plataforma. El vector se inicializa vacío.
- Métodos:
  - contar_estandar() y contar_premium(): Devuelven el número de suscripciones de cada tipo (utilice typeid).
  - buscar_por_nombre(): tiene un parámetro, texto del nombre del titular. El método devuelve cuántas suscripciones tienen ese nombre de titular.
  - add_suscripcion(): recibe un objeto del tipo c_suscripción, y añade el puntero al vector solo si el ID no existe ya. Retorna un booleano.
  - cancelar_suscripcion(): recibe un ID de suscripción. Busca, libera la memoria y elimina del vector la suscripción con dicho ID. Recuerde que necesita utilizar un iterador para borrar un elemento de un vector. El método retorna un booleano.
- Operador <<: Imprime el resumen de la plataforma y el listado de suscripciones. Un ejemplo:

StreamMaster: 3 usuarios
Suscripcion #101 (15.5€), Titular: Juan Perez
Suscripcion #102 (12€), Titular: Ana Garcia
Suscripcion #202 (20€), Titular: Luis Lopez (Premium: Soporte 24h - España + 5.5€)

Añada todas las funciones que considere necesarias. Solo puede utilizar la funcionalidad proporcionada por la librería estándar de C++. Haga también una función main() de ejemplo.

Antes de empezar: qué piden y qué teoría usas

Qué piden. Tres clases: una base (`C_Suscripcion`), una derivada (`C_Suscripcion_Premium`) y una que contiene punteros a la base (`C_Plataforma`). Es el patrón de examen de los últimos años.
Teoría que usas.

- Constructor delegado y lista de inicialización (T2).
- `operator<<` que delega en un `to_string()` virtual y protegido (T3).
- Método virtual `factura_total()` y redefinición con `override`.
- `vector<Base*>`, `typeid(*p)` para distinguir el tipo real, `find` en `string`.
- Borrar de un `vector` con iterador (T1).

1 Clase base: atributos, constructores y `to_string` virtual

Atributos `protected` (la derivada los necesita en `factura_total`) y un constructor delegado que reutiliza el completo con precio 0. El destructor es **virtual** porque la clase va a ser base de una jerarquía con punteros.

El `operator<<` es `friend` y **no** es virtual: llama a `to_string()`, que sí lo es. Así, `cout << *p` con `p` de tipo `C_Suscripcion*` imprime según el tipo real.

CPP

```cpp
class C_Suscripcion {
protected:
    int id_;
    double precio_base_;
    string titular_;
    // Método virtual intermedio: operator<< (friend) no es virtual, así que delega aquí.
    virtual string to_string() const {
        ostringstream os;
        os << "Suscripcion #" << id_ << " (" << precio_base_ << "€), Titular: " << titular_;
        return os.str();
    }
public:
    C_Suscripcion(int id, double precio, const string& titular)
        : id_{id}, precio_base_{precio}, titular_{titular} {}
    C_Suscripcion(int id, const string& titular) : C_Suscripcion{id, 0.0, titular} {}
    virtual ~C_Suscripcion() = default;           // destructor virtual: clase base polimórfica

    int get_id() const { return id_; }
    double get_precio() const { return precio_base_; }
    void set_precio(double p) { precio_base_ = p; }

    bool buscar_titular(const string& texto) const {
        return titular_.find(texto) != string::npos;
    }
    virtual double factura_total() const { return precio_base_; }

    friend ostream& operator<<(ostream& os, const C_Suscripcion& s) {
        return os << s.to_string();
    }
};
```

Por qué así: si pusieras el `operator<<` directamente en cada clase no sería polimórfico, porque los operadores libres no son virtuales.

2 Clase derivada: constructores, redefinición de `factura_total` y de `to_string`

Tres constructores: el completo, el delegado (ID, titular, soporte → precios a 0) y el que copia una suscripción (`C_Suscripcion{base}` usa el constructor de copia de la base). La redefinición de `to_string` reutiliza el de la base con **llamada calificada** (`C_Suscripcion::to_string()`), que no es polimórfica y por eso no entra en bucle.

CPP

```cpp
class C_Suscripcion_Premium : public C_Suscripcion {
    string soporte_;
    double suplemento_;
protected:
    string to_string() const override {
        ostringstream os;
        // Reutilizamos el texto de la clase base (llamada calificada, sin polimorfismo)
```

```
        os << C_Suscripcion::to_string() << " (Premium: " << soporte_ << " + " << suplemento_ << "€)";
        return os.str();
    }
public:
    C_Suscripcion_Premium(int id, double precio, const string& titular,
                          const string& soporte, double suplemento)
        : C_Suscripcion{id, precio, titular}, soporte_{soporte}, suplemento_{suplemento} {}
    C_Suscripcion_Premium(int id, const string& titular, const string& soporte)
        : C_Suscripcion_Premium{id, 0.0, titular, soporte, 0.0} {}
    // Copia los atributos de una C_Suscripcion (constructor de copia de la base)
    C_Suscripcion_Premium(const C_Suscripcion& base, const string& soporte)
        : C_Suscripcion{base}, soporte_{soporte}, suplemento_{0.0} {}

    double get_suplemento() const { return suplemento_; }
    void set_suplemento(double s) { suplemento_ = s; }
    double factura_total() const override { return precio_base_ + suplemento_; }
};
```

Por qué `override`: si te equivocas en la firma (por ejemplo, olvidas `const`), el compilador avisa en lugar de crear un método nuevo que nadie llama.

3 `C_Plataforma`: vector de punteros, añadir y cancelar

El vector es de `C_Suscripcion*`: sin punteros habría **slicing** y el premium se convertiría en estándar. `add_suscripcion` recorre buscando el ID y solo añade si no existe. `cancelar_suscripcion` necesita un **iterador** para poder usar `erase`; tras `erase` se devuelve enseguida, así que no hace falta actualizar el iterador.

CPP

```
class C_Plataforma {
    string nombre_;
    vector<C_Suscripcion*> suscripciones_;   // la plataforma NO es propietaria (ver nota)
public:
    explicit C_Plataforma(const string& nombre) : nombre_{nombre} {}

    bool add_suscripcion(C_Suscripcion& nueva) {
        for (const auto* s : suscripciones_)
            if (s->get_id() == nueva.get_id()) return false;
        suscripciones_.push_back(&nueva);
        return true;
    }
    // Quita del vector (el borrado exige iterador). No hace delete: los objetos son del llamante.
    bool cancelar_suscripcion(int id) {
        for (auto it = begin(suscripciones_); it != end(suscripciones_); ++it) {
            if ((*it)->get_id() == id) { suscripciones_.erase(it); return true; }
        }
        return false;
    }
```

Memoria

El enunciado dice «libera la memoria». Eso solo tiene sentido si la plataforma es **propietaria** de objetos creados con `new`. Aquí `main` crea las suscripciones en la pila, así que un `delete` sería un error grave. Si prefieres que la plataforma sea dueña, `add_suscripcion` debería recibir un puntero (o un `unique_ptr`) y `cancelar_suscripcion` haría `delete *it;` antes del `erase`.

4 Contar tipos con `typeid`, buscar por nombre e imprimir

`typeid(*s) == typeid(C_Suscripcion)` compara el **tipo real** (la clase es polimórfica, tiene `virtual`). Hay que desreferenciar: `typeid(s)` sería el tipo del puntero. `buscar_por_nombre` cuenta con `buscar_titular`, que usa `find` y `npos`.

CPP

```
    int contar_estandar() const {
        int n = 0;
        for (const auto* s : suscripciones_) if (typeid(*s) == typeid(C_Suscripcion)) ++n;
        return n;
    }
    int contar_premium() const {
        int n = 0;
        for (const auto* s : suscripciones_) if (typeid(*s) == typeid(C_Suscripcion_Premium)) ++n;
```

```
        return n;
    }
    int buscar_por_nombre(const string& texto) const {
        int n = 0;
        for (const auto* s : suscripciones_) if (s->buscar_titular(texto)) ++n;
        return n;
    }
    friend ostream& operator<<(ostream& os, const C_Plataforma& p) {
        os << p.nombre_ << ": " << p.suscripciones_.size() << " usuarios\n";
        for (const auto* s : p.suscripciones_) os << *s << '\n';
        return os;
    }
};
```

5 `main` de prueba y salida

Se prueba cada requisito: ID duplicado (devuelve `false`), conteo por tipos, búsqueda parcial, `factura_total` a través de una referencia a la base (polimorfismo), el constructor desde una base y cancelar un ID que existe y otro que no.

CPP

```
int main() {
    C_Plataforma plataforma{"StreamMaster"};
    C_Suscripcion s1{101, 15.5, "Juan Perez"};
    C_Suscripcion s2{102, "Ana Garcia"};
    s2.set_precio(12.0);
    C_Suscripcion_Premium s3{202, 20.0, "Luis Lopez", "Soporte 24h - España", 5.5};
    cout << boolalpha;
    cout << plataforma.add_suscripcion(s1) << ' ' << plataforma.add_suscripcion(s2) << ' '
         << plataforma.add_suscripcion(s3) << ' ' << plataforma.add_suscripcion(s3) << '\n';
    cout << plataforma;
    cout << "Estandar: " << plataforma.contar_estandar() << ", Premium: " << plataforma.contar_premium() << '\n';
    cout << "Titulares con 'Juan': " << plataforma.buscar_por_nombre("Juan") << '\n';
    const C_Suscripcion& ref = s3;
    cout << "Factura premium (via referencia base): " << ref.factura_total() << '\n';
    C_Suscripcion_Premium s4{s1, "Soporte basico"};
    cout << s4 << '\n';
    cout << "Cancelar 102: " << plataforma.cancelar_suscripcion(102)
         << ", cancelar 999: " << plataforma.cancelar_suscripcion(999) << '\n';
    cout << plataforma;
}
```

6 Comprobación mental antes de entregar

- ¿`;` tras cada `}` de clase?
- ¿`#include <typeinfo>` (para `typeid`), `<sstream>` y `<vector>`?
- ¿Los métodos que no modifican llevan `const`?
- ¿`factura_total` es `virtual` en la base y `override` en la derivada?
- ¿Se imprime exactamente el formato del enunciado (`#101 (15.5€)`)?

Ojo: errata en la solución de referencia

La solución de referencia **no compila** tal como está: en `C_SUSCRIPCION_PREMIUM::to_string` llama a `C_SUSCRIPCION::imprimir(os)`, un método que no existe (debería ser `C_SUSCRIPCION::to_string(os)`). Con la regla de 2026 («si no compila, la puntuación se divide por 2») esto cuesta la mitad de la nota. Además:

- usa `typeid` sin `#include <typeinfo>`;
- nombra los métodos `total_estandar`/`total_premium`, mientras que el enunciado pide `contar_estandar`/`contar_premium`;
- `cancelar_suscripcion` no libera nada (aquí es correcto porque los objetos viven en `main`, pero hay que decirlo).

Resultado final

Salida del programa (compilado con `-Wall -Wextra`, sin avisos):

Salida

```
true true true false
StreamMaster: 3 usuarios
```

```
Suscripcion #101 (15.5€), Titular: Juan Perez
Suscripcion #102 (12€), Titular: Ana Garcia
Suscripcion #202 (20€), Titular: Luis Lopez (Premium: Soporte 24h - España + 5.5€)
Estandar: 2, Premium: 1
Titulares con 'Juan': 1
Factura premium (via referencia base): 25.5
Suscripcion #101 (15.5€), Titular: Juan Perez (Premium: Soporte basico + 0€)
Cancelar 102: true, cancelar 999: false
StreamMaster: 2 usuarios
Suscripcion #101 (15.5€), Titular: Juan Perez
Suscripcion #202 (20€), Titular: Luis Lopez (Premium: Soporte 24h - España + 5.5€)
```

Mayo 2026 · Ejercicio 2: robots y cintas transportadoras (concurrencia)

Enunciado

Enunciado

```
EJERCICIO 2: CONCURRENCIA (30%)

Haga un programa en C++ que implemente la siguiente simulación. Añada todas las funciones que
considere necesarias. Solo puede utilizar la funcionalidad proporcionada por la librería estándar de
C++. Haga una función main() para ejecutar el código.
Se pide simular el comportamiento de una fábrica de
ensamblaje de piezas, según muestra la figura. Hay tres
robots, que se modelan como hilos, que montan una
pieza a partir de dos partes. Los parámetros que recibe la
función del hilo se describen más abajo. Estas partes les
llegan a través de dos cintas transportadoras (CT en la
figura). Por ejemplo, la pieza que monta el robot 1 utiliza
una parte que llega por la CT1 y otra parte que llega por
la CT3. Antes de montar la pieza, para asegurar la
integridad física de los equipos, el robot tiene que
conseguir acceso exclusivo a las cintas que llevan las
partes que necesita. Este acceso se va a modelar
mediante mutex. Una vez que tiene asegurado el acceso exclusivo, el hilo (1) duerme un tiempo
(simula el ensamblaje; su valor es configurable mediante parámetro), (2) imprime un mensaje por
consola, (3) libera recursos, (4) duerme otro tiempo (simula preparación para ensamblar la siguiente
pieza). Los parámetros del hilo son: su nombre, el número de piezas que tiene que fabricar, referencias
a los mutex de los recursos compartidos (incluyendo uno para usar la consola), y el tiempo que debe
dormir (milisegundos para una simulación rápida). Un ejemplo de ejecución sería:

 [Robot 1] ha procesado pieza 0                    [Robot 3] ha procesado pieza 4
 [Robot 3] ha procesado pieza 0                    [Robot 3] ha terminado
 [Robot 1] ha procesado pieza 1                    [Robot 2] ha procesado pieza 3
 [Robot 3] ha procesado pieza 1                    [Robot 1] ha procesado pieza 2
 [Robot 2] ha procesado pieza 0                    [Robot 2] ha procesado pieza 4
 [Robot 3] ha procesado pieza 2                    [Robot 1] ha procesado pieza 3
 [Robot 2] ha procesado pieza 1                    [Robot 2] ha terminado
 [Robot 3] ha procesado pieza 3                    [Robot 1] ha procesado pieza 4
 [Robot 2] ha procesado pieza 2                    [Robot 1] ha terminado
```

Antes de empezar: qué piden y qué teoría usas

Qué piden. Tres hilos (robots), cada uno necesita **dos** recursos (cintas) a la vez, que se modelan con `mutex`. Después de conseguirlos: duerme, imprime, libera y duerme otra vez.
Teoría que usas.

- `std::thread` con argumentos por referencia (`std::ref`).
- Exclusión mutua con `mutex` y varios recursos a la vez: `scoped_lock` / `std::lock` (T4).
- Un `mutex` separado para la consola.
- `join` de todos los hilos.

1 Identifica los recursos y qué robot usa cuál

Cuatro `mutex`: tres cintas (`ct1`, `ct2`, `ct3`) y la consola. Según la figura: robot 1 usa CT1 y CT3, robot 2 usa CT1 y CT2, robot 3 usa CT2 y CT3. Se pasan **por referencia** a la función del hilo.

2 ¿Por qué hay riesgo de interbloqueo?

Los tres robots comparten cintas por parejas: R1-R2 comparten CT1, R2-R3 comparten CT2 y R1-R3 comparten CT3. Si cada robot cogiera primero «su primera cinta» con `lock()` y después la segunda, podría darse la espera

circular R1→R2→R3→R1. Hay que tomar las **dos a la vez**.

3 La función del hilo

`scoped_lock cintas{m1, m2}` toma los dos `mutex` con el algoritmo de `std::lock` (sin interbloqueo, da igual el orden) y los libera al salir del bloque. El mensaje se imprime dentro de un ámbito interior con otro `mutex` (la consola), porque dos hilos con **cintas distintas** también pueden imprimir a la vez.

CPP

```cpp
// Cada robot necesita DOS cintas (mutex m1, m2) a la vez. std::lock las coge sin interbloqueo.
void f_robot(const string& nombre, int n_piezas, mutex& m1, mutex& m2, mutex& consola, int ms) {
    const chrono::milliseconds t{ms};
    for (int i = 0; i < n_piezas; ++i) {
        {
            scoped_lock cintas{m1, m2};          // (1) acceso exclusivo a las dos cintas
            this_thread::sleep_for(t);           //     ensamblaje
            {                                    // (2) mensaje por consola (otro mutex, otro ámbito)
                lock_guard<mutex> c{consola};
                cout << nombre << " ha procesado pieza " << i << '\n';
            }
        }                                        // (3) se liberan las cintas al salir del ámbito
        this_thread::sleep_for(t);               // (4) preparación de la siguiente pieza
    }
    lock_guard<mutex> c{consola};
    cout << nombre << " ha terminado\n";
}
```

4 `main`: crear los hilos y esperar

Los argumentos que son referencias se envuelven en `ref(...)`; si no, `thread` los **copiaría** y los `mutex` no se pueden copiar (no compila).

CPP

```cpp
int main() {
    mutex ct1, ct2, ct3, consola;
    thread r1{f_robot, "[Robot 1]", 3, ref(ct1), ref(ct3), ref(consola), 20};
    thread r2{f_robot, "[Robot 2]", 3, ref(ct1), ref(ct2), ref(consola), 20};
    thread r3{f_robot, "[Robot 3]", 3, ref(ct2), ref(ct3), ref(consola), 20};
    r1.join(); r2.join(); r3.join();
}
```

5 Comprobación

- ¿Se pone la liberación (`}` del ámbito) **antes** del segundo `sleep`? Sí: mientras prepara la siguiente pieza el robot no retiene las cintas.
- ¿El `mutex` de la consola se toma solo durante el `cout`? Sí.
- ¿Hay `join` para cada hilo? Sí; sin él el programa aborta.

Resultado final

Una ejecución (el orden entre robots puede variar; lo que no varía es que cada uno procesa sus 3 piezas y termina):

Salida

```
[Robot 1] ha procesado pieza 0
[Robot 3] ha procesado pieza 0
[Robot 2] ha procesado pieza 0
[Robot 1] ha procesado pieza 1
[Robot 3] ha procesado pieza 1
[Robot 2] ha procesado pieza 1
[Robot 1] ha procesado pieza 2
[Robot 1] ha terminado
[Robot 3] ha procesado pieza 2
[Robot 2] ha procesado pieza 2
[Robot 3] ha terminado
[Robot 2] ha terminado
```

Junio 2026 · Ejercicio 1: central de eventos (POO)

Enunciado

Enunciado

```
EJERCICIO 1: POO (70%)
Se desea implementar un sistema para gestionar los eventos de un sistema. Para ello, implemente la
siguiente funcionalidad:
C_RECIBIDOR (10%)
   •  Clase abstracta, sin implementación, con dos métodos virtuales abstractos:
         o recibir_valor(): recibe una cadena de texto y no retorna ningún valor.
         o get_nombre(): no recibe ningún parámetro y retorna una cadena de texto.
C_PANTALLA (20%): Hereda de C_RECIBIDOR
   •  Atributos: 'nombre' (string) y 'ancho' de la pantalla (entero).
   •  Constructores:
         o Constructor que inicializa ambos atributos.
         o Constructor delegado, que recibe el 'nombre' y asume el ancho de 'pantalla' es 80.
   •  Métodos:
         o Sobrescribir el método recibir_valor(), imprimiendo por consola la cadena de
            texto que recibe, ajustando la longitud de cada línea al valor del atributo 'ancho'.
         o Sobrescribir el método get_nombre(), retornando el atributo 'nombre'.
C_ALARMA (20%): Hereda de C_RECIBIDOR
   •  Atributos: 'nombre' de la alarma (string).
   •  Constructor: Recibe el 'nombre' de la alarma.
   •  Métodos:
         o Sobrescribir el método recibir_valor(), imprimiendo por consola la cadena de
            texto solo si contiene un valor numérico. Utilice el método int isdigit (char c),
            de la librería <cctype>, que retorna un valor distinto de cero si el carácter que se le
            pasa es un número.
         o Sobrescribir el método get_nombre(), retornando el atributo 'nombre'.
C_CENTRAL (50%):
   •  Atributos: 'nombre' de la central (string) y un mapa de temas a vector de punteros a
      C_Recibidor, llamado 'suscritos'.
   •  Constructor: recibe el 'nombre' de la central de eventos.
   •  Métodos:
         o add_tema(): recibe un string t, que representa el tema de un evento, y lo añade al
            mapa si no está ya añadido. Utilice suscritos[t] para inicializar el vector asociado.
         o suscribirse(): recibe un string, que representa el tema del evento, y un objeto de
            tipo C_Recibidor. El método añade el objeto al vector asociado al tema, pero solo si
            el tema ha sido previamente registrado. El método devuelve un booleano que indica si
            el objeto se pudo añadir.
         o suscribirse_a_todos(): recibe un objeto de tipo C_Recibidor. El método añade el
            objeto a los vectores asociados a todos los temas asociados al mapa.
         o enviar_valor(): recibe un string, el tema del evento, y otro string, el mensaje que se
            va a enviar. Este método invoca, a su vez, el método recibir_valor() de todos los
            objetos almacenados en el vector asociado al tema, pasándole el mensaje recibido. El
            método devuelve un booleano que indica si el tema existe.

   •  Operador <<: imprime el número de temas registrados en la central de eventos, así como el
      nombre de todos los objetos del tipo C_Recibidor asociados a cada tema. Los nombres que
      muestra el siguiente ejemplo son "|Pantalla| : De 15" y "|Alarma| : Beep". Un ejemplo:
Central 'Central eventos' con 3 temas:
        humedad:
                |Pantalla| : De 15,                        |Alarma| : Beep,
        presion:
                |Alarma| : Beep,
        temperatura:
                |Pantalla| : De 80,                        |Alarma| : Beep,
Añada todas las funciones que considere necesarias. Solo puede utilizar la funcionalidad
proporcionada por la librería estándar de C++. Haga también una función main() de ejemplo.
```

Antes de empezar: qué piden y qué teoría usas

Qué piden. Una interfaz (`C_Recibidor`), dos implementaciones (`C_Pantalla`, `C_Alarma`) y una **central** que guarda, para cada tema, los receptores suscritos. Es el patrón *publicación-suscripción* en versión mínima.
Teoría que usas.

- Clase **abstracta** con métodos virtuales puros y destructor virtual (T3).
- Constructor delegado, `override`.
- `map<string, vector<C_Recibidor*>>`: `find` frente a `operator[]` (T1).
- `substr` para trocear una línea; `isdigit` de `<cctype>`.

1 La interfaz

Solo métodos virtuales puros y un **destructor virtual** (aunque aquí no se haga `delete`, es la costumbre correcta en una base polimórfica). `get_nombre` es `const` porque no modifica el objeto.

CPP

```cpp
// Interfaz: clase abstracta pura (solo métodos virtuales puros + destructor virtual)
class C_Recibidor {
public:
    virtual ~C_Recibidor() = default;
    virtual void recibir_valor(const string& msg) = 0;
    virtual string get_nombre() const = 0;
};
```

2 `C_Pantalla`: ajustar el texto al ancho

Un bucle que avanza `ancho_` caracteres cada vuelta y usa `substr(i, ancho_)`. Cuando queda menos texto del ancho, `substr` devuelve solo lo que hay. El constructor delegado fija el ancho a 80.

CPP

```cpp
class C_Pantalla : public C_Recibidor {
    string nombre_;
    int ancho_;
public:
    C_Pantalla(const string& nombre, int ancho) : nombre_{nombre}, ancho_{ancho} {}
    explicit C_Pantalla(const string& nombre) : C_Pantalla{nombre, 80} {}
    void recibir_valor(const string& msg) override {
        // Imprime en líneas de como máximo 'ancho_' caracteres
        for (size_t i = 0; i < msg.size(); i += ancho_)
            cout << msg.substr(i, ancho_) << '\n';
    }
    string get_nombre() const override { return nombre_; }
};
```

Por qué `i < msg.size()` y no `<=`: con `<=`, si el texto mide justo un múltiplo del ancho, se imprimiría una línea vacía de más.

3 `C_Alarma`: ¿contiene algún dígito?

Se recorre el mensaje y se imprime si algún carácter es dígito. El bucle usa `unsigned char`: `isdigit` está definido para valores entre 0 y 255 (o `EOF`); con un `char` negativo (por ejemplo, la ñ en UTF-8) el comportamiento es indefinido.

CPP

```cpp
class C_Alarma : public C_Recibidor {
    string nombre_;
public:
    explicit C_Alarma(const string& nombre) : nombre_{nombre} {}
    void recibir_valor(const string& msg) override {
        for (unsigned char c : msg)             // isdigit exige unsigned char (o EOF)
            if (isdigit(c)) { cout << msg << '\n'; return; }
    }
    string get_nombre() const override { return nombre_; }
};
```

4 `C_Central`: registrar temas y suscribirse

`add_tema` usa `suscritos_[t];` a propósito: `operator[]` **crea** el vector vacío si no existía y no hace nada si ya estaba (justo lo que pide el enunciado). `suscribirse`, en cambio, **no puede** usar `[]`: crearía el tema que se quería comprobar que ya existía. Usa `find`.

CPP

```cpp
class C_Central {
    string nombre_;
    map<string, vector<C_Recibidor*>> suscritos_;
public:
    explicit C_Central(const string& nombre) : nombre_{nombre} {}

    void add_tema(const string& t) { suscritos_[t]; }   // operator[] crea el vector vacío si no existía
```

```
    bool suscribirse(const string& tema, C_Recibidor& r) {
        auto it = suscritos_.find(tema);               // NO usar [] aquí: crearía el tema
        if (it == end(suscritos_)) return false;
        it->second.push_back(&r);
        return true;
    }
    void suscribirse_a_todos(C_Recibidor& r) {
        for (auto& [tema, vec] : suscritos_) vec.push_back(&r);
    }
```

5 Enviar valores e imprimir

`enviar_valor` busca el tema y llama a `recibir_valor` de cada receptor: **polimorfismo**; la central no sabe si habla con una pantalla o con una alarma. `operator<<` escribe en el flujo `os` que recibe (no en `cout`).

CPP

```
    bool enviar_valor(const string& tema, const string& msg) {
        auto it = suscritos_.find(tema);
        if (it == end(suscritos_)) return false;
        for (auto* r : it->second) r->recibir_valor(msg);
        return true;
    }
    friend ostream& operator<<(ostream& os, const C_Central& c) {
        os << "Central '" << c.nombre_ << "' con " << c.suscritos_.size() << " temas:\n";
        for (const auto& [tema, vec] : c.suscritos_) {
            os << tema << ":\n";
            for (const auto* r : vec) os << r->get_nombre() << ", ";
            os << '\n';
        }
        return os;
    }
};
```

6 `main` y salida

CPP

```
int main() {
    C_Pantalla pantalla{"|Pantalla| : De 15", 15};
    C_Alarma alarma{"|Alarma| : Beep"};
    C_Central central{"Central eventos"};
    central.add_tema("temperatura");
    central.add_tema("humedad");
    central.add_tema("presion");
    central.suscribirse("temperatura", pantalla);
    central.suscribirse("humedad", pantalla);
    central.suscribirse_a_todos(alarma);
    cout << boolalpha << central.suscribirse("viento", pantalla) << '\n';
    cout << central;
    central.enviar_valor("temperatura", "Temperatura actual: 25 grados");
    central.enviar_valor("humedad", "humedad sin dato");
    cout << central.enviar_valor("viento", "x") << '\n';
}
```

Ojo: errata en la solución de referencia

En la solución de referencia:

- `operator<<` escribe con `cout << ...` en lugar de `os << ...`: si alguien imprime la central en un `ostringstream` o en un fichero, el texto sale por la consola.
- `C_Recibidor` (declarado como `struct`) **no tiene destructor virtual**: borrar un receptor a través de un puntero a la base sería comportamiento indefinido.
- `isdigit(c)` se llama con `char` con signo; con acentos o `ñ` es comportamiento indefinido.
- `for (int i=0; i<=val.size()/ancho; ++i)` imprime una línea vacía de más cuando la longitud es múltiplo exacto del ancho, y compara `int` con `size_t`.

Resultado final

La pantalla de 15 caracteres parte «Temperatura actual: 25 grados» en dos líneas, la alarma lo imprime entero (tiene dígitos) y «humedad sin dato» no se imprime en la alarma (no tiene números):

Salida

```
false
Central 'Central eventos' con 3 temas:
humedad:
|Pantalla| : De 15, |Alarma| : Beep,
presion:
|Alarma| : Beep,
temperatura:
|Pantalla| : De 15, |Alarma| : Beep,
Temperatura act
ual: 25 grados
Temperatura actual: 25 grados
humedad sin dat
o
false
```

Junio 2026 · Ejercicio 2: barreras (monitores)

Enunciado

Enunciado

```
EJERCICIO 2: CONCURRENCIA (30%)

Haga un programa en C++ que implemente las siguientes clases de utilidad concurrente y haga una
función main() para mostrar un ejemplo de uso. Añada todas las funciones que considere necesarias.
Solo puede utilizar la funcionalidad proporcionada por la librería estándar de C++.
Diseñe una clase Barrera, que se comporte como un monitor, con dos métodos públicos: void
entrar() y void abrir(). La clase implementa esta funcionalidad: los hilos invocan el método
entrar() y quedan bloqueados, hasta que otro hilo “abre” la barrera invocando el método abrir().
A partir de ese momento, todos los hilos se van desbloqueando y van saliendo del método entrar().
Una vez la barrera está abierta, ningún hilo, ni siquiera los que invoquen entrar() tras la apertura,
deberán quedar bloqueados.
Diseñe una clase Barrera_Num, que se comporte como un monitor, que tiene un constructor que
recibe el número máximo de hilos (max_hilos) que pueden quedar bloqueados y tiene un método
void entrar(). El comportamiento en este caso es que, hasta que max_hilos no hayan invocado
el método entrar(), todos ellos deben quedar bloqueados. Cuando hay max_hilos,
automáticamente se abre la barrera y se van desbloqueando todos los hilos que estaban esperando.
Una vez la barrera está abierta, ningún hilo, ni siquiera los que invoquen entrar() tras la apertura,
deberán quedar bloqueados.
Haga una función main() para mostrar un ejemplo de uso de ambas clases. La salida por consola
debe estar protegida frente acceso concurrente. Como pista, coja el mutex en el constructor de las
clases, y no se olvide de que los hilos lo vayan cogiendo y soltando.
Una posible salida por consola podría ser:
 Hilo Barrera #1 entrando                          Hilo Barrera_Num #1 entrando
 Hilo Barrera #2 entrando                          Hilo Barrera_Num #2 entrando
 Hilo Barrera #4 entrando                          Hilo Barrera_Num #3 entrando
 Hilo Barrera #5 entrando                          Hilo Barrera_Num #3 saliendo
 Hilo Barrera #3 entrando                          Hilo Barrera_Num #1 saliendo
 Hilo Barrera #1 saliendo                          Hilo Barrera_Num #2 saliendo
 Hilo Barrera #2 saliendo                          Hilo Barrera_Num #4 entrando
 Hilo Barrera #5 saliendo                          Hilo Barrera_Num #4 saliendo
 Hilo Barrera #3 saliendo                          Hilo Barrera_Num #5 entrando
 Hilo Barrera #4 saliendo                          Hilo Barrera_Num #5 saliendo
```

Antes de empezar: qué piden y qué teoría usas

Qué piden. Dos monitores: `Barrera` (todos esperan hasta que alguien llama a `abrir`) y `Barrera_Num` (se abre sola cuando han llegado `max_hilos`). En ambas, una vez abierta, **nadie** vuelve a bloquearse.
Teoría que usas.

- Monitor = datos + `mutex` + `condition_variable` (T5).
- Espera con **predicado** y `notify_all`.
- Salida por consola protegida con su propio `mutex`.

1 El estado y la condición

En `Barrera`, el estado es una sola bandera `abierta_`. La condición para seguir es «`abierta_` es `true`». Protegida

por un `mutex`. Una vez a `true`, nunca vuelve a `false`: por eso los hilos que llegan tarde pasan sin esperar (el predicado ya es cierto y `wait` ni se duerme).

2 Barrera

`entrar` hace `wait` con predicado. `abrir` cambia la bandera **con el mutex** y despierta a **todos** con `notify_all` (si usaras `notify_one`, solo saldría un hilo).

CPP

```cpp
// Monitor: los hilos esperan hasta que alguien abre; después nadie espera.
class Barrera {
    mutex m_;
    condition_variable cv_;
    bool abierta_ = false;
public:
    void entrar() {
        unique_lock<mutex> lk{m_};
        cv_.wait(lk, [this] { return abierta_; });   // el predicado evita despertares espurios
    }
    void abrir() {
        { lock_guard<mutex> lk{m_}; abierta_ = true; }
        cv_.notify_all();
    }
};
```

3 Barrera_Num

El estado ahora es un contador `llegados_` y la bandera. Cada hilo que entra cuenta; si es el último (`>= max_`), abre y avisa a todos; si no, espera el predicado. El hilo que abre **no espera** (ya sabe que está abierta).

CPP

```cpp
// Se abre sola cuando han llegado max_hilos.
class Barrera_Num {
    mutex m_;
    condition_variable cv_;
    const int max_;
    int llegados_ = 0;
    bool abierta_ = false;
public:
    explicit Barrera_Num(int max_hilos) : max_{max_hilos} {}
    void entrar() {
        unique_lock<mutex> lk{m_};
        if (++llegados_ >= max_) { abierta_ = true; cv_.notify_all(); }
        else cv_.wait(lk, [this] { return abierta_; });
    }
};
```

4 main

Un `mutex` global `consola` y una función `imprimir` que lo toma para cada línea. En el primer ejemplo, cinco hilos entran y se bloquean; el `main` espera 100 ms, abre y todos salen. En el segundo, los hilos llegan escalonados cada 30 ms y la barrera se abre al llegar el tercero: los dos últimos pasan directamente.

CPP

```cpp
int main() {
    Barrera b;
    vector<thread> hs;
    for (int i = 1; i <= 5; ++i)
        hs.emplace_back([&b, i] {
            imprimir("Hilo Barrera #" + to_string(i) + " entrando");
            b.entrar();
            imprimir("Hilo Barrera #" + to_string(i) + " saliendo");
        });
    this_thread::sleep_for(chrono::milliseconds(100));
    imprimir("--- abriendo Barrera ---");
    b.abrir();
    for (auto& h : hs) h.join();
    hs.clear();
```

```
    Barrera_Num bn{3};
    for (int i = 1; i <= 5; ++i)
        hs.emplace_back([&bn, i] {
            this_thread::sleep_for(chrono::milliseconds(30 * i));
            imprimir("Hilo Barrera_Num #" + to_string(i) + " entrando");
            bn.entrar();
            imprimir("Hilo Barrera_Num #" + to_string(i) + " saliendo");
        });
    for (auto& h : hs) h.join();
}
```

5 Comprobación

- ¿Todos los cambios de `abierta_` y `llegados_` se hacen con el `mutex` tomado? Sí.
- ¿Hay `notify_all` tras cambiar la bandera? Sí.
- ¿Puede quedar algún hilo dormido para siempre? No: o la barrera la abre `main` o el hilo número `max_` la abre solo.

Ojo: errata en la solución de referencia

La solución de referencia bloquea el `mutex` en el **constructor** y lo desbloquea desde **otro hilo** (`abrir`). En C++, un `std::mutex` solo puede liberarlo el hilo que lo bloqueó: hacerlo desde otro es comportamiento **indefinido**. Además, lee la bandera `abierta` sin protección (carrera de datos). Funciona «por casualidad» con ciertas implementaciones, pero no es correcto. La forma correcta es la de esta solución: `condition_variable` con predicado.

Resultado final

Una ejecución (el orden entre hilos puede cambiar, el patrón no): primero los cinco entran, luego `--- abriendo Barrera ---`, luego todos salen; en `Barrera_Num` los dos primeros esperan al tercero.

Salida

```
Hilo Barrera #1 entrando
Hilo Barrera #2 entrando
Hilo Barrera #4 entrando
Hilo Barrera #3 entrando
Hilo Barrera #5 entrando
--- abriendo Barrera ---
Hilo Barrera #2 saliendo
Hilo Barrera #1 saliendo
Hilo Barrera #4 saliendo
Hilo Barrera #3 saliendo
Hilo Barrera #5 saliendo
Hilo Barrera_Num #1 entrando
Hilo Barrera_Num #2 entrando
Hilo Barrera_Num #3 entrando
Hilo Barrera_Num #3 saliendo
Hilo Barrera_Num #1 saliendo
Hilo Barrera_Num #2 saliendo
Hilo Barrera_Num #4 entrando
Hilo Barrera_Num #4 saliendo
Hilo Barrera_Num #5 entrando
Hilo Barrera_Num #5 saliendo
```

Mayo 2025 · Ejercicio 1: paquetería (POO)

Enunciado

Enunciado

```
EJERCICIO 1: POO (70%)

Se pide que implemente un programa para gestionar una empresa de paquetería. Para ello, se pide que
diseñe las siguientes tres clases (las dos primeras guardan una relación de herencia):
C_ENVIO (30%)
   •  Tiene los siguientes atributos: código (valor entero), precio (precio del servicio, real), y
      dirección completa del destinatario (string).
   •  Tiene un constructor que inicializa todos los atributos mediante los valores suministrados al
      mismos. Tiene otro constructor, delegado, que recibe solo el código y el destinatario,
      inicializando el precio a 0€.
   •  Tiene métodos para obtener y cambiar el valor del precio de un envío. Tiene un método para
```

```
        obtener el código de un envío.
    •   Tiene un método buscar_destinatario(), que recibe como parámetro una parte de la dirección
        del destinatario y comprueba si, efectivamente, este valor está presente en la dirección del
        objeto.
    •   Tiene un método virtual coste_total(), que retorna el coste del envío.
    •   Tiene el operator<< para imprimir en un flujo los atributos del objeto. Por ejemplo:
                    Envio #12345 (90€), destinatario: Mario, UPCT, 30202, Cartagena

C_ ENVIO_INTERNACIONAL (25%)
    •   Hereda de la clase C_Envio.
    •   Añade como atributos el país de destino (de tipo string) y los costes de aduanas (valor real).
    •   Tiene un constructor que inicializa todos los atributos mediante los valores suministrados al
        mismo. Tiene otro constructor, delegado, que recibe solo el código del envío, el destinatario y
        el país de destino, inicializando a 0 los valores del precio y el coste de la aduana. Tiene otro
        constructor que recibe un objeto del tipo C_Envio (para copiar sus valores) y el país de destino,
        y que inicializa a 0 el coste de las aduanas.
    •   Tiene métodos para obtener y cambiar el coste de la aduana.
    •   Actualiza el método polimórfico coste_total() para incluir también el coste de las aduanas.
    •   Tiene el operator<< para imprimir en un flujo los atributos del objeto. Puede utilizar el texto
        que se genera para un C_Envio. Por ejemplo:
               Envio #54321 (50€), destinatario: Xi Ling, Gran Muralla (China + 15€)

C_CENTRO_LOGISTICO (45%)
    •   Tiene los siguientes atributos: ciudad en la que se encuentra el centro (de tipo string) y vector
        de punteros a C_Envio.
    •   Tiene un constructor, al que se le pasa la ciudad. El vector se inicializa por defecto.
    •   Tiene un método para obtener el número total de envíos normales, otro para obtener el número
        de envíos internacionales (están mezclados en el vector), y un tercero para obtener la cantidad
        de envíos cuya dirección de destinatario contiene el texto que se le pasa a este método como
        parámetro.

    •   Tiene un método add_envio(), que recibe un envío (de cualquiera de los dos tipos) y lo añade
        al vector solo si no existe ya un envío con el mismo código. El método retorna un valor
        booleano para indicar si ha podido realizar esta acción.
    •   Tiene un método borrar_envio(), que recibe el código del envío (de cualquiera de los dos
        tipos) que debe eliminar del vector. El método retorna un valor booleano para indicar si ha
        podido realizar esta acción.
    •   Tiene el operator<< para imprimir en un flujo los atributos del objeto. Por ejemplo:
        Madrid: 3 envios
              Envio #12345 (90€), destinatario: Mario, UPCT, 30202, Cartagena
              Envio #12345 (90€), destinatario: María, UPCT, 30202, Cartagena
              Envio #54321 (50€), destinatario: Xi Ling, Gran Muralla (China + 15€)
Añada todas las funciones que considere necesarias. Solo puede utilizar la funcionalidad
proporcionada por la librería estándar de C++. Haga también una función 'main' en la que muestre
cómo utilizar las clases diseñadas.
```

Antes de empezar: qué piden y qué teoría usas

Qué piden. Tres clases: `C_Envio` (base), `C_Envio_Internacional` (derivada) y `C_Centro_Logistico`, que guarda un `vector<C_Envio*>` mezclado.
Teoría que usas. Constructor delegado, `to_string()` virtual protegido con `operator<<` friend, método virtual `coste_total`, contar por tipo real con `typeid`, `find` en `string`, borrar con iterador (T1-T3).
La diferencia con el ejercicio de suscripciones es mínima: es **el mismo patrón**. Por eso conviene practicarlo hasta escribirlo sin pensar.

1 Clase base `C_Envio`

Atributos privados (la derivada los usa con los `get`), `to_string()` virtual protegido, destructor virtual, `coste_total()` virtual que devuelve el precio.

CPP

```cpp
class C_Envio {
    int codigo_;
    double precio_;
    string destinatario_;
protected:
    virtual string to_string() const {
        ostringstream os;
        os << "Envio #" << codigo_ << " (" << precio_ << "€), destinatario: " << destinatario_;
        return os.str();
    }
public:
```

```
    C_Envio(int codigo, double precio, const string& dest)
        : codigo_{codigo}, precio_{precio}, destinatario_{dest} {}
    C_Envio(int codigo, const string& dest) : C_Envio{codigo, 0.0, dest} {}
    virtual ~C_Envio() = default;

    int get_codigo() const { return codigo_; }
    double get_precio() const { return precio_; }
    void set_precio(double p) { precio_ = p; }
    bool buscar_destinatario(const string& parte) const {
        return destinatario_.find(parte) != string::npos;
    }
    virtual double coste_total() const { return precio_; }
    friend ostream& operator<<(ostream& os, const C_Envio& e) { return os << e.to_string(); }
};
```

2 Clase derivada `C_Envio_Internacional`

Reutiliza el texto de la base con llamada calificada. `coste_total()` suma la aduana usando `C_Envio::coste_total()`: si cambia la regla en la base, la derivada sigue bien. El tercer constructor copia un `C_Envio` (`C_Envio{e}`).

CPP

```
class C_Envio_Internacional : public C_Envio {
    string pais_;
    double aduana_;
protected:
    string to_string() const override {
        ostringstream os;
        os << C_Envio::to_string() << " (" << pais_ << " + " << aduana_ << "€)";
        return os.str();
    }
public:
    C_Envio_Internacional(int codigo, double precio, const string& dest, const string& pais, double aduana)
        : C_Envio{codigo, precio, dest}, pais_{pais}, aduana_{aduana} {}
    C_Envio_Internacional(int codigo, const string& dest, const string& pais)
        : C_Envio_Internacional{codigo, 0.0, dest, pais, 0.0} {}
    C_Envio_Internacional(const C_Envio& e, const string& pais)
        : C_Envio{e}, pais_{pais}, aduana_{0.0} {}
    double get_aduana() const { return aduana_; }
    void set_aduana(double a) { aduana_ = a; }
    double coste_total() const override { return C_Envio::coste_total() + aduana_; }
};
```

3 El centro logístico: contar por tipo y buscar

Para contar, `count_if` con lambda (T1) o un bucle. El tipo real se pregunta con `typeid(*p)`. Para contar destinatarios que contienen un texto, se delega en `buscar_destinatario`.

CPP

```
class C_Centro_Logistico {
    string ciudad_;
    vector<C_Envio*> envios_;
public:
    explicit C_Centro_Logistico(const string& ciudad) : ciudad_{ciudad} {}

    int contar_nacionales() const {
        return count_if(begin(envios_), end(envios_),
                        [](const C_Envio* e) { return typeid(*e) == typeid(C_Envio); });
    }
    int contar_internacionales() const {
        return count_if(begin(envios_), end(envios_),
                        [](const C_Envio* e) { return typeid(*e) == typeid(C_Envio_Internacional); });
    }
    int contar_destinatario(const string& texto) const {
        return count_if(begin(envios_), end(envios_),
                        [&texto](const C_Envio* e) { return e->buscar_destinatario(texto); });
```

4 Añadir sin duplicar y borrar

add_envio busca con find_if si ya existe el código; borrar_envio localiza el elemento y hace erase con el iterador.

CPP

```
    bool add_envio(C_Envio& e) {
        auto it = find_if(begin(envios_), end(envios_),
                          [&e](const C_Envio* x) { return x->get_codigo() == e.get_codigo(); });
        if (it != end(envios_)) return false;
        envios_.push_back(&e);
        return true;
    }
    bool borrar_envio(int codigo) {
        auto it = find_if(begin(envios_), end(envios_),
                          [codigo](const C_Envio* x) { return x->get_codigo() == codigo; });
        if (it == end(envios_)) return false;
        envios_.erase(it);
        return true;
    }
    friend ostream& operator<<(ostream& os, const C_Centro_Logistico& c) {
        os << c.ciudad_ << ": " << c.envios_.size() << " envios\n";
        for (const auto* e : c.envios_) os << *e << '\n';
        return os;
    }
};
```

5 main

CPP

```
int main() {
    C_Centro_Logistico madrid{"Madrid"};
    C_Envio a{12345, 90, "Mario, UPCT, 30202, Cartagena"};
    C_Envio b{12346, 70, "Maria, UPCT, 30202, Cartagena"};
    C_Envio_Internacional c{54321, 50, "Xi Ling, Gran Muralla", "China", 15};
    cout << boolalpha << madrid.add_envio(a) << madrid.add_envio(b) << madrid.add_envio(c)
         << madrid.add_envio(a) << '\n';
    cout << madrid;
    cout << madrid.contar_nacionales() << ' ' << madrid.contar_internacionales() << ' '
         << madrid.contar_destinatario("UPCT") << '\n';
    const C_Envio& r = c;
    cout << "coste (polimorfismo): " << r.coste_total() << '\n';
    cout << madrid.borrar_envio(12346) << madrid.borrar_envio(1) << '\n' << madrid;
}
```

Resultado final

Salida

```
truetruetruefalse
Madrid: 3 envios
Envio #12345 (90€), destinatario: Mario, UPCT, 30202, Cartagena
Envio #12346 (70€), destinatario: Maria, UPCT, 30202, Cartagena
Envio #54321 (50€), destinatario: Xi Ling, Gran Muralla (China + 15€)
2 1 2
coste (polimorfismo): 65
truefalse
Madrid: 2 envios
Envio #12345 (90€), destinatario: Mario, UPCT, 30202, Cartagena
Envio #54321 (50€), destinatario: Xi Ling, Gran Muralla (China + 15€)
```

Mayo 2025 · Ejercicio 2: puente de vía única (monitor)

Enunciado

Enunciado

```
EJERCICIO 2: CONCURRENCIA (30%)

Se pide diseñar una clase monitor C_Puente para el siguiente problema, junto con un ejemplo de uso. La
implementación de la clase tiene que estar libre de condiciones de carrera y ser segura para su utilización
en la siguiente aplicación concurrente. En una región montañosa, existe un puente de vía única que
conecta el norte con el sur. Por razones de seguridad, solo un tren puede cruzar el puente a la vez, y se
```

```
debe evitar que haya un desequilibrio excesivo en el flujo de trenes desde una sola dirección. Para ello, la
clase C_Puente implementa el siguiente comportamiento: un tren cruza solo si no hay otro cruzando y si
la cantidad de trenes esperando del sentido opuesto no supera las 3 unidades. La clase C_Puente:
    • Tiene como atributos un mutex y una variable de condición, contadores de trenes esperando a
      cruzar en cada sentido y sentido en el que está actualmente cruzando el tren (si es que algún tren
      está cruzando el puente).
    • Método que un tren invoca para solicitar cruzar el puente de norte_a_sur() y de sur_a_norte().
      Estos métodos reciben como parámetros un identificador de hilo y un valor que indica el tiempo
      en milisegundos que tarda el tren en cruzar, e implementan la sincronización requerida para
      asegurar la condición de paso por el puente (solo 1 tren a la vez) y de prioridad (si hay más de 3
      trenes esperando a cruzar en el otro sentido, se les tiene que dar prioridad). Los métodos también
      se encargan de la impresión por consola y de esperar el tiempo que tarda el tren en cruzar.
Haga una función main() en la que se creen 10 trenes en cada sentido. Cada tren será un hilo, que recibirá
como parámetros un objeto del tipo C_Puente, un número de identificación y el tiempo que tarda en
cruzar el puente. A continuación, se muestra un ejemplo de salida por consola con 2 trenes en cada
sentido (leyenda: o llegada al puente, | cruzando puente, x salida del puente):

 Tren [NS1] para cruzar: o                          Tren [NS2] cruzando:           |
 Tren [NS1] cruzando:    |                          Tren [NS2] ha cruzado:         x
 Tren [NS2] para cruzar: o                          Tren [SN2] cruzando:           |
 Tren [SN2] para cruzar: o                          Tren [SN2] ha cruzado:         x
 Tren [SN1] para cruzar: o                          Tren [SN1] cruzando:           |
 Tren [NS1] ha cruzado: x                           Tren [SN1] ha cruzado:         x
```

Antes de empezar: qué piden y qué teoría usas

Qué piden. Un monitor `C_Puente`: un solo tren a la vez y, si en el sentido opuesto esperan **más de 3** trenes, se les da prioridad.
Teoría que usas. Monitor con un `mutex`, una `condition_variable` y **predicado compuesto** (T5); contadores de espera por sentido; `unique_lock` para poder soltar el `mutex` mientras el tren cruza.

1 Estado compartido

- `esperando_[2]`: trenes en espera en cada sentido.
- `ocupado_`: ¿hay un tren cruzando?
- Un `mutex` para todo ese estado y otro para la consola.

2 La condición para cruzar

Un tren puede cruzar si **no hay otro cruzando** y **no tiene que ceder**. Con la regla literal («cedo si el otro sentido tiene más de 3 esperando») hay un problema que se ve al ejecutar el `main` pedido, con 10 trenes en cada sentido: ambos sentidos superan 3 a la vez, **ambos ceden** y nadie cruza nunca (**interbloqueo**).

La solución es un **desempate asimétrico**: cede el sentido que tiene **menos** trenes esperando, y si empatan, cede siempre el mismo (sur→norte).

3 El método que cruza

El tren se suma a la cola, espera con predicado, se quita de la cola, marca `ocupado_`, **suelta el `mutex` mientras cruza** (si lo mantuviera, nadie más podría ni registrarse como esperando) y al terminar lo vuelve a tomar, libera el puente y avisa a todos.

CPP

```cpp
// Monitor del puente: pasa UN tren cada vez; si el sentido contrario tiene MÁS de 3 esperando
// y una cola mayor que la nuestra, cedemos. (La regla literal "ceder si el otro tiene >3"
// interbloquea cuando AMBOS sentidos superan 3 a la vez; por eso se compara también el tamaño de las colas.)
class C_Puente {
    mutex m_;
    condition_variable cv_;
    int esperando_[2] = {0, 0};     // 0: norte->sur, 1: sur->norte
    bool ocupado_ = false;
    mutex consola_;

    void cruzar(int sentido, const string& id, int ms) {
        const char* nombre = sentido == 0 ? "NS" : "SN";
        const int otro = 1 - sentido;
        unique_lock<mutex> lk{m_};
        ++esperando_[sentido];
        { lock_guard<mutex> c{consola_}; cout << "Tren [" << nombre << id << "] para cruzar: º\n"; }
```

```
        // Espera mientras haya un tren cruzando O el sentido opuesto tenga prioridad
        auto otro_tiene_prioridad = [&] {
            if (esperando_[otro] <= 3) return false;
            if (esperando_[otro] != esperando_[sentido]) return esperando_[otro] > esperando_[sentido];
            return sentido == 1;                          // empate: gana siempre norte->sur
        };
        cv_.wait(lk, [&] { return !ocupado_ && !otro_tiene_prioridad(); });
        --esperando_[sentido];
        ocupado_ = true;
        { lock_guard<mutex> c{consola_}; cout << "Tren [" << nombre << id << "] cruzando: |\n"; }
        lk.unlock();                                      // cruzar NO debe hacerse con el mutex cogido
        this_thread::sleep_for(chrono::milliseconds(ms));
        lk.lock();
        ocupado_ = false;
        { lock_guard<mutex> c{consola_}; cout << "Tren [" << nombre << id << "] ha cruzado: x\n"; }
        cv_.notify_all();
    }
public:
    void norte_a_sur(const string& id, int ms) { cruzar(0, id, ms); }
    void sur_a_norte(const string& id, int ms) { cruzar(1, id, ms); }
};
```

4 `main`: 10 trenes por sentido

CPP

```
int main() {
    C_Puente puente;
    vector<thread> trenes;
    for (int i = 1; i <= 10; ++i) {
        trenes.emplace_back([&puente, i] { puente.norte_a_sur(to_string(i), 10); });
        trenes.emplace_back([&puente, i] { puente.sur_a_norte(to_string(i), 10); });
    }
    for (auto& t : trenes) t.join();
}
```

5 Comprobación

- ¿Todos los hilos terminan? Con la regla asimétrica, sí (el programa se ejecuta en pocos centenares de milisegundos).
- ¿Se cruza de uno en uno? `ocupado_` lo garantiza: la condición `!ocupado_` y el cambio de `ocupado_` se hacen con el mismo `mutex`.
- ¿Puede haber inanición (un tren que no cruza nunca)? En esta regla la prioridad cambia al vaciarse las colas; en el ejemplo, todos cruzan.

Ojo: errata en la solución de referencia

La regla literal del enunciado («cruza solo si el otro sentido tiene ≤ 3 esperando»), implementada tal cual, **interbloquea** con el `main` que pide el propio enunciado: con 10 trenes por lado, ambos sentidos tienen más de 3 esperando al principio y cada uno cede al otro. Hay que añadir un criterio de desempate (aquí, comparar colas y, si empatan, un sentido fijo).

Resultado final

Los 20 trenes cruzan, de uno en uno. Ejemplo de las primeras líneas de una ejecución:

Salida

```
Tren [NS1] para cruzar: º
Tren [NS1] cruzando: |
Tren [SN1] para cruzar: º
Tren [SN2] para cruzar: º
Tren [NS3] para cruzar: º
Tren [SN3] para cruzar: º
Tren [NS4] para cruzar: º
Tren [SN4] para cruzar: º
Tren [NS5] para cruzar: º
Tren [NS2] para cruzar: º
Tren [NS6] para cruzar: º
Tren [SN5] para cruzar: º
Tren [SN6] para cruzar: º
Tren [NS7] para cruzar: º
```

…

Junio 2025 · Ejercicio 1: nóminas de empleados (POO)

Enunciado

Enunciado

```
EJERCICIO 1: POO (70%)

Se pide que implemente un programa para gestionar las nóminas de los empleados de una empresa.
Para ello, se pide que diseñe las siguientes cuatro clases (la primera es abstracta y las dos siguientes
guardan una relación de herencia con la primera):
C_EMPLEADO (20%)
   •  Esta clase es la base de la jerarquía de herencia y se va a crear como una clase abstracta.
   •  Tiene como atributo el nombre y la dirección del empleado (ambos de tipo string).
   •  Tiene un constructor que recibe valores para inicializar todos los atributos de la clase, y un
      constructor delegado que recibe el nombre del empleado (la dirección se inicializa con “”).
   •  Tiene un método para obtener el nombre. Tiene dos métodos para obtener y cambiar la
      dirección del empleado.
   •  Tiene un método puro virtual get_sueldo(), que en esta clase no hace nada.
   •  Tiene el operator<< para imprimir en un flujo los atributos del objeto. Debe implementarse
      de forma que las clases hijas puedan también utilizarlo.

C_EMPLEADO_SUELDO_FIJO (15%)
   •  Esta clase hereda de C_Empleado.
   •  Añade como atributo el sueldo fijo del empleado (double).
   •  Tiene un constructor que recibe valores para inicializar todos los atributos de la clase, y un
      constructor delegado que recibe únicamente el nombre del empleado (el resto de atributos se
      inicializan con valores por defecto: “” y 0).
   •  Tiene un método para cambiar el valor del sueldo fijo.
   •  Sobrescribe el método virtual get_sueldo(), que retorna el valor del sueldo fijo del empleado.
   •  El operator<< imprime en un flujo los atributos del objeto. Por ejemplo:

C_EMPLEADO_SUELDO_VARIABLE (20%)
   •  Hereda de la clase C_Empleado.
   •  Añade como atributos el sueldo base y el complemento (ambos valores reales).
   •  Tiene un constructor que recibe valores para inicializar todos los atributos de la clase, y un
      constructor delegado que recibe únicamente el nombre del empleado (el resto de atributos se
      inicializan con valores por defecto: “” y 0).
   •  Tiene dos métodos para cambiar el valor del sueldo base y el complemento.
   •  Sobrescribe el método virtual get_sueldo(), que retorna el sueldo base más el complemento.
   •  El operator<< imprime en un flujo los atributos del objeto como en el caso anterior.

C_EMPRESA (45%)
   •  Tiene los siguientes atributos: nombre de la empresa (de tipo string) y vector de punteros a
      C_Empleado.
   •  Tiene un constructor, al que se le pasa el nombre de la empresa. El vector se inicializa por
      defecto.

   •  Tiene un método para obtener el número total de empleados con sueldo fijo, el número total de
      empleados con sueldo variables (ambos tipos están mezclados en el vector), y un tercer método
      para obtener un puntero al empleado con el mayor sueldo.
   •  Tiene un método add_empleado(), que recibe un empleado (de cualquiera de los dos tipos) y
      lo añade al vector.
   •  Tiene un método borrar_empleado(), que recibe el nombre de un empleado y lo elimina del
      vector. El método retorna un valor booleano para indicar si ha podido realizar esta acción.
   •  Tiene el operator<< para imprimir en un flujo los atributos del objeto. Por ejemplo:
              Empresa: UPCT, lista de empleados:
Añada todas las funciones que considere necesarias. Solo puede utilizar la funcionalidad
proporcionada por la librería estándar de C++. Haga también una función ‘main’ en la que muestre
cómo utilizar las clases diseñadas.
```

Antes de empezar: qué piden y qué teoría usas

Qué piden. Una jerarquía con **clase base abstracta** (`C_Empleado`, con `get_sueldo()` virtual puro), dos derivadas (sueldo fijo y variable) y una empresa que guarda `vector<C_Empleado*>`.
Teoría que usas. Clase abstracta, método virtual puro, `operator<<` que las hijas puedan reutilizar (gancho virtual protegido), `max_element` con lambda para el mayor sueldo, `typeid` para contar tipos, borrado con iterador.

1 La base abstracta

El `operator<<` es `friend` y delega en un método virtual protegido (`imprimir`) para que las clases hijas lo **reutilicen sin reescribirlo**. `get_sueldo()` es virtual puro: la clase ya no se puede instanciar.

CPP

```cpp
class C_Empleado {
    string nombre_, direccion_;
protected:
    virtual void imprimir(ostream& os) const {      // gancho virtual para el operator<< (friend)
        os << "Nombre: " << nombre_ << " (" << direccion_ << "), Sueldo: " << get_sueldo() << "€";
    }
public:
    C_Empleado(const string& nombre, const string& dir) : nombre_{nombre}, direccion_{dir} {}
    explicit C_Empleado(const string& nombre) : C_Empleado{nombre, ""} {}
    virtual ~C_Empleado() = default;
    const string& get_nombre() const { return nombre_; }
    const string& get_direccion() const { return direccion_; }
    void set_direccion(const string& d) { direccion_ = d; }
    virtual double get_sueldo() const = 0;          // pura virtual => clase abstracta
    friend ostream& operator<<(ostream& os, const C_Empleado& e) { e.imprimir(os); return os; }
};
```

2 Las dos derivadas

Cada una implementa `get_sueldo()`: fijo → el valor; variable → base + complemento. El constructor delegado fija el resto por defecto.

CPP

```cpp
class C_Empleado_Sueldo_Fijo : public C_Empleado {
    double sueldo_fijo_;
public:
    C_Empleado_Sueldo_Fijo(const string& n, const string& d, double s)
        : C_Empleado{n, d}, sueldo_fijo_{s} {}
    explicit C_Empleado_Sueldo_Fijo(const string& n) : C_Empleado_Sueldo_Fijo{n, "", 0.0} {}
    void set_sueldo_fijo(double s) { sueldo_fijo_ = s; }
    double get_sueldo() const override { return sueldo_fijo_; }
};

class C_Empleado_Sueldo_Variable : public C_Empleado {
    double base_, complemento_;
public:
    C_Empleado_Sueldo_Variable(const string& n, const string& d, double b, double c)
        : C_Empleado{n, d}, base_{b}, complemento_{c} {}
    explicit C_Empleado_Sueldo_Variable(const string& n) : C_Empleado_Sueldo_Variable{n, "", 0.0, 0.0} {}
    void set_base(double b) { base_ = b; }
    void set_complemento(double c) { complemento_ = c; }
    double get_sueldo() const override { return base_ + complemento_; }
};
```

3 `C_Empresa`

Contar tipos con `typeid`; el mayor sueldo con `max_element` y una lambda que compara `get_sueldo()` (devuelve un puntero al empleado); `borrar_empleado` con iterador.

CPP

```cpp
class C_Empresa {
    string nombre_;
    vector<C_Empleado*> empleados_;
public:
    explicit C_Empresa(const string& n) : nombre_{n} {}
    void add_empleado(C_Empleado& e) { empleados_.push_back(&e); }
    int total_fijos() const {
        return count_if(begin(empleados_), end(empleados_),
                        [](const C_Empleado* e) { return typeid(*e) == typeid(C_Empleado_Sueldo_Fijo); });
    }
    int total_variables() const {
        return count_if(begin(empleados_), end(empleados_),
                        [](const C_Empleado* e) { return typeid(*e) == typeid(C_Empleado_Sueldo_Variable); });
    }
    // Puntero al de mayor sueldo (nullptr si no hay empleados)
    const C_Empleado* mayor_sueldo() const {
```

```
        if (empleados_.empty()) return nullptr;
        return *max_element(begin(empleados_), end(empleados_),
                            [](const C_Empleado* a, const C_Empleado* b) { return a->get_sueldo() < b->get_sueldo();
                            ↪ });
    }
    bool borrar_empleado(const string& nombre) {
        auto it = find_if(begin(empleados_), end(empleados_),
                          [&nombre](const C_Empleado* e) { return e->get_nombre() == nombre; });
        if (it == end(empleados_)) return false;
        empleados_.erase(it);
        return true;
    }
    friend ostream& operator<<(ostream& os, const C_Empresa& e) {
        os << "Empresa: " << e.nombre_ << ", lista de empleados:\n";
        for (const auto* x : e.empleados_) os << *x << '\n';
        return os;
    }
};
```

4 main

CPP

```
int main() {
    C_Empresa upct{"UPCT"};
    C_Empleado_Sueldo_Fijo juan{"Juan", "Calle calle", 2500};
    C_Empleado_Sueldo_Fijo antonio{"Antonio", "Plaza plaza", 2500};
    C_Empleado_Sueldo_Variable maria{"Maria", "Avenida avda", 2000, 1000};
    upct.add_empleado(juan); upct.add_empleado(antonio); upct.add_empleado(maria);
    cout << upct;
    cout << upct.total_fijos() << ' ' << upct.total_variables() << '\n';
    cout << "Mayor sueldo: " << *upct.mayor_sueldo() << '\n';
    cout << boolalpha << upct.borrar_empleado("Antonio") << ' ' << upct.borrar_empleado("Nadie") << '\n';
    cout << upct;
}
```

Resultado final

Salida

```
Empresa: UPCT, lista de empleados:
Nombre: Juan (Calle calle), Sueldo: 2500€
Nombre: Antonio (Plaza plaza), Sueldo: 2500€
Nombre: Maria (Avenida avda), Sueldo: 3000€
2 1
Mayor sueldo: Nombre: Maria (Avenida avda), Sueldo: 3000€
true false
Empresa: UPCT, lista de empleados:
Nombre: Juan (Calle calle), Sueldo: 2500€
Nombre: Maria (Avenida avda), Sueldo: 3000€
```

Junio 2025 · Ejercicio 2: fábrica de patinetes (monitor)

Enunciado

Enunciado

```
EJERCICIO 2: CONCURRENCIA (30%)

Se pide desarrollar un programa que simula una fábrica de patinetes eléctricos. Para ello, programe tres
tipos de hilos: uno fabrica los cuerpos del patinete (tarda 500ms por cuerpo), otro fabrica las ruedas
(tarda 200ms por rueda) y otro termina la construcción del patinete ensamblando un cuerpo con dos
ruedas (tarda 300ms por patinete). Diseñe también una clase monitor C_Almacen, que esté libre de
condiciones de carrera y pueda utilizarse en una aplicación concurrente, y que almacene los dos tipos de
pieza que se requieren para fabricar los patinetes. Esta clase tiene un vector para almacenar cuerpos de
patinete y otro para las ruedas (para no complicar la implementación, estas partes se van a implementar
mediante strings). Añada los mutex y variables de condición necesarias para asegurar la exclusión mutua
en el acceso a los vectores y para evitar que se saquen piezas de un vector que está vacío.
Haga una función main() en la que se creen 2 instancias de cada hilo. El programa tiene que terminar
limpiamente (es decir, los 6 hilos creados tienen que terminar de forma normal, llegando al final de su
función) 2 segundos después de haber comenzado. Utilice una variable booleana en C_Almacen para
controlar el momento en que tienen que terminar todos los hilos. Una posible ejecución:

 [P1] rueda 1                                          [P1] rueda 4
```

```
[P2] rueda 1                                    [P2] rueda 4
[P1] rueda 2                                    [P1] cuerpo 2
[P2] rueda 2                                    [P2] cuerpos termina
[P1] cuerpo 1                                   [P1] cuerpos termina
[P2] cuerpo 1                                   [P1] rueda 5
[P1] rueda 3                                    [P1] ruedas termina
[P2] rueda 3                                    [P2] rueda 5
[E1] ensambla patinete 1 usando [P2]            [P2] ruedas termina
   cuerpo 1, [P1] rueda 1, [P2] rueda 1         [E1] termina
[E2] ensambla patinete 1 usando [P1]            [E2] termina
   cuerpo 1, [P2] rueda 2, [P1] rueda 2         Simulación terminada.
```

Antes de empezar: qué piden y qué teoría usas

Qué piden. Un monitor `C_Almacen` con dos vectores (cuerpos y ruedas), tres tipos de hilo (productor de cuerpos, productor de ruedas, ensamblador) y un cierre limpio a los 2 segundos mediante una bandera booleana.
Teoría que usas. Monitor con un `mutex` y una `condition_variable`; esperar «hay una pieza» con predicado; **condición de cierre** en el predicado para que ningún hilo se quede dormido; `join` de seis hilos.

1 Estado y condiciones

Un `mutex`, una `condition_variable`, `vector<string> cuerpos_`, `ruedas_` y `bool terminar_`. Condiciones: el ensamblador necesita `cuerpos_` ≥ 1 **y** `ruedas_` ≥ 2; **o** `terminar_`.

2 El monitor

Cada `poner_*` añade la pieza con el `mutex` y avisa. `sacar_piezas` espera con predicado `(hay piezas suficientes) || terminar_` y devuelve `false` si se despertó por el cierre sin piezas. `terminar()` pone la bandera con el `mutex` y despierta a **todos**.

CPP

```cpp
mutex consola;
template <class... A> void imprimir(A&&... a) {
    lock_guard<mutex> lk{consola};
    (cout << ... << a) << '\n';
}

// Monitor: dos vectores de piezas, un mutex y una variable de condición para el ensamblador.
class C_Almacen {
    mutex m_;
    condition_variable hay_piezas_;
    vector<string> cuerpos_, ruedas_;
    bool terminar_ = false;
public:
    void poner_cuerpo(const string& c) {
        { lock_guard<mutex> lk{m_}; cuerpos_.push_back(c); }
        hay_piezas_.notify_all();
    }
    void poner_rueda(const string& r) {
        { lock_guard<mutex> lk{m_}; ruedas_.push_back(r); }
        hay_piezas_.notify_all();
    }
    // Espera 1 cuerpo + 2 ruedas. Devuelve false si se ha pedido terminar (y no se saca nada).
    bool sacar_piezas(string& cuerpo, string& r1, string& r2) {
        unique_lock<mutex> lk{m_};
        hay_piezas_.wait(lk, [this] { return terminar_ || (!cuerpos_.empty() && ruedas_.size() >= 2); });
        if (terminar_) return false;
        cuerpo = cuerpos_.back(); cuerpos_.pop_back();
        r1 = ruedas_.back(); ruedas_.pop_back();
        r2 = ruedas_.back(); ruedas_.pop_back();
        return true;
    }
    void terminar() {
        { lock_guard<mutex> lk{m_}; terminar_ = true; }
        hay_piezas_.notify_all();
    }
    bool debe_terminar() { lock_guard<mutex> lk{m_}; return terminar_; }
};
```

3 Los tres tipos de hilo

Cada uno es un bucle `while (!a.debe_terminar())` con su `sleep_for` (500, 200 y 300 ms).

CPP

```
void f_cuerpos(C_Almacen& a, const string& id) {
    for (int n = 1; !a.debe_terminar(); ++n) {
        this_thread::sleep_for(500ms);
        a.poner_cuerpo("[" + id + "] cuerpo " + to_string(n));
        imprimir("[", id, "] cuerpo ", n);
    }
    imprimir("[", id, "] cuerpos termina");
}
void f_ruedas(C_Almacen& a, const string& id) {
    for (int n = 1; !a.debe_terminar(); ++n) {
        this_thread::sleep_for(200ms);
        a.poner_rueda("[" + id + "] rueda " + to_string(n));
        imprimir("[", id, "] rueda ", n);
    }
    imprimir("[", id, "] ruedas termina");
}
void f_ensamblar(C_Almacen& a, const string& id) {
    string c, r1, r2;
    for (int n = 1; a.sacar_piezas(c, r1, r2); ++n) {
        this_thread::sleep_for(300ms);
        imprimir("[", id, "] ensambla patinete ", n, " usando ", c, ", ", r1, ", ", r2);
    }
    imprimir("[", id, "] termina");
}
```

4 `main`

Crea dos hilos de cada tipo, duerme 2 s, llama a `terminar()` y hace `join` de los seis.

CPP

```
int main() {
    C_Almacen almacen;
    vector<thread> hs;
    for (int i = 1; i <= 2; ++i) {
        hs.emplace_back(f_cuerpos, ref(almacen), "P" + to_string(i));
        hs.emplace_back(f_ruedas, ref(almacen), "P" + to_string(i));
        hs.emplace_back(f_ensamblar, ref(almacen), "E" + to_string(i));
    }
    this_thread::sleep_for(2s);
    almacen.terminar();
    for (auto& h : hs) h.join();
    imprimir("Simulación terminada.");
}
```

5 Comprobación

- ¿Puede quedarse un ensamblador dormido para siempre al terminar? No: el predicado incluye `terminar_` y `terminar()` hace `notify_all`.
- ¿Se leen/escriben los vectores siempre con el `mutex` tomado? Sí, dentro del monitor.
- ¿La salida por consola está protegida? Sí, con el `mutex consola`.

Resultado final

Primeras líneas de una ejecución (el orden entre hilos puede cambiar):

Salida

```
[P1] rueda 1
[P2] rueda 1
[P2] rueda 2
[P1] rueda 2
[P1] cuerpo 1
[P2] cuerpo 1
[P2] rueda 3
[P1] rueda 3
[E2] ensambla patinete 1 usando [P1] cuerpo 1, [P2] rueda 2, [P1] rueda 2
[E1] ensambla patinete 1 usando [P2] cuerpo 1, [P2] rueda 1, [P1] rueda 1
```

```
[P1] rueda 4
[P2] rueda 4
[P1] cuerpo 2
[P2] cuerpo 2
…
```

Febrero 2017 · Planificación con techo de prioridad inmediato

Enunciado

Un sistema de tiempo real consta de 3 tareas con los siguientes parámetros temporales. Las tareas se ejecutan con planificación con prioridades fijas en plazo y con expulsión, y los accesos a los objetos compartidos se efectúan mediante el protocolo del techo de prioridad inmediato. La duración máxima de las secciones críticas que ejecutan las tareas sobre los objetos compartidos es:

Tarea	Tipo	T	C	D	Sección crítica sobre M	Sección crítica sobre N
A	Esporádica	60	4	20	—	2
B	Periódica	10	4	10	2	—
C	Periódica	50	8	50	1	2

Se pide: 1) calcular las prioridades de las tareas y los techos de prioridad de los objetos compartidos; 2) calcular el tiempo de bloqueo máximo de cada tarea; 3) calcular el tiempo de respuesta máximo de todas las tareas y decir si están garantizados los requisitos temporales.

Antes de empezar: qué piden y qué teoría usas

Qué piden. Prioridades (DM), techos, bloqueos y tiempos de respuesta. Es el cálculo completo de T6 en tres tablas.
Teoría que usas. Deadline Monotonic; techo = máxima prioridad de las tareas que usan el recurso; B_i = máxima sección crítica de tareas de **menor** prioridad sobre recursos con techo **≥** P_i; recurrencia del tiempo de respuesta.

1 Prioridades (DM)

Menor plazo ⇒ mayor prioridad. Plazos: B = 10, A = 20, C = 50. Con mayor número = mayor prioridad: $P_B = 3,\ P_A = 2,\ P_C = 1$.

2 Techos de prioridad

- M lo usan B y C ⇒ techo$(M) = \max(P_B, P_C) = 3$.
- N lo usan A y C ⇒ techo$(N) = \max(P_A, P_C) = 2$.

3 Bloqueos

Solo cuentan las secciones críticas de tareas de **menor** prioridad sobre recursos cuyo techo sea **≥ P** de la tarea que se bloquea.

- **C** (la de menor prioridad): $B_C = 0$.
- **A** ($P = 2$): las de menor prioridad es C. Usa M (techo 3 ≥ 2, dura 1) y N (techo 2 ≥ 2, dura 2). $B_A = \max(1, 2) = 2$.
- **B** ($P = 3$): las de menor prioridad son A y C. De A: N tiene techo 2 < 3, **no cuenta**. De C: M tiene techo 3 ≥ 3 (dura 1); N tiene techo 2 < 3, no cuenta. $B_B = 1$.

4 Tiempos de respuesta

$R_i = C_i + B_i + \sum_{j \in hp(i)} \lceil R_i/T_j \rceil C_j$.

- **B** (la más prioritaria): $R_B = 4 + 1 = 5 \leq 10$. Cumple.
- **A**: $R^{(0)} = 4 + 2 = 6$; $R^{(1)} = 4 + 2 + \lceil 6/10 \rceil \cdot 4 = 10$; $R^{(2)} = 4 + 2 + \lceil 10/10 \rceil \cdot 4 = 10$. $R_A = 10 \leq 20$. Cumple.
- **C**: $R^{(0)} = 8$; $R^{(1)} = 8 + \lceil 8/60 \rceil 4 + \lceil 8/10 \rceil 4 = 16$; $R^{(2)} = 8 + \lceil 16/60 \rceil 4 + \lceil 16/10 \rceil 4 = 8 + 4 + 8 = 20$; $R^{(3)} = 8 + \lceil 20/60 \rceil 4 + \lceil 20/10 \rceil 4 = 20$. $R_C = 20 \leq 50$. Cumple.

Simulador de planificación: RM / DM / EDF y tiempo de respuesta

Política: RM (periodo) + tarea C = cómputo, T = periodo, D = plazo, B = bloqueo máximo

Tarea	C	T	D	B	
τ1	4	60	20	2	×
τ2	4	10	10	1	×
τ3	8	50	50	0	×
τ4	1	10	10	0	×

Utilización U = **0,727** · cota de Liu–Layland (n=4) = **0,757** · hiperperiodo = **300**. U ≤ cota: planificable por la condición suficiente.

Tarea	Prioridad	Iteraciones de R	R	D	¿Cumple?
τ1	1	6 → 19 → 24	24	20	✘
τ2	4	5 → 5	5	10	✔
τ3	2	8 → 13 → 18 → 18	18	50	✔
τ4	3	1 → 5 → 5	5	10	✔

Cronograma (primeros 120 instantes, sin bloqueos). Ningún plazo incumplido.



Captura del simulador interactivo (disponible en la versión web de la guía).

5 Comprobación con código

Se ha verificado con un script (prioridades, techos, bloqueos y la recurrencia): los valores coinciden. El simulador de arriba usa los mismos datos (la columna B contiene los bloqueos).

Ojo: errata en la solución de referencia

La solución de referencia calcula $B_B = 2$ (toma también la sección crítica sobre N). Pero el techo de N es 2 y la prioridad de B es 3: con el techo de prioridad inmediato, B **no** se bloquea por N. El valor correcto es $B_B = 1$, y por tanto $R_B = 5$ (no 6). La conclusión (planificable) no cambia, pero el valor sí.

Resultado final

Tarea	Prioridad	Bloqueo B	Respuesta R	Plazo D	¿Cumple?
B	3	1	5	10	Sí
A	2	2	10	20	Sí
C	1	0	20	50	Sí

Techos: M = 3, N = 2. **El sistema está garantizado.**

Junio 2016 · Planificación con 4 tareas y dos recursos

Enunciado

Un sistema de tiempo real consta de 4 tareas con los siguientes parámetros temporales. Las tareas se ejecutan con planificación con prioridades fijas y expulsión, y los accesos a los objetos compartidos se efectúan mediante el protocolo del techo de prioridad inmediato. La duración máxima de las secciones críticas es:

Tarea	Tipo	T	C	D	Sección crítica sobre M	Sección crítica sobre N
A	Esporádica	100	4	15	1	—
B	Periódica	10	4	10	2	1
C	Periódica	50	8	50	—	2
D	Esporádica	40	6	25	—	—

Se pide: a) prioridades y techos; b) bloqueo máximo de cada tarea; c) tiempo de respuesta máximo de todas las tareas y si están garantizados los requisitos temporales.

Antes de empezar: qué piden y qué teoría usas

Qué piden. Lo mismo que el ejercicio anterior con una tarea más (D) que **no usa recursos**. El matiz: D también se bloquea (por C) aunque no use nada.
Teoría que usas. DM, techos, bloqueo ICPP, recurrencia.

1 Prioridades (DM)

Plazos: B = 10, A = 15, D = 25, C = 50. Prioridades: $P_B = 4,\ P_A = 3,\ P_D = 2,\ P_C = 1$.

2 Techos

- M: usan A y B ⇒ techo $= \max(3, 4) = 4$.
- N: usan B y C ⇒ techo $= \max(4, 1) = 4$.

3 Bloqueos

- **B** ($P = 4$): tareas de menor prioridad: A (M, 1; techo 4 ≥ 4 ✓), C (N, 2; techo 4 ≥ 4 ✓). $B_B = \max(1, 2) = 2$.
- **A** ($P = 3$): menores: D (nada) y C (N, 2; techo 4 ≥ 3 ✓). $B_A = 2$.
- **D** ($P = 2$): menor: C (N, 2; techo 4 ≥ 2 ✓). $B_D = 2$. Aunque D no use recursos, C puede estar en la sección crítica de N **con prioridad 4** (el techo) cuando D llega, y D tiene que esperar.
- **C**: $B_C = 0$.

4 Tiempos de respuesta

- **B**: $R = 4 + 2 = 6 \leq 10$.
- **A**: $R^{(0)} = 4 + 2 = 6$; $R^{(1)} = 6 + \lceil 6/10 \rceil \cdot 4 = 10$; $R^{(2)} = 6 + \lceil 10/10 \rceil \cdot 4 = 10$. $R_A = 10 \leq 15$.
- **D**: $R^{(0)} = 6 + 2 = 8$; $R^{(1)} = 8 + \lceil 8/10 \rceil 4 + \lceil 8/100 \rceil 4 = 16$; $R^{(2)} = 8 + \lceil 16/10 \rceil 4 + \lceil 16/100 \rceil 4 = 8 + 8 + 4 = 20$; $R^{(3)} = 8 + \lceil 20/10 \rceil 4 + 4 = 20$. $R_D = 20 \leq 25$.
- **C**: $R^{(0)} = 8$; $R^{(1)} = 8 + 4 + 4 + 6 = 22$ (cada $\lceil 8/T \rceil = 1$ para $T = 10, 100, 40$); $R^{(2)} = 8 + \lceil 22/10 \rceil 4 + \lceil 22/100 \rceil 4 + \lceil 22/40 \rceil 6 = 8 + 12 + 4 + 6 = 30$; $R^{(3)} = 8 + \lceil 30/10 \rceil 4 + 4 + \lceil 30/40 \rceil 6 = 8 + 12 + 4 + 6 = 30$. $R_C = 30 \leq 50$.

Simulador de planificación: RM / DM / EDF y tiempo de respuesta

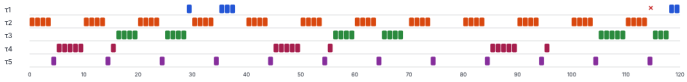
Política: RM (periodo) + tarea C = cómputo, T = periodo, D = plazo, B = bloqueo máximo

Tarea	C	T	D	B	
τ1	4	100	15	2	×
τ2	4	10	10	2	×
τ3	8	50	50	0	×
τ4	6	40	25	2	×
τ5	1	10	10	0	×

Utilización U = **0,850** · cota de Liu–Layland (n=5) = **0,743** · hiperperiodo = **200**. U > cota: la condición suficiente no concluye; usa el tiempo de respuesta.

Tarea	Prioridad	Iteraciones de R	R	D	¿Cumple?
τ1	1	6 → 25	25	15	✘
τ2	5	6 → 6	6	10	✔
τ3	2	8 → 19 → 24 → 29 → 29	29	50	✔
τ4	3	8 → 13 → 18 → 18	18	25	✔
τ5	4	1 → 5 → 5	5	10	✔

Cronograma (primeros 120 instantes, sin bloqueos). **Hay plazos incumplidos (×).**



Captura del simulador interactivo (disponible en la versión web de la guía).

Resultado final

Tarea	Prioridad	B	R	D	¿Cumple?
B	4	2	6	10	Sí
A	3	2	10	15	Sí
D	2	2	20	25	Sí
C	1	0	30	50	Sí

Techos: M = 4, N = 4. **El sistema está garantizado.** Utilización $U = 0{,}75$.

Septiembre 2016 · Planificación: un sistema que NO es planificable

Enunciado

Un sistema de tiempo real tiene 3 tareas y 2 objetos compartidos, O1 y O2. La tabla contiene los atributos temporales de las tareas, incluidos los tiempos de ejecución de las operaciones protegidas que ejecutan:

Tarea	Tipo	T	C	D	Sección crítica sobre O1	Sección crítica sobre O2
T_C	Periódica	120	12	120	2	3
T_B	Esporádica	500	8	25	3	1
T_A	Periódica	20	10	20	4	—

El método de planificación es de prioridades fijas con expulsión, monotónicas en plazo, y el acceso a los objetos protegidos se realiza mediante el techo de prioridad inmediato. Se pide: 1) asignar prioridades y calcular los techos de los objetos; 2) calcular los bloqueos máximos; 3) calcular los tiempos de respuesta en el peor caso y decir si el sistema está garantizado.

Antes de empezar: qué piden y qué teoría usas

Qué piden. El mismo procedimiento. La novedad: aquí **sí hay una tarea que falla**, y hay que reconocerlo en la iteración (el valor supera el plazo).

1 Prioridades y techos

Plazos: T_A = 20, T_B = 25, T_C = 120. $P_A = 3,\ P_B = 2,\ P_C = 1$.

- O1: lo usan las tres ⇒ techo $= 3$.
- O2: lo usan T_B y T_C ⇒ techo $= \max(2, 1) = 2$.

2 Bloqueos

- **T_A** ($P = 3$): menores T_B (O1: 3, techo 3 ≥ 3 ✓; O2: techo 2 < 3 ✗) y T_C (O1: 2 ✓; O2 ✗). $B_A = \max(3, 2) = 3$.
- **T_B** ($P = 2$): menor T_C (O1: 2, techo 3 ≥ 2 ✓; O2: 3, techo 2 ≥ 2 ✓). $B_B = \max(2, 3) = 3$.
- **T_C**: $B_C = 0$.

3 Tiempos de respuesta

- **T_A**: $R = 10 + 3 = 13 \leq 20$. Cumple.
- **T_B**: $R^{(0)} = 8 + 3 = 11$; $R^{(1)} = 11 + \lceil 11/20 \rceil \cdot 10 = 21$; $R^{(2)} = 11 + \lceil 21/20 \rceil \cdot 10 = 31 > 25$. **No cumple**: llega otra activación de T_A mientras T_B espera.
- **T_C**: $R^{(0)} = 12$; $R^{(1)} = 12 + \lceil 12/20 \rceil 10 + \lceil 12/500 \rceil 8 = 30$; $R^{(2)} = 12 + \lceil 30/20 \rceil 10 + 8 = 40$; $R^{(3)} = 12 + \lceil 40/20 \rceil 10 + 8 = 40$. $R_C = 40 \leq 120$.



Captura del simulador interactivo (disponible en la versión web de la guía).

Resultado final

Tarea	Prioridad	B	R	D	¿Cumple?
T_A	3	3	13	20	Sí
T_B	2	3	31	25	**No**
T_C	1	0	40	120	Sí

El sistema no está garantizado: T_B puede incumplir su plazo (31 > 25) aunque la utilización total sea baja ($U = 0{,}616$). Es el ejemplo de por qué la utilización no basta con plazos más cortos que el periodo.

Septiembre 2016 · Ejecutivo cíclico: ciclo secundario y código

Enunciado

Describa qué se entiende por «ejecutivo cíclico», cuáles son sus ventajas y desventajas y escriba el código necesario para crear un ejecutivo cíclico para el siguiente conjunto de tareas. Suponga que el código que deben ejecutar las tareas está definido en las funciones `void f_a()`, `void f_b()` y `void f_c()`.

Tarea	Tipo	T (ms)	C (ms)
T_C	Periódica	120	12
T_B	Periódica	80	8
T_A	Periódica	40	10

Antes de empezar: qué piden y qué teoría usas

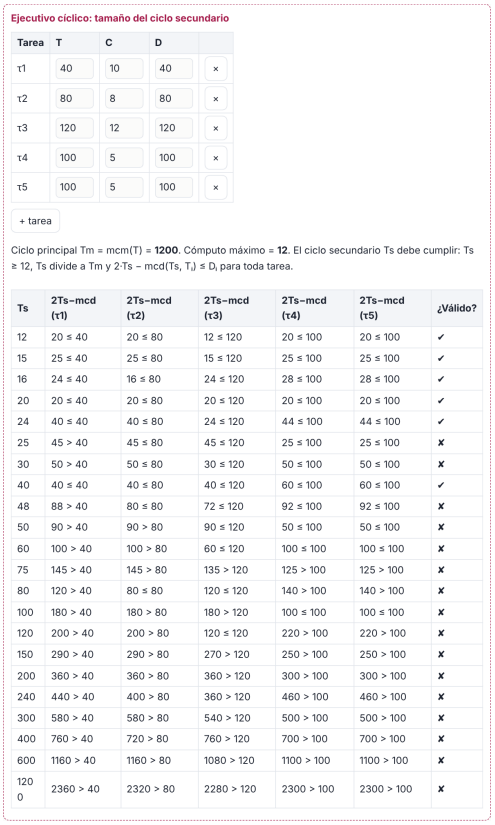
Qué piden. Diseñar el plan: hiperperiodo, ciclo secundario y qué tareas van en cada ciclo; escribir el bucle con `sleep_until`.
Teoría que usas. Ejecutivo cíclico (T6): $T_M = \text{mcm}(T_i)$; T_S divide a T_M, $T_S \geq \max C_i$ y $2T_S - \text{gcd}(T_S, T_i) \leq D_i$ (con $D_i = T_i$).

1 Utilización y hiperperiodo

$U = 10/40 + 8/80 + 12/120 = 0{,}25 + 0{,}10 + 0{,}10 = 0{,}45$, menor que la cota $3(2^{1/3} - 1) = 0{,}78$, así que hay holgura. $T_M = \text{mcm}(40, 80, 120) = 240$ ms.

2 Elección del ciclo secundario

Divisores de 240 con $T_S \geq 12$ (el mayor C): 12, 15, 16, 20, 24, 30, 40, 48, 60, 80, 120, 240. Se comprueba $2T_S - \text{gcd}(T_S, T_i) \leq T_i$ para $T_i = 40, 80, 120$: cumplen **12, 15, 16, 20, 24 y 40**. Con $T_S = 30$ falla ($60 - 10 = 50 > 40$); con 48, 60, 80… también. Se elige $T_S = 40$ **ms** (el mayor válido: menos ciclos, y coincide con el menor periodo, así que T_A ocurre en cada ciclo).



Captura del simulador interactivo (disponible en la versión web de la guía).

3 Qué va en cada ciclo

$T_M/T_S = 6$ ciclos. A (T=40) en todos; B (T=80) en los pares; C (T=120) cada 3 ciclos (0 y 3).

Ciclo	0	1	2	3	4	5
Tareas	A, B, C	A	A, B	A, C	A, B	A
Tiempo usado (ms)	30	10	18	22	18	10

Ningún ciclo supera 40 ms.

4 Código

CPP

```cpp
#include <chrono>
#include <iostream>
#include <thread>
using namespace std;
using namespace std::chrono_literals;

void f_a() { cout << 'A'; }          // 10 ms de trabajo (aquí solo se imprime)
void f_b() { cout << 'B'; }          //  8 ms
void f_c() { cout << 'C'; }          // 12 ms

int main() {
    const auto ts = 40ms;                                   // ciclo secundario
    auto siguiente = chrono::steady_clock::now() + ts;      // instante absoluto del siguiente ciclo
    for (int vuelta = 0; vuelta < 12; ++vuelta) {           // 2 ciclos principales de 240 ms (en un sistema real:
    ↪ while(true))
        switch (vuelta % 6) {                               // plan del ciclo principal (6 ciclos secundarios)
            case 0: f_a(); f_b(); f_c(); break;
            case 1: f_a(); break;
            case 2: f_a(); f_b(); break;
            case 3: f_a(); f_c(); break;
            case 4: f_a(); f_b(); break;
            case 5: f_a(); break;
        }
        cout << ' ';
        this_thread::sleep_until(siguiente);                // espera hasta el instante absoluto: sin deriva
        siguiente += ts;
    }
    cout << '\n';
}
```

5 Ventajas e inconvenientes (parte de desarrollo)

- **Ventajas:** comportamiento totalmente predecible; no necesita planificador ni sincronización entre tareas (se ejecutan en secuencia); fácil de verificar.
- **Inconvenientes:** poco flexible (cambiar una tarea obliga a rehacer el plan); el hiperperiodo puede ser enorme; difícil con tareas esporádicas y con C mayores que el ciclo secundario (hay que trocear).

Ojo: errata en la solución de referencia

La solución de referencia enumera como posibles ciclos secundarios `{120, 80, 60, 48, 40, …}`. Salvo 40, **ninguno cumple** el criterio que ella misma enuncia ($T_S \leq \min T_i = 40$), ni la condición $2T_S - \gcd(T_S, T_i) \leq T_i$: con $T_S = 120$, por ejemplo, $240 - 40 = 200 > 40$. El plan que propone (6 ciclos de 40 ms) sí es correcto.

Resultado final

$T_M = 240$ ms, $T_S = 40$ ms (también valen 12, 15, 16, 20 y 24), 6 ciclos con el reparto de la tabla, y el bucle con `sleep_until` del código.

13 Hojas de problemas

Ejercicios de las hojas de problemas, con el mismo formato que los exámenes: enunciado, qué piden, pasos plegables y resultado final. El código completo acompaña a la guía como ficheros independientes. Todos los resultados se han comprobado compilando y ejecutando.

13.1 Hoja 1 · Cálculo numérico y bucles

Leibniz, sumas parciales y criba de Eratóstenes

Enunciado

1. Calcula $\pi/4$ con la serie de Leibniz, $\sum_{n\geq 0}(-1)^n/(2n+1)$, con un millón de términos. Compara con `M_PI_4` usando `==` y usando `fabs`.
2. Dado un `vector<int>`, construye otro cuya posición `i` contenga la suma de los elementos desde `i` hasta el final.
3. Implementa la criba de Eratóstenes hasta 1 000 000 con un `vector<char>` que marque cada número como desconocido, primo o compuesto. Cuenta los primos y muestra los últimos ocho.

Antes de empezar: qué piden y qué teoría usas

Qué piden. Tres bucles clásicos. Lo importante: entender **por qué `==` falla con reales**, recorrer un vector hacia atrás con un acumulador y hacer la criba en tiempo casi lineal.
Teoría que usas. Coma flotante (T7), `vector`, `iota`, bucles con `size_t` (sin signo: cuidado con `i >= 0`).

1 Serie de Leibniz y comparación

Se alterna el signo con `n % 2`. Con N términos, el error de la serie alternada es del orden de $1/(4N)$: con $N = 10^6$, unos $2{,}5 \cdot 10^{-7}$. Por eso `==` da `false` (el resultado **no puede** coincidir bit a bit con `M_PI_4`) pero `fabs(a-b) < 1e-6` da `true`.

CPP

```cpp
// 1) Leibniz: pi/4 = sum_{n>=0} (-1)^n / (2n+1)
double pi_4 = 0;
for (int n = 0; n < 1000000; ++n) pi_4 += (n % 2 == 0 ? 1.0 : -1.0) / (2 * n + 1);
cout << boolalpha << "pi_4 == M_PI_4 ? " << (pi_4 == M_PI_4) << '\n';
cout << setprecision(12) << pi_4 << " vs " << M_PI_4 << '\n';
cout << "|diferencia| < 1e-6 ? " << (fabs(pi_4 - M_PI_4) < 1e-6) << '\n';
```

2 Sumas desde `i` hasta el final

Se recorre **de atrás hacia delante** con un acumulador: `sumas[i] = sumas[i+1] + v[i]`. Coste $O(n)$ (frente a $O(n^2)$ si sumas cada vez desde `i`). Se usa `int i` y no `size_t i` porque el bucle termina con `i >= 0`: con un tipo sin signo, `i >= 0` es siempre cierto y el bucle no acaba.

CPP

```cpp
// 2) sumas parciales hasta el final (suma desde i hasta el último)
vector<int> v(10);
iota(v.begin(), v.end(), 1);
vector<int> sumas(v.size());
int acum = 0;
for (int i = static_cast<int>(v.size()) - 1; i >= 0; --i) { acum += v[i]; sumas[i] = acum; }
for (int x : sumas) cout << x << ' ';
cout << '\n';
```

3 Criba de Eratóstenes

Marcas iniciales: `'d'` (desconocido). Se recorre desde 2: si `i` sigue desconocido es primo y se marcan como compuestos sus múltiplos. **Se empieza en `i*i`**: los múltiplos menores, $k \cdot i$ con $k < i$, ya los marcó un primo más pequeño. Coste $O(n \log \log n)$.

CPP

```
    // 3) criba de Eratóstenes ('d' desconocido, 'p' primo, 'c' compuesto)
    const size_t N = 1000000;
    vector<char> marca(N + 1, 'd');
    marca[0] = marca[1] = 'c';
    for (size_t i = 2; i <= N; ++i) {
        if (marca[i] != 'd') continue;
        marca[i] = 'p';
        for (size_t m = i * i; m <= N; m += i) marca[m] = 'c';     // desde i*i: los anteriores ya están marcados
    }
    long cuenta = 0;
    for (char c : marca) cuenta += (c == 'p');
    cout << "primos hasta 1000000: " << cuenta << '\n';
    cout << "ultimos 8:";
    int mostrados = 0;
    vector<size_t> ult;
    for (size_t i = N; i > 1 && mostrados < 8; --i) if (marca[i] == 'p') { ult.push_back(i); ++mostrados; }
    for (auto it = ult.rbegin(); it != ult.rend(); ++it) cout << ' ' << *it;
    cout << '\n';
}
```

4 Comprobación

- Hay 78 498 primos por debajo de 1 000 000 (valor conocido) y el mayor es 999 983: coincide con la salida.
- Sumas parciales de $1..10$: $55, 54, 52, \ldots, 10$ (suma total 55, último elemento 10).

Resultado final

Salida

```
pi_4 == M_PI_4 ? false
0.785397913397 vs 0.785398163397
|diferencia| < 1e-6 ? true
55 54 52 49 45 40 34 27 19 10
primos hasta 1000000: 78498
ultimos 8: 999907 999917 999931 999953 999959 999961 999979 999983
```

Interpretación. `pi_4 == M_PI_4` es falso aunque la aproximación sea buena: la serie solo converge (lentamente) y además cada suma redondea. Compara siempre con tolerancia.

13.2 Hoja 2 · Conjuntos y cadenas

Operaciones con `set<int>` y caracteres repetidos

Enunciado

1. Escribe funciones que reciban dos `set<int>` y devuelvan su unión, su intersección y su diferencia, y otra que diga si el primero es subconjunto del segundo.
2. Escribe una función que diga si una cadena tiene caracteres repetidos, de tres maneras: con un `set`, con un `vector`/`map` y sin ninguna colección.

Antes de empezar: qué piden y qué teoría usas

Qué piden. Usar `set` (ordenado, sin duplicados) y conocer sus operaciones: `insert`, `count`, rangos. Saber que `insert` **devuelve un `pair`** cuyo `.second` indica si se insertó.
Teoría que usas. `set<T>`, iteradores, `std::includes` (requiere rangos **ordenados**: un `set` lo está), `operator<<` sobrecargado.

1 Unión, intersección y diferencia

- Unión: copia de `a` y se insertan los de `b`. El `set` descarta duplicados.
- Intersección: de `a`, los que `b.count(x)` confirma.
- Diferencia: de `a`, los que **no** están en `b`.

Se pasa por `const&` para no copiar. Coste $O(n \log m)$.

CPP

```
set<int> unir(const set<int>& a, const set<int>& b) { set<int> r{a}; r.insert(b.begin(), b.end()); return r; }
set<int> interseccion(const set<int>& a, const set<int>& b) {
```

```
    set<int> r; for (int x : a) if (b.count(x)) r.insert(x); return r;
}
set<int> diferencia(const set<int>& a, const set<int>& b) {
    set<int> r; for (int x : a) if (!b.count(x)) r.insert(x); return r;
}
bool es_subconjunto(const set<int>& a, const set<int>& b) {     // ¿a está contenido en b?
    return includes(b.begin(), b.end(), a.begin(), a.end());
```

2 Subconjunto

`includes(b.begin(), b.end(), a.begin(), a.end())` devuelve `true` si todos los elementos de a están en `b`. Funciona porque ambos rangos están ordenados.

CPP

```
bool es_subconjunto(const set<int>& a, const set<int>& b) {     // ¿a está contenido en b?
    return includes(b.begin(), b.end(), a.begin(), a.end());
}
```

3 Caracteres repetidos

- **Con set**: `vistos.insert(c).second == false` significa que ya estaba: repetido. $O(n \log n)$.
- **Sin colecciones**: dos bucles anidados, $O(n^2)$, sin memoria extra.

(Con `map<char,int>` o `vector<int>(256)` se cuenta cada carácter y se busca un contador ≥ 2: $O(n)$.)

CPP

```
bool repetidos_set(const string& s) {              // con set: insert devuelve .second == false si ya estaba
    set<char> vistos;
    for (char c : s) if (!vistos.insert(c).second) return true;
    return false;
}
bool repetidos_sin_colecciones(const string& s) {  // sin colecciones: dos bucles anidados
    for (size_t i = 0; i < s.size(); ++i)
        for (size_t j = i + 1; j < s.size(); ++j) if (s[i] == s[j]) return true;
    return false;
}
```

4 Prueba

CPP

```
int main() {
    set<int> a{1, 2, 3, 4}, b{3, 4, 5};
    cout << "a=" << a << " b=" << b << '\n';
    cout << "union: " << unir(a, b) << "  interseccion: " << interseccion(a, b) << "  a-b: " << diferencia(a, b) <<
    ↪  '\n';
    cout << boolalpha << "{3,4} en a: " << es_subconjunto({3, 4}, a) << ", b en a: " << es_subconjunto(b, a) << '\n';
    for (string s : {"", "hola", "hello"})
        cout << "\"" << s << "\": set=" << repetidos_set(s) << " sin colecciones=" << repetidos_sin_colecciones(s) <<
        ↪  '\n';
}
```

Resultado final

Salida

```
a={1, 2, 3, 4} b={3, 4, 5}
union: {1, 2, 3, 4, 5}  interseccion: {3, 4}  a-b: {1, 2}
{3,4} en a: true, b en a: false
"": set=false sin colecciones=false
"hola": set=false sin colecciones=false
"hello": set=true sin colecciones=true
```

13.3 Hoja 3 · Clases

Esfera, punto y caja

Enunciado

1. Diseña la clase `Sphere` (diámetro y centro como `Point_3D`) con área, volumen, `contiene(punto)` y `en_superficie(punto)`, y `operator<<`.
2. Diseña la clase `Box` con alto, ancho y profundidad, que permita insertar objetos por volumen mientras quepan, saber si está llena y vaciarla.
3. Lee un valor positivo de `cin` sin que el programa se rompa si el usuario escribe letras.

Antes de empezar: qué piden y qué teoría usas

Qué piden. Clases con **invariantes** (el volumen libre nunca es negativo), métodos `const`, `friend operator<<` y entrada robusta.
Teoría que usas. Constructores con valores por defecto, lista de inicialización, `const` correcto, comparar reales con tolerancia, estado de error de `cin` (`clear` + `ignore`).

1 `Point_3D` y `Sphere`

La distancia entre dos puntos es $\sqrt{\Delta x^2 + \Delta y^2 + \Delta z^2}$. Un punto está **dentro** si la distancia al centro es menor que el radio, y **en la superficie** si difiere del radio en menos de una tolerancia (nunca == con reales). Área $4\pi r^2$ y volumen $\frac{4}{3}\pi r^3$, con $r = d/2$.

CPP

```
class Point_3D {
public:
    Point_3D(double x = 0, double y = 0, double z = 0) : x_{x}, y_{y}, z_{z} {}
    double distancia(const Point_3D& o) const { return sqrt(pow(x_ - o.x_, 2) + pow(y_ - o.y_, 2) + pow(z_ - o.z_,
    ↪  2)); }
    friend ostream& operator<<(ostream& os, const Point_3D& p) { return os << '(' << p.x_ << ", " << p.y_ << ", " <<
    ↪  p.z_ << ')'; }
private:
    double x_, y_, z_;
};

class Sphere {
public:
    explicit Sphere(double diametro, Point_3D centro = {}) : d_{diametro}, c_{centro} {}
    double diametro() const { return d_; }
    void set_diametro(double d) { d_ = d; }
    double area() const { double r = d_ / 2; return 4 * M_PI * r * r; }
    double volumen() const { double r = d_ / 2; return 4.0 / 3 * M_PI * r * r * r; }
    // dentro: distancia < r; sobre la superficie: |distancia - r| < eps (nunca ==, son reales)
    bool contiene(const Point_3D& p) const { return c_.distancia(p) < d_ / 2; }
    bool en_superficie(const Point_3D& p, double eps = 1e-9) const { return fabs(c_.distancia(p) - d_ / 2) < eps; }
    friend ostream& operator<<(ostream& os, const Sphere& s) {
        return os << "Esfera d=" << s.d_ << " centro " << s.c_ << " area=" << s.area() << " volumen=" << s.volumen();
    }
private:
    double d_;
    Point_3D c_;
};
```

2 `Box`

El volumen libre empieza en $a \cdot w \cdot p$. `insertar_objeto` rechaza volúmenes ≤ 0 o mayores que el libre (devuelve `false` y **no cambia el estado**). `llena_` se actualiza al llegar a 0.

CPP

```
class Box {
public:
    Box(double alto, double ancho, double prof) : a_{alto}, w_{ancho}, p_{prof}, libre_{alto * ancho * prof} {}
    bool insertar_objeto(double volumen) {
        if (volumen <= 0 || volumen > libre_) return false;
        libre_ -= volumen; llena_ = libre_ <= 0; return true;
    }
    void vaciar() { libre_ = a_ * w_ * p_; llena_ = false; }
    double libre() const { return libre_; }
    bool llena() const { return llena_; }
    friend ostream& operator<<(ostream& os, const Box& b) { return os << "Caja " << b.a_ << 'x' << b.w_ << 'x' << b.p_
    ↪  << " libre=" << b.libre_ << (b.llena_ ? " (llena)" : ""); }
```

```
private:
    double a_, w_, p_, libre_;
    bool llena_ = false;
};
```

3 Entrada robusta

Si `in >> v` falla, el flujo queda en estado de error y **todas** las lecturas siguientes fallan. Hay que `clear()` y descartar la línea con `ignore`. Se lee en un bucle hasta recibir un número válido y positivo.

CPP

```
double leer_positivo(istream& in, const string& pregunta) {       // robusto frente a «abc»
    double v;
    while (true) {
        cout << pregunta;
        if (in >> v && v > 0) return v;
        in.clear();                                                 // limpia el estado de error
        in.ignore(numeric_limits<streamsize>::max(), '\n');         // descarta la línea mala
        cout << "valor no valido\n";
    }
}
```

4 main

CPP

```
int main() {
    Sphere s{2, Point_3D{0, 0, 0}};
    cout << s << '\n';
    cout << boolalpha << "(0.5,0,0) dentro: " << s.contiene({0.5, 0, 0}) << ", (1,0,0) en superficie: " <<
    ↪  s.en_superficie({1, 0, 0}) << '\n';
    Box b{1, 1, 1};
    cout << b.insertar_objeto(0.6) << ' ' << b.insertar_objeto(0.6) << ' ' << b.insertar_objeto(0.4) << '\n' << b <<
    ↪  '\n';
    b.vaciar(); cout << b << '\n';
    istringstream simulado{"abc\n-3\n4.5\n"};                        // en el examen sería cin
    cout << "diametro leido: " << leer_positivo(simulado, "diametro? ") << '\n';
}
```

Resultado final

Salida

```
Esfera d=2 centro (0, 0, 0) area=12.5664 volumen=4.18879
(0.5,0,0) dentro: true, (1,0,0) en superficie: true
true false true
Caja 1x1x1 libre=0 (llena)
Caja 1x1x1 libre=1
diametro leido: diametro? valor no valido
diametro? valor no valido
diametro? 4.5
```

Comprobación: con $d = 2$ ($r = 1$), área $4\pi = 12{,}566$ y volumen $\frac{4}{3}\pi = 4{,}189$. La caja de volumen 1 admite 0,6 y 0,4 pero rechaza el segundo 0,6, y queda llena (libre = 0).

14 Demostraciones

Las demostraciones de los resultados teóricos que usas en el examen y en el trabajo de planificación. Cada paso está plegado: intenta primero el paso por tu cuenta y ábrelo para comprobar. Los números de cada demostración se han verificado con código (comparación por fuerza bruta o enumeración).

14.1 1 · Cota de Liu–Layland para dos tareas

Cota de utilización de Rate Monotonic: $U \le 2(\sqrt{2}-1)$ **y límite** $\ln 2$

Enunciado

Demuestra que todo conjunto de **dos** tareas periódicas con $D_i = T_i$ y prioridades Rate Monotonic es planificable si $U \le 2(\sqrt{2}-1) \approx 0{,}8284$, y que la cota general $n(2^{1/n}-1)$ tiende a $\ln 2 \approx 0{,}6931$.

Antes de empezar: qué piden y qué teoría usas

Qué piden. Encontrar la **peor** utilización de un conjunto de tareas que aun así llega justo a incumplir (conjunto «saturado»). La cota es el **mínimo** de U entre todos los conjuntos saturados: por debajo de ella nada puede fallar.
Teoría que usas. Instante crítico (todas las tareas se liberan a la vez), Rate Monotonic (menor periodo, mayor prioridad), derivadas para minimizar.

1 Planteamiento

Sean $T_1 < T_2$. Con RM, τ_1 tiene mayor prioridad. Se estudia el instante crítico $t = 0$ y se define $F = \lfloor T_2/T_1 \rfloor \ge 1$ (cuántos periodos completos de τ_1 caben en un periodo de τ_2). Fijado C_1, el mayor C_2 que mantiene el conjunto planificable es el que **satura**; si C_2 fuera menor, U sería menor y aun así planificable, así que solo hay que buscar el mínimo de U entre los saturados.

2 Caso A: $C_1 \le T_2 - F\,T_1$

La última liberación de τ_1 antes de T_2 (en $t = F\,T_1$) termina antes de T_2. En $[0, T_2]$ hay $F+1$ ejecuciones de τ_1, que ocupan $(F+1)C_1$. A τ_2 le queda $T_2 - (F+1)C_1$:

$$C_2^{max} = T_2 - (F+1)C_1, \qquad U = \frac{C_1}{T_1} + 1 - \frac{(F+1)C_1}{T_2} = 1 + C_1\,\frac{T_2 - (F+1)T_1}{T_1 T_2}.$$

Como $T_2 < (F+1)T_1$, el coeficiente de C_1 es **negativo**: U **baja** al subir C_1. Su mínimo está en el mayor C_1 permitido, $C_1 = T_2 - F\,T_1$.

3 Caso B: $C_1 > T_2 - F\,T_1$

Ahora la ejecución de τ_1 liberada en $F\,T_1$ dura hasta pasar T_2, así que τ_2 no puede ejecutar nada en $[F\,T_1, T_2]$ y debe acabar antes de $F\,T_1$. En $[0, F\,T_1]$ hay F ejecuciones de τ_1: $C_2^{max} = F\,(T_1 - C_1)$ y

$$U = \frac{C_1}{T_1} + \frac{F(T_1 - C_1)}{T_2} = \frac{F\,T_1}{T_2} + C_1\,\frac{T_2 - F\,T_1}{T_1 T_2}.$$

El coeficiente de C_1 es **positivo**: U **sube** con C_1 y su mínimo está en el menor C_1 permitido, de nuevo $C_1 = T_2 - F\,T_1$.

4 Valor mínimo en función de F y f

Los dos casos coinciden: el mínimo es en $C_1 = T_2 - F\,T_1$. Sea $T_2/T_1 = F + f$ con $0 \le f < 1$. Entonces $C_1/T_1 = f$ y $C_2 = F\,(T_1 - C_1)$ da $C_2/T_2 = F(1-f)/(F+f)$:

$$U(f) = f + \frac{F(1-f)}{F+f} = \frac{F + f^2}{F+f}.$$

5 Minimizar en f

Derivando: $U'(f) = \dfrac{f^2 + 2Ff - F}{(F+f)^2} = 0 \Rightarrow f^* = \sqrt{F^2+F} - F$. En el punto crítico, $\dfrac{d}{df}\,(F + f^2) = U\,\dfrac{d}{df}(F+f)$, es decir $2f^* = U$:

$$U_{min}(F) = 2\left(\sqrt{F^2+F} - F\right) = \frac{2F}{\sqrt{F^2+F} + F}.$$

6 El peor caso es $F = 1$

$U_{min}(F) = \dfrac{2}{\sqrt{1+1/F}+1}$ **crece** con F (el denominador decrece). Su mínimo es en $F = 1$: $U_{min} = 2(\sqrt{2}-1) \approx 0{,}8284$ (con $f^* = \sqrt{2}-1 \approx 0{,}4142$). Para $F = 2, 3, 5$ salen $0{,}8990$, $0{,}9282$, $0{,}9545$ (comprobado numéricamente).

7 Límite para n grande

La cota general (Liu y Layland, 1973) es $U_n = n(2^{1/n} - 1)$. Con $x = 1/n \to 0$: $U_n = \dfrac{2^x - 1}{x} \to \dfrac{d}{dx}2^x\Big|_{x=0} = \ln 2$. Valores: $U_1 = 1$, $U_2 = 0{,}828$, $U_3 = 0{,}780$, $U_{10} = 0{,}718$, $U_{1000} = 0{,}6934$.

8 Comprobación por fuerza bruta

Se probaron todos los conjuntos saturados con $T_1 \in [5, 40]$ y $T_2 \le 2T_1$ enteros, calculando la planificabilidad con el análisis de tiempo de respuesta: la menor utilización encontrada es $0{,}8284$ (con $T = (29, 41)$, $C = (12, 17)$, $T_2/T_1 = 1{,}414$). Coincide con la demostración.

Resultado final

Todo conjunto de dos tareas con $U \le 2(\sqrt{2}-1)$ es planificable con RM; la cota baja a $\ln 2$ con n creciente.

Ojo

La cota es **suficiente, no necesaria**. Un conjunto con U mayor puede ser planificable, p. ej. $(C, T) = (1, 4), (2, 6), (3, 13)$: $U = 0{,}814 > 0{,}780 = U_3$, y sin embargo $R_3 = 10 \le 13$ (demostración 2).

14.2 2 · Análisis de tiempo de respuesta

Por qué funciona la recurrencia $R = C + \sum\lceil R/T_j\rceil C_j$ y por qué termina

Enunciado

Justifica la ecuación del tiempo de respuesta en el peor caso, demuestra que la iteración $R^{(k+1)} = f(R^{(k)})$ termina y que converge al **menor** punto fijo, y aplícala a $(C, T) = (1, 4), (2, 6), (3, 13)$ con prioridades por periodo (RM).

Antes de empezar: qué piden y qué teoría usas

Qué piden. Entender las tres propiedades que hacen que el método sea correcto: ecuación, terminación y minimalidad.
Teoría que usas. Instante crítico, techo $\lceil x\rceil$, funciones monótonas, inducción.

1 La ecuación

En el instante crítico todas las tareas se liberan a la vez. Hasta el tiempo R, la tarea j de mayor prioridad se libera $\lceil R/T_j\rceil$ veces (en $0, T_j, 2T_j, \ldots$ mientras sea $< R$) y cada vez consume C_j. La tarea τ_i acaba cuando el procesador ha terminado su C_i **y** todo el trabajo de mayor prioridad liberado antes. Eso ocurre en el menor $R > 0$ tal que

$$R = C_i + \sum_{j \in hp(i)} \left\lceil \frac{R}{T_j} \right\rceil C_j =: f(R).$$

2 La función f es monótona no decreciente

Si $R \le R'$ entonces $\lceil R/T_j\rceil \le \lceil R'/T_j\rceil$ y por tanto $f(R) \le f(R')$.

3 La iteración es creciente y termina

Se toma $R^{(0)} = C_i$ y $R^{(k+1)} = f(R^{(k)})$. Como $f(R) \ge C_i$, $R^{(1)} \ge R^{(0)}$. Si $R^{(k)} \ge R^{(k-1)}$, por monotonía $R^{(k+1)} = f(R^{(k)}) \ge f(R^{(k-1)}) = R^{(k)}$. La sucesión es **no decreciente** y con parámetros enteros sube al menos 1 en cada paso hasta estabilizarse. Se detiene cuando $R^{(k+1)} = R^{(k)}$ (tiempo de respuesta) o cuando $R^{(k)} > D_i$ (no planificable). Como máximo hay D_i pasos.

4 Es el menor punto fijo

Sea R^* cualquier solución positiva de $R = f(R)$. Por inducción $R^{(k)} \leq R^*$: $R^{(0)} = C_i \leq f(R^*) = R^*$ y, si $R^{(k)} \leq R^*$, entonces $R^{(k+1)} = f(R^{(k)}) \leq f(R^*) = R^*$. El límite es el **menor** punto fijo.

5 Ejemplo

τ_1: $R_1 = 1$. τ_2: $R = 2 \to 2 + \lceil 2/4 \rceil 1 = 3 \to 2 + \lceil 3/4 \rceil 1 = 3$: $R_2 = 3$. τ_3:

$$3 \to 3 + 1 \cdot 1 + 1 \cdot 2 = 6 \to 3 + 2 \cdot 1 + 1 \cdot 2 = 7 \to 3 + 2 \cdot 1 + 2 \cdot 2 = 9 \to 3 + 3 \cdot 1 + 2 \cdot 2 = 10 \to 3 + 3 \cdot 1 + 2 \cdot 2 = 10.$$

$R_3 = 10 \leq 13$: planificable, aunque $U = 0{,}814 > U_3 = 0{,}780$.

Resultado final

La recurrencia es correcta (da el menor instante en que se completa el trabajo exigido), siempre termina y su resultado es exacto, no una cota.

14.3 3 · EDF es óptimo con $U \leq 1$

EDF planifica un conjunto si y solo si $U \leq 1$

Enunciado

Para tareas periódicas independientes, con $D_i = T_i$, apropiativas y en un solo procesador, demuestra que EDF cumple todos los plazos **si y solo si** $U = \sum C_i/T_i \leq 1$.

Antes de empezar: qué piden y qué teoría usas

Qué piden. Una condición necesaria y suficiente. La necesaria es casi trivial; la suficiente se demuestra por contradicción con la **demanda** de trabajo en un intervalo.
Teoría que usas. EDF (más cercano el plazo, más prioridad), demanda de procesador.

1 Necesidad

En un intervalo largo L llega trabajo $\approx L \cdot U$. Si $U > 1$, el procesador no puede ejecutar más de L unidades: tarde o temprano falla un plazo.

2 Suficiencia: suposición contraria

Supón $U \leq 1$ y que EDF falla un plazo; sea t_2 el primer instante en que ocurre. Sea $t_1 < t_2$ el último instante anterior en el que el procesador estaba ocioso o ejecutaba un trabajo con plazo $> t_2$ (si no existe, $t_1 = 0$).

3 Qué hace el procesador en $[t_1, t_2]$

Como EDF da prioridad al plazo más cercano, en $[t_1, t_2]$ solo se ejecutan trabajos con plazo $\leq t_2$ y **liberados en t_1 o después**. Está ocupado todo el intervalo (no hay ocio por la definición de t_1).

4 Demanda máxima

La demanda de trabajos liberados en $[t_1, t_2]$ con plazo $\leq t_2$ es, para la tarea i, a lo sumo $\lfloor (t_2 - t_1)/T_i \rfloor C_i$. Total:

$$W \leq \sum_i \left\lfloor \frac{t_2 - t_1}{T_i} \right\rfloor C_i \leq (t_2 - t_1)\, U \leq t_2 - t_1.$$

5 Contradicción

Si falló un plazo en t_2, el trabajo con plazo $\leq t_2$ no cupo: $W > t_2 - t_1$. Esto contradice $W \leq t_2 - t_1$. Luego con $U \leq 1$ no falla ningún plazo.

Resultado final

EDF es **óptimo**: planifica todo lo que cualquier algoritmo pueda planificar en un procesador (para $D_i = T_i$, exactamente cuando $U \leq 1$).

Ojo

Con $D_i < T_i$ la condición $U \leq 1$ **ya no es suficiente** (hace falta el análisis de demanda de procesador). Y para RM con $D = T$, $U \leq 1$ no basta: solo si los periodos son armónicos (demostración 6).

14.4 4 · Protocolo de techo inmediato (ICPP)

Con ICPP una tarea se bloquea como mucho una vez, durante una sola sección crítica, y no hay interbloqueos

Enunciado

Con el protocolo de techo de prioridad inmediato (ICPP): el techo de un recurso es la mayor prioridad de las tareas que lo usan, y quien lo toma pasa a ejecutar a esa prioridad. Demuestra que una tarea τ_H sufre bloqueo de **como máximo una** sección crítica de una tarea de menor prioridad y que no puede haber interbloqueo.

Antes de empezar: qué piden y qué teoría usas

Qué piden. Que B_i sea el **máximo** de una sola sección crítica (no la suma), y que el protocolo sea seguro.
Teoría que usas. Prioridad dinámica por techo, planificación apropiativa por prioridades fijas.

1 Cuándo se puede bloquear τ_H

Supón que τ_H está bloqueada porque una tarea de menor prioridad τ_L tiene un recurso R con $techo(R) \geq P_H$ (si el techo fuera menor, τ_L no pasaría por encima de τ_H y no habría bloqueo). Al tomarlo, τ_L ejecutó a prioridad $techo(R) \geq P_H$.

2 τ_L tomó R antes de que llegara τ_H

Si τ_L hubiera intentado tomar R **después** de la llegada de τ_H, estaría ejecutando mientras τ_H está lista, lo que solo ocurre si su prioridad es mayor: absurdo, porque $P_L < P_H$. Por tanto τ_L ya tenía R cuando τ_H llegó.

3 Solo una tarea de menor prioridad puede estar dentro de una sección crítica con techo $\geq P_H$

Supón que otra tarea de menor prioridad τ' tiene otro recurso R' con $techo(R') \geq P_H$ al llegar τ_H. Cuando τ_L (o τ') tomó el segundo recurso, estaba ejecutando mientras la otra mantenía el suyo a prioridad $\geq P_H >$ su prioridad base: no habría podido ejecutar. Contradicción. Hay a lo sumo **una** tarea así.

4 Duración del bloqueo

Una vez τ_L ejecuta a prioridad $\geq P_H$ nadie de menor prioridad la interrumpe, y τ_H solo espera a que acabe **esa** sección crítica (no puede empezar otra: para empezar tendría que tomar otro recurso, y el paso 3 lo impide para recursos con techo $\geq P_H$). Por tanto

$$B_H = \max\{c_{k,r} \ : \ k \text{ de menor prioridad, } techo(r) \geq P_H\}.$$

5 No hay interbloqueo

Un interbloqueo exige que τ_a, con R_1, pida R_2 ocupado por τ_b. Dos casos:

- τ_b tomó R_2 **antes** que τ_a tomara R_1. Entonces τ_b ejecutaba a $techo(R_2) \geq P_a$ (porque τ_a usa R_2), y τ_a no habría podido ejecutar para tomar R_1.
- τ_b tomó R_2 **después**. Entonces τ_a ejecutaba a $techo(R_1) \geq P_b$ (porque τ_b usa R_1) y τ_b no habría podido ejecutar.

Ninguno de los dos casos puede darse. En general, con ICPP **nadie ejecuta nunca pidiendo un recurso ocupado**: el bloqueo se produce antes de empezar a ejecutar. Sin espera circular, no hay interbloqueo.

Resultado final

El bloqueo máximo que entra en la recurrencia es el de **una** sección crítica (la mayor entre las de menor prioridad con techo $\geq P_i$) y con ICPP no hay inversión de prioridad no acotada ni interbloqueos.

Ojo

En el análisis, sumar todas las secciones críticas posibles da una cota **válida pero pesimista** (puede declarar no planificable lo que sí lo es). El valor correcto de B_i es el máximo, como aquí.

14.5 5 · Restricciones del ejecutivo cíclico

Por qué $T_s \geq \max C_i$, $T_s \mid H$ y $2T_s - \gcd(T_s, T_i) \leq D_i$

Enunciado

En un ejecutivo cíclico con marco secundario T_s e hiperperiodo $H = \text{mcm}(T_i)$, demuestra las tres condiciones para que un tamaño de marco sea válido.

Antes de empezar: qué piden y qué teoría usas

Qué piden. Justificar cada restricción con un argumento. La tercera es la que más se olvida en la práctica: el trabajo que llega **justo después** de empezar un marco debe esperar al siguiente.
Teoría que usas. Tareas que se ejecutan enteras dentro de un marco (sin apropiación), máximo común divisor.

1 $T_s \geq \max C_i$

Dentro de un marco las tareas no se interrumpen, así que cada una debe caber **entera** en un marco: $C_i \leq T_s$ para toda i.

2 T_s divide al hiperperiodo

El plan se repite cada H. Si $T_s \nmid H$ el último marco quedaría cortado y la repetición no sería idéntica; $H = m\,T_s$ con m entero.

3 Cuándo se libera el trabajo respecto a los marcos

Las liberaciones de τ_i están en los múltiplos de T_i y los marcos empiezan en múltiplos de T_s. La distancia $r - m\,T_s$ entre una liberación y el inicio de marco anterior toma valores **múltiplos de** $g = \gcd(T_s, T_i)$ (todos los que cumplen la identidad de Bézout al recorrer k). El valor positivo más pequeño que se alcanza es exactamente g.

4 El peor caso

Una liberación g unidades **después** del inicio de un marco no puede empezar en ese marco (ya empezó) y espera $T_s - g$ hasta el siguiente. Como mucho, la tarea terminará al final de **ese** marco siguiente, T_s más tarde. Instante de fin $\leq r + (T_s - g) + T_s = r + 2T_s - g$. Para cumplir el plazo,

$$2T_s - \gcd(T_s, T_i) \leq D_i.$$

5 Comprobación numérica

Se recorrieron todos los $T_s \in [1, 39]$ y $T_i \in [T_s, 119]$ y se calculó la espera máxima hasta el siguiente marco: coincide con $T_s - \gcd(T_s, T_i)$ en **todos** los casos.

Resultado final

Las tres restricciones son $T_s \geq \max C_i$, $T_s \mid H$ y $2T_s - \gcd(T_s, T_i) \leq D_i$ para toda tarea.

14.6 6 · RM con periodos armónicos

Con periodos armónicos, RM planifica si y solo si $U \leq 1$

Enunciado

Si cada periodo es múltiplo del anterior ($T_1 \mid T_2 \mid \cdots \mid T_n$) y $D_i = T_i$, demuestra que RM es planificable exactamente cuando $U \leq 1$.

Antes de empezar: qué piden y qué teoría usas

Qué piden. Que en el caso armónico la cota $n(2^{1/n} - 1)$ se mejora hasta 1.
Teoría que usas. Recurrencia de tiempo de respuesta; si $T_j \mid T_i$, entonces $\lceil T_i/T_j \rceil = T_i/T_j$ exactamente.

1 Se evalúa f en $R = T_i$

$$f(T_i) = C_i + \sum_{j<i} \frac{T_i}{T_j} C_j = T_i \sum_{j \leq i} \frac{C_j}{T_j} = T_i U_i,$$

donde U_i es la utilización de las i primeras tareas.

2 Se compara con el plazo

Si $U \leq 1$, entonces $U_i \leq 1$ y $f(T_i) \leq T_i$. Por monotonía de f y minimalidad del punto fijo (demostración 2), el menor punto fijo cumple $R_i \leq T_i = D_i$. Todas las tareas cumplen el plazo.

3 Reciprocidad

Si $U > 1$ el procesador no puede con la carga (demostración 3, necesidad): no es planificable con ningún algoritmo.

4 Comprobación

Con 2 000 conjuntos armónicos aleatorios (2 a 5 tareas) la planificabilidad por tiempo de respuesta coincidió siempre con $U \leq 1$.

Resultado final

Con periodos armónicos, RM llega al 100 % de utilización.

14.7 7 · Pérdida de actualización

Por qué `++contador` desde dos hilos pierde incrementos, y cuánto puede perder

Enunciado

Dos hilos hacen cada uno n veces `++contador` sobre un `int` compartido sin protección. Explica por qué el resultado no es siempre $2n$, da un entrelazado concreto y encuentra el menor valor posible.

Antes de empezar: qué piden y qué teoría usas

Qué piden. Entender que ++x no es una operación única, sino **leer, sumar y escribir**.
Teoría que usas. Condición de carrera, entrelazados de instrucciones, exclusión mutua.

1 Qué hace ++x

Tres pasos: `r = x` (leer), `r = r + 1` (sumar, en un registro propio) y `x = r` (escribir). Si dos hilos leen el mismo valor, ambos escriben ese valor + 1 y **un incremento se pierde**.

2 Entrelazado con $n = 1$

Hilo A	Hilo B	x
lee 0		0
	lee 0	0
escribe 1		1
	escribe 1	1

Resultado 1 en lugar de 2.

3 **Cota superior**

Cada escritura aumenta x en 1 como mucho respecto a lo que su hilo leyó: nunca se supera $2n$. Con exclusión mutua se alcanza siempre.

4 **El peor caso, $n \geq 2$: valor 2**

A lee 0; B ejecuta $n-1$ incrementos completos ($x = n-1$); A escribe 1; B lee 1; A ejecuta sus $n-1$ incrementos restantes ($x = n$); B escribe 2. Resultado: **2**.

5 **No se puede bajar de 2 (verificación)**

Se enumeraron **todos** los entrelazados de las instrucciones leer/escribir para $n = 1, 2, 3, 4, 5$ y el valor mínimo final es $1, 2, 2, 2, 2$. (Prueba general: es un resultado clásico; aquí se verifica por enumeración exhaustiva.)

6 **Solución**

Proteger la sección con un `mutex` (`lock_guard`) o usar `atomic<int>`: el incremento deja de poder intercalarse y el resultado es siempre $2n$.

Resultado final

Sin protección, el resultado final está entre 2 y $2n$ (para $n \geq 2$). Con `mutex` o `atomic`, vale $2n$.

14.8 8 · `push_back` en $O(1)$ amortizado

Si la capacidad se duplica, n inserciones cuestan menos de $3n$ operaciones

Enunciado

Un `vector` guarda sus elementos contiguos. Cuando se llena, reserva el doble de espacio y **copia** todos los elementos. Demuestra que n llamadas a `push_back` cuestan $O(n)$ en total (coste amortizado $O(1)$) y compara con crecer una cantidad fija k.

Antes de empezar: qué piden y qué teoría usas

Qué piden. Razonar con el **coste total** y no con el peor caso de una sola inserción.
Teoría que usas. Serie geométrica $\sum_{i=0}^{m} 2^i = 2^{m+1} - 1$.

1 **Cuántas copias**

Las copias ocurren cuando el tamaño llega a $1, 2, 4, \ldots, 2^m$ con $2^m < n$ (último crecimiento antes de acabar). Copias totales: $\sum_{i=0}^{m} 2^i = 2^{m+1} - 1 < 2n$.

2 **Coste total**

Cada inserción cuesta 1 (escribir) y las copias suman menos de $2n$: total $< 3n$, es decir menos de 3 operaciones por inserción: **amortizado** $O(1)$.

3 **Con factor f general**

Las copias son una serie geométrica de razón $f > 1$: $< n \dfrac{f}{f-1}$. Con $f = 2$, $< 2n$; con $f = 1{,}5$, $< 3n$.

4 **Crecer una cantidad fija k**

Se copian $k, 2k, 3k, \ldots$ elementos: $k\,(1 + 2 + \cdots + \frac{n}{k}) \approx \dfrac{n^2}{2k}$ copias, $O(n^2)$: por inserción, $\frac{n}{2k}$ en promedio, que crece con n.

5 **Comprobación numérica**

Contando copias para $n = 10^3, 10^4, 10^5$: con factor 2, entre 1,0 y 1,7 copias por inserción (siempre < 2); con factor 1,5, de 2,1 a 2,8 (< 3); con $+10$ fijo, 49,5, 499,5 y 4 999,5 copias por inserción ($= n/(2k)$).

Resultado final

Duplicar capacidad hace que `push_back` sea $O(1)$ amortizado. Por eso `reserve` evita copias cuando conoces el tamaño final, pero no cambia la complejidad asintótica.

14.9 9 · El error del Patriot

De 0,1 s truncado a 575 m de error

Enunciado

El reloj del Patriot contaba décimas de segundo y almacenaba $0{,}1$ con 23 bits fraccionarios truncados. Calcula el error por tic y la deriva tras 100 horas de funcionamiento, y el error de posición de un misil a 1 676 m/s.

Antes de empezar: qué piden y qué teoría usas

Qué piden. Una cuenta con potencias de 2 que explique por qué un error de 10^{-7} s acaba siendo un fallo de cientos de metros.
Teoría que usas. Representación binaria de fracciones, truncamiento, acumulación de errores.

1 0,1 en binario

$0{,}1 = 0{,}0001100110011\ldots_2$: fracción **periódica infinita** (el bloque 0011 se repite), como $1/3 = 0{,}333\ldots$ en base 10. No cabe en un número finito de bits.

2 Truncar a 23 bits

$0{,}1 \cdot 2^{23} = 838\,860{,}8$. Se guarda la parte entera $838\,860$ y se pierde $0{,}8$ unidades de 2^{-23}:

$$e = 0{,}8 \cdot 2^{-23} = 0{,}1 \cdot 2^{-20} \approx 9{,}537 \cdot 10^{-8} \text{ s por tic.}$$

3 Acumulación en 100 h

100 h $= 360\,000$ s $= 3{,}6 \cdot 10^6$ tics (10 por segundo). Deriva: $3{,}6 \cdot 10^6 \cdot 9{,}537 \cdot 10^{-8} \approx 0{,}3433$ s.

4 En metros

A $1\,676$ m/s: $0{,}3433 \cdot 1\,676 \approx 575$ m. El sistema buscaba el misil en el sitio equivocado y lo perdió.

Resultado final

Error por tic $9{,}54 \cdot 10^{-8}$ s; deriva en 100 h $0{,}343$ s; error de posición ≈ 575 m (la bibliografía suele citar 0,34 s y entre 573 y 575 m; la diferencia es solo de redondeo).

15 Autoevaluación

15.1 Test con corrección

Responde primero sin mirar; luego comprueba la respuesta y la explicación.

1. ¿Qué es un sistema de tiempo real firme?

a) El resultado tardío vale poco, pero se acepta
b) El resultado tardío no sirve y se descarta, pero el sistema continúa
c) Perder un plazo es catastrófico
d) Ningún plazo es importante

Respuesta: b). En el tiempo real firme un resultado fuera de plazo se descarta pero no hay catástrofe; en el duro sí la hay, y en el blando sirve con menos valor.

2. ¿Qué mide el jitter?

a) La utilización de la CPU
b) La variación del instante de activación respecto al ideal
c) El tiempo de cómputo medio
d) El número de hilos

Respuesta: b). Es la variabilidad temporal de una tarea periódica.

3. Para verificar los plazos de un sistema de tiempo real duro hay que usar…

a) El tiempo de cómputo medio
b) El tiempo de cómputo del peor caso
c) Un solo intento de prueba
d) El tiempo de compilación

Respuesta: b). Los plazos duros se garantizan con el peor caso (C), no con la media.

4. ¿Qué hace `m["x"]` en un `map<string,int>` si «x» no existe?

a) Lanza una excepción
b) Devuelve un valor sin crear nada
c) Crea la entrada con valor 0
d) Devuelve `end()`

Respuesta: c). `operator[]` inserta la clave con el valor por defecto. Para consultar sin modificar, usa `find` o `at`.

5. Tras `it = v.erase(it)`, ¿qué apunta `it`?

a) Al elemento borrado
b) Al siguiente elemento
c) A `begin()`
d) Es inválido

Respuesta: b). `erase` devuelve el iterador al elemento siguiente al borrado.

6. Un `vector` tiene size 8 y capacity 8. Haces `push_back`. Con factor 2, ¿qué capacity queda?

a) 9
b) 16
c) 10
d) 8

Respuesta: b). Se reserva el doble (16), se copian los 8 elementos y se libera el bloque antiguo.

7. ¿Cómo se pasa un `string` grande a una función que solo lo lee?

a) `string s`
b) `string& s`
c) `const string& s`
d) `string* s`

Respuesta: c). `const T&` evita la copia y garantiza que no se modifica.

8. ¿Qué devuelve un `operator<<` bien escrito?

a) `void`
b) `ostream&`
c) `string`
d) `bool`

Respuesta: b). Devolviendo el mismo flujo se pueden encadenar varias salidas: `cout << a << b`.

9. Una clase tiene un `int*` obtenido con `new[]`. Sin escribir copia, `Buffer b = a;`…

a) Copia el contenido
b) Comparte el bloque y lo libera dos veces
c) No compila
d) Mueve el bloque

Respuesta: b). La copia por defecto copia el puntero; ambos destructores hacen `delete[]` del mismo bloque.

10. ¿En qué orden se inicializan los atributos?

a) En el de la lista de inicialización
b) En el orden de declaración en la clase
c) Alfabético
d) En el que decida el compilador

Respuesta: b). Siempre en el orden de declaración, aunque la lista los escriba en otro.

11. ¿Qué atributos NO necesitan gestión manual de copia (regla de 0)?

a) `int*` con `new`
b) `string` y `vector`
c) Descriptores de fichero C
d) Memoria con `malloc`

Respuesta: b). `string` y `vector` ya gestionan su memoria y su copia correctamente.

12. ¿Qué hace falta para que `p->area()` ejecute la versión de la clase derivada?

a) Solo herencia
b) Método `virtual` y llamada vía puntero o referencia a la base
c) Que la clase sea `final`
d) Que `area` sea `static`

Respuesta: b). Sin `virtual`, o con copia por valor, el enlace es estático.

13. ¿Por qué la base de una jerarquía necesita destructor virtual?

a) Para compilar
b) Para destruir correctamente un objeto derivado a través de un puntero a la base
c) Para evitar el slicing
d) Para que sea abstracta

Respuesta: b). Sin él, `delete p` solo llama al destructor de la base.

14. `vector<C_Figura> v; v.push_back(C_Circulo{"c",1});`

a) Mantiene el círculo completo
b) Provoca slicing: solo queda la parte C_Figura (y no compila si es abstracta)
c) Es la forma recomendada
d) Llama a `to_string` del círculo

Respuesta: b). Un contenedor de objetos base copia por valor y recorta. Usa punteros.

15. ¿Cuál de estas clases es abstracta?

a) `class A { virtual void f() = 0; };`
b) `class A { virtual void f() {} };`
c) `class A { void f() {} };`
d) `class A {};`

Respuesta: a). `= 0` en un método virtual lo hace puro y la clase abstracta.

16. ¿Por qué `++x` desde dos hilos pierde incrementos?

a) Porque `++` es lento
b) Porque son tres pasos (leer, sumar, escribir) que se pueden intercalar
c) Porque `x` es `int`
d) Porque el compilador lo optimiza mal siempre

Respuesta: b). Si los dos leen el mismo valor antes de escribir, uno de los incrementos se pisa.

17. ¿Qué pasa si destruyes un `thread` sin hacer `join` ni `detach`?

a) Nada
b) Se llama a `std::terminate`
c) Se hace join solo
d) Se libera la CPU

Respuesta: b). El estándar obliga a unir o soltar antes de destruir.

18. ¿Qué evita el interbloqueo con dos mutex?

a) `lock_guard` sobre cada uno por separado
b) `scoped_lock lk{m1, m2};`
c) `sleep_for`
d) `detach`

Respuesta: b). Adquiere todos a la vez con un algoritmo seguro; cada `lock_guard` por separado no lo garantiza.

19. ¿Qué es mejor para una tarea periódica?

a) `sleep_for(T)`
b) `sleep_until(siguiente); siguiente += T;`
c) `while` activo
d) `detach`

Respuesta: b). `sleep_until` con instante absoluto evita la deriva acumulada.

20. ¿Por qué se usa un predicado en `cv.wait(lk, pred)`?

a) Es más rápido
b) Para tolerar despertares espurios y avisos adelantados
c) Para evitar el mutex
d) Es obligatorio por sintaxis

Respuesta: b). `wait` recomprueba la condición al despertar, así que no dependes de que el aviso llegue justo a tiempo ni de que sea cierto.

21. ¿Qué tipo de bloqueo exige `condition_variable::wait`?

a) `lock_guard`
b) `unique_lock`
c) `scoped_lock` de dos mutex
d) Ninguno

Respuesta: b). `wait` debe poder desbloquear y volver a bloquear el mutex.

22. Un hilo cambia `terminar_ = true` sin el mutex y llama a `notify_all`. ¿Qué riesgo hay?

a) Ninguno
b) Carrera con el predicado y aviso perdido
c) Interbloqueo seguro
d) El hilo se destruye

Respuesta: b). La modificación debe hacerse con el mutex para que el predicado vea el valor y no se pierda el aviso.

23. ¿Qué hace `wait` mientras el hilo duerme?

a) Mantiene el mutex bloqueado
b) Libera el mutex
c) Hace espera activa
d) Cierra la variable

Respuesta: b). Libera el mutex al dormir y lo vuelve a tomar al despertar.

24. En RM, ¿qué tarea tiene la mayor prioridad?

a) La de mayor cómputo
b) La de menor periodo
c) La de mayor plazo
d) La de mayor utilización

Respuesta: b). Rate Monotonic: tasa más alta (periodo más pequeño), prioridad más alta.

25. Si U supera la cota de Liu y Layland, el sistema…

a) No es planificable
b) Puede serlo o no: hay que calcular el tiempo de respuesta
c) Es planificable con EDF siempre
d) Es de tiempo real blando

Respuesta: b). La cota es solo condición suficiente.

26. Tareas (C,T): (1,4) y (2,6), D=T. ¿Cuál es R de la segunda con RM?

a) 2
b) 3
c) 4
d) 6

Respuesta: b). R = 2 + ⌈R/4⌉·1: 2→3→3 (⌈3/4⌉=1).

27. Con ICPP, ¿cuántas veces se bloquea como máximo una tarea por activación?

a) Una vez
b) Tantas como recursos use
c) Indefinidas
d) Dos

Respuesta: a). Como mucho una vez y una sección crítica; además, no hay interbloqueo.

28. ¿Por qué `0.1 + 0.2 == 0.3` es falso en `double`?

a) Por un error del compilador
b) Porque 0,1 y 0,2 no son exactos en binario y el error se acumula
c) Porque == no funciona con `double`
d) Porque `0.3` es `float`

Respuesta: b). 0,1 y 0,2 se aproximan; la suma no coincide exactamente con la aproximación de 0,3.

29. El error del Patriot fue grande porque…

a) El error por tic era enorme
b) Un error pequeño se acumuló durante ~100 h
c) El radar fallaba
d) Usaba `float`

Respuesta: b). 3,6 millones de tics con $9{,}5 \cdot 10^{-8}$ s de error dan 0,34 s.

30. En Pathfinder, ¿qué tarea expulsó a la que tenía el mutex y alargó el bloqueo de la tarea más prioritaria?

a) Una tarea de prioridad media
b) La de menor prioridad
c) El watchdog
d) El planificador cíclico

Respuesta: a). Las tareas medias expulsan a la baja y la alta espera por ellas: inversión de prioridad.

31. ¿Cuál es la forma correcta de comparar dos `double`?

a) `a == b`
b) `fabs(a-b) < eps`
c) `a - b == 0`
d) `(int)a == (int)b`

Respuesta: b). Comparar con una tolerancia adecuada al problema.

32. En un ejecutivo cíclico con $T_s = 8$ y una tarea de periodo 12, ¿se cumple $2T_s - \gcd(T_s, T_i) \leq D_i$ con $D_i = 12$?

a) No, da 14
b) Sí, da exactamente 12
c) No, da 16
d) Sí, da 4

Respuesta: b). $2 \cdot 8 - \gcd(8, 12) = 16 - 4 = 12 \leq 12$: se cumple con igualdad.

33. Un conjunto de 3 tareas con $U = 0{,}814$ y RM…

a) No es planificable (supera la cota)
b) Puede serlo: hay que calcular los tiempos de respuesta
c) Es planificable seguro
d) Solo lo es con EDF

Respuesta: b). La cota 0,780 es suficiente, no necesaria: sobre ella el test no decide.

34. Con ICPP, el bloqueo B_i de una tarea es…

a) La suma de todas las secciones críticas posibles
b) El máximo de **una** sección crítica de menor prioridad con techo $\geq P_i$
c) Siempre 0
d) La duración de la sección crítica más larga de cualquier tarea

Respuesta: b). ICPP garantiza bloqueo único y acotado por una sección.

35. ¿Por qué se usa `sleep_until` en una tarea periódica en vez de `sleep_for`?

a) Porque consume menos CPU
b) Para no acumular la deriva del tiempo de trabajo
c) Porque `sleep_for` no existe en C++17
d) Para que el hilo pueda ser interrumpido

Respuesta: b). `sleep_for` suma el trabajo al periodo; `sleep_until` fija instantes absolutos.

36. En `vector<unique_ptr<Item>> v`, para añadir un elemento hay que…

a) `v.push_back(p)` con una copia
b) `v.push_back(move(p))`
c) `v.push_back(&p)`
d) `v = p`

Respuesta: b). Un `unique_ptr` no se copia; se mueve.

37. Para borrar de un `vector` todos los elementos que cumplen una condición se usa…

a) `v.remove_if(...)`
b) `v.erase(remove_if(v.begin(), v.end(), f), v.end())`
c) `delete v`
d) `v.clear()`

Respuesta: b). `remove_if` reordena; `erase` elimina la cola sobrante.

38. Un monitor tiene un `wait` con predicado que incluye ‘

a)
b) cerrado_‘. ¿Qué hay que hacer al cerrar?
c) `notify_one`
d) `notify_all` bajo/tras tomar el `mutex`
e) Nada, el hilo se despierta solo
f) `terminate`

Respuesta: b). Todos los hilos que esperan deben comprobar la nueva condición.

39. ¿Qué hace `in.clear(); in.ignore(numeric_limits<streamsize>::max(), '\n');` tras una lectura fallida?

a) Borra el fichero
b) Limpia el estado de error y descarta el resto de la línea
c) Cierra el flujo
d) Reinicia el programa

Respuesta: b). Sin `clear()` las lecturas siguientes seguirían fallando.

40. Con una criba de Eratóstenes, ¿por qué el bucle interior empieza en `i*i`?

a) Por rendimiento solo
b) Los múltiplos menores ya los marcó un primo más pequeño
c) Para evitar el desbordamiento
d) Porque 1 no es primo

Respuesta: b). $k \cdot i$ con $k < i$ tiene un factor primo menor que i.

41. En un `vector` que duplica su capacidad, `push_back` es…

a) $O(n)$ siempre
b) $O(1)$ amortizado
c) $O(\log n)$
d) $O(n^2)$

Respuesta: b). Las copias suman menos de $2n$ en n inserciones.

15.2 Tarjetas de repaso

Pregunta	Respuesta
¿Cuánto vale el ejercicio de concurrencia en los exámenes recientes?	El 30 % de la nota del examen (POO el 70 %).
¿Qué pasa si el programa no compila (examen 2026)?	La puntuación se divide por 2.
¿Cuánto dura el examen reciente?	2 h 30 min.
Tiempo real duro vs firme vs blando	Duro: fallo catastrófico. Firme: resultado tardío inútil. Blando: resultado tardío con menos valor.
¿Qué es C, T y D?	C = cómputo en peor caso, T = periodo, D = plazo relativo.
¿Qué devuelve `v.end()`?	Un iterador a una posición después del último elemento (no se puede desreferenciar).
Diferencia entre `find` y `[]` en un map	`find` no modifica el mapa; `[]` crea la entrada si no existe.
¿Cuándo se invalidan iteradores de un vector?	Tras `push_back` con realocación y tras `erase` (el iterador borrado y los posteriores).
¿Para qué sirve `const` al final de un método?	Garantiza que no modifica el objeto y permite llamarlo sobre objetos `const`.
¿Qué es copy-and-swap?	Asignar copiando a un temporal e intercambiando sus datos con los del objeto; seguro ante excepciones y autoasignación.
¿Qué es la vtable?	Tabla de punteros a métodos virtuales por clase; cada objeto polimórfico apunta a la de su clase real.
¿Qué es slicing?	Copiar un objeto derivado a una variable base por valor: se pierde la parte derivada y el polimorfismo.
¿Para qué sirve `override`?	Para que el compilador verifique que realmente redefines un virtual de la base.
¿Cómo pasas una variable por referencia a `thread`?	Con `std::ref(var)`; si no, se copia.
Cuatro condiciones del interbloqueo	Exclusión mutua, retener y esperar, no expulsión, espera circular.
¿`lock_guard` o `scoped_lock`?	`lock_guard` para un mutex; `scoped_lock` para varios a la vez.
Patrón de espera	`unique_lock lk{m}; cv.wait(lk, [&]{ return cond; });`
Patrón de aviso	Cambiar el estado con el mutex, soltar y `cv.notify_one()` / `notify_all()`.
¿Cuándo `notify_all` en lugar de `notify_one`?	Cuando varios hilos con predicados distintos comparten la variable o puede quedar más de uno listo.
Recurrencia del tiempo de respuesta	R = C + B + Σ ⌈R/Tj⌉·Cj sobre las tareas más prioritarias; iterar hasta punto fijo.
Bloqueo con ICPP	B_i = máx. sección crítica de tareas de menor prioridad sobre recursos con techo ≥ P_i.
Condición del ciclo secundario	Ts divide a Tm, Ts ≥ máx C, 2Ts − mcd(Ts,Ti) ≤ Di.
¿Por qué 0,1 no es exacto en binario?	Es una fracción periódica infinita en base 2; solo cabe una aproximación.
¿Qué falló en Mars Pathfinder?	Inversión de prioridad con un mutex sin herencia de prioridad; el watchdog reinició el sistema. Se arregló activando la herencia.
Fórmula del tiempo de respuesta con bloqueo	$R_i = C_i + B_i + \sum_{j \in hp(i)} \lceil R_i/T_j \rceil C_j$, iterando desde $R^{(0)} = C_i + B_i$ hasta que no cambie o supere D_i.
Restricciones de marco en el ejecutivo cíclico	$T_s \geq \max C_i$; $T_s \mid H$; $2T_s - \gcd(T_s, T_i) \leq D_i$ para toda tarea.
Cota de Liu–Layland	$U \leq n(2^{1/n} - 1)$ (suficiente, no necesaria); tiende a $\ln 2 \approx 0{,}693$. Con periodos armónicos, $U \leq 1$.
¿Cuándo es planificable con EDF?	Con $D_i = T_i$, apropiativo y un procesador: si y solo si $U \leq 1$.
¿Qué garantiza ICPP?	Bloqueo único y acotado por una sección crítica, y ausencia de interbloqueos.

Pregunta	Respuesta
`sleep_for` frente a `sleep_until`	`sleep_for` acumula deriva; `sleep_until` con instante teórico + periodo, no.
copy-and-swap	`operator=(T o) { swap(*this, o); return *this; }`: copia por valor, intercambio, y el destructor limpia lo viejo.

16 Simulacros

Tres simulacros para practicar con reloj. Cada uno imita el examen: **2 h 30 min**, parte de programación orientada a objetos (≈ 70 %) y parte de concurrencia (≈ 30 %); el tercero es de planificación. Hazlos **sin mirar** las soluciones: cada paso está plegado, y el resultado final también. Escribe el código a mano o en un editor sin ayuda, y compílalo con `g++ -std=c++17 -Wall -Wextra -pthread`.

Cómo usarlos

1. Pon un temporizador de 75 minutos por ejercicio (150 min para el simulacro completo).
2. Si el programa **no compila**, la nota de ese ejercicio se divide por 2 en el examen reciente: compila siempre antes de dar por bueno.
3. Al terminar, abre los pasos y compara. Anota qué te ha fallado en la lista de «Qué suele fallar» del tema.

16.1 Simulacro 1 · Biblioteca y parking

Simulacro 1 · Ejercicio 1: préstamos de una biblioteca (POO, 70 %)

Enunciado

Una biblioteca presta **libros** y **DVD**. Todo elemento tiene título y año, un número de días de préstamo y una multa por día de retraso.

1. Diseña una clase base abstracta `Item` con título, año, `dias_prestamo()` y `multa_por_dia()` (virtuales puros). Los libros (14 días, 0,50 €/día) guardan además las páginas; los DVD (7 días, 1 €/día), los minutos.
2. Sobrecarga `operator<<` para que todo elemento se imprima con su título, año y días, y cada hija añada sus datos **sin repetir** el código de la base.
3. Diseña `Biblioteca`, que guarda los elementos en un `vector` con punteros inteligentes y permite: añadir; calcular la multa de un título por unos días de retraso; devolver el elemento de mayor multa diaria; contar cuántos hay de un tipo (con `typeid`) y eliminar un título.
4. Escribe un `main` que pruebe todo.

Antes de empezar: qué piden y qué teoría usas

Qué piden. Herencia con clase abstracta, polimorfismo (colección de punteros a la base), `operator<<` con gancho virtual, y algoritmos STL con lambdas.
Teoría que usas. T3: clase abstracta, destructor virtual, `override`, `unique_ptr` en `vector`, `max_element`, `remove_if` + `erase`, `typeid`.

1 La base abstracta

El destructor es **virtual** (se borran objetos hijos a través de la base). Los métodos de política son virtuales puros. `operator<<` es `friend` y delega en un método **protegido virtual** `imprimir`, que las hijas reutilizan llamando a `Item::imprimir`.

CPP

```cpp
class Item {
public:
    Item(string titulo, int anio) : titulo_{move(titulo)}, anio_{anio} {}
    virtual ~Item() = default;                          // destructor virtual: se borra a través de la base
    virtual double multa_por_dia() const = 0;           // virtual puro: Item es abstracta
    virtual int dias_prestamo() const = 0;
    const string& titulo() const { return titulo_; }
    friend ostream& operator<<(ostream& os, const Item& i) { i.imprimir(os); return os; }
protected:
    virtual void imprimir(ostream& os) const {          // gancho reutilizable por las hijas
        os << '"' << titulo_ << "\" (" << anio_ << "), " << dias_prestamo() << " dias";
    }
private:
    string titulo_;
    int anio_;
};
```

2 Libro y DVD

Cada una implementa los dos métodos y amplía `imprimir`. Usa `override` para que el compilador te avise si te equivocas en la firma.

CPP

```cpp
class Libro : public Item {
public:
    Libro(string t, int a, int paginas) : Item{move(t), a}, paginas_{paginas} {}
    double multa_por_dia() const override { return 0.5; }
    int dias_prestamo() const override { return 14; }
protected:
    void imprimir(ostream& os) const override { Item::imprimir(os); os << ", " << paginas_ << " pags"; }
private:
    int paginas_;
};

class Dvd : public Item {
public:
    Dvd(string t, int a, int minutos) : Item{move(t), a}, minutos_{minutos} {}
    double multa_por_dia() const override { return 1.0; }
    int dias_prestamo() const override { return 7; }
protected:
    void imprimir(ostream& os) const override { Item::imprimir(os); os << ", " << minutos_ << " min"; }
private:
    int minutos_;
};
```

3 Biblioteca

- `find_if` con lambda para localizar un título (si no hay, o los días no son positivos, la multa es 0).
- `max_element` con comparador sobre `multa_por_dia()`.
- `typeid(*p) == t` compara el **tipo dinámico**; `typeid(p)` sería el del puntero.
- `remove_if` mueve los elementos que se mantienen al principio; `erase` borra la cola. Con `unique_ptr`, la memoria se libera sola.

CPP

```cpp
class Biblioteca {
public:
    void anadir(unique_ptr<Item> i) { items_.push_back(move(i)); }
    double multa(const string& titulo, int dias_tarde) const {
        auto it = find_if(items_.begin(), items_.end(), [&](const auto& p) { return p->titulo() == titulo; });
        if (it == items_.end() || dias_tarde <= 0) return 0;
        return (*it)->multa_por_dia() * dias_tarde;
    }
    const Item* mayor_multa() const {
        auto it = max_element(items_.begin(), items_.end(),
                              [](const auto& a, const auto& b) { return a->multa_por_dia() < b->multa_por_dia(); });
        return it == items_.end() ? nullptr : it->get();
    }
    int contar(const type_info& t) const {
        return count_if(items_.begin(), items_.end(), [&](const auto& p) { return typeid(*p) == t; });
    }
    bool eliminar(const string& titulo) {
        auto it = remove_if(items_.begin(), items_.end(), [&](const auto& p) { return p->titulo() == titulo; });
        bool borrado = it != items_.end();
        items_.erase(it, items_.end());                // unique_ptr libera la memoria solo
        return borrado;
    }
    friend ostream& operator<<(ostream& os, const Biblioteca& b) {
        for (const auto& p : b.items_) os << "  - " << *p << '\n';
        return os;
    }
private:
    vector<unique_ptr<Item>> items_;
};
```

4 main

CPP

```
int main() {
    Biblioteca b;
    b.anadir(make_unique<Libro>("Quijote", 1605, 1100));
    b.anadir(make_unique<Dvd>("Matrix", 1999, 136));
    b.anadir(make_unique<Libro>("Ulises", 1922, 730));
    cout << b;
    cout << "multa Quijote 4 dias: " << b.multa("Quijote", 4) << " EUR\n";
    cout << "multa Matrix 3 dias: " << b.multa("Matrix", 3) << " EUR\n";
    cout << "mayor multa/dia: " << *b.mayor_multa() << '\n';
    cout << "libros: " << b.contar(typeid(Libro)) << ", dvd: " << b.contar(typeid(Dvd)) << '\n';
    cout << boolalpha << "eliminar Ulises: " << b.eliminar("Ulises") << ", eliminar X: " << b.eliminar("X") << '\n';
    cout << b;
}
```

5 Lista de comprobación

- ¿Compila sin avisos? ¿Destructor virtual? ¿`override` en las hijas?
- ¿`vector<unique_ptr<Item>>` y `move` al añadir (no se puede copiar un `unique_ptr`)?
- ¿`typeid(*p)` y no `typeid(p)`?

Resultado final

Salida

```
  - "Quijote" (1605), 14 dias, 1100 pags
  - "Matrix" (1999), 7 dias, 136 min
  - "Ulises" (1922), 14 dias, 730 pags
multa Quijote 4 dias: 2 EUR
multa Matrix 3 dias: 3 EUR
mayor multa/dia: "Matrix" (1999), 7 dias, 136 min
libros: 2, dvd: 1
eliminar Ulises: true, eliminar X: false
  - "Quijote" (1605), 14 dias, 1100 pags
  - "Matrix" (1999), 7 dias, 136 min
```

Simulacro 1 · Ejercicio 2: parking con plazas limitadas (concurrencia, 30 %)

Enunciado

Un parking tiene `n` plazas. Los coches (hilos) llaman a `entrar()`, que **espera** si no hay plaza libre, y a `salir()`. Cuando el parking **cierra**, ningún coche que espere puede quedarse bloqueado: `entrar()` devuelve `false`. Implementa el monitor `Parking` y un `main` con 6 coches y 2 plazas que compruebe que nunca hay más de 2 coches dentro.

Antes de empezar: qué piden y qué teoría usas

Qué piden. Un monitor con `mutex` y `condition_variable`, una condición de espera con predicado y una condición de cierre.
Teoría que usas. T5: monitor, `unique_lock`, `wait` con predicado, `notify_one` / `notify_all`, bandera de cierre.

1 Estado

`libres_` (plazas libres), `cerrado_`, `ocupadas_max_` (para comprobar la invariante) y el `mutex`.

2 `entrar` y `salir`

`entrar` espera con el predicado `libres_ > 0 || cerrado_`; si se despertó por el cierre, devuelve `false` **sin tomar plaza**. `salir` incrementa bajo el `mutex` y avisa **a uno** (hay un solo hueco). `cerrar` activa la bandera y avisa a **todos**.

CPP

```
class Parking {
public:
    explicit Parking(int plazas) : libres_{plazas} {}
    bool entrar() {                                    // false si el parking cerró mientras esperaba
        unique_lock<mutex> lk{m_};
        hay_hueco_.wait(lk, [this] { return libres_ > 0 || cerrado_; });
        if (cerrado_) return false;
```

```
        --libres_;
        ocupadas_max_ = max(ocupadas_max_, total_ - libres_);
        return true;
    }
    void salir() {
        { lock_guard<mutex> lk{m_}; ++libres_; }
        hay_hueco_.notify_one();                          // un hueco: despierta a uno
    }
    void cerrar() {
        { lock_guard<mutex> lk{m_}; cerrado_ = true; }
        hay_hueco_.notify_all();                          // el cierre interesa a todos
    }
    int max_ocupadas() { lock_guard<mutex> lk{m_}; return ocupadas_max_; }
private:
    mutex m_;
    condition_variable hay_hueco_;
    int libres_;
    const int total_ = libres_;
    int ocupadas_max_ = 0;
    bool cerrado_ = false;
};
```

3 main

CPP

```
int main() {
    Parking p{2};
    atomic<int> entradas{0};
    vector<thread> coches;
    for (int i = 0; i < 6; ++i)
        coches.emplace_back([&] {
            if (p.entrar()) { ++entradas; this_thread::sleep_for(50ms); p.salir(); }
        });
    for (auto& c : coches) c.join();
    p.cerrar();
    cout << "entradas = " << entradas << '\n';
    cout << boolalpha << "max ocupadas <= 2: " << (p.max_ocupadas() <= 2) << '\n';
    cout << "entrada tras cierre: " << p.entrar() << '\n';
}
```

4 Por qué es correcto

- Toda lectura/escritura de `libres_` y `cerrado_` se hace con el `mutex` tomado.
- El predicado se vuelve a comprobar tras despertar: un coche que llega de fuera puede quitarte el hueco antes de que reacciones.
- `notify_all` en `cerrar()` + condición `|| cerrado_`: ningún hilo queda dormido para siempre.

Resultado final

Salida

```
entradas = 6
max ocupadas <= 2: true
entrada tras cierre: false
```

16.2 Simulacro 2 · Matriz y tarea periódica

Simulacro 2 · Ejercicio 1: clase `Matriz` con memoria dinámica (POO, 70 %)

Enunciado

Diseña una clase `Matriz` de números reales que gestione su propia memoria (`double*`) con:

1. Constructor `(filas, columnas)` con ceros, y acceso `m(i, j)` (versiones modificable y `const`).
2. Regla de los cinco: destructor, constructor de copia (copia **profunda**), constructor de movimiento y asignación con **copy-and-swap**.
3. `operator+`, `operator*` (producto), `traspuesta()` y `operator<<`. Dimensiones incompatibles → excepción.

4. Un `main` que construya $A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$, calcule $A\,A^T$, demuestre que la copia es independiente y capture un error.

Antes de empezar: qué piden y qué teoría usas

Qué piden. Una clase con recursos propios: si no escribes la copia profunda, dos matrices compartirían el mismo `double*` y se borrarían dos veces.
Teoría que usas. T2: reglas de 0/3/5, copy-and-swap, `friend swap`, `noexcept`, `operator()`, excepciones.

1 Memoria y regla de los cinco

- Constructor: `new double[n]{}` (inicializa a 0).
- Copia: reserva nueva y `copy`.
- Movimiento: roba el puntero y deja el origen **vacío** (`nullptr`, 0×0), para que su destructor no borre lo robado.
- Asignación por **valor** + `swap`: gestiona a la vez la copia y la asignación a uno mismo, y es segura ante excepciones.

CPP

```cpp
class Matriz {
public:
    Matriz(int filas, int cols) : f_{filas}, c_{cols}, d_{new double[filas * cols]{}} {}
    Matriz(const Matriz& o) : f_{o.f_}, c_{o.c_}, d_{new double[o.f_ * o.c_]} { copy(o.d_, o.d_ + f_ * c_, d_); }
    Matriz(Matriz&& o) noexcept : f_{o.f_}, c_{o.c_}, d_{o.d_} { o.d_ = nullptr; o.f_ = o.c_ = 0; }
    Matriz& operator=(Matriz o) noexcept { swap(*this, o); return *this; }       // copy-and-swap
    ~Matriz() { delete[] d_; }
    friend void swap(Matriz& a, Matriz& b) noexcept { swap(a.f_, b.f_); swap(a.c_, b.c_); swap(a.d_, b.d_); }

    double& operator()(int i, int j) { return d_[i * c_ + j]; }
    double operator()(int i, int j) const { return d_[i * c_ + j]; }
    int filas() const { return f_; }
    int cols() const { return c_; }
```

2 Operaciones

La suma exige mismas dimensiones; el producto, columnas de la izquierda = filas de la derecha. Se acumula en `r(i,j)` (la matriz resultado empieza en 0).

CPP

```cpp
    Matriz operator+(const Matriz& o) const {
        if (f_ != o.f_ || c_ != o.c_) throw invalid_argument{"dimensiones distintas"};
        Matriz r{f_, c_};
        for (int k = 0; k < f_ * c_; ++k) r.d_[k] = d_[k] + o.d_[k];
        return r;
    }
    Matriz operator*(const Matriz& o) const {
        if (c_ != o.f_) throw invalid_argument{"no se pueden multiplicar"};
        Matriz r{f_, o.c_};
        for (int i = 0; i < f_; ++i) for (int j = 0; j < o.c_; ++j) for (int k = 0; k < c_; ++k) r(i, j) += (*this)(i,
        ↪  k) * o(k, j);
        return r;
    }
    Matriz traspuesta() const {
        Matriz r{c_, f_};
        for (int i = 0; i < f_; ++i) for (int j = 0; j < c_; ++j) r(j, i) = (*this)(i, j);
        return r;
    }
    friend ostream& operator<<(ostream& os, const Matriz& m) {
        for (int i = 0; i < m.f_; ++i) { for (int j = 0; j < m.c_; ++j) os << m(i, j) << (j + 1 < m.c_ ? " " : ""); os
        ↪  << '\n'; }
        return os;
    }
private:
    int f_, c_;
    double* d_;
};
```

3 main

CPP

```
int main() {
    Matriz a{2, 3};
    int v = 1;
    for (int i = 0; i < 2; ++i) for (int j = 0; j < 3; ++j) a(i, j) = v++;
    Matriz b = a.traspuesta();                  // 3x2
    cout << "A:\n" << a << "A^T:\n" << b << "A*A^T:\n" << a * b;
    Matriz c = a;                               // copia profunda
    c(0, 0) = 99;
    cout << "A(0,0) tras modificar la copia: " << a(0, 0) << '\n';
    c = b.traspuesta() + a;                     // asignación (copy-and-swap)
    cout << "A+A:\n" << c;
    try { a * a; } catch (const exception& e) { cout << "error: " << e.what() << '\n'; }
}
```

4 Comprobación

$A\,A^T = \begin{pmatrix} 1+4+9 & 4+10+18 \\ 4+10+18 & 16+25+36 \end{pmatrix} = \begin{pmatrix} 14 & 32 \\ 32 & 77 \end{pmatrix}$: coincide con la salida. $A \cdot A$ es 2×3 por 2×3: error de dimensiones.

Resultado final

Salida

```
A:
1 2 3
4 5 6
A^T:
1 4
2 5
3 6
A*A^T:
14 32
32 77
A(0,0) tras modificar la copia: 1
A+A:
2 4 6
8 10 12
error: no se pueden multiplicar
```

Simulacro 2 · Ejercicio 2: tarea periódica y contador compartido (concurrencia, 30 %)

Enunciado

1. Escribe un hilo que ejecute 5 veces una tarea cada 20 ms **sin acumular deriva** (usa `sleep_until`) y cuenta cuántas veces se ejecutó. Comprueba que la duración total es de unos 100 ms.
2. Lanza 4 hilos que incrementen 1 000 veces un contador compartido y comprueba que el resultado es 4 000. Usa el mecanismo mínimo adecuado.

Antes de empezar: qué piden y qué teoría usas

Qué piden. Diferenciar `sleep_for` (deriva) de `sleep_until` (sin deriva) y proteger un contador con `atomic`.
Teoría que usas. T4: `sleep_until`, `steady_clock`, `atomic`, `join`.

1 Tarea periódica

El instante siguiente se calcula **sumando el periodo al instante teórico anterior**, no a `now()`. Con `sleep_for(20ms)` cada vuelta duraría 20 ms **más** lo que tardara el trabajo, y el error se acumularía.

CPP

```
int main() {
    // 1) tarea periódica sin deriva: sleep_until sobre el instante anterior + periodo
    const auto periodo = 20ms;
    int ticks = 0;
    auto inicio = steady_clock::now();
    thread t{[&] {
```

```
        auto siguiente = steady_clock::now();
        for (int k = 0; k < 5; ++k) {
            ++ticks;                                    // «trabajo» de la tarea
            siguiente += periodo;                       // se suma al instante teórico, no a now()
            this_thread::sleep_until(siguiente);
        }
    }};
    t.join();
    auto ms = duration_cast<milliseconds>(steady_clock::now() - inicio).count();
    cout << "ticks = " << ticks << '\n';
```

2 Contador compartido

`atomic<int>` basta: la operación es un incremento simple. Un `mutex` también valdría, pero es más pesado. Sin protección, el resultado sería menor que 4 000 (demostración de «Pérdida de actualización»).

CPP

```
    // 2) cuatro hilos suman 1000 veces con atomic
    atomic<int> total{0};
    vector<thread> hilos;
    for (int i = 0; i < 4; ++i) hilos.emplace_back([&] { for (int k = 0; k < 1000; ++k) ++total; });
    for (auto& h : hilos) h.join();
    cout << "total = " << total << '\n';
}
```

Resultado final

Salida

```
ticks = 5
duracion >= 100 ms: true, < 150 ms: true
total = 4000
```

16.3 Simulacro 3 · Planificación

Simulacro 3 · Planificación de tiempo real

Enunciado

Tres tareas periódicas con plazo igual al periodo: τ_1 ($C = 2, T = 8$), τ_2 ($C = 3, T = 12$) y τ_3 ($C = 4, T = 24$). La tarea τ_1 y la τ_3 comparten el recurso `r1`; τ_3 lo mantiene durante 2 unidades de tiempo como máximo y τ_1 durante 1.

1. Calcula la utilización y aplica el test de Liu–Layland con Rate Monotonic.
2. Calcula los tiempos de respuesta (sin bloqueo).
3. Con el protocolo de techo de prioridad inmediato, calcula los bloqueos B_i y los tiempos de respuesta.
4. Diseña un ejecutivo cíclico: hiperperiodo, tamaños de marco válidos y plan de ejecución.

Antes de empezar: qué piden y qué teoría usas

Qué piden. Los tres análisis clásicos de T6: test de utilización, tiempo de respuesta con bloqueo y ejecutivo cíclico.
Teoría que usas. $U \le n(2^{1/n} - 1)$, $R = C + B + \sum \lceil R/T_j \rceil C_j$, techos y bloqueo único, restricciones de marco.

1 Utilización

$U = \frac{2}{8} + \frac{3}{12} + \frac{4}{24} = 0{,}25 + 0{,}25 + 0{,}1667 = 0{,}667$. La cota para $n = 3$ es $3(2^{1/3} - 1) = 0{,}780$. Como $0{,}667 \le 0{,}780$, es **planificable** con RM (condición suficiente).

2 Tiempos de respuesta sin bloqueo

Prioridades RM: $\tau_1 > \tau_2 > \tau_3$.

- $R_1 = 2$.
- R_2: $3 \to 3 + \lceil 3/8 \rceil 2 = 5 \to 3 + \lceil 5/8 \rceil 2 = 5$: $R_2 = 5$.

- R_3: $4 \to 4+2+3=9 \to 4+\lceil 9/8\rceil 2+\lceil 9/12\rceil 3 = 4+4+3=11 \to 4+\lceil 11/8\rceil 2+\lceil 11/12\rceil 3 = 11$: $R_3 = 11$.

Todos $\leq D_i$.

3 Bloqueos con ICPP

Techo de `r1` = prioridad de τ_1 (la mayor de las que lo usan).

- B_1: bloquea la sección crítica de τ_3 con techo $\geq P_1$: $B_1 = 2$.
- B_2: τ_3 (menor prioridad) tiene `r1` con techo $P_1 \geq P_2$: $B_2 = 2$ (aunque τ_2 no usa `r1`: es el bloqueo **por techo**, típico del ICPP).
- $B_3 = 0$ (no hay tareas de menor prioridad).

Recurrencias: $R_1 = 2+2 = 4 \leq 8$; R_2: $3+2=5 \to 5+\lceil 5/8\rceil 2 = 7 \to 5+2=7$: $R_2 = 7 \leq 12$; $R_3 = 11$ (igual).

4 Ejecutivo cíclico

$H = \text{mcm}(8,12,24) = 24$, $C_{max} = 4$. Candidatos $T_s \mid 24$ con $T_s \geq 4$: 4, 6, 8, 12, 24. Condición $2T_s - \text{gcd}(T_s, T_i) \leq D_i$:

- $T_s = 4$: $8-4=4 \leq 8$, $8-4=4 \leq 12$, $8-4=4 \leq 24$. ✓.
- $T_s = 6$: para $T_i = 8$, $12-2=10 > 8$. ✗
- $T_s = 8$: $16-8=8 \leq 8$, $16-4=12 \leq 12$, $16-8=8 \leq 24$. ✓.
- $T_s = 12$: para $T_i = 8$, $24-4=20 > 8$. ✗
- $T_s = 24$: ✗.

Válidos: **4 y 8**.

5 Plan con $T_s = 8$ (3 marcos de 8)

- Marco 1 (0–8): τ_1 (2) + τ_2 (3) = 5.
- Marco 2 (8–16): τ_1 (2) + τ_3 (4) = 6.
- Marco 3 (16–24): τ_1 (2) + τ_2 (3) = 5.

Comprobación: τ_2 se libera en 0 y 12: se ejecuta en 0–8 y 16–24 (ambas dentro de su ventana, el segundo trabajo empieza tras su liberación en 12). τ_3 (plazo 24) en 8–16. Ningún marco pasa de 8.

Resultado final

Salida

```
U = 0.666667, cota n=3: 0.779763
Sin bloqueo: R1=2 R2=5 R3=11
Con ICPP (B=2,2,0): R1=4 R2=7 R3=11
H = 24, Cmax = 4
Marcos validos: 4 8
```

17 Formularios

Las fórmulas clave de cada tema, juntas. En el archivo aparte tienes una versión de una página para imprimir.

17.1 T0 · Sistemas de tiempo real

Definiciones clave

- Tarea periódica: (C, T, D) con $D \leq T$ habitual.
- Utilización de una tarea: $U_i = C_i/T_i$.
- Utilización del sistema: $U = \sum_i C_i/T_i$.

17.2 T1 · Repaso de C y C++

Plantillas rápidas

- Contar palabras: `++hist[w];`
- ¿Existe la clave?: `if (m.find(k) != m.end())`
- Recorrer mapa: `for (const auto& [k, v] : m)`
- Borrar con condición: `it = v.erase(it);` o `v.erase(remove_if(...), v.end());`
- Ordenar por criterio: `sort(b, e, [](const T& a, const T& b){ return ...; });`

17.3 T2 · Clases y objetos

Plantilla de clase para el examen

CPP

```cpp
class C_X {
public:
    C_X(tipo a, tipo b) : a_{a}, b_{b} {}          // constructor
    tipo a() const { return a_; }                   // getters const
    friend ostream& operator<<(ostream& os, const C_X& x) { return os << x.a_; }
private:
    tipo a_, b_;
};   // <- el ; final
```

17.4 T3 · Herencia y polimorfismo

Plantilla de jerarquía

CPP

```cpp
class Base {
public:
    virtual ~Base() = default;
    virtual double f() const = 0;                       // pura
    friend ostream& operator<<(ostream& o, const Base& b) { return o << b.to_string(); }
protected:
    virtual string to_string() const { return "base"; }
};
class Der : public Base {
public:
    double f() const override { return 1; }
protected:
    string to_string() const override { return "der " + Base::to_string(); }
};
// uso: vector<Base*> v; for (auto* p : v) cout << *p;
```

17.5 T4 · Hilos y exclusión mutua

Plantillas de concurrencia

- Lanzar y esperar: `vector<thread> v; for(...) v.emplace_back(f, args); for (auto& h : v) h.join();`
- Sección crítica: `{ lock_guard<mutex> lk{m}; /* tocar datos protegidos */ }`
- Dos recursos: `scoped_lock lk{m1, m2};`
- Periódica: `auto sig = steady_clock::now() + T; loop { trabajo(); sleep_until(sig); sig += T; }`

17.6 T5 · Monitores y variables condición

Plantilla de monitor

CPP

```
class Monitor {
public:
    void operacion() {
        unique_lock<mutex> lk{m_};
        cv_.wait(lk, [this]{ return puedo_; });
        /* actualizar estado */
        cv_.notify_all();
    }
private:
    mutex m_;
    condition_variable cv_;
    /* estado protegido por m_ */
};
```

17.7 T6 · Planificación

Planificación: fórmulas clave

- Utilización: $U = \sum C_i/T_i$.
- Cota RM (D=T): $U \leq n(2^{1/n} - 1)$ — $n = 1$: 1; $n = 2$: $0{,}828$; $n = 3$: $0{,}780$.
- EDF (D=T): $U \leq 1$.
- Hiperperiodo: $T_M = \text{mcm}(T_i)$.
- Ciclo secundario: $T_S \mid T_M$, $T_S \geq \max C_i$, $2T_S - \gcd(T_S, T_i) \leq D_i$.
- Tiempo de respuesta: $R_i = C_i + B_i + \sum_{j \in hp(i)} \lceil R_i/T_j \rceil C_j$.
- ICPP: techo$(r) = \max P_k$ de quienes usan r; $B_i = \max c_{k,r}$ con $P_k < P_i$ y techo$(r) \geq P_i$.

17.8 T7 · Coma flotante y casos reales

Coma flotante

- `double`: 1 bit de signo + 11 de exponente + 52 de mantisa (≈ 15-16 cifras decimales).
- `float`: 1 + 8 + 23 (≈ 7 cifras decimales).
- Comparar: `fabs(a - b) < tolerancia`.
- Error acumulado ≈ (error por operación) × (número de operaciones).
- Patriot: $e = 0{,}1 - \lfloor 0{,}1 \cdot 2^{23} \rfloor / 2^{23} \approx 9{,}5 \cdot 10^{-8}$ s; en 100 h: 0,34 s.

17.9 Prácticas

Resumen de prácticas

- P1: `++hist[p]`, `find` para consultar, `sort` con lambda para ordenar por valor.
- P2: constructor delegado, `operator<<` con `to_string()` virtual, destructor virtual, contenedor recursivo con punteros.

- P3: filas disjuntas ⇒ sin mutex; contador compartido ⇒ mutex; varios recursos ⇒ `scoped_lock`; esperar condición ⇒ `condition_variable` con predicado.

18 Glosario

Amortizado
: Coste medio por operación sobre una secuencia larga, aunque alguna operación aislada sea cara.

Atomic
: Tipo cuyas operaciones son indivisibles.

Barrera
: Punto en el que varios hilos esperan hasta que se cumple una condición común.

Bloqueo
: Tiempo que una tarea espera a otra de menor prioridad por un recurso compartido.

Bloqueo (B)
: Tiempo máximo que una tarea espera por una tarea de menor prioridad que usa un recurso.

Clase abstracta
: Clase con algún método virtual puro; no se puede instanciar.

Condición de carrera
: Resultado que depende del orden en que se intercalan hilos que acceden a un dato compartido.

const (método)
: Método que promete no modificar el objeto.

Constructor delegado
: Constructor que llama a otro constructor de la misma clase.

Copy-and-swap
: Técnica para la asignación: copiar a un temporal e intercambiar.

Criba de Eratóstenes
: Algoritmo para marcar los primos hasta N tachando múltiplos de cada primo.

Deadline Monotonic (DM)
: Prioridad fija: a menor plazo, mayor prioridad.

Deriva
: Error de tiempo que se acumula tic tras tic.

Despertar espurio
: `wait` que se despierta sin que haya `notify`; por eso se usa predicado.

Destructor virtual
: Obligatorio en la base de una jerarquía polimórfica para destruir bien con `delete` a través de la base.

Ejecutivo cíclico
: Planificación estática: un plan fijo se repite cada ciclo principal.

Enlace dinámico
: Elección de método en tiempo de ejecución con la vtable (método virtual vía puntero/referencia).

Épsilon de máquina
: Distancia entre 1 y el siguiente número representable ($\approx 2{,}2 \cdot 10^{-16}$ en `double`).

Espera activa
: Comprobar una condición en bucle consumiendo CPU, en lugar de dormir.

friend
: Función externa con acceso a los miembros privados de una clase.

Herencia de prioridad
: Quien tiene el recurso hereda la prioridad del más prioritario que espera por él.

Hiperperiodo
: Mínimo común múltiplo de los periodos; el plan se repite cada hiperperiodo.

Hiperperiodo
: Mínimo común múltiplo de los periodos; el plan cíclico se repite cada H.

Instante crítico
: Instante en que todas las tareas se liberan a la vez; es el peor caso para RM.

Interbloqueo (deadlock)
: Varios hilos esperan indefinidamente recursos que tienen los otros.

Inversión de prioridad
: Una tarea de alta prioridad espera a una de baja que a su vez es retrasada por una de prioridad media.

Iterador
: Objeto que apunta a un elemento de un contenedor y permite avanzar.

Jitter
: Variación del instante real de activación o finalización respecto al ideal.

join Bloquea al hilo llamador hasta que el otro hilo termina.

Lambda
Función anónima escrita en el sitio de uso: `[captura](params){...}`.

Lista de inicialización
Forma de construir atributos antes de entrar al cuerpo del constructor.

Mantisa
Parte fraccionaria de un número en coma flotante (52 bits en `double`).

Marco secundario
Intervalo de tamaño T_s en el que se ejecutan enteras las tareas asignadas, sin apropiación.

Monitor
Clase que encapsula datos compartidos, su mutex y sus variables condición.

Mutex
Objeto de exclusión mutua: solo un hilo lo posee a la vez.

override
Palabra clave que obliga al compilador a comprobar que se redefine un método virtual.

Periodo (T)
Tiempo entre dos activaciones consecutivas de una tarea periódica.

Plantilla (template)
Código genérico que se instancia para cada tipo concreto.

Plazo (D)
Tiempo máximo desde la activación de una tarea hasta que debe terminar.

Polimorfismo
Misma llamada, distinto código según el tipo real del objeto.

Productor–consumidor
Patrón en el que unos hilos generan datos y otros los consumen a través de un buffer.

Puntero
Variable que guarda una dirección de memoria. Se desreferencia con *.

RAII
Adquirir un recurso en la construcción y liberarlo en la destrucción (lock_guard).

Rate Monotonic (RM)
Prioridad fija: a menor periodo, mayor prioridad.

Referencia
Otro nombre para una variable ya existente. No se puede reasignar ni es nula.

Regla de 0/3/5
Si no gestionas recursos, no escribas nada; si escribes destructor/copia/asignación, escribe los tres (o los cinco con movimiento).

Sección crítica
Fragmento de código que accede a datos compartidos y debe ejecutarse en exclusión mutua.

size / capacity
Elementos que hay frente a espacio reservado en un vector.

Slicing
Pérdida de la parte derivada al copiar un objeto derivado a uno base por valor.

static (atributo)
Atributo compartido por todos los objetos de la clase.

Techo de prioridad
Mayor prioridad entre las tareas que usan un recurso.

Techo de prioridad inmediato
Al tomar un recurso la tarea sube a su techo (prioridad máxima de sus usuarios).

Tiempo de cómputo (C)
Tiempo máximo de ejecución de una tarea (peor caso).

Tiempo de respuesta
Tiempo máximo desde la activación de una tarea hasta que termina.

Tiempo real blando
Un resultado fuera de plazo vale, pero menos.

Tiempo real duro
Sistema donde perder un plazo es un fallo catastrófico.

Tiempo real firme
Un resultado fuera de plazo no vale y se descarta; el sistema puede seguir.

Truncamiento
: Descartar los bits que no caben, sin redondear.

unique_ptr
: Puntero inteligente con un único dueño; libera al destruirse.

Variable condición
: Primitiva que permite a un hilo dormir hasta que otro avise de que cambió una condición.

Watchdog
: Temporizador que reinicia el sistema si no se le «alimenta» a tiempo.

19 Plan de estudio

19.1 Plan por semanas

Semana 1 · Cimientos de C++ (T0 y T1)

Lee «Empieza aquí» y el tema de tiempo real. Después, T1: referencias, punteros, `string`, `vector`, `map`, `set`, iteradores, lambdas y `algorithm`. Haz la práctica 1 (palabras e histograma) y las hojas 1 y 2. Meta de la semana: escribir sin ayuda un programa que lea un fichero, cuente palabras con `map` y las ordene con `sort` + lambda. Marca la lista de seguimiento al terminar cada apartado.

Semana 2 · Clases (T2)

Constructores, lista de inicialización, `const`, `friend`, `operator<<` y las reglas de 0, 3 y 5 con copy-and-swap. Haz la práctica 2 y la hoja 3 (esfera, caja). Resuelve **sin mirar** el ejercicio 1 de un examen reciente y compara. Meta: escribir una clase con memoria dinámica (la `Matriz` del simulacro 2) que compile a la primera.

Semana 3 · Herencia y polimorfismo (T3)

Clase abstracta, `virtual`, `override`, destructor virtual, `unique_ptr` en colecciones, `typeid`, troceado (slicing) y gancho `imprimir` para `operator<<`. Haz el simulacro 1 (ejercicio 1) con reloj. Repasa en «Qué suele fallar» los errores de compilación más comunes: ellos pueden dividir tu nota por 2.

Semana 4 · Concurrencia (T4 y T5)

Hilos, `join`, carreras, `mutex`, `lock_guard`, `atomic`, interbloqueos y monitores con `condition_variable` (predicado, `notify_all`, bandera de cierre). Haz la práctica 3 y los ejercicios 2 de los exámenes de 2025 y 2026. Meta: escribir un monitor productor-consumidor con cierre limpio en 20 minutos.

Semana 5 · Planificación y coma flotante (T6 y T7)

Test de utilización, tiempo de respuesta con bloqueo, ejecutivo cíclico, EDF. Estudia las demostraciones 1, 2 y 5. Haz el simulacro 3 y los ejercicios de planificación de exámenes antiguos. Repasa el caso Patriot y la comparación de reales con tolerancia.

Semana 6 · Repaso y simulacros

Haz los simulacros 1 y 2 completos (2 h 30) en un solo bloque, sin mirar nada salvo la hoja de referencia de la STL. Rehaz los test de autoevaluación y las tarjetas. Lee solo el formulario de una página y los recuadros «Qué suele fallar». Último día: descanso y revisar lo que más has fallado.

Si solo te quedan 3 días

Día 1: T1 y T2 con sus ejercicios de examen más recientes (son los temas que más aparecen en el recuento de los exámenes: STL y clases). **Día 2:** T3 y T5 (herencia y monitores; de ahí salen casi siempre el ejercicio de POO con jerarquía y el de concurrencia). **Día 3:** un simulacro completo con reloj y repaso de errores. Deja la planificación para el final y solo si te piden ese tipo de ejercicio en tu convocatoria.

Reglas para el día del examen

Lee el enunciado completo antes de escribir. Empieza por lo que mejor sepas. **Compila siempre antes de entregar** (si no compila, la nota puede dividirse por 2 en el examen reciente). Comenta qué hace cada clase en una línea. No dejes ningún ejercicio en blanco: un esqueleto correcto da puntos parciales.